



Security Engineering

**A Guide to Building Dependable
Distributed Systems
Third Edition**

Ross Anderson

WILEY

Copyright © 2020 by Ross Anderson
Published by John Wiley & Sons, Inc., Indianapolis, Indiana
Published simultaneously in Canada and the United Kingdom

ISBN: 978-1-119-64278-7
ISBN: 978-1-119-64283-1 (ebk)
ISBN: 978-1-119-64281-7 (ebk)

Manufactured in the United States of America

No part of this publication may be reproduced, stored in a retrieval system or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, scanning or otherwise, except as permitted under Sections 107 or 108 of the 1976 United States Copyright Act, without either the prior written permission of the Publisher, or authorization through payment of the appropriate per-copy fee to the Copyright Clearance Center, 222 Rosewood Drive, Danvers, MA 01923, (978) 750-8400, fax (978) 646-8600. Requests to the Publisher for permission should be addressed to the Permissions Department, John Wiley & Sons, Inc., 111 River Street, Hoboken, NJ 07030, (201) 748-6011, fax (201) 748-6008, or online at www.wiley.com/go/permissions.

Limit of Liability/Disclaimer of Warranty: The publisher and the author make no representations or warranties with respect to the accuracy or completeness of the contents of this work and specifically disclaim all warranties, including without limitation warranties of fitness for a particular purpose. No warranty may be created or extended by sales or promotional materials. The advice and strategies contained herein may not be suitable for every situation. This work is sold with the understanding that the publisher is not engaged in rendering legal, accounting, or other professional services. If professional assistance is required, the services of a competent professional person should be sought. Neither the publisher nor the author shall be liable for damages arising herefrom. The fact that an organization or Web site is referred to in this work as a citation and/or a potential source of further information does not mean that the author or the publisher endorses the information the organization or website may provide or recommendations it may make. Further, readers should be aware that Internet websites listed in this work may have changed or disappeared between when this work was written and when it is read.

For general information on our other products and services please contact our Customer Care Department within the United States at (877) 762-2974, outside the United States at (317) 572-3993 or fax (317) 572-4002.

Wiley publishes in a variety of print and electronic formats and by print-on-demand. Some material included with standard print versions of this book may not be included in e-books or in print-on-demand. If this book refers to media such as a CD or DVD that is not included in the version you purchased, you may download this material at booksupport.wiley.com. For more information about Wiley products, visit www.wiley.com.

Library of Congress Control Number: 2020948679

Trademarks: Wiley and the Wiley logo are trademarks or registered trademarks of John Wiley & Sons, Inc. and/or its affiliates, in the United States and other countries, and may not be used without written permission. All other trademarks are the property of their respective owners. John Wiley & Sons, Inc. is not associated with any product or vendor mentioned in this book.

For Shireen, Bavani, Nav, Ivan, Lily-Rani, Veddie and Bella



About the Author

I've worked with systems for over forty years. I graduated in mathematics and natural science from Cambridge in the 1970s, and got a qualification in computer engineering; my first proper job was in avionics; and after getting interested in cryptology and computer security, I worked in the banking industry in the 1980s. I then started working for companies who designed equipment for banks, and then on related applications such as prepayment electricity meters.

I moved to academia in 1992 but continued to consult to industry on security technology. During the 1990s, the number of applications that used cryptology rose rapidly: burglar alarms, car door locks, road toll tags and satellite TV systems all made their appearance. The first legal disputes about these systems came along, and I was lucky enough to be an expert witness in some of the important cases. The research team I lead had the good fortune to be in the right place at the right time when technologies such as peer-to-peer systems, tamper-resistance and digital watermarking became hot topics.

After I'd taught security and cryptology to students for a few years, it became clear to me that the existing textbooks were too narrow and theoretical: the security textbooks focused on the access control mechanisms in operating systems, while the cryptology books developed the theory behind cryptographic algorithms and protocols. These topics are interesting, and important. But they're only part of the story. Most working engineers are not overly concerned with crypto or operating system internals, but with getting good tools and learning how to use them effectively. The inappropriate use of protection mechanisms is one of the main causes of security failure. I was encouraged by the positive reception of a number of articles I wrote on security engineering (starting with 'Why Cryptosystems Fail' in 1993).

Finally, in 1999, I got round to rewriting my class lecture notes and a number of real-world case studies into a book for a general technical audience.

The first edition of the book, which appeared in 2001, helped me consolidate my thinking on the economics of information security, as I found that when I pulled my experiences about some field together into a narrative, the backbone of the story was often the incentives that the various players had faced. As the first edition of this book established itself as the standard textbook in the field, I worked on establishing security economics as a discipline. In 2002, we started the Workshop on the Economics of Information Security to bring researchers and practitioners together.

By the time the second edition came out in 2008, it was clear we'd not paid enough attention to the psychology of security either. Although we'd worked on security usability from the 1990s, there's much more to it than that. We need to understand everything from the arts of deception to how people's perception of risk is manipulated. So in 2008 we started the Workshop on Security and Human Behaviour to get security engineers talking to psychologists, anthropologists, philosophers and even magicians.

A sabbatical in 2011, which I spent partly at Google and partly at Carnegie Mellon University, persuaded me to broaden our research group to hire psychologists and criminologists. Eventually in 2015 we set up the Cambridge Cybercrime Centre to collect lots of data on the bad things that happen online and make them available to over a hundred researchers worldwide. This hasn't stopped us doing research on technical security; in fact it's helped us pick more relevant technical research topics.

A medic needs to understand a whole series of subjects including anatomy, physiology, biochemistry, pharmacy and psychology, and then temper this knowledge with experience of working on hundreds of cases with experienced colleagues. So also a security engineer needs to understand technical subjects like crypto, access controls, protocols and side channels; but this knowledge also needs to be honed by studying real cases. My goal in my academic career has been to pull all this together. The result you now hold in your hands.

I have learned a lot in the process; writing down what you think you know is a good way of finding out what you don't. I have also had a lot of fun. I hope you have as much fun reading it!

Ross Anderson
Cambridge, November 2020



Acknowledgements

A great many people have helped in various ways with the third edition of this book. I put the chapters online for comment as I wrote them, and I owe thanks to the many people who read them and pointed out assorted errors and obscurities. They are: Mansoor Ahmed, Sam Ainsworth, Peter Allan, Amit Seal Ami, James Andrews, Tom Auger, Asokan, Maria Bada, Daniel Bates, Craig Bauer, Pilgrim Beart, Gerd Beuster, Johann Bezuidenhout, Fred Bone, Matt Brockman, Nick Bohm, Fred Bone, Phil Booth, Lorenzo Cavallaro, David Chaiken, Yi Ting Chua, Valerio Cini, Ben Collier, Hugo Connery, Lachlan Cooper, Franck Courbon, Christopher Cowan, Ot van Daalen, Ezra Darshan, Roman Dickmann, Saar Drimer, Charles Duffy, Marlena Erdos, Andy Farnell, Bob Fenichel, David Fernée, Alexis FitzGerald, Jean-Alain Fournier, Jordan Frank, Steve Friedl, Jerry Gamache, Alex Gantman, Ben Gardiner, Jon Geater, Stuart Gentry, Cam Gerlach, John Gilmore, Jan Goette, Ralph Gross, Cyril Guerin, Pedram Hayati, Chengying He, Matt Hermannson, Alex Hicks, Ross Hinds, Timothy Howell, Nick Humphrey, James Humphry, Duncan Hurwood, Gary Irvine, Erik Itland, Christian Jeschke, Gary Johnson, Doug Jones, Henrik Karlzen, Joud Khoury, Jon Kilian, Timm Korte, Ronny Kuckuck, Mart Kung, Jay Lala, Jack Lang, Susan Landau, Peter Landrock, Carl Landwehr, Peter Lansley, Jeff Leese, Jochen Leidner, Tom de Leon, Andrew Lewis, David Lewis, Steve Lipner, Jim Lippard, Liz Louis, Simon Luyten, Christian Mainka, Dhruv Malik, Ivan Marsa-Maestra, Phil Maud, Patrick McCorry, TJ McIntyre, Marco Mesturino, Luke Mewburn, Spencer Moss, Steven Murdoch, Arvind Narayanan, Lakshmi Narayanan, Kristi Nikolla, Greg Norcie, Stanislav Ochotnický, Andy Ozment, Deborah Peel, Stephen Perlmutter, Tony Plank, William Porquet, David Pottage, Mark Quevedo, Roderick Rees, Larry Reeves, Philipp Reisinger, Mark Richards, Niklas Rosencrantz, Andy Sayler, Philipp

Schaumann, Christian Schneider, Ben Scott, Jean-Pierre Seifert, Mark Shawyer, Adam Shostack, Ilia Shumailov, Barbara Simons, Sam Smith, Saija Sorsa, Michael Specter, Chris Tarnowski, Don Taylor, Andrew Thaeler, Kurt Thomas, Anthony Vance, Jonas Vautherin, Alex Vetterl, Jeffrey Walton, Andrew Watson, Debora Weber-Wulff, Nienke Weiland, David White, Blake Wiggs, Robin Wilton, Ron Woerner, Bruno Wolff, Stuart Wray, Jeff Yan, Tom Yates, Andrew Yeomans, Haarooun Yousaf, Tim Zander and Yiren Zhao. I am also grateful to my editors at Wiley, Tom Dinse, Jim Minatel and Pete Gaughan, and to my copyeditors Judy Flynn and Kim Wimpsett, who have all helped make the process run smoothly.

The people who contributed in various ways to the first and second editions included the late Anne Anderson, Adam Atkinson, Jean Bacon, Robin Ball, Andreas Bender, Alastair Beresford, Johann Bezuidenhout, Maximilian Blochberger, David Boddie, Kristof Boeynaems, Nick Bohm, Mike Bond, Richard Bondi, Robert Brady, Martin Brain, John Brazier, Ian Brown, Mike Brown, Nick Bohm, Richard Bondi, the late Caspar Bowden, Duncan Campbell, Piotr Carlson, Peter Chambers, Valerio Cini, Richard Clayton, Frank Clish, Jolyon Clulow, Richard Cox, Dan Cvrcek, George Danezis, James Davenport, Peter Dean, John Daugman, Whit Diffie, Roger Dingledine, Nick Drage, Austin Donnelly, Ben Dougall, Saar Drimer, Orr Dunkelman, Steve Early, Dan Eble, Mike Ellims, Jeremy Epstein, Rasit Eskicioğlu, Robert Fenichel, Fleur Fisher, Shawn Fitzgerald, Darren Foong, Shailendra Fuloria, Dan Geer, Gary Geldart, Paul Gillingwater, John Gilmore, Brian Gladman, Virgil Gligor, Bruce Godfrey, John Gordon, Gary Graunke, Rich Graveman, Wendy Grossman, Dan Hagon, Feng Hao, Tony Harminc, Pieter Hartel, David Häsäther, Bill Hey, Fay Hider, Konstantin Hyppönen, Ian Jackson, Neil Jenkins, Simon Jenkins, Roger Johnston, Oliver Jorns, Nikolaos Karapanos, the late Paul Karger, Ian Kelly, Grant Kelly, Alistair Kelman, Ronald De Keulenaer, Hyoungh Joong Kim, Patrick Koeberl, Oliver Kömmerling, Simon Kramer, Markus Kuhn, Peter Landrock, Susan Landau, Jack Lang, Jong-Hyeon Lee, the late Owen Lewis, Stephen Lewis, Paul Leyland, Jim Lippard, Willie List, Dan Lough, John McHugh, the late David MacKay, Garry McKay, Udi Manber, John Martin, Nick Mathewson, Tyler Moore, the late Bob Morris, Ira Moskowitz, Steven Murdoch, Shishir Nagaraja, Roger Nebel, the late Roger Needham, Stephan Neuhaus, Andrew Odlyzko, Mark Oeltjenbruns, Joe Osborne, Andy Ozment, Alexandros Papadopoulos, Roy Paterson, Chris Pepper, Oscar Pereira, Fabien Petitcolas, Raphael Phan, Mike Roe, Mark Rotenberg, Avi Rubin, Jerry Saltzer, Marv Schaefer, Denise Schmandt-Besserat, Gus Simmons, Sam Simpson, Sergei Skorobogatov, Matthew Slyman, Rick Smith, Sijbrand Spannenburg, the late Karen Spärck Jones, Mark Staples, Frank Stajano, Philipp Steinmetz, Nik Sultana, Don Taylor, Martin Taylor, Peter Taylor, Daniel Thomas, Paul Thomas,

Vlasios Tsiatsis, Marc Tobias, Hal Varian, Nick Volenec, Daniel Wagner-Hall, Randall Walker, Robert Watson, Keith Willis, Simon Wiseman, Stuart Wray, Jeff Yan and the late Stefek Zaba. I also owe a lot to my first publisher, Carol Long.

Through the whole process I have been supported by my family, and especially by my long-suffering wife Shireen. Each edition of the book meant over a year when I was constantly distracted. Huge thanks to all for putting up with me!



Contents at a Glance

Preface to the Third Edition	xxxvii
Preface to the Second Edition	xli
Preface to the First Edition	xlili
For my daughter, and other lawyers ...	xlvi
Foreword	xlix
Part I	
Chapter 1 What Is Security Engineering?	3
Chapter 2 Who Is the Opponent?	17
Chapter 3 Psychology and Usability	63
Chapter 4 Protocols	119
Chapter 5 Cryptography	145
Chapter 6 Access Control	207
Chapter 7 Distributed Systems	243
Chapter 8 Economics	275

Part II

Chapter 9	Multilevel Security	315
Chapter 10	Boundaries	341
Chapter 11	Inference Control	375
Chapter 12	Banking and Bookkeeping	405
Chapter 13	Locks and Alarms	471
Chapter 14	Monitoring and Metering	497
Chapter 15	Nuclear Command and Control	529
Chapter 16	Security Printing and Seals	549
Chapter 17	Biometrics	571
Chapter 18	Tamper Resistance	599
Chapter 19	Side Channels	639
Chapter 20	Advanced Cryptographic Engineering	667
Chapter 21	Network Attack and Defence	699
Chapter 22	Phones	737
Chapter 23	Electronic and Information Warfare	777
Chapter 24	Copyright and DRM	815
Chapter 25	New Directions?	865

Part III

Chapter 26	Surveillance or Privacy?	909
Chapter 27	Secure Systems Development	965
Chapter 28	Assurance and Sustainability	1015
Chapter 29	Beyond “Computer Says No”	1059

Bibliography	1061
---------------------	-------------

Index	1143
--------------	-------------



Contents

Preface to the Third Edition	xxxvii
Preface to the Second Edition	xli
Preface to the First Edition	xlili
For my daughter, and other lawyers ...	xlvii
Foreword	xlix
Part I	
Chapter 1 What Is Security Engineering?	3
1.1 Introduction	3
1.2 A framework	4
1.3 Example 1 – a bank	6
1.4 Example 2 – a military base	7
1.5 Example 3 – a hospital	8
1.6 Example 4 – the home	10
1.7 Definitions	11
1.8 Summary	16
Chapter 2 Who Is the Opponent?	17
2.1 Introduction	17
2.2 Spies	19
2.2.1 The Five Eyes	19
2.2.1.1 Prism	19
2.2.1.2 Tempora	20
2.2.1.3 Muscular	21
2.2.1.4 Special collection	22

	2.2.1.5	Bullrun and Edgehill	22
	2.2.1.6	Xkeyscore	23
	2.2.1.7	Longhaul	24
	2.2.1.8	Quantum	25
	2.2.1.9	CNE	25
	2.2.1.10	The analyst's viewpoint	27
	2.2.1.11	Offensive operations	28
	2.2.1.12	Attack scaling	29
	2.2.2	China	30
	2.2.3	Russia	35
	2.2.4	The rest	38
	2.2.5	Attribution	40
2.3	Crooks		41
	2.3.1	Criminal infrastructure	42
		2.3.1.1 Botnet herders	42
		2.3.1.2 Malware devs	44
		2.3.1.3 Spam senders	45
		2.3.1.4 Bulk account compromise	45
		2.3.1.5 Targeted attackers	46
		2.3.1.6 Cashout gangs	46
		2.3.1.7 Ransomware	47
	2.3.2	Attacks on banking and payment systems	47
	2.3.3	Sectoral cybercrime ecosystems	49
	2.3.4	Internal attacks	49
	2.3.5	CEO crimes	49
	2.3.6	Whistleblowers	50
2.4	Geeks		52
2.5	The swamp		53
	2.5.1	Hacktivism and hate campaigns	54
	2.5.2	Child sex abuse material	55
	2.5.3	School and workplace bullying	57
	2.5.4	Intimate relationship abuse	57
2.6	Summary		59
		Research problems	60
		Further reading	61
Chapter 3	Psychology and Usability		63
	3.1	Introduction	63
	3.2	Insights from psychology research	64
		3.2.1 Cognitive psychology	65
		3.2.2 Gender, diversity and interpersonal variation	68

3.2.3	Social psychology	70
3.2.3.1	Authority and its abuse	71
3.2.3.2	The bystander effect	72
3.2.4	The social-brain theory of deception	73
3.2.5	Heuristics, biases and behavioural economics	76
3.2.5.1	Prospect theory and risk misperception	77
3.2.5.2	Present bias and hyperbolic discounting	78
3.2.5.3	Defaults and nudges	79
3.2.5.4	The default to intentionality	79
3.2.5.5	The affect heuristic	80
3.2.5.6	Cognitive dissonance	81
3.2.5.7	The risk thermostat	81
3.3	Deception in practice	81
3.3.1	The salesman and the scamster	82
3.3.2	Social engineering	84
3.3.3	Phishing	86
3.3.4	Opsec	88
3.3.5	Deception research	89
3.4	Passwords	90
3.4.1	Password recovery	92
3.4.2	Password choice	94
3.4.3	Difficulties with reliable password entry	94
3.4.4	Difficulties with remembering the password	95
3.4.4.1	Naïve choice	96
3.4.4.2	User abilities and training	96
3.4.4.3	Design errors	98
3.4.4.4	Operational failures	100
3.4.4.5	Social-engineering attacks	101
3.4.4.6	Customer education	102
3.4.4.7	Phishing warnings	103
3.4.5	System issues	104
3.4.6	Can you deny service?	105
3.4.7	Protecting oneself or others?	105
3.4.8	Attacks on password entry	106
3.4.8.1	Interface design	106
3.4.8.2	Trusted path, and bogus terminals	107
3.4.8.3	Technical defeats of password retry counters	107
3.4.9	Attacks on password storage	108
3.4.9.1	One-way encryption	109
3.4.9.2	Password cracking	109
3.4.9.3	Remote password checking	109

	3.4.10 Absolute limits	110
	3.4.11 Using a password manager	111
	3.4.12 Will we ever get rid of passwords?	113
3.5	CAPTCHAs	115
3.6	Summary	116
	Research problems	117
	Further reading	118
Chapter 4	Protocols	119
4.1	Introduction	119
4.2	Password eavesdropping risks	120
4.3	Who goes there? – simple authentication	122
	4.3.1 Challenge and response	124
	4.3.2 Two-factor authentication	128
	4.3.3 The MITM-in-the-middle attack	129
	4.3.4 Reflection attacks	132
4.4	Manipulating the message	133
4.5	Changing the environment	134
4.6	Chosen protocol attacks	135
4.7	Managing encryption keys	136
	4.7.1 The resurrecting duckling	137
	4.7.2 Remote key management	137
	4.7.3 The Needham-Schroeder protocol	138
	4.7.4 Kerberos	139
	4.7.5 Practical key management	141
4.8	Design assurance	141
4.9	Summary	143
	Research problems	143
	Further reading	144
Chapter 5	Cryptography	145
5.1	Introduction	145
5.2	Historical background	146
	5.2.1 An early stream cipher – the Vigenère	147
	5.2.2 The one-time pad	148
	5.2.3 An early block cipher – Playfair	150
	5.2.4 Hash functions	152
	5.2.5 Asymmetric primitives	154
5.3	Security models	155
	5.3.1 Random functions – hash functions	157
	5.3.1.1 Properties	157
	5.3.1.2 The birthday theorem	158
	5.3.2 Random generators – stream ciphers	159
	5.3.3 Random permutations – block ciphers	161

5.3.4	Public key encryption and trapdoor one-way permutations	163
5.3.5	Digital signatures	164
5.4	Symmetric crypto algorithms	165
5.4.1	SP-networks	165
5.4.1.1	Block size	166
5.4.1.2	Number of rounds	166
5.4.1.3	Choice of S-boxes	167
5.4.1.4	Linear cryptanalysis	167
5.4.1.5	Differential cryptanalysis	168
5.4.2	The Advanced Encryption Standard (AES)	169
5.4.3	Feistel ciphers	171
5.4.3.1	The Luby-Rackoff result	173
5.4.3.2	DES	173
5.5	Modes of operation	175
5.5.1	How not to use a block cipher	176
5.5.2	Cipher block chaining	177
5.5.3	Counter encryption	178
5.5.4	Legacy stream cipher modes	178
5.5.5	Message authentication code	179
5.5.6	Galois counter mode	180
5.5.7	XTS	180
5.6	Hash functions	181
5.6.1	Common hash functions	181
5.6.2	Hash function applications – HMAC, commitments and updating	183
5.7	Asymmetric crypto primitives	185
5.7.1	Cryptography based on factoring	185
5.7.2	Cryptography based on discrete logarithms	188
5.7.2.1	One-way commutative encryption	189
5.7.2.2	Diffie-Hellman key establishment	190
5.7.2.3	ElGamal digital signature and DSA	192
5.7.3	Elliptic curve cryptography	193
5.7.4	Certification authorities	194
5.7.5	TLS	195
5.7.5.1	TLS uses	196
5.7.5.2	TLS security	196
5.7.5.3	TLS 1.3	197
5.7.6	Other public-key protocols	197
5.7.6.1	Code signing	197
5.7.6.2	PGP/GPG	198
5.7.6.3	QUIC	199
5.7.7	Special-purpose primitives	199

5.7.8	How strong are asymmetric cryptographic primitives?	200
5.7.9	What else goes wrong	202
5.8	Summary	203
	Research problems	204
	Further reading	204
Chapter 6	Access Control	207
6.1	Introduction	207
6.2	Operating system access controls	209
6.2.1	Groups and roles	210
6.2.2	Access control lists	211
6.2.3	Unix operating system security	212
6.2.4	Capabilities	214
6.2.5	DAC and MAC	215
6.2.6	Apple's macOS	217
6.2.7	iOS	217
6.2.8	Android	218
6.2.9	Windows	219
6.2.10	Middleware	222
	6.2.10.1 Database access controls	222
	6.2.10.2 Browsers	223
6.2.11	Sandboxing	224
6.2.12	Virtualisation	225
6.3	Hardware protection	227
6.3.1	Intel processors	228
6.3.2	Arm processors	230
6.4	What goes wrong	231
6.4.1	Smashing the stack	232
6.4.2	Other technical attacks	234
6.4.3	User interface failures	236
6.4.4	Remedies	237
6.4.5	Environmental creep	238
6.5	Summary	239
	Research problems	240
	Further reading	240
Chapter 7	Distributed Systems	243
7.1	Introduction	243
7.2	Concurrency	244
7.2.1	Using old data versus paying to propagate state	245
7.2.2	Locking to prevent inconsistent updates	246
7.2.3	The order of updates	247
7.2.4	Deadlock	248

	7.2.5	Non-convergent state	249
	7.2.6	Secure time	250
7.3		Fault tolerance and failure recovery	251
	7.3.1	Failure models	252
	7.3.1.1	Byzantine failure	252
	7.3.1.2	Interaction with fault tolerance	253
	7.3.2	What is resilience for?	254
	7.3.3	At what level is the redundancy?	255
	7.3.4	Service-denial attacks	257
7.4		Naming	259
	7.4.1	The Needham naming principles	260
	7.4.2	What else goes wrong	263
	7.4.2.1	Naming and identity	264
	7.4.2.2	Cultural assumptions	265
	7.4.2.3	Semantic content of names	267
	7.4.2.4	Uniqueness of names	268
	7.4.2.5	Stability of names and addresses	269
	7.4.2.6	Restrictions on the use of names	269
	7.4.3	Types of name	270
7.5		Summary	271
		Research problems	272
		Further reading	273
Chapter 8		Economics	275
	8.1	Introduction	275
	8.2	Classical economics	276
	8.2.1	Monopoly	278
	8.3	Information economics	281
	8.3.1	Why information markets are different	281
	8.3.2	The value of lock-in	282
	8.3.3	Asymmetric information	284
	8.3.4	Public goods	285
	8.4	Game theory	286
	8.4.1	The prisoners' dilemma	287
	8.4.2	Repeated and evolutionary games	288
	8.5	Auction theory	291
	8.6	The economics of security and dependability	293
	8.6.1	Why is Windows so insecure?	294
	8.6.2	Managing the patching cycle	296
	8.6.3	Structural models of attack and defence	298
	8.6.4	The economics of lock-in, tying and DRM	300
	8.6.5	Antitrust law and competition policy	302
	8.6.6	Perversely motivated guards	304

8.6.7	Economics of privacy	305
8.6.8	Organisations and human behaviour	307
8.6.9	Economics of cybercrime	308
8.7	Summary	310
	Research problems	311
	Further reading	311

Part II

Chapter 9	Multilevel Security	315
9.1	Introduction	315
9.2	What is a security policy model?	316
9.3	Multilevel security policy	318
9.3.1	The Anderson report	319
9.3.2	The Bell-LaPadula model	320
9.3.3	The standard criticisms of Bell-LaPadula	321
9.3.4	The evolution of MLS policies	323
9.3.5	The Biba model	325
9.4	Historical examples of MLS systems	326
9.4.1	SCOMP	326
9.4.2	Data diodes	327
9.5	MAC: from MLS to IFC and integrity	329
9.5.1	Windows	329
9.5.2	SELinux	330
9.5.3	Embedded systems	330
9.6	What goes wrong	331
9.6.1	Composability	331
9.6.2	The cascade problem	332
9.6.3	Covert channels	333
9.6.4	The threat from malware	333
9.6.5	Polyinstantiation	334
9.6.6	Practical problems with MLS	335
9.7	Summary	337
	Research problems	338
	Further reading	339
Chapter 10	Boundaries	341
10.1	Introduction	341
10.2	Compartmentation and the lattice model	344
10.3	Privacy for tigers	346
10.4	Health record privacy	349
10.4.1	The threat model	351
10.4.2	The BMA security policy	353
10.4.3	First practical steps	356

10.4.4	What actually goes wrong	357
10.4.4.1	Emergency care	358
10.4.4.2	Resilience	359
10.4.4.3	Secondary uses	359
10.4.5	Confidentiality – the future	362
10.4.6	Ethics	365
10.4.7	Social care and education	367
10.4.8	The Chinese Wall	369
10.5	Summary	371
	Research problems	372
	Further reading	373
Chapter 11	Inference Control	375
11.1	Introduction	375
11.2	The early history of inference control	377
11.2.1	The basic theory of inference control	378
11.2.1.1	Query set size control	378
11.2.1.2	Trackers	379
11.2.1.3	Cell suppression	379
11.2.1.4	Other statistical disclosure control mechanisms	380
11.2.1.5	More sophisticated query controls	381
11.2.1.6	Randomization	382
11.2.2	Limits of classical statistical security	383
11.2.3	Active attacks	384
11.2.4	Inference control in rich medical data	385
11.2.5	The third wave: preferences and search	388
11.2.6	The fourth wave: location and social	389
11.3	Differential privacy	392
11.4	Mind the gap?	394
11.4.1	Tactical anonymity and its problems	395
11.4.2	Incentives	398
11.4.3	Alternatives	399
11.4.4	The dark side	400
11.5	Summary	401
	Research problems	402
	Further reading	402
Chapter 12	Banking and Bookkeeping	405
12.1	Introduction	405
12.2	Bookkeeping systems	406
12.2.1	Double-entry bookkeeping	408
12.2.2	Bookkeeping in banks	408
12.2.3	The Clark-Wilson security policy model	410

12.2.4	Designing internal controls	411
12.2.5	Insider frauds	415
12.2.6	Executive frauds	416
12.2.6.1	The post office case	418
12.2.6.2	Other failures	419
12.2.6.3	Ecological validity	420
12.2.6.4	Control tuning and corporate governance	421
12.2.7	Finding the weak spots	422
12.3	Interbank payment systems	424
12.3.1	A telegraphic history of E-commerce	424
12.3.2	SWIFT	425
12.3.3	What goes wrong	427
12.4	Automatic teller machines	430
12.4.1	ATM basics	430
12.4.2	What goes wrong	433
12.4.3	Incentives and injustices	437
12.5	Credit cards	438
12.5.1	Credit card fraud	439
12.5.2	Online card fraud	440
12.5.3	3DS	443
12.5.4	Fraud engines	444
12.6	EMV payment cards	445
12.6.1	Chip cards	445
12.6.1.1	Static data authentication	446
12.6.1.2	ICVVs, DDA and CDA	450
12.6.1.3	The No-PIN attack	451
12.6.2	The preplay attack	452
12.6.3	Contactless	454
12.7	Online banking	457
12.7.1	Phishing	457
12.7.2	CAP	458
12.7.3	Banking malware	459
12.7.4	Phones as second factors	459
12.7.5	Liability	461
12.7.6	Authorised push payment fraud	462
12.8	Nonbank payments	463
12.8.1	M-Pesa	463
12.8.2	Other phone payment systems	464
12.8.3	Sofort, and open banking	465
12.9	Summary	466
	Research problems	466
	Further reading	468

Chapter 13	Locks and Alarms	471
13.1	Introduction	471
13.2	Threats and barriers	472
13.2.1	Threat model	473
13.2.2	Deterrence	474
13.2.3	Walls and barriers	476
13.2.4	Mechanical locks	478
13.2.5	Electronic locks	482
13.3	Alarms	484
13.3.1	How not to protect a painting	485
13.3.2	Sensor defeats	486
13.3.3	Feature interactions	488
13.3.4	Attacks on communications	489
13.3.5	Lessons learned	493
13.4	Summary	494
	Research problems	495
	Further reading	495
 Chapter 14	 Monitoring and Metering	 497
14.1	Introduction	497
14.2	Prepayment tokens	498
14.2.1	Utility metering	499
14.2.2	How the STS system works	501
14.2.3	What goes wrong	502
14.2.4	Smart meters and smart grids	504
14.2.5	Ticketing fraud	508
14.3	Taxi meters, tachographs and truck speed limiters	509
14.3.1	The tachograph	509
14.3.2	What goes wrong	511
	14.3.2.1 How most tachograph manipulation is done	511
	14.3.2.2 Tampering with the supply	512
	14.3.2.3 Tampering with the instrument	512
	14.3.2.4 High-tech attacks	513
14.3.3	Digital tachographs	514
	14.3.3.1 System-level problems	515
	14.3.3.2 Other problems	516
14.3.4	Sensor defeats and third-generation devices	518
14.3.5	The fourth generation – smart tachographs	518
14.4	Curfew tags: GPS as policeman	519
14.5	Postage meters	522

14.6	Summary	526
	Research problems	527
	Further reading	527
Chapter 15	Nuclear Command and Control	529
15.1	Introduction	529
15.2	The evolution of command and control	532
15.2.1	The Kennedy memorandum	532
15.2.2	Authorization, environment, intent	534
15.3	Unconditionally secure authentication	534
15.4	Shared control schemes	536
15.5	Tamper resistance and PALs	538
15.6	Treaty verification	540
15.7	What goes wrong	541
15.7.1	Nuclear accidents	541
15.7.2	Interaction with cyberwar	542
15.7.3	Technical failures	543
15.8	Secrecy or openness?	544
15.9	Summary	545
	Research problems	546
	Further reading	546
Chapter 16	Security Printing and Seals	549
16.1	Introduction	549
16.2	History	550
16.3	Security printing	551
16.3.1	Threat model	552
16.3.2	Security printing techniques	553
16.4	Packaging and seals	557
16.4.1	Substrate properties	558
16.4.2	The problems of glue	558
16.4.3	PIN mailers	559
16.5	Systemic vulnerabilities	560
16.5.1	Peculiarities of the threat model	562
16.5.2	Anti-gundecking measures	563
16.5.3	The effect of random failure	564
16.5.4	Materials control	564
16.5.5	Not protecting the right things	565
16.5.6	The cost and nature of inspection	566
16.6	Evaluation methodology	567
16.7	Summary	569
	Research problems	569
	Further reading	570

Chapter 17	Biometrics	571
	17.1 Introduction	571
	17.2 Handwritten signatures	572
	17.3 Face recognition	575
	17.4 Fingerprints	579
	17.4.1 Verifying positive or negative identity claims	581
	17.4.2 Crime scene forensics	584
	17.5 Iris codes	588
	17.6 Voice recognition and morphing	590
	17.7 Other systems	591
	17.8 What goes wrong	593
	17.9 Summary	596
	Research problems	597
	Further reading	597
Chapter 18	Tamper Resistance	599
	18.1 Introduction	599
	18.2 History	601
	18.3 Hardware security modules	601
	18.4 Evaluation	607
	18.5 Smartcards and other security chips	609
	18.5.1 History	609
	18.5.2 Architecture	610
	18.5.3 Security evolution	611
	18.5.4 Random number generators and PUFs	621
	18.5.5 Larger chips	624
	18.5.6 The state of the art	628
	18.6 The residual risk	630
	18.6.1 The trusted interface problem	630
	18.6.2 Conflicts	631
	18.6.3 The lemons market, risk dumping and evaluation games	632
	18.6.4 Security-by-obscurity	632
	18.6.5 Changing environments	633
	18.7 So what should one protect?	634
	18.8 Summary	636
	Research problems	636
	Further reading	636
Chapter 19	Side Channels	639
	19.1 Introduction	639
	19.2 Emission security	640
	19.2.1 History	641
	19.2.2 Technical surveillance and countermeasures	642

19.3	Passive attacks	645
19.3.1	Leakage through power and signal cables	645
19.3.2	Leakage through RF signals	645
19.3.3	What goes wrong	649
19.4	Attacks between and within computers	650
19.4.1	Timing analysis	651
19.4.2	Power analysis	652
19.4.3	Glitching and differential fault analysis	655
19.4.4	Rowhammer, CLKscrew and Plundervolt	656
19.4.5	Meltdown, Spectre and other enclave side channels	657
19.5	Environmental side channels	659
19.5.1	Acoustic side channels	659
19.5.2	Optical side channels	661
19.5.3	Other side-channels	661
19.6	Social side channels	663
19.7	Summary	663
	Research problems	664
	Further reading	664
Chapter 20	Advanced Cryptographic Engineering	667
20.1	Introduction	667
20.2	Full-disk encryption	668
20.3	Signal	670
20.4	Tor	674
20.5	HSMs	677
20.5.1	The xor-to-null-key attack	677
20.5.2	Attacks using backwards compatibility and time-memory tradeoffs	678
20.5.3	Differential protocol attacks	679
20.5.4	The EMV attack	681
20.5.5	Hacking the HSMs in CAs and clouds	681
20.5.6	Managing HSM risks	681
20.6	Enclaves	682
20.7	Blockchains	685
20.7.1	Wallets	688
20.7.2	Miners	689
20.7.3	Smart contracts	689
20.7.4	Off-chain payment mechanisms	691
20.7.5	Exchanges, cryptocrime and regulation	692
20.7.6	Permissioned blockchains	695
20.8	Crypto dreams that failed	695
20.9	Summary	696
	Research problems	698
	Further reading	698

Chapter 21	Network Attack and Defence	699
21.1	Introduction	699
21.2	Network protocols and service denial	701
21.2.1	BGP security	701
21.2.2	DNS security	703
21.2.3	UDP, TCP, SYN floods and SYN reflection	704
21.2.4	Other amplifiers	705
21.2.5	Other denial-of-service attacks	706
21.2.6	Email – from spies to spammers	706
21.3	The malware menagerie – Trojans, worms and RATs	708
21.3.1	Early history of malware	709
21.3.2	The Internet worm	710
21.3.3	Further malware evolution	711
21.3.4	How malware works	713
21.3.5	Countermeasures	714
21.4	Defense against network attack	715
21.4.1	Filtering: firewalls, censorware and wiretaps	717
21.4.1.1	Packet filtering	718
21.4.1.2	Circuit gateways	718
21.4.1.3	Application proxies	719
21.4.1.4	Ingress versus egress filtering	719
21.4.1.5	Architecture	720
21.4.2	Intrusion detection	722
21.4.2.1	Types of intrusion detection	722
21.4.2.2	General limitations of intrusion detection	724
21.4.2.3	Specific problems detecting network attacks	724
21.5	Cryptography: the ragged boundary	725
21.5.1	SSH	726
21.5.2	Wireless networking at the periphery	727
21.5.2.1	WiFi	727
21.5.2.2	Bluetooth	728
21.5.2.3	HomePlug	729
21.5.2.4	VPNs	729
21.6	CAs and PKI	730
21.7	Topology	733
21.8	Summary	734
	Research problems	734
	Further reading	735
Chapter 22	Phones	737
22.1	Introduction	737
22.2	Attacks on phone networks	738

22.2.1	Attacks on phone-call metering	739
22.2.2	Attacks on signaling	742
22.2.3	Attacks on switching and configuration	743
22.2.4	Insecure end systems	745
22.2.5	Feature interaction	746
22.2.6	VOIP	747
22.2.7	Frauds by phone companies	748
22.2.8	Security economics of telecomms	749
22.3	Going mobile	750
22.3.1	GSM	751
22.3.2	3G	755
22.3.3	4G	757
22.3.4	5G and beyond	758
22.3.5	General MNO failings	760
22.4	Platform security	761
22.4.1	The Android app ecosystem	763
22.4.1.1	App markets and developers	764
22.4.1.2	Bad Android implementations	764
22.4.1.3	Permissions	766
22.4.1.4	Android malware	767
22.4.1.5	Ads and third-party services	768
22.4.1.6	Pre-installed apps	770
22.4.2	Apple's app ecosystem	770
22.4.3	Cross-cutting issues	774
22.5	Summary	775
	Research problems	776
	Further reading	776
Chapter 23	Electronic and Information Warfare	777
23.1	Introduction	777
23.2	Basics	778
23.3	Communications systems	779
23.3.1	Signals intelligence techniques	781
23.3.2	Attacks on communications	784
23.3.3	Protection techniques	785
23.3.3.1	Frequency hopping	786
23.3.3.2	DSSS	787
23.3.3.3	Burst communications	788
23.3.3.4	Combining covertness and jam resistance	789
23.3.4	Interaction between civil and military uses	790
23.4	Surveillance and target acquisition	791
23.4.1	Types of radar	792

23.4.2	Jamming techniques	793
23.4.3	Advanced radars and countermeasures	795
23.4.4	Other sensors and multisensor issues	796
23.5	IFF systems	797
23.6	Improvised explosive devices	800
23.7	Directed energy weapons	802
23.8	Information warfare	803
23.8.1	Attacks on control systems	805
23.8.2	Attacks on other infrastructure	808
23.8.3	Attacks on elections and political stability	809
23.8.4	Doctrine	811
23.9	Summary	812
	Research problems	813
	Further reading	813
Chapter 24	Copyright and DRM	815
24.1	Introduction	815
24.2	Copyright	817
24.2.1	Software	817
24.2.2	Free software, free culture?	823
24.2.3	Books and music	827
24.2.4	Video and pay-TV	828
24.2.4.1	Typical system architecture	829
24.2.4.2	Video scrambling techniques	830
24.2.4.3	Attacks on hybrid scrambling systems	832
24.2.4.4	DVB	836
24.2.5	DVD	837
24.3	DRM on general-purpose computers	838
24.3.1	Windows media rights management	839
24.3.2	FairPlay, HTML5 and other DRM systems	840
24.3.3	Software obfuscation	841
24.3.4	Gaming, cheating, and DRM	843
24.3.5	Peer-to-peer systems	845
24.3.6	Managing hardware design rights	847
24.4	Information hiding	848
24.4.1	Watermarks and copy generation management	849
24.4.2	General information hiding techniques	849
24.4.3	Attacks on copyright marking schemes	851
24.5	Policy	854
24.5.1	The IP lobby	857
24.5.2	Who benefits?	859
24.6	Accessory control	860

24.7	Summary	862
	Research problems	862
	Further reading	863
Chapter 25	New Directions?	865
25.1	Introduction	865
25.2	Autonomous and remotely-piloted vehicles	866
25.2.1	Drones	866
25.2.2	Self-driving cars	867
25.2.3	The levels and limits of automation	869
25.2.4	How to hack a self-driving car	872
25.3	AI / ML	874
25.3.1	ML and security	875
25.3.2	Attacks on ML systems	876
25.3.3	ML and society	879
25.4	PETS and operational security	882
25.4.1	Anonymous messaging devices	885
25.4.2	Social support	887
25.4.3	Living off the land	890
25.4.4	Putting it all together	891
25.4.5	The name's Bond. James Bond	893
25.5	Elections	895
25.5.1	The history of voting machines	896
25.5.2	Hanging chads	896
25.5.3	Optical scan	898
25.5.4	Software independence	899
25.5.5	Why electronic elections are hard	900
25.6	Summary	904
	Research problems	904
	Further reading	905
Part III		
Chapter 26	Surveillance or Privacy?	909
26.1	Introduction	909
26.2	Surveillance	912
26.2.1	The history of government wiretapping	912
26.2.2	Call data records (CDRs)	916
26.2.3	Search terms and location data	919
26.2.4	Algorithmic processing	920
26.2.5	ISPs and CSPs	921
26.2.6	The Five Eyes' system of systems	922
26.2.7	The crypto wars	925
26.2.7.1	The back story to crypto policy	926
26.2.7.2	DES and crypto research	927

26.2.7.3	Crypto War 1 – the Clipper chip	928
26.2.7.4	Crypto War 2 – going spotty	931
26.2.8	Export control	934
26.3	Terrorism	936
26.3.1	Causes of political violence	936
26.3.2	The psychology of political violence	937
26.3.3	The role of institutions	938
26.3.4	The democratic response	940
26.4	Censorship	941
26.4.1	Censorship by authoritarian regimes	942
26.4.2	Filtering, hate speech and radicalisation	944
26.5	Forensics and rules of evidence	948
26.5.1	Forensics	948
26.5.2	Admissibility of evidence	950
26.5.3	What goes wrong	951
26.6	Privacy and data protection	953
26.6.1	European data protection	953
26.6.2	Privacy regulation in the USA	956
26.6.3	Fragmentation?	958
26.7	Freedom of information	960
26.8	Summary	961
	Research problems	962
	Further reading	962
Chapter 27	Secure Systems Development	965
27.1	Introduction	965
27.2	Risk management	966
27.3	Lessons from safety-critical systems	969
27.3.1	Safety engineering methodologies	970
27.3.2	Hazard analysis	971
27.3.3	Fault trees and threat trees	971
27.3.4	Failure modes and effects analysis	972
27.3.5	Threat modelling	973
27.3.6	Quantifying risks	975
27.4	Prioritising protection goals	978
27.5	Methodology	980
27.5.1	Top-down design	981
27.5.2	Iterative design: from spiral to agile	983
27.5.3	The secure development lifecycle	985
27.5.4	Gated development	987
27.5.5	Software as a Service	988
27.5.6	From DevOps to DevSecOps	991
	27.5.6.1 The Azure ecosystem	991

27.5.6.2	The Google ecosystem	992
27.5.6.3	Creating a learning system	994
27.5.7	The vulnerability cycle	995
27.5.7.1	The CVE system	997
27.5.7.2	Coordinated disclosure	998
27.5.7.3	Security incident and event management	999
27.5.8	Organizational mismanagement of risk	1000
27.6	Managing the team	1004
27.6.1	Elite engineers	1004
27.6.2	Diversity	1005
27.6.3	Nurturing skills and attitudes	1007
27.6.4	Emergent properties	1008
27.6.5	Evolving your workflow	1008
27.6.6	And finally ...	1010
27.7	Summary	1010
	Research problems	1011
	Further reading	1012
Chapter 28	Assurance and Sustainability	1015
28.1	Introduction	1015
28.2	Evaluation	1018
28.2.1	Alarms and locks	1019
28.2.2	Safety evaluation regimes	1019
28.2.3	Medical device safety	1020
28.2.4	Aviation safety	1023
28.2.5	The Orange book	1025
28.2.6	FIPS 140 and HSMs	1026
28.2.7	The common criteria	1026
28.2.7.1	The gory details	1027
28.2.7.2	What goes wrong with the Common Criteria	1029
28.2.7.3	Collaborative protection profiles	1031
28.2.8	The 'Principle of Maximum Complacency'	1032
28.2.9	Next steps	1034
28.3	Metrics and dynamics of dependability	1036
28.3.1	Reliability growth models	1036
28.3.2	Hostile review	1039
28.3.3	Free and open-source software	1040
28.3.4	Process assurance	1042
28.4	The entanglement of safety and security	1044
28.4.1	The electronic safety and security of cars	1046
28.4.2	Modernising safety and security regulation	1049
28.4.3	The Cybersecurity Act 2019	1050

28.5 Sustainability	1051
28.5.1 The Sales of goods directive	1052
28.5.2 New research directions	1053
28.6 Summary	1056
Research problems	1057
Further reading	1058
Chapter 29 Beyond “Computer Says No”	1059
Bibliography	1061
Index	1143



Preface to the Third Edition

The first edition of *Security Engineering* was published in 2001 and the second in 2008. Since then there have been huge changes.

The most obvious is that the smartphone has displaced the PC and laptop. Most of the world's population now walk around with a computer that's also a phone, a camera and a satnav; and the apps that run on these magic devices have displaced many of the things we were building ten years ago. Taxi rides are now charged by ride-hailing apps rather than by taxi meters. Banking has largely gone online, with phones starting to displace credit cards. Energy saving is no longer about your meter talking to your heating system but about both talking to your phone. Social networking has taken over many people's lives, driving everything from advertising to politics.

A related but less visible change is the move to large server farms. Sensitive data have moved from servers in schools, doctors' offices and law firms to cloud service providers. Many people no longer do their writing on word processing software on their laptop but on Google Docs or Office365 (I'm writing this book on Overleaf). This has consequences. Security breaches can happen at a scale no-one would have imagined twenty years ago. Compromises of tens of millions of passwords, or credit cards, have become almost routine. And in 2013, we discovered that fifteen years' worth of UK hospital medical records had been sold to 1200 organisations worldwide without the consent of the patients (who were still identifiable via their postcodes and dates of birth).

A real game-changer of the last decade was the Snowden revelations, also in 2013, when over 50,000 Top Secret documents about the NSA's signals intelligence activities were leaked to the press. The scale and intrusiveness of government surveillance surprised even cynical security engineers. It followed on from Stuxnet, where America attacked Iran's nuclear weapons program using malware, and was followed by NotPetya, where a Russian

cyberweapon, deployed against the Ukraine, inflicted hundreds of millions of dollars' worth of collateral damage on firms elsewhere. This brings us to the third big change, which is a much better understanding of nation-state security threats. In addition to understanding the capabilities and priorities of western intelligence agencies, we have a reasonably good idea of what the Chinese, the Russians and even the Syrians get up to.

And where the money is, the crooks follow too. The last decade has also seen the emergence of a cyber-crime ecosystem, with malware writers providing the tools to subvert millions of machines, many of which are used as criminal infrastructure while others are subverted in various ways into defrauding their users. We have a team at Cambridge that studies this, and so do dozens of other research groups worldwide. The rise of cybercrime is changing policing, and other state activity too: cryptocurrencies are not just making it easier to write ransomware, but undermining financial regulation. And then there are non-financial threats from cyber-bullying up through hate speech to election manipulation and videos of rape and murder.

So online harms now engage all sorts of people from teachers and the police to banks and the military. It is ever more important to measure the costs of these harms, and the effectiveness of the measures we deploy to mitigate them.

Some of the changes would have really surprised someone who read my book ten years ago and then spent a decade in solitary confinement. For example, the multilevel security industry is moribund, despite being the beneficiary of billions of dollars of US government funding over forty years; the Pentagon's entire information security philosophy – of mandating architectures to stop information flowing downward from Top Secret to Secret to Confidential to Unclassified – has been abandoned as unworkable. While architecture still matters, the emphasis has shifted to ecosystems. Given that bugs are ubiquitous and exploits inevitable, we had better be good at detecting exploits, fixing bugs and recovering from attacks. The game is no longer trusted systems but coordinated disclosure, DevSecOps and resilience.

What might the future hold? A likely game-changer is that as we put software into safety-critical systems like cars and medical devices, and connect them to the Internet, safety and security engineering are converging. This is leading to real strains; while security engineers fix bugs quickly, safety engineers like to test systems rigorously against standards that change slowly if at all. A wicked problem is how we will patch durable goods. At present, you might get security patches for your phone for three years and your laptop for five; you're expected to buy a new one after that. But cars last for fifteen years on average and if we're suddenly asked to scrap them after five the environmental costs won't be acceptable. So tell me, if you're writing navigation software today in 2020 for a car that will launch in 2023, how will you ensure that you can keep on shipping security patches in 2033, 2043 and 2053? What tools will you choose today?

Finally, there has been a sea change in the political environment. After decades in which political leaders considered technology policy to be for men in anoraks, and generally took the line of least resistance, the reports of Russian interference in the Brexit referendum and the Trump election got their attention. The prospect of losing your job can concentrate the mind wonderfully. The close attention of lawmakers is changing the game, first with tighter general rules such as Europe's General Data Protection Regulation; and second as products that are already regulated for safety, from cars and railway signals to children's toys acquire software and online connectivity, which has led to rules in Europe about how long software has to be maintained.

The questions the security engineer has to ask today are just the same as a decade ago: what are we seeking to prevent, and will the proposed mechanisms actually work? However, the canvas on which we work is now much broader. Almost all human life is there.

Ross Anderson
Cambridge, October 2020



Preface to the Second Edition

The first edition of *Security Engineering* was published in May 2001. Since then the world has changed.

System security was one of Microsoft's lowest priorities then; it's now one of the highest. The volume of malware continues to increase along with the nuisance that it causes. Although a lot of effort has gone into defence – we have seen Windows NT replaced by XP and then Vista, and occasional service packs replaced by monthly security patches – the effort put into attacks has increased far more. People who write viruses no longer do so for fun, but for profit; the last few years have seen the emergence of a criminal economy that supports diverse specialists. Spammers, virus writers, phishermen, money launderers and spies trade busily with each other.

Cryptography has also moved on. The Advanced Encryption Standard is being embedded into more and more products, and we have some interesting developments on the public-key side of things too. But just as our algorithm problems get solved, so we face a host of implementation issues. Side channels, poorly designed APIs and protocol failures continue to break systems. Applied cryptography is harder than ever to do well.

Pervasive computing also opens up new challenges. As computers and communications become embedded invisibly everywhere, so problems that used to only afflict 'proper computers' crop up in all sorts of other devices too. What does it mean for a thermometer to be secure, or an air-conditioner?

The great diversity of intelligent devices brings with it a great diversity of interests and actors. Security is not just about keeping the bad guys out, but increasingly concerned with tussles for power and control. DRM pits the content and platform industries against consumers, and against each other; accessory control is used to tie printers to their vendors' cartridges, but leads to antitrust lawsuits and government intervention. Security also interacts with

safety in applications from cars through utilities to electronic healthcare. The security engineer needs to understand not just crypto and operating systems, but economics and human factors as well.

And the ubiquity of digital devices means that ‘computer security’ is no longer just a problem for a few systems specialists. Almost all white-collar crime (and much crime of the serious violent sort) now involves computers or mobile phones, so a detective needs to understand computer forensics just as she needs to know how to drive. More and more lawyers, accountants, managers and other people with no formal engineering training are going to have to understand system security in order to do their jobs well.

The rapid growth of online services, from Google and Facebook to massively multiplayer games, has also changed the world. Bugs in online applications can be fixed rapidly once they’re noticed, but the applications get ever more complex and their side-effects harder to predict. We may have a reasonably good idea what it means for an operating system or even a banking service to be secure, but we can’t make any such claims for online lifestyles that evolve all the time. We’re entering a novel world of evolving socio-technical systems, and that raises profound questions about how the evolution is driven and who is in control.

The largest changes, however, may be those driven by the tragic events of September 2001 and by our reaction to them. These have altered perceptions and priorities in many ways, and changed the shape of the security industry. Terrorism is not just about risk, but about the perception of risk, and about the manipulation of perception. This adds psychology and politics to the mix. Security engineers also have a duty to contribute to the political debate. Where inappropriate reactions to terrorist crimes have led to major waste of resources and unforced policy errors, we have to keep on educating people to ask a few simple questions: what are we seeking to prevent, and will the proposed mechanisms actually work?

Ross Anderson
Cambridge, January 2008



Preface to the First Edition

For generations, people have defined and protected their property and their privacy using locks, fences, signatures, seals, account books, and meters. These have been supported by a host of social constructs ranging from international treaties through national laws to manners and customs.

This is changing, and quickly. Most records are now electronic, from bank accounts to registers of real property; and transactions are increasingly electronic, as shopping moves to the Internet. Just as important, but less obvious, are the many everyday systems that have been quietly automated. Burglar alarms no longer wake up the neighborhood, but send silent messages to the police; students no longer fill their dormitory washers and dryers with coins, but credit them using a smartcard they recharge at the college bookstore; locks are no longer simple mechanical affairs, but are operated by electronic remote controls or swipe cards; and instead of renting videocassettes, millions of people get their movies from satellite or cable channels. Even the humble banknote is no longer just ink on paper, but may contain digital watermarks that enable many forgeries to be detected by machine.

How good is all this new security technology? Unfortunately, the honest answer is 'nowhere near as good as it should be.' New systems are often rapidly broken, and the same elementary mistakes are repeated in one application after another. It often takes four or five attempts to get a security design right, and that is far too many.

The media regularly report security breaches on the Internet; banks fight their customers over 'phantom withdrawals' from cash machines; VISA reports huge increases in the number of disputed Internet credit card transactions; satellite TV companies hound pirates who copy their smartcards; and law enforcement agencies try to stake out territory in cyberspace with laws controlling the use of encryption. Worse still, features interact. A mobile phone

that calls the last number again if one of the keys is pressed by accident may be just a minor nuisance – until someone invents a machine that dispenses a can of soft drink every time its phone number is called. When all of a sudden you find 50 cans of Coke on your phone bill, who is responsible, the phone company, the handset manufacturer, or the vending machine operator? Once almost every electronic device that affects your life is connected to the Internet – which Microsoft expects to happen by 2010 – what does ‘Internet security’ mean to you, and how do you cope with it?

As well as the systems that fail, many systems just don’t work well enough. Medical record systems don’t let doctors share personal health information as they would like, but still don’t protect it against inquisitive private eyes. Zillion-dollar military systems prevent anyone without a “top secret” clearance from getting at intelligence data, but are often designed so that almost everyone needs this clearance to do any work. Passenger ticket systems are designed to prevent customers cheating, but when trustbusters break up the railroad, they cannot stop the new rail companies cheating each other. Many of these failures could have been foreseen if designers had just a little bit more knowledge of what had been tried, and had failed, elsewhere.

Security engineering is the new discipline that is starting to emerge out of all this chaos.

Although most of the underlying technologies (cryptology, software reliability, tamper resistance, security printing, auditing, etc.) are relatively well understood, the knowledge and experience of how to apply them effectively is much scarcer. And since the move from mechanical to digital mechanisms is happening everywhere at once, there just has not been time for the lessons learned to percolate through the engineering community. Time and again, we see the same old square wheels being reinvented.

The industries that have managed the transition most capably are often those that have been able to borrow an appropriate technology from another discipline. Examples include the reuse of technology designed for military identify-friend-or-foe equipment in bank cash machines and even prepayment gas meters. So even if a security designer has serious expertise in some particular speciality – whether as a mathematician working with ciphers or a chemist developing banknote inks – it is still prudent to have an overview of the whole subject. The essence of good security engineering is understanding the potential threats to a system, then applying an appropriate mix of protective measures – both technological and organizational – to control them. Knowing what has worked, and more importantly what has failed, in other applications is a great help in developing judgment. It can also save a lot of money.

The purpose of this book is to give a solid introduction to security engineering, as we understand it at the beginning of the twenty-first century. My goal is that it works at four different levels:

1. as a textbook that you can read from one end to the other over a few days as an introduction to the subject. The book is to be used mainly by the working IT professional who needs to learn about the subject, but it can also be used in a one-semester course in a university;
2. as a reference book to which you can come for an overview of the workings of some particular type of system (such as cash machines, taxi meters, radar jammers, anonymous medical record databases or whatever);
3. as an introduction to the underlying technologies, such as crypto, access control, inference control, tamper resistance, and seals. Space prevents me from going into great depth; but I provide a basic road map for each subject, plus a reading list for the curious (and a list of open research problems for the prospective graduate student);
4. as an original scientific contribution in which I have tried to draw out the common principles that underlie security engineering, and the lessons that people building one kind of system should have learned from others. In the many years I have been working in security, I keep coming across these. For example, a simple attack on stream ciphers wasn't known to the people who designed a common anti-aircraft fire control radar so it was easy to jam; while a trick well known to the radar community wasn't understood by banknote printers and people who design copyright marking schemes, which led to a quite general attack on most digital watermarks.

I have tried to keep this book resolutely mid-Atlantic. A security engineering book has to be, as many of the fundamental technologies are American, while many of the interesting applications are European. (This isn't surprising given the better funding of US universities and research labs, and the greater diversity of nations and markets in Europe.) What's more, many of the successful European innovations – from the smartcard to the GSM mobile phone to the pay-per-view TV service – have crossed the Atlantic and now thrive in the Americas. Both the science, and the case studies, are necessary.

This book grew out of the security engineering courses I teach at Cambridge University, but I have rewritten my notes to make them self-contained and added at least as much material again. It should be useful to the established professional security manager or consultant as a first-line reference; to the computer science professor doing research in cryptology; to the working police detective trying to figure out the latest computer scam; and to policy wonks struggling with the conflicts involved in regulating cryptography and

anonymity. Above all, it is aimed at Dilbert. My main audience is the working programmer or engineer who is trying to design real systems that will keep on working despite the best efforts of customers, managers, and everybody else.

This book is divided into three parts.

- The first looks at basic concepts, starting with the central concept of a security protocol, and going on to the human-computer interface, access controls, cryptology and distributed system issues. It does not assume any particular technical background other than basic computer literacy. It is based on an 'Introduction to Security' course which we teach to second year undergraduates.
- The second part looks in much more detail at a number of important applications such as military communications, medical record systems, cash machines, mobile phones and pay-TV. These are used to introduce more of the advanced technologies and concepts. It also considers information security from the viewpoint of a number of different interest groups such as companies, consumers, criminals, the police and spies. This material is drawn from my senior course on security, from research work, and from experience consulting.
- The third part looks at the organizational and policy issues: how computer security interacts with law, with evidence, and with corporate politics; how we can gain confidence that a system will perform as intended; and how the whole business of security engineering can best be managed.

I believe that building systems which continue to perform robustly in the face of malice is one of the most important, interesting, and difficult tasks facing engineers in the twenty-first century.

Ross Anderson
Cambridge, January 2001



For my daughter, and other lawyers ...

The tricks taught in this book are intended only to enable you to build better systems. They are not in any way given as a means of helping you to break into systems or do anything else illegal. So where possible I have tried to give case histories at a level of detail that illustrates the underlying principles without giving a ‘hacker’s cookbook’.

Governments fought to restrict knowledge of cryptography until the turn of the century, and there may still be people who believe that the knowledge contained in this book should not be published.

Their fears were answered in the first book in English that discussed cryptology, a 1641 treatise on optical and acoustic telegraphy written by Oliver Cromwell’s cryptographer and son-in-law John Wilkins [2025]. He traced scientific censorship back to the Egyptian priests who forbade the use of alphabetic writing on the grounds that it would spread literacy among the common people and thus foster dissent. As he said:

‘It will not follow that everything must be suppressed which may be abused ... If all those useful inventions that are liable to abuse should therefore be concealed there is not any Art of Science which may be lawfully professed.’

The question was raised again in the nineteenth century, when some well-meaning people wanted to ban books on locksmithing. In 1853, a contemporary writer replied [1899]:

‘Many well-meaning persons suppose that the discussion respecting the means for baffling the supposed safety of locks offers a premium for dishonesty, by showing others how to be dishonest. This is a fallacy. Rogues are very keen in their profession, and already know much more than we can teach them respecting their several kinds of roguery. Rogues knew a good deal about lockpicking long before

locksmiths discussed it among themselves ... if there be harm, it will be much more than counterbalanced by good.'

Thirty years later, in the first book on cryptographic engineering, Auguste Kerckhoffs explained that you must always assume that the other side knows the system, so security must reside in the choice of a key.

His wisdom has been borne out by long experience since. The relative benefits of 'Open' versus 'Closed' security systems have also been studied by researchers applying the tools of dependability analysis and security economics. We discuss their findings in this book.

In short, while some bad guys will benefit from a book such as this, they mostly know it already – and the good guys benefit much more.

Ross Anderson
Cambridge, November 2020



Foreword

In a paper he wrote with Roger Needham, Ross Anderson coined the phrase ‘programming Satan’s computer’ to describe the problems faced by computer-security engineers. It’s the sort of evocative image I’ve come to expect from Ross, and a phrase I’ve used ever since.

Programming a computer is straightforward: keep hammering away at the problem until the computer does what it’s supposed to do. Large application programs and operating systems are a lot more complicated, but the methodology is basically the same. Writing a reliable computer program is much harder, because the program needs to work even in the face of random errors and mistakes: Murphy’s computer, if you will. Significant research has gone into reliable software design, and there are many mission-critical software applications that are designed to withstand Murphy’s Law.

Writing a secure computer program is another matter entirely. Security involves making sure things work, not in the presence of random faults, but in the face of an intelligent and malicious adversary trying to ensure that things fail in the worst possible way at the worst possible time ... again and again. It truly is programming Satan’s computer.

Security engineering is different from any other kind of programming. It’s a point I made over and over again: in my own book, *Secrets and Lies*, in my monthly newsletter *Crypto-Gram*, and in my other writings. And it’s a point Ross makes in every chapter of this book. This is why, if you’re doing any security engineering ... if you’re even thinking of doing any security engineering, you need to read this book. It’s the first, and only, end-to-end modern security design and engineering book ever written.

And it comes just in time. You can divide the history of the Internet into three waves. The first wave centered around mainframes and terminals. Computers

were expensive and rare. The second wave, from about 1992 until now, centered around personal computers, browsers, and large application programs. And the third, starting now, will see the connection of all sorts of devices that are currently in proprietary networks, standalone, and non-computerized. By 2003, there will be more mobile phones connected to the Internet than computers. Within a few years we'll see many of the world's refrigerators, heart monitors, bus and train ticket dispensers, burglar alarms, and electricity meters talking IP. Personal computers will be a minority player on the Internet.

Security engineering, especially in this third wave, requires you to think differently. You need to figure out not how something works, but how something can be made to not work. You have to imagine an intelligent and malicious adversary inside your system (remember Satan's computer), constantly trying new ways to subvert it. You have to consider all the ways your system can fail, most of them having nothing to do with the design itself. You have to look at everything backwards, upside down, and sideways. You have to think like an alien.

As the late great science fiction editor John W. Campbell, said: "An alien thinks as well as a human, but not like a human." Computer security is a lot like that. Ross is one of those rare people who can think like an alien, and then explain that thinking to humans. Have fun reading.

Bruce Schneier
January 2001

PART

In the first section of the book, I cover the basics. The first chapter sets out to clarify concepts and terminology by describing the secure distributed systems commonly found in four environments: a bank, an air force base, a hospital, and the home. The second chapter then plunges into the thick of things by describing the threat actors and how they operate. We look at state actors such as the US, Chinese and Russian intelligence communities, about which we now know quite a lot thanks to disclosures by Ed Snowden and others; we describe the cybercrime ecosystem, which we've been studying for some years now; and we also describe non-financial abuses from cyber-bullying and intimate partner abuse up to election manipulation and political radicalisation. This teaches that a wide range of attackers use similar techniques, not just at the technical level but increasingly to deceive and manipulate people.

In the third chapter we therefore turn to psychology. Phishing is a key technique for both online crime and national intelligence gathering; usability failures are exploited all the time, and are really important for safety as well as security. One of the most fruitful areas of security research in recent years has therefore been psychology. Security engineers need to understand how people can be deceived, so we can design systems that make deception harder. We also need to understand how risk perceptions and realities have drifted ever further apart.

The following chapters dig deeper into the technical meat. The fourth chapter is on security protocols, which specify how the players in a system – whether people, computers, phones or other electronic devices – establish and maintain trust. The fifth is on the ‘duct tape’ that underlies most of the protocols and holds distributed systems together: cryptography. This is the art (and science) of codes and ciphers; but it is much more than a clever means for keeping messages secret from an eavesdropper. Nowadays its job is taking trust from where it exists to where it’s needed, maintaining the integrity of security contexts, and much more besides.

The sixth chapter is on access control: how can we keep apart the different apps on a phone, or the different virtual machines or containers on a server, and how can we control the data flows we want to permit between them. Sometimes this can be done cleanly, but often it’s hard; web browsers deal with JavaScript code from multiple untrustworthy websites, while home assistants have to deal with multiple people.

The next chapter is on distributed systems. Systems that run on multiple devices have to deal with coordination problems such as concurrency control, fault tolerance, and naming. These take on subtle new meanings when systems must be made resilient against malice as well as against accidental failure. Many systems perform poorly or even fail because their designers don’t think through these issues.

The final chapter in this part is on economics. Security economics has grown hugely since this book first appeared in 2001 and helped to launch it as a subject. We now know that many security failures are due to perverse incentives rather than to deficient technical protection mechanisms. (Indeed, the former often explain the latter.) The dependability of a system is increasingly an emergent property that depends on the self-interested striving of large numbers of players; in effect it’s an equilibrium in a market. Security mechanisms are not just used to keep ‘bad’ people out of ‘good’ systems, but to enable one principal to exert power over another; they are often abused to capture or distort markets. If we want to understand such plays, or to design systems that resist strategic manipulation, we need some game theory and auction theory.

These chapters cover basic material, and largely follow what we teach first-year and second-year undergraduates at Cambridge. But I hope that even experts will find the case studies of interest and value.

What Is Security Engineering?

Out of the crooked timber of humanity, no straight thing was ever made.

– IMMANUEL KANT

The world is never going to be perfect, either on- or offline; so let's not set impossibly high standards for online.

– ESTHER DYSON

1.1 Introduction

Security engineering is about building systems to remain dependable in the face of malice, error, or mischance. As a discipline, it focuses on the tools, processes, and methods needed to design, implement, and test complete systems, and to adapt existing systems as their environment evolves.

Security engineering requires cross-disciplinary expertise, ranging from cryptography and computer security through hardware tamper-resistance to a knowledge of economics, applied psychology, organisations and the law. System engineering skills, from business process analysis through software engineering to evaluation and testing, are also important; but they are not sufficient, as they deal only with error and mischance rather than malice. The security engineer also needs some skill at adversarial thinking, just like a chess player; you need to have studied lots of attacks that worked in the past, from their openings through their development to the outcomes.

Many systems have critical assurance requirements. Their failure may endanger human life and the environment (as with nuclear safety and control systems), do serious damage to major economic infrastructure (cash machines and online payment systems), endanger personal privacy (medical record systems), undermine the viability of whole business sectors (prepayment utility meters), and facilitate crime (burglar and car alarms). Security and safety are becoming ever more intertwined as we get software in everything.

Even the perception that a system is more vulnerable or less reliable than it really is can have real social costs.

The conventional view is that while software engineering is about ensuring that certain things happen (“John can read this file”), security is about ensuring that they don’t (“The Chinese government can’t read this file”). Reality is much more complex. Security requirements differ greatly from one system to another. You typically need some combination of user authentication, transaction integrity and accountability, fault-tolerance, message secrecy, and covertness. But many systems fail because their designers protect the wrong things, or protect the right things but in the wrong way.

Getting protection right thus depends on several different types of process. You have to figure out what needs protecting, and how to do it. You also need to ensure that the people who will guard the system and maintain it are properly motivated. In the next section, I’ll set out a framework for thinking about this. Then, in order to illustrate the range of different things that security and safety systems have to do, I will take a quick look at four application areas: a bank, a military base, a hospital, and the home. Once we’ve given concrete examples of the stuff that security engineers have to understand and build, we will be in a position to attempt some definitions.

1.2 A framework

To build really dependable systems, you need four things to come together. There’s policy: what you’re supposed to achieve. There’s mechanism: the ciphers, access controls, hardware tamper-resistance and other machinery that you use to implement the policy. There’s assurance: the amount of reliance you can place on each particular mechanism, and how well they work together. Finally, there’s incentive: the motive that the people guarding and maintaining the system have to do their job properly, and also the motive that the attackers have to try to defeat your policy. All of these interact (see Figure 1.1).

As an example, let’s think of the 9/11 terrorist attacks. The hijackers’ success in getting knives through airport security was not a mechanism failure but a policy one; the screeners did their job of keeping out guns and explosives, but at that time, knives with blades up to three inches were permitted. Policy changed quickly: first to prohibit all knives, then most weapons (baseball bats are now forbidden but whiskey bottles are OK); it’s flip-flopped on many details (butane lighters forbidden then allowed again). Mechanism is weak, because of things like composite knives and explosives that don’t contain nitrogen. Assurance is always poor; many tons of harmless passengers’ possessions are consigned to the trash each month, while less than half of all the real weapons taken through screening (whether accidentally or for test purposes) are spotted and confiscated.

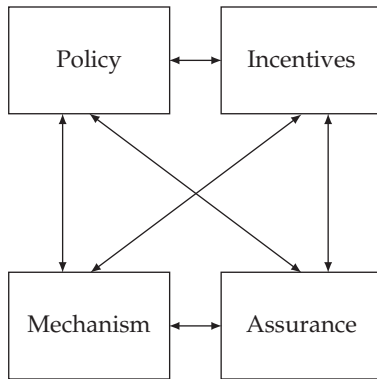


Figure 1.1: – Security Engineering Analysis Framework

Most governments have prioritised visible measures over effective ones. For example, the TSA has spent billions on passenger screening, which is fairly ineffective, while the \$100m spent on reinforcing cockpit doors removed most of the risk [1526]. The President of the Airline Pilots Security Alliance noted that most ground staff aren't screened, and almost no care is taken to guard aircraft parked on the ground overnight. As most airliners don't have door locks, there's not much to stop a bad guy wheeling steps up to a plane and placing a bomb on board; if he had piloting skills and a bit of chutzpah, he could file a flight plan and make off with it [1204]. Yet screening staff and guarding planes are just not a priority.

Why are such policy choices made? Quite simply, the incentives on the decision makers favour visible controls over effective ones. The result is what Bruce Schneier calls 'security theatre' – measures designed to produce a feeling of security rather than the reality. Most players also have an incentive to exaggerate the threat from terrorism: politicians to 'scare up the vote' (as President Obama put it), journalists to sell more papers, companies to sell more equipment, government officials to build their empires, and security academics to get grants. The upshot is that most of the damage done by terrorists to democratic countries comes from the overreaction. Fortunately, electorates figure this out over time, and now – nineteen years after 9/11 – less money is wasted. Of course, we now know that much more of our society's resilience budget should have been spent on preparing for pandemic disease. It was at the top of Britain's risk register, but terrorism was politically more sexy. The countries that managed their priorities more rationally got much better outcomes.

Security engineers need to understand all this; we need to be able to put risks and threats in context, make realistic assessments of what might go wrong, and give our clients good advice. That depends on a wide understanding of what has gone wrong over time with various systems; what sort of attacks have worked, what their consequences were, and how they were stopped (if it was

worthwhile to do so). History also matters because it leads to complexity, and complexity causes many failures. Knowing the history of modern information security enables us to understand its complexity, and navigate it better.

So this book is full of case histories. To set the scene, I'll give a few brief examples here of interesting security systems and what they're designed to prevent.

1.3 Example 1 – a bank

Banks operate a lot of security-critical computer systems.

1. A bank's operations rest on a core bookkeeping system. This keeps customer account master files plus a number of journals that record incoming and outgoing transactions. The main threat here is the bank's own staff; about one percent of bank branch staff are fired each year, mostly for petty dishonesty (the average theft is only a few thousand dollars). The traditional defence comes from book-keeping procedures that have evolved over centuries. For example, each debit against one account must be matched by a credit against another; so money can only be moved within a bank, never created or destroyed. In addition, large transfers typically need two people to authorize them. There are also alarms that look for unusual volumes or patterns of transactions, and staff are required to take regular vacations with no access to the bank's systems.
2. One public face is the bank's automatic teller machines. Authenticating transactions based on a customer's card and personal identification number – so as to defend against both outside and inside attack – is harder than it looks! There have been many epidemics of 'phantom withdrawals' in various countries when local villains (or bank staff) have found and exploited loopholes in the system. Automatic teller machines are also interesting as they were the first large-scale commercial use of cryptography, and they helped establish a number of crypto standards. The mechanisms developed for ATMs have been extended to point-of-sale terminals in shops, where card payments have largely displaced cash; and they've been adapted for other applications such as prepayment utility meters.
3. Another public face is the bank's website and mobile phone app. Most customers now do their routine business, such as bill payments and transfers between savings and checking accounts, online rather than at a branch. Bank websites have come under heavy attack since 2005 from *phishing* – where customers are invited to enter their passwords

at bogus websites. The standard security mechanisms designed in the 1990s turned out to be less effective once criminals started attacking the customers rather than the bank, so many banks now send you a text message with an authentication code. The crooks' reaction is to go to a phone shop, pretend to be you, and buy a new phone that takes over your phone number. This arms race poses many fascinating security engineering problems mixing elements from authentication, usability, psychology, operations and economics.

4. Behind the scenes are high-value messaging systems, used to move large sums between banks; to trade in securities; to issue letters of credit and guarantees; and so on. An attack on such a system is the dream of the high-tech criminal – and we hear that the government of North Korea has stolen many millions by attacks on banks. The defence is a mixture of bookkeeping controls, access controls, and cryptography.
5. The bank's branches may seem large, solid and prosperous, reassuring customers that their money is safe. But the stone facade is theatre rather than reality. If you walk in with a gun, the tellers will give you all the cash you can see; and if you break in at night, you can cut into the safe in minutes with an abrasive wheel. The effective controls center on alarm systems, which are connected to a security company's control center, whose staff check things out by video and call the police if they have to. Cryptography is used to prevent a robber manipulating the communications and making the alarm appear to say 'all's well' when it isn't.

I'll look at these applications in later chapters. Banking computer security is important: until the early 2000s, banks were the main civilian market for many computer security products, so they had a huge influence on security standards.

1.4 Example 2 – a military base

Military systems were the other technology driver back in the 20th century, as they motivated much of the academic research that governments funded into computer security from the early 1980s onwards. As with banking, there's not one application but many.

1. Military communications drove the development of cryptography, going right back to ancient Egypt and Mesopotamia. But it is often not enough to just encipher messages: an enemy who sees traffic encrypted with somebody else's keys may simply locate and attack the transmitter. *Low-probability-of-intercept* (LPI) radio links are one answer; they use tricks that are now adopted in everyday communications such as Bluetooth.

2. Starting in the 1940s, governments spent a lot of money on electronic warfare systems. The arms race of trying to jam enemy radars while preventing the enemy from jamming yours has led to many sophisticated deception tricks, countermeasures, and counter-countermeasures – with a depth, subtlety and range of strategies that are still not found elsewhere. Spoofing and service-denial attacks were a reality there long before blackmailers started targeting the websites of bankers, bookmakers and gamers.
3. Military organisations need to hold some information close, such as intelligence sources and plans for future operations. These are typically labeled ‘Top Secret’ and handled on separate systems; they may be further restricted in compartments, so that the most sensitive information is known to only a handful of people. For years, attempts were made to enforce information flow rules, so you could copy a file from a *Secret* stores system to a *Top Secret* command system, but not vice versa. Managing multiple systems with information flow restrictions is a hard problem, and the billions that were spent on attempting to automate military security helped develop the access-control technology you now have in your mobile phone and laptop.
4. The problems of protecting nuclear weapons led to the invention of a lot of cool security technology, ranging from provably-secure authentication systems, through optical-fibre alarm sensors, to methods of identifying people using biometrics – including the iris patterns now used to identify all citizens of India.

The security engineer can still learn a lot from this. For example, the military was until recently one of the few customers for software systems that had to be maintained for decades. Now that software and Internet connectivity are finding their way into safety-critical consumer goods such as cars, software sustainability is becoming a much wider concern. In 2019, the European Union passed a law demanding that if you sell goods with digital components, you must maintain those components for two years, or for longer if that’s a reasonable expectation of the customer – which will mean ten years for cars and white goods. If you’re writing software for a car or fridge that will be on sale for seven years, you’ll have to maintain it for almost twenty years. What tools should you use?

1.5 Example 3 – a hospital

From bankers and soldiers we move on to healthcare. Hospitals have a number of interesting protection requirements – mostly to do with patient safety and privacy.

1. Safety usability is important for medical equipment, and is by no means a solved problem. Safety usability failures are estimated to kill about as many people as road traffic accidents – a few tens of thousands a year in the USA, for example, and a few thousand in the UK. The biggest single problem is with the infusion pumps used to drip-feed patients with drugs; a typical hospital might have half-a-dozen makes, all with somewhat different controls, making fatal errors more likely. Safety usability interacts with security: unsafe devices that are also found to be hackable are much more likely to have product recalls ordered as regulators know that the public's appetite for risk is lower when hostile action becomes a possibility. So as more and more medical devices acquire not just software but radio communications, security sensitivities may lead to better safety.
2. Patient record systems should not let all the staff see every patient's record, or privacy violations can be expected. In fact, since the second edition of this book, the European Court has ruled that patients have a right to restrict their personal health information to the clinical staff involved in their care. That means that systems have to implement rules such as "nurses can see the records of any patient who has been cared for in their department at any time during the previous 90 days". This can be harder than it looks. (The US HIPAA legislation sets easier standards for compliance but is still a driver of information security investment.)
3. Patient records are often anonymized for use in research, but this is hard to do well. Simply encrypting patient names is not enough: an enquiry such as "show me all males born in 1953 who were treated for atrial fibrillation on October 19th 2003" should be enough to target former Prime Minister Tony Blair, who was rushed to hospital that day to be treated for an irregular heartbeat. Figuring out what data can be anonymized effectively is hard, and it's also a moving target as we get more and more social and contextual data – not to mention the genetic data of relatives near and far.
4. New technology can introduce poorly-understood risks. Hospital administrators understand the need for backup procedures to deal with outages of power; hospitals are supposed to be able to deal with casualties even if their mains electricity and water supplies fail. But after several hospitals in Britain had machines infected by the Wannacry malware in May 2017, they closed down their networks to limit further infection, and then found that they had to close their accident and emergency departments – as X-rays no longer travel from the X-ray machine to the operating theatre in an envelope, but via a server in a distant town. So a network failure can stop doctors operating when a power failure would not. There were standby generators, but no standby

network. Cloud services can make things more reliable on average, but the failures can be bigger, more complex, and correlated. An issue surfaced by the coronavirus pandemic is accessory control: some medical devices authenticate their spare parts, just as printers authenticate ink cartridges. Although the vendors claim this is for safety, it's actually so they can charge more money for spares. But it introduces fragility: when the supply chain gets interrupted, things are a lot harder to fix.

We'll look at medical system security (and safety too) in more detail later. This is a younger field than banking IT or military systems, but as healthcare accounts for a larger proportion of GNP than either of them in all developed countries, its importance is growing. It's also consistently the largest source of privacy breaches in countries with mandatory reporting.

1.6 Example 4 – the home

You might not think that the typical family operates any secure systems. But just stop and think.

1. You probably use some of the systems I've already described. You may use a web-based electronic banking system to pay bills, and you may have online access to your doctor's surgery so you can order repeat prescriptions. If you're diabetic then your insulin pump may communicate with a docking station at your bedside. Your home burglar alarm may send an encrypted 'all's well' signal to the security company every few minutes, rather than waking up the neighborhood when something happens.
2. Your car probably has an electronic immobilizer. If it was made before about 2015, the car unlocks when you press a button on the key, which sends an encrypted unlock command. If it's a more recent model, where you don't have to press any buttons but just have the key in your pocket, the car sends an encrypted challenge to the key and waits for the right response. But eliminating the button press meant that if you leave your key near the front door, a thief might use a radio relay to steal your car. Car thefts have shot up since this technology was introduced.
3. Your mobile phone authenticates itself to the network by a cryptographic challenge-response protocol similar to the ones used in car door locks and immobilizers, but the police can use a false base station (known in Europe as an IMSI-catcher, and in America as a Stingray) to listen in. And, as I mentioned above, many phone companies are relaxed about selling new SIM cards to people who claim their phones have been stolen; so a crook might steal your phone number and use this to raid your bank account.

4. In over 100 countries, households can get prepayment meters for electricity and gas, which they top up using a 20-digit code that they buy from an ATM or an online service. It even works off-grid; in Kenyan villages, people who can't afford \$200 to buy a solar panel can get one for \$2 a week and unlock the electricity it generates using codes they buy with their mobile phones.
5. Above all, the home provides a haven of physical security and seclusion. This is changing in a number of ways. Burglars aren't worried by locks as much as by occupants, so alarms and monitoring systems can help; but monitoring is also becoming pervasive, with many households buying systems like Alexa and Google Home that listen to what people say. All sorts of other gadgets now have microphones and cameras as voice and gesture interfaces become common, and the speech processing is typically done in the cloud to save battery life. By 2015, President Obama's council of advisers on science and technology was predicting that pretty soon every inhabited space on earth would have microphones that were connected to a small number of cloud service providers. (The USA and Europe have quite different views on how privacy law should deal with this.) One way or another, the security of your home may come to depend on remote systems over which you have little control.

Over the next few years, the number of such systems is going to increase rapidly. On past experience, many of them will be badly designed. For example, in 2019, Europe banned a children's watch that used unencrypted communications to the vendor's cloud service; a wiretapper could download any child's location history and cause their watch to phone any number in the world. When this was discovered, the EU ordered the immediate safety recall of all watches [903].

This book aims to help you avoid such outcomes. To design systems that are safe and secure, an engineer needs to know about what systems there are, how they work, and – at least as important – how they have failed in the past. Civil engineers learn far more from the one bridge that falls down than from the hundred that stay up; exactly the same holds in security engineering.

1.7 Definitions

Many of the terms used in security engineering are straightforward, but some are misleading or even controversial. There are more detailed definitions of technical terms in the relevant chapters, which you can find using the index. In this section, I'll try to point out where the main problems lie.

The first thing we need to clarify is what we mean by *system*. In practice, this can denote:

1. a product or component, such as a cryptographic protocol, a smartcard, or the hardware of a phone, a laptop or server;
2. one or more of the above plus an operating system, communications and other infrastructure;
3. the above plus one or more applications (banking app, health app, media player, browser, accounts/payroll package, and so on – including both client and cloud components);
4. any or all of the above plus IT staff;
5. any or all of the above plus internal users and management;
6. any or all of the above plus customers and other external users.

Confusion between the above definitions is a fertile source of errors and vulnerabilities. Broadly speaking, the vendor and evaluator communities focus on the first and (occasionally) the second of them, while a business will focus on the sixth (and occasionally the fifth). We will come across many examples of systems that were advertised or even certified as secure because the hardware was, but that broke badly when a particular application was run, or when the equipment was used in a way the designers didn't anticipate. Ignoring the human components, and thus neglecting usability issues, is one of the largest causes of security failure. So we will generally use definition 6; when we take a more restrictive view, it should be clear from the context.

The next set of problems comes from lack of clarity about who the players are and what they're trying to prove. In the literature on security and cryptology, it's a convention that principals in security protocols are identified by names chosen with (usually) successive initial letters – much like hurricanes, except that we use alternating genders. So we see lots of statements such as "Alice authenticates herself to Bob". This makes things much more readable, but can come at the expense of precision. Do we mean that Alice proves to Bob that her name actually is Alice, or that she proves she's got a particular credential? Do we mean that the authentication is done by Alice the human being, or by a smartcard or software tool acting as Alice's agent? In that case, are we sure it's Alice, and not perhaps Carol to whom Alice lent her card, or David who stole her phone, or Eve who hacked her laptop?

By a *subject* I will mean a physical person in any role including that of an operator, principal or victim. By a *person*, I will mean either a physical person or a legal person such as a company or government¹.

¹The law around companies may come in handy when we start having to develop rules around AI. A company, like a robot, may be immortal and have some functional intelligence – but without consciousness. You can't jail a company but you can fine it.

A *principal* is an entity that participates in a security system. This entity can be a subject, a person, a role, or a piece of equipment such as a laptop, phone, smartcard, or card reader. A principal can also be a communications channel (which might be a port number, or a crypto key, depending on the circumstance). A principal can also be a compound of other principals; examples are a group (Alice or Bob), a conjunction (Alice and Bob acting together), a compound role (Alice acting as Bob's manager) and a delegation (Bob acting for Alice in her absence).

Beware that groups and roles are not the same. By a *group* I will mean a set of principals, while a *role* is a set of functions assumed by different persons in succession (such as 'the officer of the watch on the USS Nimitz' or 'the president for the time being of the Icelandic Medical Association'). A principal may be considered at more than one level of abstraction: e.g. 'Bob acting for Alice in her absence' might mean 'Bob's smartcard representing Bob who is acting for Alice in her absence' or even 'Bob operating Alice's smartcard in her absence'. When we have to consider more detail, I'll be more specific.

The meaning of the word *identity* is controversial. When we have to be careful, I will use it to mean a correspondence between the names of two principals signifying that they refer to the same person or equipment. For example, it may be important to know that the Bob in 'Alice acting as Bob's manager' is the same as the Bob in 'Bob acting as Charlie's manager' and in 'Bob as branch manager signing a bank draft jointly with David'. Often, identity is abused to mean simply 'name', an abuse entrenched by such phrases as 'user identity' and 'citizen identity card'.

The definitions of *trust* and *trustworthy* are often confused. The following example illustrates the difference: if an NSA employee is observed in a toilet stall at Baltimore Washington International airport selling key material to a Chinese diplomat, then (assuming his operation was not authorized) we can describe him as 'trusted but not trustworthy'. I use the NSA definition that a *trusted* system or component is one whose failure can break the security policy, while a *trustworthy* system or component is one that won't fail.

There are many alternative definitions of trust. In the corporate world, trusted system might be 'a system which won't get me fired if it gets hacked on my watch' or even 'a system which we can insure'. But when I mean an approved system, an insurable system or an insured system, I'll say so.

The definition of *confidentiality* versus *privacy* versus *secrecy* opens another can of worms. These terms overlap, but are not exactly the same. If my neighbor cuts down some ivy at our common fence with the result that his kids can look into my garden and tease my dogs, it's not my confidentiality that has been invaded. And the duty to keep quiet about the affairs of a former employer is a duty of confidence, not of privacy.

The way I'll use these words is as follows.

- *Secrecy* is an engineering term that refers to the effect of the mechanisms used to limit the number of principals who can access information, such as cryptography or computer access controls.
- *Confidentiality* involves an obligation to protect some other person's or organisation's secrets if you know them.
- *Privacy* is the ability and/or right to protect your personal information and extends to the ability and/or right to prevent invasions of your personal space (the exact definition of which varies from one country to another). Privacy can extend to families but not to legal persons such as corporations.

For example, hospital patients have a right to privacy, and in order to uphold this right the doctors, nurses and other staff have a duty of confidence towards their patients. The hospital has no right of privacy in respect of its business dealings but those employees who are privy to them may have a duty of confidence (unless they invoke a whistleblowing right to expose wrongdoing). Typically, privacy is secrecy for the benefit of the individual while confidentiality is secrecy for the benefit of the organisation.

There is a further complexity in that it's often not sufficient to protect data, such as the contents of messages; we also have to protect metadata, such as logs of who spoke to whom. For example, many countries have laws making the treatment of sexually transmitted diseases secret, and yet if a private eye could observe you exchanging encrypted messages with a sexually-transmitted disease clinic, he might infer that you were being treated there. In fact, a key privacy case in the UK turned on such a fact: a model in Britain won a privacy lawsuit against a tabloid newspaper which printed a photograph of her leaving a meeting of Narcotics Anonymous. So *anonymity* can be just as important a factor in privacy (or confidentiality) as secrecy. But anonymity is hard. It's difficult to be anonymous on your own; you usually need a crowd to hide in. Also, our legal codes are not designed to support anonymity: it's much easier for the police to get itemized billing information from the phone company, which tells them who called whom, than it is to get an actual wiretap. (And it's often more useful.)

The meanings of *authenticity* and *integrity* can also vary subtly. In the academic literature on security protocols, authenticity means integrity plus freshness: you have established that you are speaking to a genuine principal, not a replay of previous messages. We have a similar idea in banking protocols. If local banking laws state that checks are no longer valid after six months, a seven month old uncashed check has integrity (assuming it's not been altered) but is no longer valid. However, there are some strange edge cases. For example, a police crime scene officer will preserve the integrity of a forged check – by

placing it in an evidence bag. (The meaning of integrity has changed in the new context to include not just the signature but any fingerprints.)

The things we don't want are often described as hacking. I'll follow Bruce Schneier and define a *hack* as something a system's rules permit, but which was unanticipated and unwanted by its designers [1682]. For example, tax attorneys study the tax code to find loopholes which they develop into tax avoidance strategies; in exactly the same way, black hats study software code to find loopholes which they develop into exploits. Hacks can target not just the tax system and computer systems, but the market economy, our systems for electing leaders and even our cognitive systems. They can happen at multiple layers: lawyers can hack the tax code, or move up the stack and hack the legislature, or even the media. In the same way, you might try to hack a cryptosystem by finding a mathematical weakness in the encryption algorithm, or you can go down a level and measure the power drawn by a device that implements it in order to work out the key, or up a level and deceive the device's custodian into using it when they shouldn't. This book contains many examples. In the broader context, hacking is sometimes a source of significant innovation. If a hack becomes popular, the rules may be changed to stop it; but it may also become normalised (examples range from libraries through the filibuster to search engines and social media).

The last matter I'll clarify here is the terminology that describes what we're trying to achieve. A *vulnerability* is a property of a system or its environment which, in conjunction with an internal or external *threat*, can lead to a *security failure*, which is a breach of the system's security policy. By *security policy* I will mean a succinct statement of a system's protection strategy (for example, "in each transaction, sums of credits and debits are equal, and all transactions over \$1,000,000 must be authorized by two managers"). A *security target* is a more detailed specification which sets out the means by which a security policy will be implemented in a particular product – encryption and digital signature mechanisms, access controls, audit logs and so on – and which will be used as the yardstick to evaluate whether the engineers have done a proper job. Between these two levels you may find a *protection profile* which is like a security target, except written in a sufficiently device-independent way to allow comparative evaluations among different products and different versions of the same product. I'll elaborate on security policies, security targets and protection profiles in Part 3. In general, the word *protection* will mean a property such as confidentiality or integrity, defined in a sufficiently abstract way for us to reason about it in the context of general systems rather than specific implementations.

This somewhat mirrors the terminology we use for safety-critical systems, and as we are going to have to engineer security and safety together in ever more applications it is useful to keep thinking of the two side by side.

In the safety world, a *critical* system or component is one whose failure could lead to an accident, given a *hazard* – a set of internal conditions or external circumstances. *Danger* is the probability that a hazard will lead to an accident, and *risk* is the overall probability of an accident. Risk is thus hazard level combined with danger and *latency* – the hazard exposure and duration. *Uncertainty* is where the risk is not quantifiable, while *safety* is freedom from accidents. We then have a *safety policy* which gives us a succinct statement of how risks will be kept below an acceptable threshold (and this might range from succinct, such as “don’t put explosives and detonators in the same truck”, to the much more complex policies used in medicine and aviation); at the next level down, we might find a *safety case* having to be made for a particular component such as an aircraft, an aircraft engine or even the control software for an aircraft engine.

1.8 Summary

‘Security’ is a terribly overloaded word, which often means quite incompatible things to different people. To a corporation, it might mean the ability to monitor all employees’ email and web browsing; to the employees, it might mean being able to use email and the web without being monitored.

As time goes on, and security mechanisms are used more and more by the people who control a system’s design to gain some commercial advantage over the other people who use it, we can expect conflicts, confusion and the deceptive use of language to increase.

One is reminded of a passage from Lewis Carroll:

“When I use a word,” Humpty Dumpty said, in a rather scornful tone, “it means just what I choose it to mean – neither more nor less.” “The question is,” said Alice, “whether you can make words mean so many different things.” “The question is,” said Humpty Dumpty, “which is to be master – that’s all.”

The security engineer must be sensitive to the different nuances of meaning that words acquire in different applications, and be able to formalize what the security policy and target actually are. That may sometimes be inconvenient for clients who wish to get away with something, but, in general, robust security design requires that the protection goals are made explicit.

CHAPTER 4 Protocols

It is impossible to foresee the consequences of being clever.

– CHRISTOPHER STRACHEY

If it's provably secure, it probably isn't.

– LARS KNUDSEN

4.1 Introduction

Passwords are just one example of a more general concept, the security protocol. If security engineering has a core theme, it may be the study of security protocols. They specify the steps that principals use to establish trust relationships. They are where the cryptography and the access controls meet; they are the tools we use to link up human users with remote machines, to synchronise security contexts, and to regulate key applications such as payment. We've come across a few protocols already, including challenge-response authentication and Kerberos. In this chapter, I'll dig down into the details, and give many examples of how protocols fail.

A typical security system consists of a number of principals such as people, companies, phones, computers and card readers, which communicate using a variety of channels including fibre, wifi, the cellular network, bluetooth, infrared, and by carrying data on physical devices such as bank cards and transport tickets. The security protocols are the rules that govern these communications. They are designed so that the system will survive malicious acts such as people telling lies on the phone, hostile governments jamming radio, or forgers altering the data on train tickets. Protection against all possible attacks is often too expensive, so protocol designs make assumptions about threats. For example, when we get a user to log on by entering a password into a machine, we implicitly assume that she can enter it into the right machine. In the old days of hard-wired terminals in the workplace, this was reasonable;

now that people log on to websites over the Internet, it is much less obvious. Evaluating a protocol thus involves two questions: first, is the threat model realistic? Second, does the protocol deal with it?

Protocols may be very simple, such as swiping a badge through a reader to enter a building. They often involve interaction, and are not necessarily technical. For example, when we order a bottle of fine wine in a restaurant, the standard protocol is that the wine waiter offers us the menu (so that we see the prices but our guests don't); they bring the bottle, so we can check the label, the seal and the temperature; they open it so we can taste it; and then serve it. This has evolved to provide some privacy (our guests don't learn the price), some integrity (we can be sure we got the right bottle and that it wasn't refilled with cheap plonk) and non-repudiation (we can't complain afterwards that the wine was off). Matt Blaze gives other non-technical protocol examples from ticket inspection, aviation security and voting in [261]. Traditional protocols like these often evolved over decades or centuries to meet social expectations as well as technical threats.

At the technical end of things, protocols get a lot more complex, and they don't always get better. As the car industry moved from metal keys to electronic keys with buttons you press, theft fell, since the new keys were harder to copy. But the move to keyless entry has seen car crime rise again, as the bad guys figured out how to build relay devices that would make a key seem closer to the car than it actually was. Another security upgrade that's turned out to be tricky is the move from magnetic-strip cards to smartcards. Europe made this move in the late 2000s while the USA is only catching up in the late 2010s. Fraud against cards issued in Europe actually went up for several years; clones of European cards were used in magnetic-strip cash machines in the USA, as the two systems' protection mechanisms didn't quite mesh. And there was a protocol failure that let a thief use a stolen chipcard in a store even if he didn't know the PIN, which took the banks several years to fix.

So we need to look systematically at security protocols and how they fail.

4.2 Password eavesdropping risks

Passwords and PINs are still the foundation for much of computer security, as the main mechanism used to authenticate humans to machines. We discussed their usability in the last chapter; now let's consider the kinds of technical attack we have to block when designing protocols that operate between one machine and another.

Remote key entry is a good place to start. The early systems, such as the remote control used to open your garage or to unlock cars manufactured up

to the mid-1990's, just broadcast a serial number. The attack that killed them was the 'grabber', a device that would record a code and replay it later. The first grabbers, seemingly from Taiwan, arrived on the market in about 1995; thieves would lurk in parking lots or outside a target's house, record the signal used to lock the car and then replay it once the owner had gone¹.

The first countermeasure was to use separate codes for lock and unlock. But the thief can lurk outside your house and record the unlock code before you drive away in the morning, and then come back at night and help himself. Second, sixteen-bit passwords are too short. Occasionally people found they could unlock the wrong car by mistake, or even set the alarm on a car whose owner didn't know he had one [309]. And by the mid-1990's, devices appeared that could try all possible codes one after the other. A code will be found on average after about 2^{15} tries, and at ten per second that takes under an hour. A thief operating in a parking lot with a hundred vehicles within range would be rewarded in less than a minute with a car helpfully flashing its lights.

The next countermeasure was to double the length of the password from 16 to 32 bits. The manufacturers proudly advertised 'over 4 billion codes'. But this only showed they hadn't really understood the problem. There were still only one or two codes for each car, and grabbers still worked fine.

Using a serial number as a password has a further vulnerability: lots of people have access to it. In the case of a car, this might mean all the dealer staff, and perhaps the state motor vehicle registration agency. Some burglar alarms have also used serial numbers as master passwords, and here it's even worse: when a bank buys a burglar alarm, the serial number may appear on the order, the delivery note and the invoice. And banks don't like sending someone out to buy something for cash.

Simple passwords are sometimes the appropriate technology. For example, a monthly season ticket for our local swimming pool simply has a barcode. I'm sure I could make a passable forgery, but as the turnstile attendants get to know the 'regulars', there's no need for anything more expensive. For things that are online, however, static passwords are hazardous; the Mirai botnet got going by recruiting wifi-connected CCTV cameras which had a password that couldn't be changed. And for things people want to steal, like cars, we also need something better. This brings us to cryptographic authentication protocols.

¹With garage doors it's even worse. A common chip is the Princeton PT2262, which uses 12 tri-state pins to encode 3^{12} or 531,441 address codes. However implementers often don't read the data sheet carefully enough to understand tri-state inputs and treat them as binary instead, getting 2^{12} . Many of them only use eight inputs, as the other four are on the other side of the chip. And as the chip has no retry-lockout logic, an attacker can cycle through the combinations quickly and open your garage door after 2^7 attempts on average. Twelve years after I noted these problems in the second edition of this book, the chip has not been withdrawn. It's now also sold for home security systems and for the remote control of toys.

4.3 Who goes there? – simple authentication

A simple modern authentication device is the token that some multistorey parking garages give subscribers to raise the barrier. The token has a single button; when you press it, it first transmits its serial number and then sends an authentication block consisting of the same serial number, followed by a random number, all encrypted using a key unique to the device, and sent to the garage barrier (typically by radio at 434MHz, though infrared is also used). We will postpone discussion of how to encrypt data to the next chapter, and simply write $\{X\}_K$ for the message X encrypted under the key K .

Then the protocol between the access token and the parking garage can be written as:

$$T \rightarrow G : T, \{T, N\}_{KT}$$

This is standard protocol notation, so we'll take it slowly.

The token T sends a message to the garage G consisting of its name T followed by the encrypted value of T concatenated with N , where N stands for 'number used once', or *nonce*. Everything within the braces is encrypted, and the encryption binds T and N together as well as obscuring their values. The purpose of the nonce is to assure the recipient that the message is *fresh*, that is, it is not a replay of an old message. Verification is simple: the garage reads T , gets the corresponding key KT , deciphers the rest of the message, checks that the nonce N has not been seen before, and finally that the plaintext contains T .

One reason many people get confused is that to the left of the colon, T identifies one of the principals (the token that represents the subscriber) whereas to the right it means the name (that is, the unique device number) of the token. Another is that once we start discussing attacks on protocols, we may find that a message intended for one principal was intercepted and played back by another. So you might think of the $T \rightarrow G$ to the left of the colon as a hint as to what the protocol designer had in mind.

A *nonce* can be anything that guarantees the freshness of a message. It can be a random number, a counter, a random challenge received from a third party, or even a timestamp. There are subtle differences between them, such as in the level of resistance they offer to various kinds of replay attack, and the ways in which they increase system cost and complexity. In very low-cost systems, random numbers and counters predominate as it's cheaper to communicate in one direction only, and cheap devices usually don't have clocks.

Key management in such devices can be very simple. In a typical garage token product, each token's key is just its unique device number encrypted under a global master key KM known to the garage:

$$KT = \{T\}_{KM}$$

This is known as *key diversification* or *key derivation*. It's a common way of implementing access tokens, and is widely used in smartcards too. The goal is that someone who compromises a token by drilling into it and extracting the key cannot masquerade as any other token; all he can do is make a copy of one particular subscriber's token. In order to do a complete break of the system, and extract the master key that would enable him to pretend to be any of the system's users, an attacker has to compromise the central server at the garage (which might protect this key in a tamper-resistant smartcard or hardware security module).

But there is still room for error. A common failure mode is for the serial numbers – whether unique device numbers or protocol counters – not to be long enough, so that someone occasionally finds that their remote control works for another car in the car park as well. This can be masked by cryptography. Having 128-bit keys doesn't help if the key is derived by encrypting a 16-bit device number, or by taking a 16-bit key and repeating it eight times. In either case, there are only 2^{16} possible keys, and that's unlikely to be enough even if they appear to be random².

Protocol vulnerabilities usually give rise to more, and simpler, attacks than cryptographic weaknesses do. An example comes from the world of prepayment utility meters. Over a million households in the UK, plus over 400 million in developing countries, have an electricity or gas meter that accepts encrypted tokens: the householder buys a magic number and types it into the meter, which then dispenses the purchased quantity of energy. One early meter that was widely used in South Africa checked only that the nonce was different from last time. So the customer could charge their meter indefinitely by buying two low-value power tickets and then feeding them in one after the other; given two valid codes *A* and *B*, the series *ABABAB...* was seen as valid [94].

So the question of whether to use a random number or a counter is not as easy as it looks. If you use random numbers, the lock has to remember a lot of past codes. There's the *valet attack*, where someone with temporary access, such as a valet parking attendant, records some access codes and replays them later to steal your car. In addition, someone might rent a car, record enough unlock codes, and then go back later to the rental lot to steal it. Providing enough non-volatile memory to remember thousands of old codes might add a few cents to the cost of your lock.

If you opt for counters, the problem is synchronization. The key might be used for more than one lock; it may also be activated repeatedly by accident (I once took an experimental token home where it was gnawed by my dogs). So you need a way to recover after the counter has been incremented hundreds or possibly even thousands of times. One common product uses a sixteen bit

²We'll go into this in more detail in section 5.3.1.2 where we discuss the birthday theorem in probability theory.

counter, and allows access when the deciphered counter value is the last valid code incremented by no more than sixteen. To cope with cases where the token has been used more than sixteen times elsewhere (or gnawed by a family pet), the lock will open on a second press provided that the counter value has been incremented between 17 and 32,767 times since a valid code was entered (the counter rolls over so that 0 is the successor of 65,535). This is fine in many applications, but a thief who can get six well-chosen access codes – say for values 0, 1, 20,000, 20,001, 40,000 and 40,001 – can break the system completely. In your application, would you be worried about that?

So designing even a simple token authentication mechanism is not as easy as it looks, and if you assume that your product will only attract low-grade adversaries, this assumption might fail over time. An example is *accessory control*. Many printer companies embed authentication mechanisms in printers to ensure that genuine toner cartridges are used. If a competitor's product is loaded instead, the printer may quietly downgrade from 1200 dpi to 300 dpi, or simply refuse to work at all. All sorts of other industries are getting in on the act, from scientific instruments to games consoles. The cryptographic mechanisms used to support this started off in the 1990s being fairly rudimentary, as vendors thought that any competitor who circumvented them on an industrial scale could be sued or even jailed under copyright law. But then a judge found that while a vendor had the right to hire the best cryptographer they could find to lock their customers in, a competitor also had the right to hire the best cryptanalyst they could find to set them free to buy accessories from elsewhere. This set off a serious arms race, which we'll discuss in section 24.6. Here I'll just remark that security isn't always a good thing. Security mechanisms are used to support many business models, where they're typically stopping the device's owner doing things she wants to rather than protecting her from the bad guys. The effect may be contrary to public policy; one example is cellphone locking, which results in hundreds of millions of handsets ending up in landfills each year, with toxic heavy metals as well as the embedded carbon cost.

4.3.1 Challenge and response

Since 1995, all cars sold in Europe were required to have a 'cryptographically enabled immobiliser' and by 2010, most cars had remote-controlled door unlocking too, though most also have a fallback metal key so you can still get into your car even if the key fob battery is flat. The engine immobiliser is harder to bypass using physical means and uses a two-pass *challenge-response protocol* to authorise engine start. As the car key is inserted into the steering lock, the engine controller sends a challenge consisting of a random n -bit number to the key using short-range radio. The car key computes a response

by encrypting the challenge; this is often done by a separate RFID chip that's powered by the incoming radio signal and so keeps on working even if the battery is flat. The frequency is low (125kHz) so the car can power the transponder directly, and the exchange is also relatively immune to a noisy RF environment.

Writing E for the engine controller, T for the transponder in the car key, K for the cryptographic key shared between the transponder and the engine controller, and N for the random challenge, the protocol may look something like:

$$\begin{aligned} E \rightarrow T : & \quad N \\ T \rightarrow E : & \quad T, \{T, N\}_K \end{aligned}$$

This is sound in theory, but implementations of security mechanisms often fail the first two or three times people try it.

Between 2005 and 2015, all the main remote key entry and immobiliser systems were broken, whether by security researchers, car thieves or both. The attacks involved a combination of protocol errors, poor key management, weak ciphers, and short keys mandated by export control laws.

The first to fall was TI's DST transponder chip, which was used by at least two large car makers and was also the basis of the SpeedPass toll payment system. Stephen Bono and colleagues found in 2005 that it used a block cipher with a 40-bit key, which could be calculated by brute force from just two responses [298]. This was one side-effect of US cryptography export controls, which I discuss in 26.2.7.1. From 2010, Ford, Toyota and Hyundai adopted a successor product, the DST80. The DST80 was broken in turn in 2020 by Lennert Wouters and colleagues, who found that as well as side-channel attacks on the chip, there are serious implementation problems with key management: Hyundai keys have only 24 bits of entropy, while Toyota keys are derived from the device serial number that an attacker can read (Tesla was also vulnerable but unlike the older firms it could fix the problem with a software upgrade) [2050]. Next was Keeloq, which was used for garage door openers as well as by some car makers; in 2007, Eli Biham and others found that given an hour's access to a token they could collect enough data to recover the key [244]. Worse, in some types of car, there is also a protocol bug, in that the key diversification used exclusive-or: $KT = T \oplus KM$. So you can rent a car of the type you want to steal and work out the key for any other car of that type.

Also in 2007, someone published the Philips Hitag 2 cipher, which also had a 48-bit secret key. But this cipher is also weak, and as it was attacked by various cryptanalysts, the time needed to extract a key fell from days to hours to minutes. By 2016, attacks took 8 authentication attempts and a minute of computation on a laptop; they worked against cars from all the French and Italian makers, along with Nissan, Mitsubishi and Chevrolet [748].

The last to fall was the Megamos Crypto transponder, used by Volkswagen and others. Car locksmithing tools appeared on the market from 2008, which included the Megamos cipher and were reverse engineered by researchers from Birmingham and Nijmegen – Roel Verdult, Flavio Garcia and Barış Ege – who cracked it [1956]. Although it has a 96-bit secret key, the effective key length is only 49 bits, about the same as Hitag 2. Volkswagen got an injunction in the High Court in London to stop them presenting their work at Usenix 2013, claiming that their trade secrets had been violated. The researchers resisted, arguing that the locksmithing tool supplier had extracted the secrets. After two years of argument, the case settled without admission of liability on either side. Closer study then threw up a number of further problems. There's also a protocol attack as an adversary can rewrite each 16-bit word of the 96-bit key, one after another, and search for the key 16 bits at a time; this reduces the time needed for an attack from days to minutes [1957].

Key management was pervasively bad. A number of Volkswagen implementations did not diversify keys across cars and transponders, but used a fixed global master key for millions of cars at a time. Up till 2009, this used a cipher called AUT64 to generate device keys; thereafter they moved to a stronger cipher called XTEA but kept on using global master keys, which were found in 23 models from the Volkswagen-Audi group up till 2016 [748]³.

It's easy to find out if a car is vulnerable: just try to buy a spare key. If the locksmith companies have figured out how to duplicate the key, your local garage will sell you a spare for a few bucks. We have a spare key for my wife's 2005 Lexus, bought by the previous owner. But when we lost one of the keys for my 2012 Mercedes, we had to go to a main dealer, pay over £200, show my passport and the car log book, have the mechanic photograph the vehicle identification number on the chassis, send it all off to Mercedes and wait for a week. We saw in Chapter 3 that the hard part of designing a password system was recovering from compromise without the recovery mechanism itself becoming either a vulnerability or a nuisance. Exactly the same applies here!

But the worst was still to come: passive keyless entry systems (PKES). Challenge-response seemed so good that car vendors started using it with just a push button on the dashboard to start the car, rather than with a metal key. Then they increased the radio frequency to extend the range, so that it worked not just for short-range authentication once the driver was sitting in the car, but as a keyless entry mechanism. The marketing pitch was that so long as

³There are some applications where universal master keys are inevitable, such as in communicating with a heart pacemaker – where a cardiologist may need to tweak the pacemaker of any patient who walks in, regardless of where it was first fitted, and regardless of whether the network's up – so the vendor puts the same key in all its equipment. Another example is the subscriber smartcard in a satellite-TV set-top box, which we'll discuss later. But they often result in a break-once-run-anywhere (BORA) attack. To install universal master keys in valuable assets like cars in a way that facilitated theft and without even using proper tamper-resistant chips to protect them was an egregious error.

you keep the key in your pocket or handbag you don't have to worry about it; the car will unlock when you walk up to it, lock as you walk away, and start automatically when you touch the controls. What's not to like?

Well, now you don't have to press a button to unlock your car, it's easy for thieves to use devices that amplify or relay the signals. The thief sneaks up to your front door with one relay while leaving the other next to your car. If you left your keys on the table in the hall, the car door opens and away he goes. Even if the car is immobilised he can still steal your stuff. And after many years of falling car thefts, the statistics surged in 2017 with 56% more vehicles stolen in the UK, followed by a further 9% in 2018 [824]⁴.

The takeaway message is that the attempt since about 1990 to use cryptography to make cars harder to steal had some initial success, as immobilisers made cars harder to steal and insurance premiums fell. It has since backfired, as the politicians and then the marketing people got in the way. The politicians said it would be disastrous for law enforcement if people were allowed to use cryptography they couldn't crack, even for stopping car theft. Then the immobiliser vendors' marketing people wanted proprietary algorithms to lock in the car companies, whose own marketing people wanted passive keyless entry as it seemed cool.

What can we do? Well, at least two car makers have put an accelerometer in the key fob, so it won't work unless the key is moving. One of our friends left her key on the car seat while carrying her child indoors, and got locked out. The local police advise us to use old-fashioned metal steering-wheel locks; our residents' association recommends keeping keys in a biscuit tin. As for me, we bought such a car but found that the keyless entry was simply too flaky; my wife got stranded in a supermarket car park when it just wouldn't work at all. So we took that car back, and got a second-hand one with a proper push-button remote lock. There are now chips using AES from NXP, Atmel and TI – of which the Atmel is open source with an open protocol stack.

However crypto by itself can't fix relay attacks; the proper fix is a new radio protocol based on ultrawideband (UWB) with intrinsic ranging, which measures the distance from the key fob to the car with a precision of 10cm up to a range of 150m. This is fairly complex to do properly, and the design of the new 802.15.4z Enhanced Impulse Radio is described by Srdjan Capkun and colleagues [1768]; the first chip became available in 2019, and it will ship in cars from 2020. Such chips have the potential to replace both the Bluetooth and NFC protocols, but they might not all be compatible; there's a low-rate pulse (LRP) mode that has an open design, and a high-rate pulse (HRP) variant that's partly proprietary. Were I advising a car startup, LRP would be my starting point.

⁴To be fair this was not due solely to relay attacks, as about half of the high-value thefts seem to involve connecting a car theft kit to the onboard diagnostic port under the glove box. As it happens, the authentication protocols used on the CAN bus inside the vehicle are also vulnerable in a number of ways [893]. Updating these protocols will take many years because of the huge industry investment.

Locks are not the only application of challenge-response protocols. In HTTP Digest Authentication, a web server challenges a client or proxy, with whom it shares a password, by sending it a nonce. The response consists of the hash of the nonce, the password, and the requested URI [715]. This provides a mechanism that's not vulnerable to password snooping. It's used, for example, to authenticate clients and servers in SIP, the protocol for Voice-Over-IP (VOIP) telephony. It's much better than sending a password in the clear, but like key-less entry it suffers from middleperson attacks (the beneficiaries seem to be mostly intelligence agencies).

4.3.2 Two-factor authentication

The most visible use of challenge-response is probably in *two-factor authentication*. Many organizations issue their staff with password generators to let them log on to corporate computer systems, and many banks give similar devices to customers. They may look like little calculators (and some even work as such) but their main function is as follows. When you want to log in, you are presented with a random nonce of maybe seven digits. You key this into your password generator, together with a PIN of maybe four digits. The device encrypts these eleven digits using a secret key shared with the corporate security server, and displays the first seven digits of the result. You enter these seven digits as your password. This protocol is illustrated in Figure 4.1. If you had a password generator with the right secret key, and you entered the PIN right, and you typed in the result correctly, then you get in.

Formally, with S for the server, P for the password generator, PIN for the user's Personal Identification Number, U for the user and N for the nonce:

$$\begin{aligned} S &\rightarrow U : N \\ U &\rightarrow P : N, PIN \\ P &\rightarrow U : \{N, PIN\}_K \\ U &\rightarrow S : \{N, PIN\}_K \end{aligned}$$

These devices appeared from the early 1980s and caught on first with phone companies, then in the 1990s with banks for use by staff. There are simplified versions that don't have a keyboard, but just generate new access codes by encrypting a counter or a clock. And they work; the US Defense Department announced in 2007 that an authentication system based on the DoD Common Access Card had cut network intrusions by 46% in the previous year [321].

This was just when crooks started phishing bank customers at scale, so many banks adopted the technology. One of my banks gives me a small calculator that generates a new code for each logon, and also allows me to authenticate new payees by using the last four digits of their account number in place of the challenge. My other bank uses the Chip Authentication Program (CAP), a calculator in which I can insert my bank card to do the crypto.

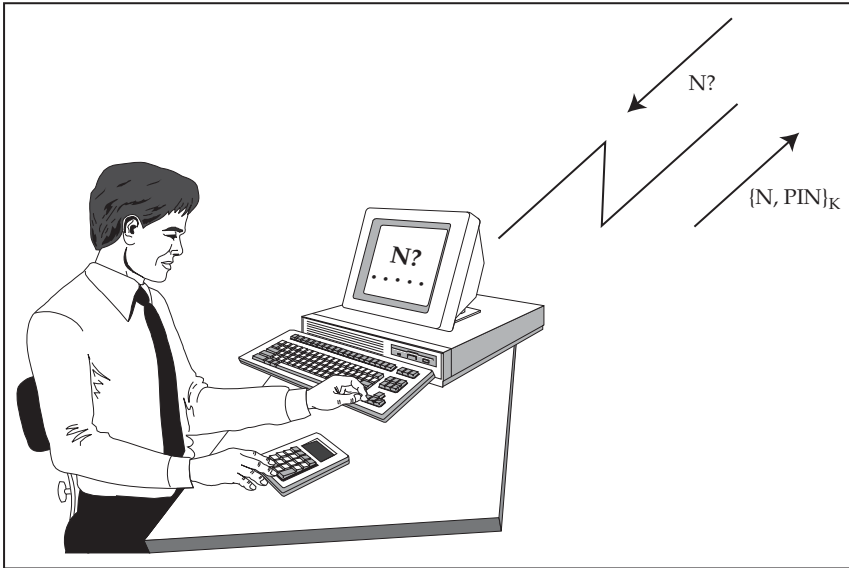


Figure 4.1: Password generator use

But this still isn't foolproof. In the second edition of this book, I noted 'someone who takes your bank card from you at knifepoint can now verify that you've told them the right PIN', and this now happens. I also noted that 'once lots of banks use one-time passwords, the phishermen will just rewrite their scripts to do real-time man-in-the-middle attacks' and this has also become widespread. To see how such attacks work, let's look at a military example.

4.3.3 The MIG-in-the-middle attack

The first use of challenge-response authentication protocols was probably in the military, with 'identify-friend-or-foe' (IFF) systems. The ever-increasing speeds of warplanes in the 1930s and 1940s, together with the invention of the jet engine, radar and rocketry, made it ever more difficult for air defence forces to tell their own craft apart from the enemy's. This led to a risk of pilots shooting down their colleagues by mistake and drove the development of automatic systems to prevent this. These were first fielded in World War II, and enabled an airplane illuminated by radar to broadcast an identifying number to signal friendly intent. In 1952, this system was adopted to identify civil aircraft to air traffic controllers and, worried about the loss of security once it became widely used, the US Air Force started a research program to incorporate cryptographic protection in the system. Nowadays, the typical air defense system sends random challenges with its radar signals, and friendly aircraft can identify themselves with correct responses.

It's tricky to design a good IFF system. One of the problems is illustrated by the following story, which I heard from an officer in the South African Air Force (SAAF). After it was published in the first edition of this book, the story was disputed – as I'll discuss below. Be that as it may, similar games have been played with other electronic warfare systems since World War 2. The 'MIG-in-the-middle' story has since become part of the folklore, and it nicely illustrates how attacks can be carried out in real time on challenge-response protocols.

In the late 1980's, South African troops were fighting a war in northern Namibia and southern Angola. Their goals were to keep Namibia under white rule, and impose a client government (UNITA) on Angola. Because the South African Defence Force consisted largely of conscripts from a small white population, it was important to limit casualties, so most South African soldiers remained in Namibia on policing duties while the fighting to the north was done by UNITA troops. The role of the SAAF was twofold: to provide tactical support to UNITA by bombing targets in Angola, and to ensure that the Angolans and their Cuban allies did not return the compliment in Namibia.

Suddenly, the Cubans broke through the South African air defenses and carried out a bombing raid on a South African camp in northern Namibia, killing a number of white conscripts. This proof that their air supremacy had been lost helped the Pretoria government decide to hand over Namibia to the insurgents –itself a huge step on the road to majority rule in South Africa several years later. The raid may also have been the last successful military operation ever carried out by Soviet bloc forces.

Some years afterwards, a SAAF officer told me how the Cubans had pulled it off. Several MIGs had loitered in southern Angola, just north of the South African air defense belt, until a flight of SAAF Impala bombers raided a target in Angola. Then the MIGs turned sharply and flew openly through the SAAF's air defenses, which sent IFF challenges. The MIGs relayed them to the Angolan air defense batteries, which transmitted them at a SAAF bomber; the responses were relayed back to the MIGs, who retransmitted them and were allowed through – as in Figure 4.2. According to my informant, this shocked the general staff in Pretoria. Being not only outfought by black opponents, but actually outsmarted, was not consistent with the world view they had held up till then.

After this tale was published in the first edition of my book, I was contacted by a former officer in SA Communications Security Agency who disputed the story's details. He said that their IFF equipment did not use cryptography yet at the time of the Angolan war, and was always switched off over enemy territory. Thus, he said, any electronic trickery must have been of a more primitive kind. However, others tell me that 'Mig-in-the-middle' tricks were significant in Korea, Vietnam and various Middle Eastern conflicts.

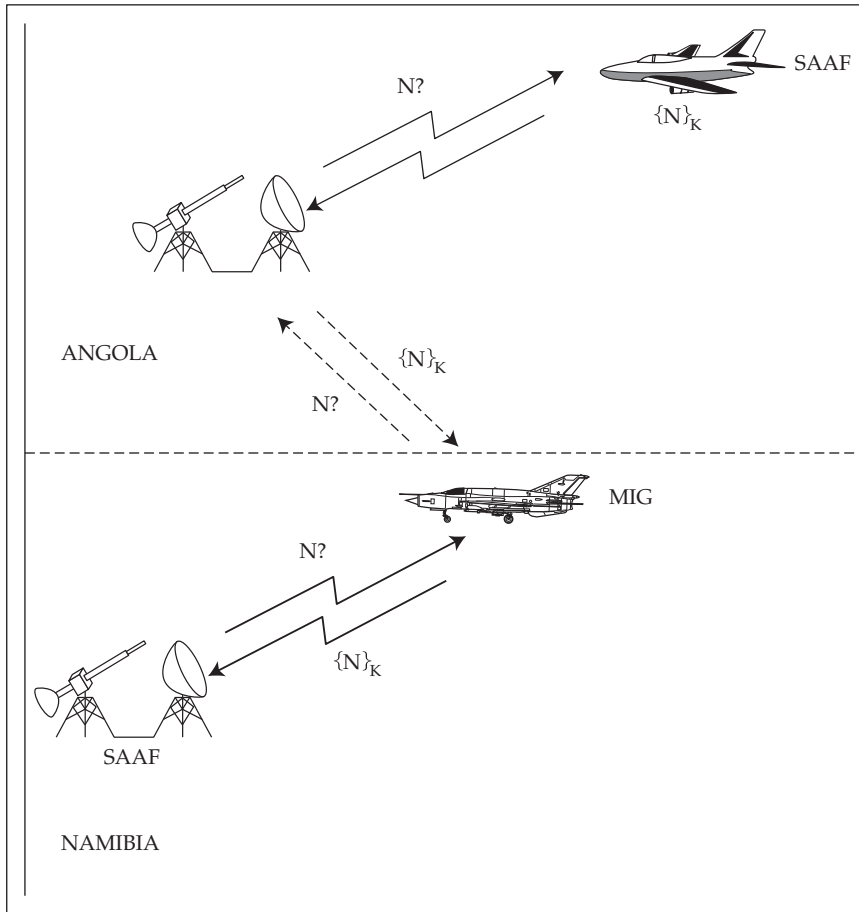


Figure 4.2: The MIG-in-the middle attack

In any case, the tale gives us another illustration of the man-in-the-middle attack. The relay attack against cars is another example. It also works against password calculators: the phishing site invites the mark to log on and simultaneously opens a logon session with his bank. The bank sends a challenge; the phisherman relays this to the mark, who uses his device to respond to it; the phisherman relays the response to the bank, and the bank now accepts the phisherman as the mark.

Stopping a middleperson attack is harder than it looks, and may involve multiple layers of defence. Banks typically look for a known machine, a password, a second factor such as an authentication code from a CAP reader, and a risk assessment of the transaction. For high-risk transactions, such as adding a new payee to an account, both my banks demand that I compute an authentication code on the payee account number. But they only authenticate the last four

digits, because of usability. If it takes two minutes and the entry of dozens of digits to make a payment, then a lot of customers will get digits wrong, give up, and then either call the call center or get annoyed and bank elsewhere. Also, the bad guys may be able to exploit any fallback mechanisms, perhaps by spoofing customers into calling phone numbers that run a middleperson attack between the customer and the call center. I'll discuss all this further in the chapter on Banking and Bookkeeping.

We will come across such attacks again and again in applications ranging from Internet security protocols to Bluetooth. They even apply in gaming. As the mathematician John Conway once remarked, it's easy to get at least a draw against a grandmaster at postal chess: just play two grandmasters at once, one as white and the other as black, and relay the moves between them!

4.3.4 Reflection attacks

Further interesting problems arise when two principals have to identify each other. Suppose that a challenge-response IFF system designed to prevent anti-aircraft gunners attacking friendly aircraft had to be deployed in a fighter-bomber too. Now suppose that the air force simply installed one of their air gunners' challenge units in each aircraft and connected it to the fire-control radar.

But now when a fighter challenges an enemy bomber, the bomber might just reflect the challenge back to the fighter's wingman, get a correct response, and then send that back as its own response:

$$\begin{aligned} F \rightarrow B & : N \\ B \rightarrow F' & : N \\ F' \rightarrow B & : \{N\}_K \\ B \rightarrow F & : \{N\}_K \end{aligned}$$

There are a number of ways of stopping this, such as including the names of the two parties in the exchange. In the above example, we might require a friendly bomber to reply to the challenge:

$$F \rightarrow B : N$$

with a response such as:

$$B \rightarrow F : \{B, N\}_K$$

Thus a reflected response $\{F', N\}$ from the wingman F' could be detected⁵.

This serves to illustrate the subtlety of the trust assumptions that underlie authentication. If you send out a challenge N and receive, within 20 milliseconds, a response $\{N\}_K$, then – since light can travel a bit under 3,730 miles in 20 ms – you know that there is someone with the key K within 2000 miles.

⁵And don't forget: you also have to check that the intruder didn't just reflect your own challenge back at you. You must be able to remember or recognise your own messages!

But that's all you know. If you can be sure that the response was not computed using your own equipment, you now know that there is someone *else* with the key K within two thousand miles. If you make the further assumption that all copies of the key K are securely held in equipment which may be trusted to operate properly, and you see $\{B, N\}_K$, you might be justified in deducing that the aircraft with callsign B is within 2000 miles. A careful analysis of trust assumptions and their consequences is at the heart of security protocol design.

By now you might think that we understand all the protocol design aspects of IFF. But we've omitted one of the most important problems – and one which the designers of early IFF systems didn't anticipate. As radar is passive the returns are weak, while IFF is active and so the signal from an IFF transmitter will usually be audible at a much greater range than the same aircraft's radar return. The Allies learned this the hard way; in January 1944, decrypts of Enigma messages revealed that the Germans were plotting British and American bombers at twice the normal radar range by interrogating their IFF. So more modern systems authenticate the challenge as well as the response. The NATO mode XII, for example, has a 32 bit encrypted challenge, and a different valid challenge is generated for every interrogation signal, of which there are typically 250 per second. Theoretically there is no need to switch off over enemy territory, but in practice an enemy who can record valid challenges can replay them as part of an attack. Relays are made difficult in mode XII using directionality and time-of-flight.

Other IFF design problems include the difficulties posed by neutrals, error rates in dense operational environments, how to deal with equipment failure, how to manage keys, and how to cope with multinational coalitions. I'll return to IFF in Chapter 23. For now, the spurious-challenge problem serves to reinforce an important point: that the correctness of a security protocol depends on the assumptions made about the requirements. A protocol that can protect against one kind of attack (being shot down by your own side) but which increases the exposure to an even more likely attack (being shot down by the other side) might not help. In fact, the spurious-challenge problem became so serious in World War II that some experts advocated abandoning IFF altogether, rather than taking the risk that one bomber pilot in a formation of hundreds would ignore orders and leave his IFF switched on while over enemy territory.

4.4 Manipulating the message

We've now seen a number of middleperson attacks that reflect or spoof the information used to authenticate a participant. However, there are more complex attacks where the attacker doesn't just impersonate someone, but manipulates the message content.

One example we saw already is the prepayment meter that remembers only the last ticket it saw, so it can be recharged without limit by copying in the codes from two tickets *A* and *B* one after another: *ABABAB....* Another is when dishonest cabbies insert pulse generators in the cable that connects their taximeter to a sensor in their taxi's gearbox. The sensor sends pulses as the prop shaft turns, which lets the meter work out how far the taxi has gone. A pirate device can insert extra pulses, making the taxi appear to have gone further. A truck driver who wants to drive faster or further than regulations allow can use a similar device to discard some pulses, so he seems to have been driving more slowly or not at all. We'll discuss such attacks in the chapter on 'Monitoring Systems', in section 14.3.

As well as monitoring systems, control systems often need to be hardened against message-manipulation attacks. The Intelsat satellites used for international telephone and data traffic have mechanisms to prevent a command being accepted twice – otherwise an attacker could replay control traffic and repeatedly order the same maneuver to be carried out until the satellite ran out of fuel [1529]. We will see lots of examples of protocol attacks involving message manipulation in later chapters on specific applications.

4.5 Changing the environment

A common cause of protocol failure is that the environment changes, so that the design assumptions no longer hold and the security protocols cannot cope with the new threats.

A nice example comes from the world of cash machine fraud. In 1993, Holland suffered an epidemic of 'phantom withdrawals'; there was much controversy in the press, with the banks claiming that their systems were secure while many people wrote in to the papers claiming to have been cheated. Eventually the banks noticed that many of the victims had used their bank cards at a certain filling station near Utrecht. This was staked out and one of the staff was arrested. It turned out that he had tapped the line from the card reader to the PC that controlled it; his tap recorded the magnetic stripe details from their cards while he used his eyeballs to capture their PINs [55]. Exactly the same fraud happened in the UK after the move to 'chip and PIN' smartcards in the mid-2000s; a gang wiretapped perhaps 200 filling stations, collected card data from the wire, observed the PINs using CCTV cameras, then made up thousands of magnetic-strip clone cards that were used in countries whose ATMs still used magnetic strip technology. At our local filling station, over 200 customers suddenly found that their cards had been used in ATMs in Thailand.

Why had the system been designed so badly, and why did the design error persist for over a decade through a major technology change? Well, when the standards for managing magnetic stripe cards and PINs were developed in the

early 1980's by organizations such as IBM and VISA, the engineers had made two assumptions. The first was that the contents of the magnetic strip – the card number, version number and expiration date – were not secret, while the PIN was [1303]. (The analogy used was that the magnetic strip was your name and the PIN your password.) The second assumption was that bank card equipment would only be operated in trustworthy environments, such as in a physically robust automatic teller machine, or by a bank clerk at a teller station. So it was 'clearly' only necessary to encrypt the PIN, on its way from the PIN pad to the server; the magnetic strip data could be sent in clear from the card reader.

Both of these assumptions had changed by 1993. An epidemic of card forgery, mostly in the Far East in the late 1980's, drove banks to introduce authentication codes on the magnetic strips. Also, the commercial success of the bank card industry led banks in many countries to extend the use of debit cards from ATMs to terminals in all manner of shops. The combination of these two environmental changes destroyed the assumptions behind the original system architecture. Instead of putting a card whose magnetic strip contained no security data into a trusted machine, people were putting a card with clear security data into an untrusted machine. These changes had come about so gradually, and over such a long period, that the industry didn't see the problem coming.

4.6 Chosen protocol attacks

Governments keen to push ID cards have tried to get them used for many other transactions; some want a single card to be used for ID, banking and even transport ticketing. Singapore went so far as to experiment with a bank card that doubled as military ID. This introduced some interesting new risks: if a Navy captain tries to withdraw some cash from an ATM after a good dinner and forgets his PIN, will he be unable to take his ship to sea until Monday morning when they open the bank and give him his card back?

Some firms are pushing multifunction authentication devices that could be used in a wide range of transactions to save you having to carry around dozens of different cards and keys. A more realistic view of the future may be that people's phones will be used for most private-sector authentication functions.

But this too may not be as simple as it looks. The idea behind the 'Chosen Protocol Attack' is that given a target protocol, you design a new protocol that will attack it if the users can be inveigled into reusing the same token or crypto key. So how might the Mafia design a protocol to attack the authentication of bank transactions?

Here's one approach. It used to be common for people visiting a porn website to be asked for 'proof of age,' which usually involves giving a credit card number, whether to the site itself or to an age checking service. If

smartphones are used to authenticate everything, it would be natural for the porn site to ask the customer to authenticate a random challenge as proof of age. A porn site might then mount a ‘Mafia-in-the-middle’ attack as shown in Figure 4.3. They wait until an unsuspecting customer visits their site, then order something resellable (such as gold coins) from a dealer, playing the role of the coin dealer’s customer. When the coin dealer sends them the transaction data for authentication, they relay it through their porn site to the waiting customer. The poor man OKs it, the Mafia gets the gold coins, and when thousands of people suddenly complain about the huge charges to their cards at the end of the month, the porn site has vanished – along with the gold [1034].

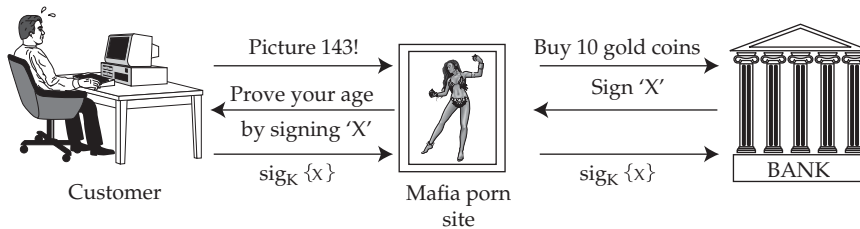


Figure 4.3: The Mafia-in-the-middle attack

In the 1990s a vulnerability of this kind found its way into international standards: the standards for digital signature and authentication could be run back-to-back in this way. It has since been shown that many protocols, though secure in themselves, can be broken if their users can be inveigled into reusing the same keys in other applications [1034]. This is why, if we’re going to use our phones to authenticate everything, it will be really important to keep the banking apps and the porn apps separate. That will be the subject in Chapter 6 on Access Control.

In general, using crypto keys (or other authentication mechanisms) in more than one application is dangerous, while letting other people bootstrap their own application security off yours can be downright foolish. The classic case is where a bank relies for two-factor authentication on sending SMSes to customers as authentication codes. As I discussed in section 3.4.1, the bad guys have learned to attack that system by SIM-swap fraud – pretending to the phone company that they’re the target, claiming to have lost their phone, and getting a replacement SIM card.

4.7 Managing encryption keys

The examples of security protocols that we’ve discussed so far are mostly about authenticating a principal’s name, or application data such as the impulses driving a taximeter. There is one further class of authentication protocols that is very important – the protocols used to manage cryptographic keys.

4.7.1 The resurrecting duckling

In the Internet of Things, keys can sometimes be managed directly and physically, by local setup and a policy of *trust-on-first-use* or TOFU.

Vehicles provided an early example. I mentioned above that crooked taxi drivers used to put interruptors in the cable from their car's gearbox sensor to the taximeter, to add additional mileage. The same problem happened in reverse with tachographs, the devices used by trucks to monitor drivers' hours and speed. When tachographs went digital in the late 1990s, we decided to encrypt the pulse train from the sensor. But how could keys be managed? The solution was that whenever a new tachograph is powered up after a factory reset, it trusts the first crypto key it receives over the sensor cable. I'll discuss this further in section 14.3.

A second example is Homeplug AV, the standard used to encrypt data communications over domestic power lines, and widely used in LAN extenders. In the default, 'just-works' mode, a new Homeplug device trusts the first key it sees; and if your new wifi extender mates with the neighbour's wifi instead, you just press the reset button and try again. There is also a 'secure mode' where you open a browser to the network management node and manually enter a crypto key printed on the device packaging, but when we designed the Homeplug protocol we realised that most people have no reason to bother with that [1439].

The TOFU approach is also known as the 'resurrecting duckling' after an analysis that Frank Stajano and I did in the context of pairing medical devices [1822]. The idea is that when a baby duckling hatches, it imprints on the first thing it sees that moves and quacks, even if this is the farmer – who can end up being followed everywhere by a duck that thinks he's mummy. If such false imprinting happens with an electronic device, you need a way to kill it and resurrect it into a newborn state – which the reset button does in a device such as a LAN extender.

4.7.2 Remote key management

The more common, and interesting, case is the management of keys in remote devices. The basic technology was developed from the late 1970s to manage keys in distributed computer systems, with cash machines being an early application. In this section we'll discuss shared-key protocols such as Kerberos, leaving public-key protocols such as TLS and SSH until after we've discussed public-key cryptology in Chapter 5.

The basic idea behind key-distribution protocols is that where two principals want to communicate, they may use a trusted third party to introduce them. It's customary to give them human names in order to avoid getting lost in too much algebra. So we will call the two communicating principals 'Alice' and 'Bob', and the trusted third party 'Sam'. Alice, Bob and Sam are likely to be programs running on different devices. (For example, in a protocol to let a car

dealer make a replacement key with a car, Alice might be the car, Bob the key and Sam the car maker.)

A simple authentication protocol could run as follows.

1. Alice first calls Sam and asks for a key for communicating with Bob.
2. Sam responds by sending Alice a pair of certificates. Each contains a copy of a key, the first encrypted so only Alice can read it, and the second encrypted so only Bob can read it.
3. Alice then calls Bob and presents the second certificate as her introduction. Each of them decrypts the appropriate certificate under the key they share with Sam and thereby gets access to the new key. Alice can now use the key to send encrypted messages to Bob, and to receive messages from him in return.

We've seen that replay attacks are a known problem, so in order that both Bob and Alice can check that the certificates are fresh, Sam may include a timestamp in each of them. If certificates never expire, there might be serious problems dealing with users whose privileges have been revoked.

Using our protocol notation, we could describe this as

$$\begin{aligned}
 A \rightarrow S &: A, B \\
 S \rightarrow A &: \{A, B, K_{AB}, T\}_{K_{AS}}, \{A, B, K_{AB}, T\}_{K_{BS}} \\
 A \rightarrow B &: \{A, B, K_{AB}, T\}_{K_{BS}}, \{M\}_{K_{AB}}
 \end{aligned}$$

Expanding the notation, Alice calls Sam and says she'd like to talk to Bob. Sam makes up a message consisting of Alice's name, Bob's name, a session key for them to use, and a timestamp. He encrypts all this under the key he shares with Alice, and he encrypts another copy of it under the key he shares with Bob. He gives both ciphertexts to Alice. Alice retrieves the session key from the ciphertext that was encrypted to her, and passes on to Bob the ciphertext encrypted for him. She now sends him whatever message she wanted to send, encrypted using this session key.

4.7.3 The Needham-Schroeder protocol

Many things can go wrong, and here is a famous historical example. Many existing key distribution protocols are derived from the Needham-Schroeder protocol, which appeared in 1978 [1428]. It is somewhat similar to the above, but uses nonces rather than timestamps. It runs as follows:

Message 1	$A \rightarrow S :$	A, B, N_A
Message 2	$S \rightarrow A :$	$\{N_A, B, K_{AB}, \{K_{AB}, A\}_{K_{BS}}\}_{K_{AS}}$
Message 3	$A \rightarrow B :$	$\{K_{AB}, A\}_{K_{BS}}$
Message 4	$B \rightarrow A :$	$\{N_B\}_{K_{AB}}$
Message 5	$A \rightarrow B :$	$\{N_B - 1\}_{K_{AB}}$

Here Alice takes the initiative, and tells Sam: ‘I’m Alice, I want to talk to Bob, and my random nonce is N_A .’ Sam provides her with a session key, encrypted using the key she shares with him. This ciphertext also contains her nonce so she can confirm it’s not a replay. He also gives her a certificate to convey this key to Bob. She passes it to Bob, who then does a challenge-response to check that she is present and alert.

There is a subtle problem with this protocol – Bob has to assume that the key K_{AB} he receives from Sam (via Alice) is fresh. This is not necessarily so: Alice could have waited a year between steps 2 and 3. In many applications this may not be important; it might even help Alice to cache keys against possible server failures. But if an opponent – say Charlie – ever got hold of Alice’s key, he could use it to set up session keys with many other principals. And if Alice ever got fired, then Sam had better have a list of everyone in the firm to whom he issued a key for communicating with her, to tell them not to believe it any more. In other words, revocation is a problem: Sam may have to keep complete logs of everything he’s ever done, and these logs would grow in size forever unless the principals’ names expired at some fixed time in the future.

Almost 40 years later, this example is still controversial. The simplistic view is that Needham and Schroeder just got it wrong; the view argued by Susan Pancho and Dieter Gollmann (for which I have some sympathy) is that this is a protocol failure brought on by shifting assumptions [781, 1493]. 1978 was a kinder, gentler world; computer security then concerned itself with keeping ‘bad guys’ out, while nowadays we expect the ‘enemy’ to be among the users of our system. The Needham-Schroeder paper assumed that all principals behave themselves, and that all attacks came from outsiders [1428]. Under those assumptions, the protocol remains sound.

4.7.4 Kerberos

The most important practical derivative of the Needham-Schroeder protocol is Kerberos, a distributed access control system that originated at MIT and is now one of the standard network authentication tools [1829]. It has become part of the basic mechanics of authentication for both Windows and Linux, particularly when machines share resources over a local area network. Instead of a single trusted third party, Kerberos has two kinds: authentication servers to which users log on, and ticket granting servers which give them tickets allowing access to various resources such as files. This enables scalable access management. In a university, for example, one might manage students through their colleges or halls of residence but manage file servers by departments; in a company, the personnel people might register users to the payroll system while departmental administrators manage resources such as servers and printers.

First, Alice logs on to the authentication server using a password. The client software in her PC fetches a ticket from this server that is encrypted under her password and that contains a session key K_{AS} . Assuming she gets the password right, she now controls K_{AS} and to get access to a resource B controlled by the ticket granting server S , the following protocol takes place. Its outcome is a key K_{AB} with timestamp T_S and lifetime L , which will be used to authenticate Alice's subsequent traffic with that resource:

$$\begin{aligned}
 A \rightarrow S &: A, B \\
 S \rightarrow A &: \{T_S, L, K_{AB}, B, \{T_S, L, K_{AB}, A\}_{K_{BS}}\}_{K_{AS}} \\
 A \rightarrow B &: \{T_S, L, K_{AB}, A\}_{K_{BS}}, \{A, T_A\}_{K_{AB}} \\
 B \rightarrow A &: \{T_A + 1\}_{K_{AB}}
 \end{aligned}$$

Translating this into English: Alice asks the ticket granting server for access to B . If this is permissible, the ticket $\{T_S, L, K_{AB}, A\}_{K_{BS}}$ is created containing a suitable key K_{AB} and given to Alice to use. She also gets a copy of the key in a form readable by her, namely encrypted under K_{AS} . She now verifies the ticket by sending a timestamp T_A to the resource, which confirms it's alive by sending back the timestamp incremented by one (this shows it was able to decrypt the ticket correctly and extract the key K_{AB}).

The revocation issue with the Needham-Schroeder protocol has been fixed by introducing timestamps rather than random nonces. But, as in most of life, we get little in security for free. There is now a new vulnerability, namely that the clocks on our various clients and servers might get out of sync; they might even be desynchronized deliberately as part of a more complex attack.

What's more, Kerberos is a *trusted third-party* (TTP) protocol in that S is trusted: if the police turn up with a warrant, they can get Sam to turn over the keys and read the traffic. Protocols with this feature were favoured during the 'crypto wars' of the 1990s, as I will discuss in section 26.2.7. Protocols that involve no or less trust in a third party generally use public-key cryptography, which I describe in the next chapter.

A rather similar protocol to Kerberos is OAuth, a mechanism to allow secure delegation. For example, if you log into Doodle using Google and allow Doodle to update your Google calendar, Doodle's website redirects you to Google, which gets you to log in (or relies on a master cookie from a previous login) and asks you for consent for Doodle to write to your calendar. Doodle then gives you an access token for the calendar service [864]. I mentioned in section 3.4.9.3 that this poses a cross-site phishing risk. OAuth was not designed for user authentication, and access tokens are not strongly bound to clients. It's a complex framework within which delegation mechanisms can be built, with both short-term and long-term access tokens; the details are tied up with how cookies and web redirects operate and optimised to enable servers to be stateless, so they scale well for modern web services. In the example above, you want to

be able to revoke Doodle's access at Google, so behind the scenes Doodle only gets short-lived access tokens. Because of this complexity, the OpenID Connect protocol is a 'profile' of OAuth which ties down the details for the case where the only service required is authentication. OpenID Connect is what you use when you log into your newspaper using your Google or Facebook account.

4.7.5 Practical key management

So we can use a protocol like Kerberos to set up and manage working keys between users given that each user shares one or more long-term keys with a server that acts as a key distribution centre. But there may be encrypted passwords for tens of thousands of staff and keys for large numbers of devices too. That's a lot of key material. How is it to be managed?

Key management is a complex and difficult business and is often got wrong because it's left as an afterthought. You need to sit down and think about how many keys are needed, how they're to be generated, how long they need to remain in service and how they'll eventually be destroyed. There is a much longer list of concerns – many of them articulated in the Federal Information Processing Standard for key management [1410]. And things go wrong as applications evolve; it's important to provide headroom to support next year's functionality. It's also important to support recovery from security failure. Yet there are no standard ways of doing either.

Public-key cryptography, which I'll discuss in Chapter 5, can simplify the key-management task slightly. In banking the usual answer is to use dedicated cryptographic processors called hardware security modules, which I'll describe in detail later. Both of these introduce further complexities though, and even more subtle ways of getting things wrong.

4.8 Design assurance

Subtle difficulties of the kind we have seen above, and the many ways in which protection properties depend on subtle assumptions that may be misunderstood, have led researchers to apply formal methods to protocols. The goal of this exercise was originally to decide whether a protocol was right or wrong: it should either be proved correct, or an attack should be exhibited. We often find that the process helps clarify the assumptions that underlie a given protocol.

There are several different approaches to verifying the correctness of protocols. One of the best known is the *logic of belief*, or *BAN logic*, named after its inventors Burrows, Abadi and Needham [352]. It reasons about what a principal might reasonably believe having seen certain messages, timestamps and so

on. Other researchers have applied mainstream formal methods such as CSP and verification tools such as Isabelle.

Some history exists of flaws being found in protocols that had been proved correct using formal methods; I described an example in Chapter 3 of the second edition, of how the BAN logic was used to verify a bank card used for stored-value payments. That's still used in Germany as the 'Geldkarte' but elsewhere its use has died out (it was Net1 in South Africa, Proton in Belgium, Moneo in France and a VISA product called COPAC). I've therefore decided to drop the gory details from this edition; the second edition is free online, so you can download and read the details.

Formal methods can be an excellent way of finding bugs in security protocol designs as they force the designer to make everything explicit and thus confront difficult design choices that might otherwise be fudged. But they have their limitations, too.

We often find bugs in verified protocols; they're just not in the part that we verified. For example, Larry Paulson verified the SSL/TLS protocol using his Isabelle theorem prover in 1998, and about one security bug has been found every year since then. These have not been flaws in the basic design but exploited additional features that had been added later, and implementation issues such as timing attacks, which we'll discuss later. In this case there was no failure of the formal method; that simply told the attackers where they needn't bother looking.

For these reasons, people have explored alternative ways of assuring the design of authentication protocols, including the idea of *protocol robustness*. Just as structured programming techniques aim to ensure that software is designed methodically and nothing of importance is left out, so robust protocol design is largely about explicitness. Robustness principles include that the interpretation of a protocol should depend only on its content, not its context; so everything of importance (such as principals' names) should be stated explicitly in the messages. It should not be possible to interpret data in more than one way; so the message formats need to make clear what's a name, what's an address, what's a timestamp, and so on; string formats have to be unambiguous and it should be impossible to use the protocol itself to mount attacks on the software that handles it, such as by buffer overflows. There are other issues concerning the freshness provided by counters, timestamps and random challenges, and on the way encryption is used. If the protocol uses public key cryptography or digital signature mechanisms, there are more subtle attacks and further robustness issues, which we'll start to tackle in the next chapter. To whet your appetite, randomness in protocol often helps robustness at other layers, since it makes it harder to do a whole range of attacks – from those based on mathematical cryptanalysis through those that exploit side-channels such as power consumption and timing to physical attacks that involve microprobes or lasers.

4.9 Summary

Passwords are just one example of a more general concept, the security protocol. Protocols specify the steps that principals use to establish trust relationships in a system, such as authenticating a claim to identity, demonstrating ownership of a credential, or establishing a claim on a resource. Cryptographic authentication protocols are used for a wide range of purposes, from basic entity authentication to providing infrastructure for distributed systems that allows trust to be taken from where it exists to where it is needed. Security protocols are fielded in all sorts of systems from remote car door locks through military IFF systems to authentication in distributed computer systems.

Protocols are surprisingly difficult to get right. They can suffer from a number of problems, including middleperson attacks, modification attacks, reflection attacks, and replay attacks. These threats can interact with implementation vulnerabilities and poor cryptography. Using mathematical techniques to verify the correctness of protocols can help, but it won't catch all the bugs. Some of the most pernicious failures are caused by creeping changes in the environment for which a protocol was designed, so that the protection it gives is no longer relevant. The upshot is that attacks are still found frequently on protocols that we've been using for years, and sometimes even on protocols for which we thought we had a security proof. Failures have real consequences, including the rise in car crime worldwide since car makers started adopting passive key-less entry systems without stopping to think about relay attacks. Please don't design your own protocols; get a specialist to help, and ensure that your design is published for thorough peer review by the research community. Even specialists get the first versions of a protocol wrong (I have, more than once). It's a lot cheaper to fix the bugs before the protocol is actually deployed, both in terms of cash and in terms of reputation.

Research problems

At several times during the past 30 years, some people have thought that protocols had been 'done' and that we should turn to new research topics. They have been repeatedly proved wrong by the emergence of new applications with a new crop of errors and attacks to be explored. Formal methods blossomed in the early 1990s, then key management protocols; during the mid-1990's the flood of proposals for electronic commerce mechanisms kept us busy. Since 2000, one strand of protocol research has acquired an economic flavour as security mechanisms are used more and more to support business models; the designer's 'enemy' is often a commercial competitor, or even the customer. Another has applied protocol analysis tools to look at the security of application programming interfaces (APIs), a topic to which I'll return later.

Much protocol research is problem-driven, but there are still deep questions. How much can we get out of formal methods, for example? And how do we manage the tension between the principle that robust protocols are generally those in which everything is completely specified and checked and the system engineering principle that a good specification should not overconstrain the implementer?

Further reading

Research papers on security protocols are scattered fairly widely throughout the literature. For the historical background you might read the original Needham-Schroeder paper [1428], the Burrows-Abadi-Needham authentication logic [352], papers on protocol robustness [2, 113] and a survey paper by Anderson and Needham [114]. Beyond that, there are many papers scattered around a wide range of conferences; you might also start by studying the protocols used in a specific application area, such as payments, which we cover in more detail in Part 2. As for remote key entry and other security issues around cars, a good starting point is a tech report by Charlie Miller and Chris Valasek on how to hack a Jeep Cherokee [1318].

Cryptography

ZHQM ZMGM ZMFM

– G JULIUS CAESAR

KXJEY UREBE ZWEHE WRYTU HEYFS KREHE GOYFI WTTTU OLKSY CAJPO BOTEI ZONTX BYBWT
GONEY CUZWR GDSON SXBOU YWRHE BAAHY USEDQ

– JOHN F KENNEDY

5.1 Introduction

Cryptography is where security engineering meets mathematics. It gives us the tools that underlie most modern security protocols. It is the key technology for protecting distributed systems, yet it is surprisingly hard to do right. As we’ve already seen in Chapter 4, “Protocols,” cryptography has often been used to protect the wrong things, or to protect them in the wrong way. Unfortunately, the available crypto tools aren’t always very usable.

But no security engineer can ignore cryptology. A medical friend once told me that while she was young, she worked overseas in a country where, for economic reasons, they’d shortened their medical degrees and concentrated on producing specialists as quickly as possible. One day, a patient who’d had both kidneys removed and was awaiting a transplant needed her dialysis shunt redone. The surgeon sent the patient back from the theater on the grounds that there was no urinalysis on file. It just didn’t occur to him that a patient with no kidneys couldn’t produce any urine.

Just as a doctor needs to understand physiology as well as surgery, so a security engineer needs to be familiar with at least the basics of crypto (and much else). There are, broadly speaking, three levels at which one can approach crypto. The first consists of the underlying intuitions; the second of the mathematics that we use to clarify these intuitions, provide security proofs where possible and tidy up the constructions that cause the most confusion;

and the third is the cryptographic engineering – the tools we commonly use, and the experience of what can go wrong with them. In this chapter, I assume you have no training in crypto and set out to explain the basic intuitions. I illustrate them with engineering, and sketch enough of the mathematics to help give you access to the literature when you need it. One reason you need some crypto know-how is that many common constructions are confusing, and many tools offer unsafe defaults. For example, Microsoft’s Crypto API (CAPI) nudges engineers to use electronic codebook mode; by the end of this chapter you should understand what that is, why it’s bad, and what you should do instead.

Many crypto textbooks assume that their readers are pure maths graduates, so let me start off with non-mathematical definitions. *Cryptography* refers to the science and art of designing ciphers; *cryptanalysis* to the science and art of breaking them; while *cryptology*, often shortened to just crypto, is the study of both. The input to an encryption process is commonly called the *plaintext* or *cleartext*, and the output the *ciphertext*. Thereafter, things get somewhat more complicated. There are a number of basic building blocks, such as *block ciphers*, *stream ciphers*, and *hash functions*. Block ciphers may either have one key for both encryption and decryption, in which case they’re called *shared-key* (also *secret-key* or *symmetric*), or have separate keys for encryption and decryption, in which case they’re called *public-key* or *asymmetric*. A *digital signature scheme* is a special type of asymmetric crypto primitive.

I will first give some historical examples to illustrate the basic concepts. I’ll then fine-tune definitions by introducing the security models that cryptologists use, including perfect secrecy, concrete security, indistinguishability and the random oracle model. Finally, I’ll show how the more important cryptographic algorithms actually work, and how they can be used to protect data. En route, I’ll give examples of how people broke weak ciphers, and weak constructions using strong ciphers.

5.2 Historical background

Suetonius tells us that Julius Caesar enciphered his dispatches by writing ‘D’ for ‘A’, ‘E’ for ‘B’ and so on [1847]. When Augustus Caesar ascended the throne, he changed the imperial cipher system so that ‘C’ was now written for ‘A’, ‘D’ for ‘B’ etcetera. In modern terminology, we would say that he changed the key from ‘D’ to ‘C’. Remarkably, a similar code was used by Bernardo Provenzano, allegedly the *capo di tutti capi* of the Sicilian mafia, who wrote ‘4’ for ‘a’, ‘5’ for ‘b’ and so on. This led directly to his capture by the Italian police in 2006 after they intercepted and deciphered some of his messages [1538].

The Arabs generalised this idea to the *monoalphabetic substitution*, in which a keyword is used to permute the cipher alphabet. We will write the plaintext in lower case letters, and the ciphertext in upper case, as shown in Figure 5.1:

```
abcdefghijklmnopqrstuvwxyz
SECURITYABDFGHJKLMNPOQVWXZ
```

Figure 5.1: Monoalphabetic substitution cipher

OYAN RWSGKFR AN AH RHTFANY MSOYRM OYSH SMSEAC NCMako; but it's a pencil and paper puzzle to break ciphers of this kind. The trick is that some letters, and combinations of letters, are much more common than others; in English the most common letters are e,t,a,i,o,n,s,h,r,d,l,u in that order. Artificial intelligence researchers have experimented with programs to solve monoalphabetic substitutions. Using letter and digram (letter pair) frequencies alone, they typically need about 600 letters of ciphertext; smarter strategies such as guessing probable words can cut this to about 150 letters; and state-of-the-art systems that use neural networks and approach the competence of human analysts are also tested on deciphering ancient scripts such as Ugaritic and Linear B [1196].

There are basically two ways to make a stronger cipher – the *stream cipher* and the *block cipher*. In the former, you make the encryption rule depend on a plaintext symbol's position in the stream of plaintext symbols, while in the latter you encrypt several plaintext symbols at once in a block.

5.2.1 An early stream cipher – the Vigenère

This early stream cipher is commonly ascribed to the Frenchman Blaise de Vigenère, a diplomat who served King Charles IX. It works by adding a key repeatedly into the plaintext using the convention that 'A' = 0, 'B' = 1, ..., 'Z' = 25, and addition is carried out modulo 26 – that is, if the result is greater than 25, we subtract as many multiples of 26 as are needed to bring it into the range [0, ..., 25], that is, [A, ..., Z]. Mathematicians write this as

$$C = P + K \bmod 26$$

So, for example, when we add P (15) to U (20) we get 35, which we reduce to 9 by subtracting 26. 9 corresponds to J, so the encryption of P under the key U (and of U under the key P) is J, or more simply $U + P = J$. In this notation, Julius Caesar's system used a fixed key $K = D$, while Augustus Caesar's used $K = C$ and Vigenère used a repeating key, also known as a *running key*. Techniques were developed to do this quickly, ranging from printed tables to brass cipher wheels. Whatever the technology, the encryption using a repeated keyword for the key would look as shown in Figure 5.2:

<i>Plain</i>	tobeornottobethatisthequestion
<i>Key</i>	runrunrunrunrunrunrunrunrunrunrun
<i>Cipher</i>	KIOVIEEIGKIOVNURNVJNUVKHVMGZIA

Figure 5.2: Vigenère (polyalphabetic substitution cipher)

A number of people appear to have worked out how to solve polyalphabetic ciphers, from the womaniser Giacomo Casanova to the computing pioneer Charles Babbage. But the first published solution was in 1863 by Friedrich Kasiski, a Prussian infantry officer [1023]. He noticed that given a long enough piece of ciphertext, repeated patterns will appear at multiples of the keyword length.

In Figure 5.2, for example, we see ‘KIOV’ repeated after nine letters, and ‘NU’ after six. Since three divides both six and nine, we might guess a keyword of three letters. Then ciphertext letters one, four, seven and so on were all enciphered under the same keyletter; so we can use frequency analysis techniques to guess the most likely values of this letter, and then repeat the process for the remaining letters of the key.

5.2.2 The one-time pad

One way to make a stream cipher of this type proof against attacks is for the key sequence to be as long as the plaintext, and to never repeat. This is known as the *one-time pad* and was proposed by Gilbert Vernam during World War I [1003]; given any ciphertext, and any plaintext of the same length, there’s a key that decrypts the ciphertext to the plaintext. So regardless of the amount of computation opponents can do, they’re none the wiser, as given any ciphertext, all possible plaintexts of that length are equally likely. This system therefore has *perfect secrecy*.

Here’s an example. Suppose you had intercepted a message from a wartime German agent which you knew started with ‘Heil Hitler’, and the first ten letters of ciphertext were DGTYI BWPJA. So the first ten letters of the one-time pad were wclnb tdefj, as shown in Figure 5.3:

<i>Plain</i>	heilhitler
<i>Key</i>	wclnb tdefj
<i>Cipher</i>	DGTYIBWPJA

Figure 5.3: A spy’s message

But once he’s burnt the piece of silk with his key material, the spy can claim that he’s actually a member of the underground resistance, and the message actually said ‘Hang Hitler’. This is also possible, as the key material could just as easily have been wggsh tdefj, as shown in Figure 5.4:

<i>Cipher</i>	DGTYIBWBJA
<i>Key</i>	wggsbtdefj
<i>Plain</i>	hanghitler

Figure 5.4: What the spy can claim he said

Now we rarely get anything for nothing in cryptography, and the price of the perfect secrecy of the one-time pad is that it fails completely to protect message integrity. So if you wanted to get this spy into trouble, you could change the ciphertext to DCYTI BWPJA (Figure 5.5):

<i>Cipher</i>	DCYTIBWBJA
<i>Key</i>	wclnbtdefj
<i>Plain</i>	hanghitler

Figure 5.5: Manipulating the message to entrap the spy

Leo Marks' engaging book on cryptography in the Special Operations Executive in World War II [1226] relates how one-time key material was printed on silk, which agents could conceal inside their clothing; whenever a key had been used it was torn off and burnt. In fact, during the war, Claude Shannon proved that a cipher has perfect secrecy if and only if there are as many possible keys as possible plaintexts, and every key is equally likely; so the one-time pad is the only kind of system that offers perfect secrecy. He was finally allowed to publish this in 1948 [1717, 1718].

The one-time tape was used for top-level communications by both sides from late in World War II, then for strategic communications between NATO allies, and for the US-USSR hotline from 1963. Thousands of machines were produced in total, using paper tapes for key material, until they were eventually replaced by computers from the mid-1980s¹. But such cryptography is too expensive for most applications as it consumes as much key material as there is traffic. It's more common for stream ciphers to use a pseudorandom number generator to expand a short key into a long keystream. The data is then encrypted by combining the keystream, one symbol at a time, with the data. It's not enough for the keystream to appear "random" in the sense of passing the standard statistical randomness tests: it must also have the property that an opponent who gets his hands on even quite a lot of keystream symbols should not be able to predict any more of them.

¹Information about the machines can be seen at the Crypto Museum, <https://www.cryptomuseum.com>.

An early example was *rotor machines*, mechanical stream-cipher devices that produce a very long sequence of pseudorandom states² and combine them with plaintext to get ciphertext. These machines were independently invented by a number of people from the 1920s, many of whom tried to sell them to the banking industry. Banks weren't in general interested, for reasons we'll discuss below, but rotor machines were very widely used by the combatants in World War II to encipher radio traffic, and the efforts made by the Allies to decipher German traffic included the work by Alan Turing and others on Colossus, which helped kickstart the computer industry after the war.

Stream ciphers have been widely used in hardware applications where the number of gates had to be minimised to save power. However, block ciphers are more flexible and are more common in systems being designed now, so let's look at them next.

5.2.3 An early block cipher – Playfair

The Playfair cipher was invented in 1854 by Sir Charles Wheatstone, a telegraph pioneer who also invented the concertina and the Wheatstone bridge. The reason it's not called the Wheatstone cipher is that he demonstrated it to Baron Playfair, a politician; Playfair in turn demonstrated it to Prince Albert and to Viscount Palmerston (later Prime Minister), on a napkin after dinner.

This cipher uses a 5 by 5 grid, in which we place the alphabet, permuted by the key word, and omitting the letter 'J' (see Figure 5.6):

P	A	L	M	E
R	S	T	O	N
B	C	D	F	G
H	I	K	Q	U
V	W	X	Y	Z

Figure 5.6: The Playfair enciphering table

The plaintext is first conditioned by replacing 'J' with 'I' wherever it occurs, then dividing it into letter pairs, preventing double letters occurring in a pair by separating them with an 'x', and finally adding a 'z' if necessary to complete the last letter pair. The example Playfair wrote on his napkin was 'Lord Granville's letter' which becomes 'lo rd gr an vi lx le sl et te rz'.

²letters in the case of the Hagelin machine used by the USA, permutations in the case of the German Enigma and the British Typex

<i>Plain</i>	l o r d g r a n v i l x l e s l e t t e r z
<i>Cipher</i>	M T T B B N E S W H T L M P T A L N N L N V

Figure 5.7: Example of Playfair enciphering

It is then enciphered two letters at a time using the following rules:

- if the two letters are in the same row or column, they are replaced by the succeeding letters. For example, ‘am’ enciphers to ‘LE’;
- otherwise the two letters stand at two of the corners of a rectangle in the table, and we replace them with the letters at the other two corners of this rectangle. For example, ‘lo’ enciphers to ‘MT’.

We can now encipher our specimen text as follows:

Variants of this cipher were used by the British army as a field cipher in World War I, and by the Americans and Germans in World War II. It’s a substantial improvement on Vigenère as the statistics that an analyst can collect are of *digraphs* (letter pairs) rather than single letters, so the distribution is much flatter and more ciphertext is needed for an attack.

Again, it’s not enough for the output of a block cipher to just look intuitively “random”. Playfair ciphertexts look random; but they have the property that if you change a single letter of a plaintext pair, then often only a single letter of the ciphertext will change. Thus using the key in Figure 5.7, *rd* enciphers to *TB* while *rf* enciphers to *OB* and *rg* enciphers to *NB*. One consequence is that given enough ciphertext, or a few probable words, the table (or an equivalent one) can be reconstructed [740]. In fact, the quote at the head of this chapter is a Playfair-encrypted message sent by the future President Jack Kennedy when he was a young lieutenant holed up on a small island with ten other survivors after his motor torpedo boat had been sunk in a collision with a Japanese destroyer. Had the Japanese intercepted it, they might possibly have decrypted it, and history could be different. For a stronger cipher, we will want the effects of small changes in the cipher’s input to diffuse completely through its output. Changing one input bit should, on average, cause half of the output bits to change. We’ll tighten these ideas up in the next section.

The security of a block cipher can also be greatly improved by choosing a longer block length than two characters. For example, the *Data Encryption Standard* (DES), which is widely used in payment systems, has a block length of 64 bits and the *Advanced Encryption Standard* (AES), which has replaced it in most other applications, has a block length of twice this. I discuss the internal details of DES and AES below; for the time being, I’ll just remark that we need more than just an adequate block size.

For example, if a bank account number always appears at the same place in a transaction, then it’s likely to produce the same ciphertext every time a

transaction involving it is encrypted with the same key. This might allow an opponent to cut and paste parts of two different ciphertexts in order to produce a valid but unauthorised transaction. Suppose a crook worked for a bank's phone company, and monitored an enciphered transaction that he knew said "Pay IBM \$10,000,000". He might wire \$1,000 to his brother causing the bank computer to insert another transaction saying "Pay John Smith \$1,000", intercept this instruction, and make up a false instruction from the two ciphertexts that decrypted as "Pay John Smith \$10,000,000". So unless the cipher block is as large as the message, the ciphertext will contain more than one block and we'll need some way of binding the blocks together.

5.2.4 Hash functions

The third classical type of cipher is the *hash function*. This evolved to protect the integrity and authenticity of messages, where we don't want someone to be able to manipulate the ciphertext in such a way as to cause a predictable change in the plaintext.

After the invention of the telegraph in the mid-19th century, banks rapidly became its main users and developed systems for transferring money electronically. What's 'wired' is a payment instruction, such as:

'To Lombard Bank, London. Please pay from our account with you no. 1234567890 the sum of £1000 to John Smith of 456 Chesterton Road, who has an account with HSBC Bank Cambridge no. 301234 4567890123, and notify him that this was for "wedding present from Doreen Smith". From First Cowboy Bank of Santa Barbara, CA, USA. Charges to be paid by us.'

Since telegraph messages were relayed from one office to another by human operators, it was possible for an operator to manipulate a payment message.

In the nineteenth century, banks, telegraph companies and shipping companies developed *code books* that could not only protect transactions but also shorten them – which was important given the costs of international telegrams at the time. A code book was essentially a block cipher that mapped words or phrases to fixed-length groups of letters or numbers. So "Please pay from our account with you no." might become 'AFVCT'. Sometimes the codes were also enciphered.

The banks realised that neither stream ciphers nor code books protect message authenticity. If, for example, the codeword for '1000' is 'mauve' and for '1,000,000' is 'magenta', then the crooked telegraph clerk who can compare the coded traffic with known transactions should be able to figure this out and substitute one for the other.

The critical innovation, for the banks' purposes, was to use a code book but to make the coding one-way by adding the code groups together into a number called a *test key*. (Modern cryptographers would describe it as a *hash value* or *message authentication code*, terms I'll define more carefully later.)

Here is a simple example. Suppose the bank has a code book with a table of numbers corresponding to payment amounts as in Figure 5.8.

	0	1	2	3	4	5	6	7	8	9
x 1000	14	22	40	87	69	93	71	35	06	58
x 10,000	73	38	15	46	91	82	00	29	64	57
x 100,000	95	70	09	54	82	63	21	47	36	18
x 1,000,000	53	77	66	29	40	12	31	05	87	94

Figure 5.8: A simple test key system

Now in order to authenticate a transaction for £376,514 we might add together 53 (no millions), 54 (300,000), 29 (70,000) and 71 (6,000) ignoring the less significant digits. This gives us a test key of 207.

Most real systems were more complex than this; they usually had tables for currency codes, dates and even recipient account numbers. In the better systems, the code groups were four digits long rather than two, and in order to make it harder for an attacker to reconstruct the tables, the test keys were compressed: a key of '7549' might become '23' by adding the first and second digits, and the third and fourth digits, ignoring the carry.

This made such test key systems into *one-way functions* in that although it was possible to compute a test from a message, given knowledge of the key, it was not possible to reverse the process and recover either a message or a key from a single test – the test just did not contain enough information. Indeed, one-way functions had been around since at least the seventeenth century. The scientist Robert Hooke published in 1678 the sorted anagram 'ceiinossttuu' and revealed two years later that it was derived from 'Ut tensio sic uis' – 'the force varies as the tension', or what we now call Hooke's law for a spring. (The goal was to establish priority for the idea while giving him time to do more work on it.)

Banking test keys are not strong by the standards of modern cryptography. Given between a few dozen and a few hundred tested messages, depending on the design details, a patient analyst could reconstruct enough of the tables to forge a transaction. With a few carefully chosen messages inserted into the banking system by an accomplice, it's even easier. But the banks got away with it: test keys worked fine from the late nineteenth century through the 1980s. In several years working as a bank security consultant, and listening to elderly auditors' tales over lunch, I only ever heard of two cases of fraud that exploited it: one external attempt involving cryptanalysis, which failed because the attacker didn't understand bank procedures, and one successful but small fraud involving a crooked staff member. I'll discuss the systems that replaced test keys in the chapter on Banking and Bookkeeping.

However, test keys are our historical example of an algebraic function used for authentication. They have important modern descendants in the authentication codes used in the command and control of nuclear weapons, and also with modern block ciphers. The idea in each case is the same: if you can use a unique key to authenticate each message, simple algebra can give you ideal security. Suppose you have a message M of arbitrary length and want to compute an authentication code or tag A of 128 bits length, and the property you want is that nobody should be able to find a different message M' whose authentication code under the same key will also be A , unless they know the key, except by a lucky guess for which the probability is 2^{-128} . You can simply choose a 128-bit prime number p and compute $A = k_1M + k_2 \pmod{p}$ where the key consists of two 128-bit numbers k_1 and k_2 .

This is secure for the same reason the one-time pad is: given any other message M' you can find another key (k'_1, k'_2) that authenticates M' to A . So without knowledge of the key, the adversary who sees M and A simply has no information of any use in creating a valid forgery. As there are 256 bits of key and only 128 bits of tag, this holds even for an adversary with unlimited computing power: such an adversary can find the 2^{128} possible keys for each pair of message and tag but has no way to choose between them. I'll discuss how this *universal hash function* is used with block ciphers below, and how it's used in nuclear command and control in Part 2.

5.2.5 Asymmetric primitives

Finally, some modern cryptosystems are asymmetric, in that different keys are used for encryption and decryption. So, for example, most web sites nowadays have a certificate containing a *public key* with which people can encrypt their session using a protocol called TLS; the owner of the web page can decrypt the traffic using the corresponding *private key*. We'll go into the details later.

There are some pre-computer examples of this too; perhaps the best is the postal service. You can send me a private message by addressing it to me and dropping it into a post box. Once that's done, I'm the only person who'll be able to read it. Of course, many things can go wrong: you might get the wrong address for me (whether by error or as a result of deception); the police might get a warrant to open my mail; the letter might be stolen by a dishonest postman; a fraudster might redirect my mail without my knowledge; or a thief might steal the letter from my doormat. Similar things can go wrong with public key cryptography: false public keys can be inserted into the system, computers can be hacked, people can be coerced and so on. We'll look at these problems in more detail in later chapters.

Another asymmetric application of cryptography is the *digital signature*. The idea here is that I can sign a message using a private *signature key* and then

anybody can check this using my public *signature verification key*. Again, there are pre-computer analogues in the form of manuscript signatures and seals; and again, there is a remarkably similar litany of things that can go wrong, both with the old way of doing things and with the new.

5.3 Security models

Before delving into the detailed design of modern ciphers, I want to look more carefully at the various types of cipher and the ways in which we can reason about their security.

Security models seek to formalise the idea that a cipher is “good”. We’ve already seen the model of *perfect secrecy*: given any ciphertext, all possible plaintexts of that length are equally likely. Similarly, an authentication scheme that uses a key only once can be designed so that the best forgery attack on it is a random guess, whose probability of success can be made as low as we want by choosing a long enough tag.

The second model is *concrete security*, where we want to know how much actual work an adversary has to do. At the time of writing, it takes the most powerful adversary in existence – the community of bitcoin miners, burning about as much electricity as the state of Denmark – about ten minutes to solve a 68-bit cryptographic puzzle and mine a new block. So an 80-bit key would take them 2^{12} times as long, or about a month; a 128-bit key, the default in modern systems, is 2^{48} times harder again. So even in 1000 years the probability of finding the right key by chance is 2^{-35} or one in many billion. In general, a system is (t, ϵ) -secure if an adversary working for time t succeeds in breaking the cipher with probability at most ϵ .

The third model, which many theoreticians now call the standard model, is about *indistinguishability*. This enables us to reason about the specific properties of a cipher we care about. For example, most cipher systems don’t hide the length of a message, so we can’t define a cipher to be secure by just requiring that an adversary not be able to distinguish ciphertexts corresponding to two messages; we have to be more explicit and require that the adversary not be able to distinguish between two messages M_1 and M_2 of the same length. This is formalised by having the cryptographer and the cryptanalyst play a game in which the analyst wins by finding an efficient discriminator of something she shouldn’t be able to discriminate with more than negligible probability. If the cipher doesn’t have perfect security this can be *asymptotic*, where we typically want the effort to grow faster than any polynomial function of a security parameter n – say the length of the key in bits. A security proof typically consists of a *reduction* where we show that if there exists a randomised (i.e., probabilistic) algorithm running in time polynomial

in n that learns information it shouldn't with non-negligible probability, then this would give an efficient discriminator for an underlying cryptographic primitive that we already trust. Finally, a construction is said to have *semantic security* if there's no efficient distinguisher for the plaintext regardless of any side information the analyst may have about it; even if she knows all but one bit of it, and even if she can get a decryption of any other ciphertext, she can't learn anything more from the target ciphertext. This skips over quite a few mathematical details, which you can find in a standard text such as Katz and Lindell [1025].

The fourth model is the random oracle model, which is not as general as the standard model but which often leads to more efficient constructions. We call a cryptographic primitive *pseudorandom* if there's no efficient way of distinguishing it from a random function of that type, and in particular it passes all the statistical and other randomness tests we apply. Of course, the cryptographic primitive will actually be an algorithm, implemented as an array of gates in hardware or a program in software; but the outputs should "look random" in that they're indistinguishable from a suitable random oracle given the type and the number of tests that our model of computation permits.

To visualise a random oracle, we might imagine an elf sitting in a black box with a source of physical randomness and some means of storage (see Figure 5.9) – represented in our picture by the dice and the scroll. The elf will accept inputs of a certain type, then look in the scroll to see whether this query has ever been answered before. If so, it will give the answer it finds there; if not, it will generate an answer at random by throwing the dice, and keep a record for future reference. We'll further assume finite bandwidth – the elf

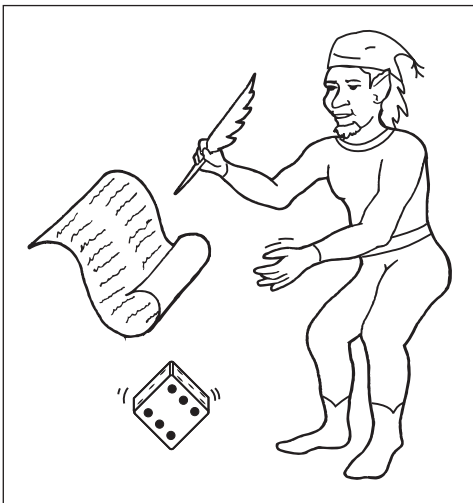


Figure 5.9: The random oracle

will only answer so many queries every second. What's more, our oracle can operate according to several different rules.

5.3.1 Random functions – hash functions

The first type of random oracle is the random function. A random function accepts an input string of any length and outputs a string of fixed length, say n bits long. The same input gives the same output, but the set of outputs appears random. So the elf just has a simple list of inputs and outputs, which grows steadily as it works.

Random functions are our model for *cryptographic hash functions*. These were first used in computer systems for one-way encryption of passwords in the 1960s and have many more uses today. For example, if the police seize your laptop, the standard forensic tools will compute checksums on all the files, to identify which files are already known (such as system files) and which are novel (such as user data). These hash values will change if a file is corrupted and so can assure the court that the police haven't tampered with evidence. And if we want evidence that we possessed a given electronic document by a certain date, we might submit it to an online time-stamping service or have it mined into the Bitcoin blockchain. However, if the document is still secret – for example an invention for which we want to establish a priority date – then we would not upload the whole document, but just the message hash. This is the modern equivalent of Hooke's anagram that we discussed in section 5.2.4 above.

5.3.1.1 Properties

The first main property of a random function is one-wayness. Given knowledge of an input x we can easily compute the hash value $h(x)$, but it is very difficult given $h(x)$ to find x if such an input is not already known. (The elf will only pick outputs for given inputs, not the other way round.) As the output is random, the best an attacker can do to invert a random function is to keep on feeding in more inputs until he gets lucky; with an n -bit output this will take about 2^{n-1} guesses on average. A pseudorandom function will have the same properties, or they could be used to distinguish it from a random function, contrary to our definition. So a pseudorandom function will also be a *one-way function*, provided there are too many possible outputs for the opponent to guess an input that has a desired target output by chance. This means choosing n so that the opponent can't do anything near 2^n computations. If we claim, for example, that SHA256 is a pseudorandom function, then we're saying that there's no practical way to find an input that hashes to a given 256-bit value, unless you knew it already and used it to compute that value.

A second property of pseudorandom functions is that the output will not give any information at all about even part of the input. So we can get a one-way encryption of the value x by concatenating it with a secret key k and computing $h(x, k)$. If the hash function isn't random enough, though, using it for one-way encryption in this manner is asking for trouble. (I'll discuss an example later in section 22.3.1: the hash function used by many phone companies in the 1990s and early 2000s to authenticate mobile phone users wasn't random enough, which led to attacks.)

A third property of pseudorandom functions with sufficiently long outputs is that it is hard to find *collisions*, that is, different messages $M_1 \neq M_2$ with $h(M_1) = h(M_2)$. Unless the opponent can find a shortcut attack (which would mean the function wasn't pseudorandom) then the best way of finding a collision is to collect a large set of messages M_i and their corresponding hashes $h(M_i)$, sort the hashes, and look for a match. If the hash function output is an n -bit number, so that there are 2^n possible hash values, then the number of hashes the enemy will need to compute before he can expect to find a match will be about the square root of this, namely $2^{n/2}$ hashes. This fact is of huge importance in security engineering, so let's look at it more closely.

5.3.1.2 The birthday theorem

The birthday theorem gets its name from the following problem. A maths teacher asks a class of 30 pupils what they think is the probability that two of them have the same birthday. Most pupils intuitively think it's unlikely, and the maths teacher then asks the pupils to state their birthdays one after another. The odds of a match exceed 50% once 23 pupils have been called. As this surprises most people, it's also known as the 'birthday paradox'.

The birthday theorem was first used in the 1930's to count fish, so it's also known as *capture-recapture statistics* [1668]. Suppose there are N fish in a lake and you catch m of them, ring them and throw them back, then when you first catch a fish you've ringed already, m should be 'about' the square root of N . The intuitive reason why this holds is that once you have $\sqrt{N}$ samples, each could potentially match any of the others, so the number of possible matches is about $\sqrt{N} \times \sqrt{N}$ or N , which is what you need³.

This theorem has many applications for the security engineer. For example, if we have a biometric system that can authenticate a person's claim to identity with a probability of only one in a million that two randomly selected subjects will be falsely identified as the same person, this doesn't mean that we can use it as a reliable means of identification in a university with a user population of

³More precisely, the probability that m fish chosen randomly from N fish are different is $p = N(N-1) \dots (N-m+1)/N^m$ which is asymptotically solved by $N \simeq m^2/2\log(1/p)$ [1039].

twenty thousand staff and students. This is because there will be almost two hundred million possible pairs. In fact, you expect to find the first *collision* – the first pair of people who can be mistaken for each other by the system – once you have somewhat over a thousand people enrolled. It may well, however, be OK to use it to verify a claimed identity (though many other things can go wrong; see the chapter on Biometrics in Part 2 for a discussion).

There are some applications where collision-search attacks aren't a problem, such as in challenge-response protocols where an attacker has to find the answer to the challenge just issued, and where you can prevent challenges repeating. In identify-friend-or-foe (IFF) systems, for example, common equipment has a response length of 48 to 80 bits. You can't afford much more than that, as it costs radar accuracy.

But there are other applications in which collisions are unacceptable. When we design digital signature systems, we typically pass the message M through a cryptographic hash function first, and then sign the hash $h(M)$, for a number of reasons we'll discuss later. In such an application, if it were possible to find collisions with $h(M_1) = h(M_2)$ but $M_1 \neq M_2$, then a Mafia owned bookstore's web site might precalculate suitable pairs M_1, M_2 , get you to sign an M_1 saying something like "I hereby order a copy of Rubber Fetish volume 7 for \$32.95" and then present the signature together with an M_2 saying something like "I hereby mortgage my house for \$75,000 and please send the funds to Mafia Holdings Inc., Bermuda."

For this reason, hash functions used with digital signature schemes have n large enough to make them collision-free. Historically, the two most common hash functions have been MD5, which has a 128-bit output and will thus require at most 2^{64} computations to break, and SHA1 with a 160-bit output and a work factor for the cryptanalyst of at most 2^{80} . However, collision search gives at best an upper bound on the strength of a hash function, and both these particular functions have turned out to be disappointing, with cryptanalytic attacks that I'll describe later in section 5.6.2.

To sum up: if you need a cryptographic hash function to be collision resistant, then you'd better choose a function with an output of at least 256 bits, such as SHA-2 or SHA-3. However if you only need to be sure that nobody will find a second preimage for an existing, externally given hash, then you can perhaps make do with less.

5.3.2 Random generators – stream ciphers

The second basic cryptographic primitive is the *random generator*, also known as a *keystream generator* or *stream cipher*. This is also a random function, but it's the reverse of the hash function in that it has a short input and a long output. If we had a good pseudorandom function whose input and output were long

enough, we could turn it into a hash function by throwing away all but a few hundred bits of the output, and turn it into a stream cipher by padding all but a few hundred bits of the input with a constant and using the output as a keystream.

It can be used to protect the confidentiality of our backup data as follows: we go to the keystream generator, enter a key, get a long file of random bits, and exclusive-or it with our plaintext data to get ciphertext, which we then send to our backup service in the cloud. (This is also called an *additive stream cipher* as exclusive-or is addition modulo 2.) We can think of the elf generating a random tape of the required length each time he is presented with a new key, giving it to us and keeping a copy on his scroll for reference in case he's given the same input key again. If we need to recover the data, we go back to the generator, enter the same key, get the same keystream, and exclusive-or it with our ciphertext to get our plaintext back again. Other people with access to the keystream generator won't be able to generate the same keystream unless they know the key. Note that this would not give us any guarantee of file integrity; as we saw in the discussion of the one-time pad, adding a keystream to plaintext can protect confidentiality, but it can't detect modification of the file. For that, we might make a hash of the file and keep that somewhere safe. It may be easier to protect the hash from modification than the whole file.

One-time pad systems are a close fit for our theoretical model, except in that they are used to secure communications across space rather than time: the two communicating parties have shared a copy of a keystream in advance. Vernam's original telegraph cipher machine used punched paper tape; Marks describes how SOE agents' silken keys were manufactured in Oxford by retired ladies shuffling counters; we'll discuss modern hardware random number generators in the chapter on Physical Security.

A real problem with keystream generators is to prevent the same keystream being used more than once, whether to encrypt more than one backup tape or to encrypt more than one message sent on a communications channel. During World War II, the amount of Russian diplomatic traffic exceeded the quantity of one-time tape they had distributed in advance to their embassies, so it was reused. But if $M_1 + K = C_1$ and $M_2 + K = C_2$, then the opponent can combine the two ciphertexts to get a combination of two messages: $C_1 - C_2 = M_1 - M_2$, and if the messages M_i have enough redundancy then they can be recovered. Text messages do in fact contain enough redundancy for much to be recovered; in the case of the Russian traffic this led to the Venona project in which the US and UK decrypted large amounts of wartime Russian traffic from 1943 onwards and broke up a number of Russian spy rings. In the words of one former NSA chief scientist, it became a "two-time tape".

To avoid this, the normal engineering practice is to have not just a key but also a *seed* (also known as an *initialisation vector* or IV) so we start the keystream at a different place each time. The seed N may be a sequence number, or generated

from a protocol in a more complex way. Here, you need to ensure that both parties synchronise on the right working key even in the presence of an adversary who may try to get you to reuse old keystream.

5.3.3 Random permutations – block ciphers

The third type of primitive, and the most important in modern cryptography, is the block cipher, which we model as a *random permutation*. Here, the function is invertible, and the input plaintext and the output ciphertext are of a fixed size. With Playfair, both input and output are two characters; with DES, they're both bit strings of 64 bits. Whatever the number of symbols and the underlying alphabet, encryption acts on a block of fixed length. (So if you want to encrypt a shorter input, you have to pad it as with the final 'z' in our Playfair example.)

We can visualise block encryption as follows. As before, we have an elf in a box with dice and a scroll. This has on the left a column of plaintexts and on the right a column of ciphertexts. When we ask the elf to encrypt a message, it checks in the left-hand column to see if it has a record of it. If not, it rolls the dice to generate a random ciphertext of the appropriate size (and which doesn't appear yet in the right-hand column of the scroll), and then writes down the plaintext/ciphertext pair in the scroll. If it does find a record, it gives us the corresponding ciphertext from the right-hand column.

When asked to decrypt, the elf does the same, but with the function of the columns reversed: he takes the input ciphertext, looks for it on the right-hand scroll, and if he finds it he gives the message with which it was previously associated. If not, he generates a new message at random, notes it down and gives it to us.

A *block cipher* is a keyed family of pseudorandom permutations. For each key, we have a single permutation that's independent of all the others. We can think of each key as corresponding to a different scroll. The intuitive idea is that a cipher machine should output the ciphertext given the plaintext and the key, and output the plaintext given the ciphertext and the key, but given only the plaintext and the ciphertext it should output nothing. Furthermore, nobody should be able to infer any information about plaintexts or ciphertexts that it has not yet produced.

We will write a block cipher using the notation established for encryption in the chapter on protocols:

$$C = \{M\}_K$$

The random permutation model also allows us to define different types of attack on block ciphers. In a *known plaintext attack*, the opponent is just given a number of randomly chosen inputs and outputs from the oracle corresponding to a target key. In a *chosen plaintext attack*, the opponent is allowed to put a

certain number of plaintext queries and get the corresponding ciphertexts. In a *chosen ciphertext attack* he gets to make a number of ciphertext queries. In a *chosen plaintext/ciphertext attack* he is allowed to make queries of either type. Finally, in a *related key attack* he can make queries that will be answered using keys related to the target key K , such as $K + 1$ and $K + 2$.

In each case, the objective of the attacker may be either to deduce the answer to a query he hasn't already made (a *forgery attack*), or to recover the key (unsurprisingly known as a *key recovery attack*).

This precision about attacks is important. When someone discovers a vulnerability in a cryptographic primitive, it may or may not be relevant to your application. Often it won't be, but will have been hyped by the media – so you will need to be able to explain clearly to your boss and your customers why it's not a problem. So you have to look carefully to find out exactly what kind of attack has been found, and what the parameters are. For example, the first major attack announced on the Data Encryption Standard algorithm (differential cryptanalysis) required 2^{47} chosen plaintexts to recover the key, while the next major attack (linear cryptanalysis) improved this to 2^{43} known plaintexts. While these attacks were of huge scientific importance, their practical engineering effect was zero, as no practical systems make that much known text (let alone chosen text) available to an attacker. Such impractical attacks are often referred to as *certificational* as they affect the cipher's security certification rather than providing a practical exploit. They can have a commercial effect, though: the attacks on DES undermined confidence and started moving people to other ciphers. In some other cases, an attack that started off as *certificational* has been developed by later ideas into an exploit.

Which sort of attacks you should be worried about depends on your application. With a broadcast entertainment system, for example, a hacker can buy a decoder, watch a lot of movies and compare them with the enciphered broadcast signal; so a *known-plaintext attack* might be the main threat. But there are surprisingly many applications where *chosen-plaintext attacks* are possible. A historic example is from World War II, where US analysts learned of Japanese intentions for an island 'AF' which they suspected meant Midway. So they arranged for Midway's commander to send an unencrypted message reporting problems with its fresh water condenser, and then intercepted a Japanese report that 'AF is short of water'. Knowing that Midway was the Japanese objective, Admiral Chester Nimitz was waiting for them and sank four Japanese carriers, turning the tide of the war [1003].

The other attacks are more specialised. *Chosen plaintext/ciphertext attacks* may be a worry where the threat is a *lunchtime attack*: someone who gets temporary access to a cryptographic device while its authorised user is out, and tries out the full range of permitted operations for a while with data of their choice. *Related-key attacks* are a concern where the block cipher is used as a building block in the construction of a hash function (which we'll discuss below).

To exclude all such attacks, the goal is semantic security, as discussed above; the cipher should not allow the inference of unauthorised information (whether of plaintexts, ciphertexts or keys) other than with negligible probability.

5.3.4 Public key encryption and trapdoor one-way permutations

A *public-key encryption* algorithm is a special kind of block cipher in which the elf will perform the encryption corresponding to a particular key for anyone who requests it, but will do the decryption operation only for the key's owner. To continue with our analogy, the user might give a secret name to the scroll that only she and the elf know, use the elf's public one-way function to compute a hash of this secret name, publish the hash, and instruct the elf to perform the encryption operation for anybody who quotes this hash. This means that a principal, say Alice, can publish a key and if Bob wants to, he can now encrypt a message and send it to her, even if they have never met. All that is necessary is that they have access to the oracle.

The simplest variation is the *trapdoor one-way permutation*. This is a computation that anyone can perform, but which can be reversed only by someone who knows a *trapdoor* such as a secret key. This model is like the 'one-way function' model of a cryptographic hash function. Let us state it formally nonetheless: a public key encryption primitive consists of a function which given a random input R will return two keys, KR (the public encryption key) and KR^{-1} (the private decryption key) with the properties that

1. Given KR , it is infeasible to compute KR^{-1} (so it's not possible to compute R either);
2. There is an encryption function $\{ \dots \}$ which, applied to a message M using the encryption key KR , will produce a ciphertext $C = \{M\}_{KR}$; and
3. There is a decryption function which, applied to a ciphertext C using the decryption key KR^{-1} , will produce the original message $M = \{C\}_{KR^{-1}}$.

For practical purposes, we will want the oracle to be replicated at both ends of the communications channel, and this means either using tamper-resistant hardware or (more commonly) implementing its functions using mathematics rather than metal.

In most real systems, the encryption is randomised, so that every time someone uses the same public key to encrypt the same message, the answer is different; this is necessary for semantic security, so that an opponent cannot check whether a guess of the plaintext of a given ciphertext is correct. There are even more demanding models than this, for example to analyse security in the case where the opponent can get ciphertexts of their choice decrypted, with the exception of the target ciphertext. But this will do for now.

5.3.5 Digital signatures

The final cryptographic primitive we'll define here is the *digital signature*. The basic idea is that a signature on a message can be created by only one principal, but checked by anyone. It can thus perform the same function in the electronic world that ordinary signatures do in the world of paper. Applications include signing software updates, so that a PC can tell that an update to Windows was really produced by Microsoft rather than by a foreign intelligence agency.

Signature schemes, too, can be deterministic or randomised: in the first, computing a signature on a message will always give the same result and in the second, it will give a different result. (The latter is more like handwritten signatures; no two are ever alike but the bank has a means of deciding whether a given specimen is genuine or forged.) Also, signature schemes may or may not support *message recovery*. If they do, then given the signature, anyone can recover the message on which it was generated; if they don't, then the verifier needs to know or guess the message before they can perform the verification.

Formally, a signature scheme, like a public key encryption scheme, has a key-pair generation function which given a random input R will return two keys, σR (the private signing key) and VR (the public signature verification key) with the properties that

1. Given the public signature verification key VR , it is infeasible to compute the private signing key σR ;
2. There is a digital signature function which given a message M and a private signature key σR , will produce a signature $Sig_{\sigma R}\{M\}$; and
3. There is a verification function which, given a signature $Sig_{\sigma R}\{M\}$ and the public signature verification key VR , will output TRUE if the signature was computed correctly with σR and otherwise output FALSE.

Where we don't need message recovery, we can model a simple digital signature algorithm as a random function that reduces any input message to a one-way hash value of fixed length, followed by a special kind of block cipher in which the elf will perform the operation in one direction, known as *signature*, for only one principal. In the other direction, it will perform verification for anybody.

For this simple scheme, signature verification means that the elf (or the signature verification algorithm) only outputs TRUE or FALSE depending on whether the signature is good. But in a scheme with *message recovery*, anyone can input a signature and get back the message corresponding to it. In our elf model, this means that if the elf has seen the signature before, it will give the message corresponding to it on the scroll, otherwise it will give a random value (and record the input and the random output as a signature and message pair). This is sometimes desirable: when sending short messages over a low bandwidth channel, it can save space if only the signature has to

be sent rather than the signature plus the message. An application that uses message recovery is machine-printed postage stamps, or *indicia*: the stamp consists of a 2-d barcode with a digital signature made by the postal meter and which contains information such as the value, the date and the sender's and recipient's post codes. We discuss this at the end of section 16.3.2.

In the general case we do not need message recovery; the message to be signed may be of arbitrary length, so we first pass it through a hash function and then sign the hash value. We need the hash function to be not just one-way, but also collision resistant.

5.4 Symmetric crypto algorithms

Now that we've tidied up the definitions, we'll look under the hood to see how they can be implemented in practice. While most explanations are geared towards graduate mathematics students, the presentation I'll give here is based on one I developed over the years with computer science undergraduates, to help the non-specialist grasp the essentials. In fact, even at the research level, most of cryptography is as much computer science as mathematics: modern attacks on ciphers are put together from guessing bits, searching for patterns, sorting possible results and so on, and require ingenuity and persistence rather than anything particularly highbrow.

5.4.1 SP-networks

Claude Shannon suggested in the 1940s that strong ciphers could be built by combining substitution with transposition repeatedly. For example, one might add some key material to a block of input text, and then shuffle subsets of the input, and continue in this way a number of times. He described the properties of a cipher as being *confusion* and *diffusion* – adding unknown key values will confuse an attacker about the value of a plaintext symbol, while diffusion means spreading the plaintext information through the ciphertext. Block ciphers need diffusion as well as confusion.

The earliest block ciphers were simple networks which combined substitution and permutation circuits, and so were called SP-networks [1011]. Figure 5.10 shows an SP-network with sixteen inputs, which we can imagine as the bits of a sixteen-bit number, and two layers of four-bit invertible substitution boxes (or *S-boxes*), each of which can be visualised as a lookup table containing some permutation of the numbers 0 to 15.

The point of this arrangement is that if we were to implement an arbitrary 16 bit to 16 bit function in digital logic, we would need 2^{20} bits of memory – one lookup table of 2^{16} bits for each single output bit. That's hundreds of thousands

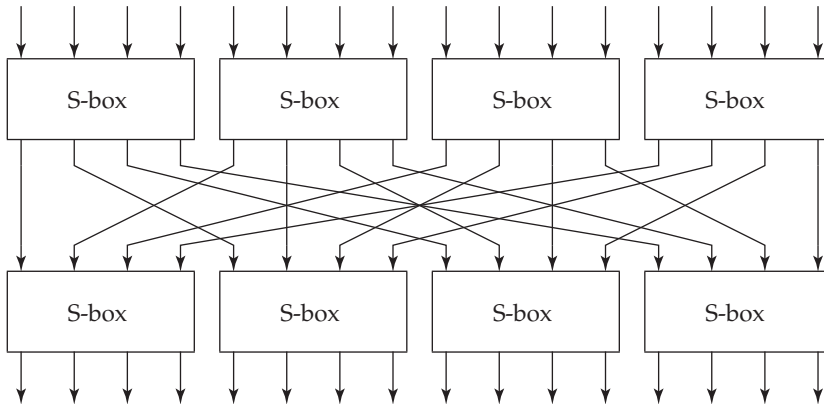


Figure 5.10: A simple 16-bit SP-network block cipher

of gates, while a four bit to four bit function takes only 4×2^4 or 64 bits of memory. One might hope that with suitable choices of parameters, the function produced by iterating this simple structure would be indistinguishable from a random 16 bit to 16 bit function to an opponent who didn't know the value of the key. The key might consist of some choice of a number of four-bit S-boxes, or it might be added at each round to provide confusion and the resulting text fed through the S-boxes to provide diffusion.

Three things need to be done to make such a design secure:

1. the cipher needs to be “wide” enough
2. it needs to have enough rounds, and
3. the S-boxes need to be suitably chosen.

5.4.1.1 Block size

First, a block cipher which operated on sixteen bit blocks would be rather limited, as an opponent could just build a dictionary of plaintext and ciphertext blocks as they were observed. The birthday theorem tells us that even if the input plaintexts were random, he'd expect to find a match as soon as he had seen a few hundred blocks. So a practical block cipher will usually deal with plaintexts and ciphertexts of 64 bits, 128 bits or even more. So if we are using four-bit to four-bit S-boxes, we may have 16 of them (for a 64 bit block size) or 32 of them (for a 128 bit block size).

5.4.1.2 Number of rounds

Second, we have to have enough rounds. The two rounds in Figure 5.10 are completely inadequate, as an opponent can deduce the values of the S-boxes

by tweaking input bits in suitable patterns. For example, he could hold the rightmost 12 bits constant and try tweaking the leftmost four bits, to deduce the values in the top left S-box. (The attack is slightly more complicated than this, as sometimes a tweak in an input bit to an S-box won't produce a change in any output bit, so we have to change one of its other inputs and tweak again. But it is still a basic student exercise.)

The number of rounds we need depends on the speed with which data diffuse through the cipher. In our simple example, diffusion is very slow because each output bit from one round of S-boxes is connected to only one input bit in the next round. Instead of having a simple permutation of the wires, it is more efficient to have a linear transformation in which each input bit in one round is the exclusive-or of several output bits in the previous round. If the block cipher is to be used for decryption as well as encryption, this linear transformation will have to be invertible. We'll see some concrete examples below in the sections on AES and DES.

5.4.1.3 Choice of S-boxes

The design of the S-boxes also affects the number of rounds required for security, and studying bad choices gives us our entry into the deeper theory of block ciphers. Suppose that the S-box were the permutation that maps the inputs $(0,1,2,\dots,15)$ to the outputs $(5,7,0,2,4,3,1,6,8,10,15,12,9,11,14,13)$. Then the most significant bit of the input would come through unchanged as the most significant bit of the output. If the same S-box were used in both rounds in the above cipher, then the most significant bit of the input would pass through to become the most significant bit of the output. We certainly couldn't claim that our cipher was pseudorandom.

5.4.1.4 Linear cryptanalysis

Attacks on real block ciphers are usually harder to spot than in this example, but they use the same ideas. It might turn out that the S-box had the property that bit one of the input was equal to bit two plus bit four of the output; more commonly, there will be linear approximations to an S-box which hold with a certain probability. *Linear cryptanalysis* [897, 1246] proceeds by collecting a number of relations such as "bit 2 plus bit 5 of the input to the first S-box is equal to bit 1 plus bit 8 of the output, with probability $13/16$ ", then searching for ways to glue them together into an algebraic relation between input bits, output bits and key bits that holds with a probability different from one half. If we can find a linear relationship that holds over the whole cipher with probability $p = 0.5 + 1/M$, then according to the sampling theorem in probability theory we can expect to start recovering keybits once we have about M^2 known texts.

If the value of M^2 for the best linear relationship is greater than the total possible number of known texts (namely 2^n where the inputs and outputs are n bits wide), then we consider the cipher to be secure against linear cryptanalysis.

5.4.1.5 Differential cryptanalysis

Differential Cryptanalysis [246, 897] is similar but is based on the probability that a given change in the input to an S-box will give rise to a certain change in the output. A typical observation on an 8-bit S-box might be that “if we flip input bits 2, 3, and 7 at once, then with probability 11/16 the only output bits that will flip are 0 and 1”. In fact, with any nonlinear Boolean function, tweaking some combination of input bits will cause some combination of output bits to change with a probability different from one half. The analysis procedure is to look at all possible input difference patterns and look for those values δ_i , δ_o such that an input change of δ_i will produce an output change of δ_o with particularly high (or low) probability.

As in linear cryptanalysis, we then search for ways to join things up so that an input difference which we can feed into the cipher will produce a known output difference with a useful probability over a number of rounds. Given enough chosen inputs, we will see the expected output and be able to make deductions about the key. As in linear cryptanalysis, it's common to consider the cipher to be secure if the number of texts required for an attack is greater than the total possible number of different texts for that key. (We have to be careful of pathological cases, such as if you had a cipher with a 32-bit block and a 128-bit key with a differential attack whose success probability given a single pair was 2^{-40} . Given a lot of text under a number of keys, we'd eventually solve for the current key.)

There are many variations on these two themes. For example, instead of looking for high probability differences, we can look for differences that can't happen (or that happen only rarely). This has the charming name of *impossible cryptanalysis*, but it is quite definitely possible against many systems [243]⁴.

Block cipher design involves a number of trade-offs. For example, we can reduce the per-round information leakage, and thus the required number of rounds, by designing the rounds carefully. But a complex design might be slow in software, or need a lot of gates in hardware, so using simple rounds but more of them might have been better. Simple rounds may also be easier to analyse. A prudent designer will also use more rounds than are strictly necessary to block the attacks known today, in order to give some safety margin, as attacks only ever get better. But while we may be able to show that a cipher resists all the attacks we know of, and with some safety margin, this says little about

⁴This may have been used first at Bletchley in World War II where a key insight into breaking the German Enigma machine was that no letter ever enciphered to itself.

whether it will resist novel types of attack. (A general security proof for a block cipher would appear to imply a result such as $P \neq NP$ that would revolutionise computer science.)

5.4.2 The Advanced Encryption Standard (AES)

The Advanced Encryption Standard (AES) is an algorithm originally known as Rijndael after its inventors Vincent Rijmen and Joan Daemen [507]. It acts on 128-bit blocks and can use a key of 128, 192 or 256 bits in length. It is an SP-network; in order to specify it, we need to fix the S-boxes, the linear transformation between the rounds, and the way in which the key is added into the computation.

AES uses a single S-box that acts on a byte input to give a byte output. For implementation purposes it can be regarded simply as a lookup table of 256 bytes; it is actually defined by the equation $S(x) = M(1/x) + b$ over the field $GF(2^8)$ where M is a suitably chosen matrix and b is a constant. This construction gives tight differential and linear bounds.

The linear transformation is based on arranging the 16 bytes of the value being enciphered in a square and then doing bitwise shuffling and mixing operations. The first step is the *shuffle*, in which the top row of four bytes is left unchanged while the second row is shifted one place to the left, the third row by two places and the fourth row by three places. The second step is a column-mixing step in which the four bytes in a column are mixed using matrix multiplication. This is illustrated in Figure 5.11, which shows, as an example, how a change in the value of the third byte in the first column is propagated. The effect of this combination is that a change in the input to the cipher can potentially affect all of the output after just two rounds – an *avalanche* effect that makes both linear and differential attacks harder.

The key material is added byte by byte after the linear transformation. This means that 16 bytes of key material are needed per round; they are derived from the user supplied key material by means of a recurrence relation.

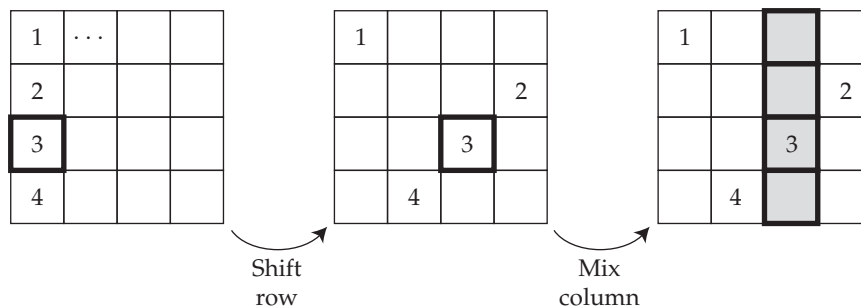


Figure 5.11: The AES linear transformation, illustrated by its effect on byte 3 of the input

The algorithm uses 10 rounds with 128-bit keys, 12 rounds with 192-bit keys and 14 rounds with 256-bit keys. These are enough to give practical, but not certificational, security – as indeed we expected at the time of the AES competition, and as I described in earlier editions of this chapter. The first key-recovery attacks use a technique called biclique cryptanalysis and were discovered in 2009 by Andrey Bogdanov, Dmitry Khovratovich, and Christian Rechberger [274]; they give only a very small advantage, with complexity now estimated at 2^{126} for 128-bit AES and $2^{254.3}$ for 256-bit AES, as opposed to 2^{127} and 2^{255} for brute-force search. Faster shortcut attacks are known for the case where we have related keys. But none of these attacks make any difference in practice, as they require infeasibly large numbers of texts or very special combinations of related keys.

Should we trust AES? The governments of Russia, China and Japan try to get firms to use local ciphers instead, and the Japanese offering, Camellia, is found in a number of crypto libraries alongside AES and another AES competition finalist, Bruce Schneier's Twofish. (Camellia was designed by a team whose own AES candidate was knocked out at the first round.) Conspiracy theorists note that the US government picked the weakest of the five algorithms that were finalists in the AES competition. Well, I was one of the designers of the AES finalist Serpent [95], which came second in the competition: the winner Rijndael got 86 votes, Serpent 59 votes, Twofish 31 votes, RC6 23 votes and MARS 13 votes. Serpent has a simple structure that makes it easy to analyse – the structure of Figure 5.10, but modified to be wide enough and to have enough rounds – and was designed to have a much larger security margin than Rijndael in anticipation of the attacks that have now appeared. Yet the simple fact is that while Serpent is more secure, Rijndael is faster; industry and crypto researchers voted for it at the last AES conference, and NIST approved it as the standard.

Having been involved in the whole process, and having worked on the analysis and design of shared-key ciphers for much of the 1990s, I have a high level of confidence that AES is secure against practical attacks based on mathematical cryptanalysis. And even though AES is less secure than Serpent, practical security is all about implementation, and we now have enormous experience at implementing AES. Practical attacks include timing analysis and power analysis. In the former, the main risk is that an opponent observes cache misses and uses them to work out the key. In the latter, an opponent uses measurements of the current drawn by the device doing the crypto – think of a bank smartcard that a customer places in a terminal in a Mafia-owned shop. I discuss both in detail in Part 2, in the chapter on Emission Security; countermeasures include special operations in many CPUs to do AES, which are available precisely because the algorithm is now a standard. It does not make sense to implement Serpent as well, 'just in case AES is broken': having swappable algorithms is

known as *pluggable cryptography*, yet the risk of a fatal error in the algorithm negotiation protocol is orders of magnitude greater than the risk that anyone will come up with a production attack on AES. (We'll see a number of examples later where using multiple algorithms caused something to break horribly.)

The back story is that, back in the 1970s, the NSA manipulated the choice and parameters of the previous standard block cipher, the *Data Encryption Standard* (DES) in such a way as to deliver a cipher that was good enough for US industry at the time, while causing foreign governments to believe it was insecure, so they used their own weak designs instead. I'll discuss this in more detail below, once I've described the design of DES. AES seems to have followed this playbook; by selecting an algorithm that was only just strong enough mathematically and whose safe implementation requires skill and care, the US government saw to it that firms in Russia, China, Japan and elsewhere will end up using systems that are less secure because less skill and effort has been invested in the implementation. However, this was probably luck rather than Machiavellian cunning: the relevant committee at NIST would have had to have a lot of courage to disregard the vote and choose another algorithm instead. Oh, and the NSA has since 2005 approved AES with 128-bit keys for protecting information up to SECRET and with 192-bit or 256-bit keys for TOP SECRET. So I recommend that you use AES instead of GOST, or Camellia, or even Serpent. The definitive specification of AES is Federal Information Processing Standard 197, and its inventors have written a book describing its design in detail [507].

5.4.3 Feistel ciphers

Many block ciphers use a more complex structure, which was invented by Feistel and his team while they were developing the Mark XII IFF in the late 1950s and early 1960s. Feistel then moved to IBM and founded a research group that produced the Data Encryption Standard (DES) algorithm, which is still a mainstay of payment system security.

A Feistel cipher has the ladder structure shown in Figure 5.12. The input is split up into two blocks, the left half and the right half. A *round function* f_1 of the left half is computed and combined with the right half using exclusive-or (binary addition without carry), though in some Feistel ciphers addition with carry is also used. (We use the notation $\oplus$ for exclusive-or.) Then, a function f_2 of the right half is computed and combined with the left half, and so on. Finally (if the number of rounds is even) the left half and right half are swapped.

A notation which you may see for the Feistel cipher is $\psi(f, g, h, \dots)$ where $f, g, h, \dots$ are the successive round functions. Under this notation, the above

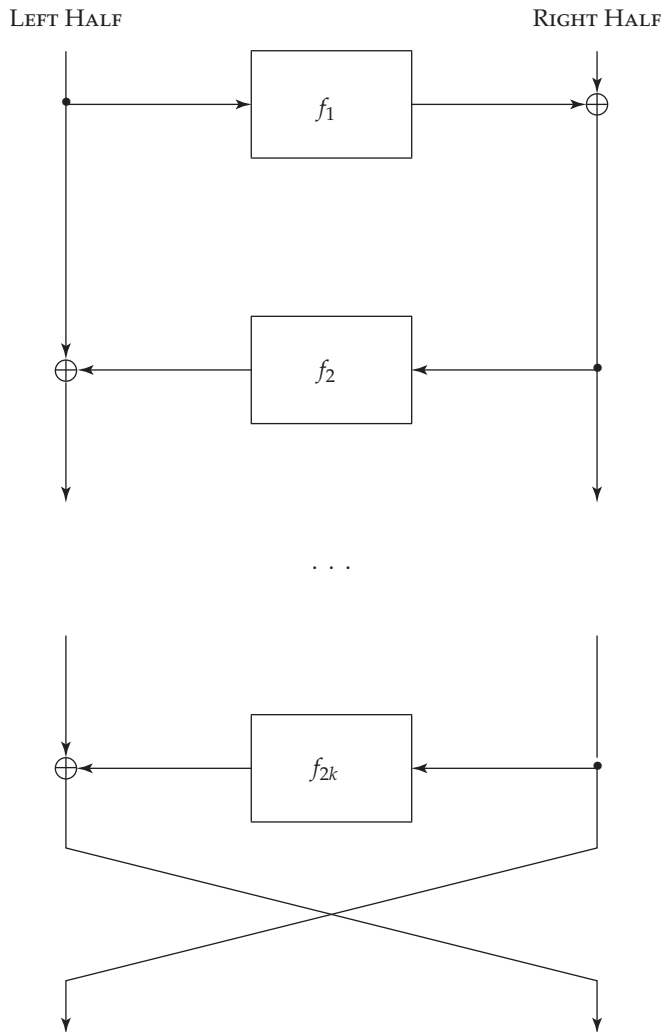


Figure 5.12: The Feistel cipher structure

cipher is $\psi(f_1, f_2, \dots, f_{2k-1}, f_{2k})$. The basic result that enables us to decrypt a Feistel cipher – and indeed the whole point of his design – is that:

$$\psi^{-1}(f_1, f_2, \dots, f_{2k-1}, f_{2k}) = \psi(f_{2k}, f_{2k-1}, \dots, f_2, f_1)$$

In other words, to decrypt, we just use the round functions in the reverse order. Thus the round functions f_i do not have to be invertible, and the Feistel structure lets us turn any one-way function into a block cipher. This means that we are less constrained in trying to choose a round function with good diffusion and confusion properties, and which also satisfies any other design constraints such as code size, software speed or hardware gate count.

5.4.3.1 The Luby-Rackoff result

The key theoretical result on Feistel ciphers was proved by Mike Luby and Charlie Rackoff in 1988. They showed that if f_i were random functions, then $\psi(f_1, f_2, f_3)$ was indistinguishable from a random permutation under chosen-plaintext attack, and this result was soon extended to show that $\psi(f_1, f_2, f_3, f_4)$ was indistinguishable under chosen plaintext/ciphertext attack – in other words, it was a pseudorandom permutation. (I omit a number of technicalities.)

In engineering terms, the effect is that given a really good round function, four rounds of Feistel are enough. So if we have a hash function in which we have confidence, it is straightforward to construct a block cipher from it: use four rounds of keyed hash in a Feistel network.

5.4.3.2 DES

The DES algorithm is widely used in banking and other payment applications. The ‘killer app’ that got it widely deployed was ATM networks; from there it spread to prepayment meters, transport tickets and much else. In its classic form, it is a Feistel cipher, with a 64-bit block and 56-bit key. Its round function operates on 32-bit half blocks and consists of three operations:

- first, the block is expanded from 32 bits to 48;
- next, 48 bits of round key are mixed in using exclusive-or;
- the result is passed through a row of eight S-boxes, each of which takes a six-bit input and provides a four-bit output;
- finally, the bits of the output are permuted according to a fixed pattern.

The effect of the expansion, key mixing and S-boxes is shown in Figure 5.13:

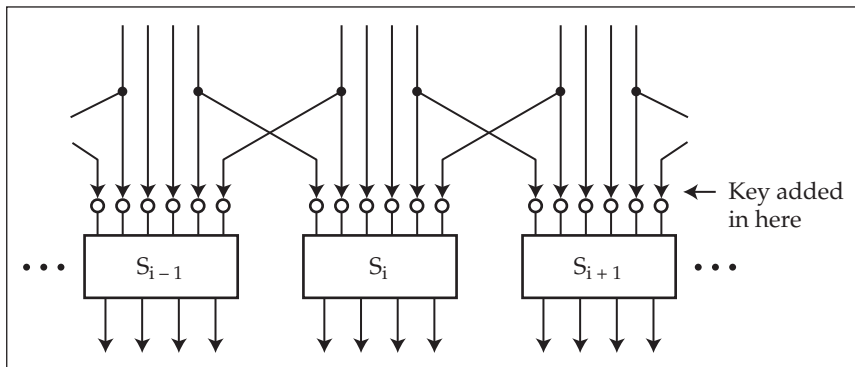


Figure 5.13: The DES round function

The round keys are derived from the user-supplied key by using each user key bit in twelve different rounds according to a slightly irregular pattern. A full specification of DES is given in [1399].

DES was introduced in 1974 and immediately caused controversy. The most telling criticism was that the key is too short. Someone who wants to find a 56 bit key using brute force, that is by trying all possible keys, will have a *total exhaust time* of 2^{56} encryptions and an *average solution time* of half that, namely 2^{55} encryptions. Whit Diffie and Martin Hellman argued in 1977 that a DES key-search machine could be built with a million chips, each testing a million keys a second; as a million is about 2^{20} , this would take on average 2^{15} seconds, or a bit over 9 hours, to find the key. They argued that such a machine could be built for \$20 million in 1977 [557]. IBM, whose scientists invented DES, retorted that they would charge the US government \$200 million to build such a machine. (In hindsight, both were right.)

During the 1980's, there were persistent rumors of DES keysearch machines being built by various intelligence agencies, but the first successful public key-search attack took place in 1997. In a distributed effort organised over the net, 14,000 PCs took more than four months to find the key to a challenge. In 1998, the Electronic Frontier Foundation (EFF) built a DES keysearch machine called Deep Crack for under \$250,000, which broke a DES challenge in 3 days. It contained 1,536 chips run at 40MHz, each chip containing 24 search units which each took 16 cycles to do a test decrypt. The search rate was thus 2.5 million test decryptions per second per search unit, or 60 million keys per second per chip. The design of the cracker is public and can be found at [619]. By 2006, Sandeep Kumar and colleagues at the universities of Bochum and Kiel built a machine using 120 FPGAs and costing \$10,000, which could break DES in 7 days on average [1110]. A modern botnet with 100,000 machines would take a few hours. So the key length of single DES is now inadequate.

Another criticism of DES was that, since IBM kept its design principles secret at the request of the US government, perhaps there was a 'trapdoor' which would give them easy access. However, the design principles were published in 1992 after differential cryptanalysis was invented and published [473]. The story was that IBM had discovered these techniques in 1972, and the US National Security Agency (NSA) even earlier. IBM kept the design details secret at the NSA's request. We'll discuss the political aspects of all this in 26.2.7.1.

We now have a fairly thorough analysis of DES. The best known *shortcut attack*, that is, a cryptanalytic attack involving less computation than keysearch, is a linear attack using 2^{42} known texts. DES would be secure with more than 20 rounds, but for practical purposes its security is limited by its keylength. I don't know of any real applications where an attacker might get hold of even 2^{40} known texts. So the known shortcut attacks are not an issue. However, its

vulnerability to keysearch makes single DES unusable in most applications. As with AES, there are also attacks based on timing analysis and power analysis.

The usual way of dealing with the DES key length problem is to use the algorithm multiple times with different keys. Banking networks have largely moved to *triple-DES*, a standard since 1999 [1399]. Triple-DES does an encryption, then a decryption, and then a further encryption, all with independent keys. Formally:

$$3DES(k_0, k_1, k_2; M) = DES(k_2; DES^{-1}(k_1; DES(k_0; M)))$$

By setting the three keys equal, you get the same result as a single DES encryption, thus giving a backwards compatibility mode with legacy equipment. (Some banking systems use *two-key triple-DES* which sets $k_2 = k_0$; this gives an intermediate step between single and triple DES.) Most new systems use AES as the default choice, but many banking systems are committed to using block ciphers with an eight-byte block, because of the message formats used in the many protocols by which ATMs, point-of-sale terminals and bank networks talk to each other, and because of the use of block ciphers to generate and protect customer PINs (which I discuss in the chapter on Banking and Bookkeeping). Triple DES is a perfectly serviceable block cipher for such purposes for the foreseeable future.

Another way of preventing keysearch (and making power analysis harder) is *whitening*. In addition to the 56-bit key, say k_0 , we choose two 64-bit whitening keys k_1 and k_2 , xor'ing the first with the plaintext before encryption and the second with the output of the encryption to get the ciphertext afterwards. This composite cipher is known as DESX. Formally,

$$DESX(k_0, k_1, k_2; M) = DES(k_0; M \oplus k_1) \oplus k_2$$

It can be shown that, on reasonable assumptions, DESX has the properties you'd expect; it inherits the differential strength of DES but its resistance to keysearch is increased by the amount of the whitening [1049]. Whitened block ciphers are used in some applications, most specifically in the XTS mode of operation which I discuss below. Nowadays, it's usually used with AES, and AESX is defined similarly, with the whitening keys used to make each block encryption operation unique – as we shall see below in section 5.5.7.

5.5 Modes of operation

A common failure is that cryptographic libraries enable or even encourage developers to use an inappropriate *mode of operation*. This specifies how a block cipher with a fixed block size (8 bytes for DES, 16 for AES) can be extended to process messages of arbitrary length.

There are several standard modes of operation for using a block cipher on multiple blocks [1406]. It is vital to understand them, so you can choose the right one for the job, especially as some common tools provide a weak one by default. This weak mode is electronic code book (ECB) mode, which we discuss next.

5.5.1 How not to use a block cipher

In electronic code book mode, we just encrypt each succeeding block of plaintext with our block cipher to get ciphertext, as with the Playfair example above. This is adequate for protocols using single blocks such as challenge-response and some key management tasks; it's also used to encrypt PINs in cash machine systems. But if we use it to encrypt redundant data the patterns will show through, giving an opponent information about the plaintext. For example, figure 5.14 shows what happens to a cartoon image when encrypted using DES in ECB mode. Repeated blocks of plaintext all encrypt to the same ciphertext, leaving the image quite recognisable.

In one popular corporate email system from the last century, the encryption used was DES ECB with the key derived from an eight-character password. If you looked at a ciphertext generated by this system, you saw that a certain block was far more common than the others – the one corresponding to a plaintext of nulls. This gave one of the simplest attacks ever on a fielded DES encryption system: just encrypt a null block with each password in a dictionary and sort the answers. You can now break at sight any ciphertext whose password was one of those in your dictionary.

In addition, using ECB mode to encrypt messages of more than one block length which require authenticity – such as bank payment messages – is

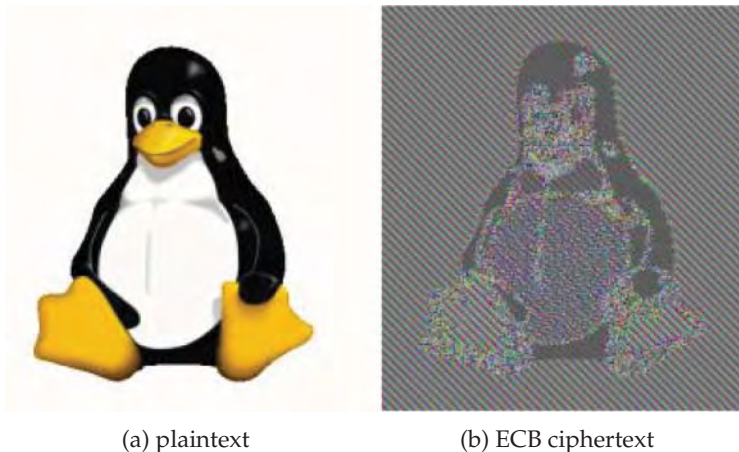


Figure 5.14: The Linux penguin, in clear and ECB encrypted (from Wikipedia, derived from images created by Larry Ewing).

particularly foolish, as it opens you to a *cut and splice* attack along the block boundaries. For example, if a bank message said “Please pay account number X the sum Y , and their reference number is Z ” then an attacker might initiate a payment designed so that some of the digits of X are replaced with some of the digits of Z .

5.5.2 Cipher block chaining

Most commercial applications which encrypt more than one block used to use cipher block chaining, or CBC, mode. Like ECB, this was one of the original modes of operation standardised with DES. In it, we exclusive-or the previous block of ciphertext to the current block of plaintext before encryption (see Figure 5.15).

This mode disguises patterns in the plaintext: the encryption of each block depends on all the previous blocks. The input initialisation vector (IV) ensures that stereotyped plaintext message headers won’t leak information by encrypting to identical ciphertexts, just as with a stream cipher.

However, an opponent who knows some of the plaintext may be able to cut and splice a message (or parts of several messages encrypted under the same key). In fact, if an error is inserted into the ciphertext, it will affect only two blocks of plaintext on decryption, so if there isn’t any integrity protection on the plaintext, an enemy can insert two-block garbles of random data at locations of their choice. For that reason, CBC encryption usually has to be used with a separate authentication code.

More subtle things can go wrong, too; systems have to pad the plaintext to a multiple of the block size, and if a server that decrypts a message and finds incorrect padding signals this fact, whether by returning an ‘invalid padding’ message or just taking longer to respond, then this opens a *padding oracle attack*

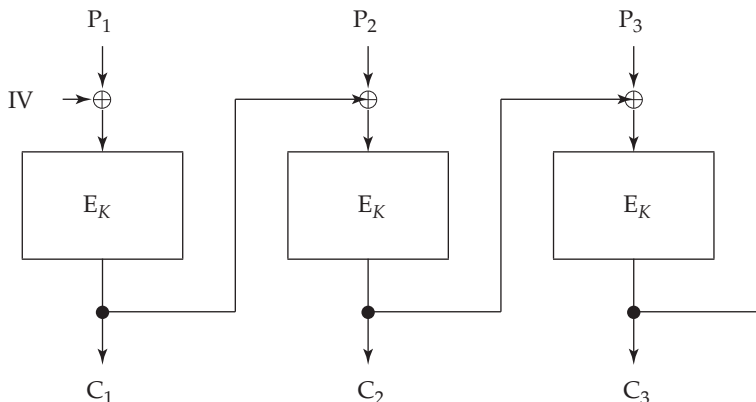


Figure 5.15: Cipher Block Chaining (CBC) mode

in which the attacker tweaks input ciphertexts, one byte at a time, watches the error messages, and ends up being able to decrypt whole messages. This was discovered by Serge Vaudenay in 2002; variants of it were used against SSL, IPSEC and TLS as late as 2016 [1953].

5.5.3 Counter encryption

Feedback modes of block cipher encryption are falling from fashion, and not just because of cryptographic issues. They are hard to parallelise. With CBC, a whole block of the cipher must be computed between each block input and each block output. This can be inconvenient in high-speed applications, such as protecting traffic on backbone links. As silicon is cheap, we would rather pipeline our encryption chip, so that it encrypts a new block (or generates a new block of keystream) in as few clock ticks as possible.

The simplest solution is to use AES as a stream cipher. We generate a keystream by encrypting a counter starting at an initialisation vector: $K_i = \{IV + i\}_K$, thus expanding the key K into a long stream of blocks K_i of keystream, which is typically combined with the blocks of a message M_i using exclusive-or to give ciphertext $C_i = M_i \oplus K_i$.

Additive stream ciphers have two systemic vulnerabilities, as we noted in section 5.2.2 above. The first is an attack in depth: if the same keystream is used twice, then the xor of the two ciphertexts is the xor of the two plaintexts, from which plaintext can often be deduced, as with Venona. The second is that they fail to protect message integrity. Suppose that a stream cipher were used to encipher fund transfer messages. These messages are highly structured; you might know, for example, that bytes 37–42 contain the sum being transferred. You could then cause the data traffic from a local bank to go via your computer, for example by an SS7 exploit. You go into the bank and send \$500 to an accomplice. The ciphertext $C_i = M_i \oplus K_i$, duly arrives in your machine. You know M_i for bytes 37–42, so you can recover K_i and construct a modified message which instructs the receiving bank to pay not \$500 but \$500,000! This is an example of an *attack in depth*; it is the price not just of the perfect secrecy we get from the one-time pad, but of much more humble stream ciphers, too.

The usual way of dealing with this is to add an authentication code, and the most common standard uses a technique called Galois counter mode, which I describe later.

5.5.4 Legacy stream cipher modes

You may find two old stream-cipher modes of operation, output feedback mode (OFB) and less frequently ciphertext feedback mode (CFB).

Output feedback mode consists of repeatedly encrypting an initial value and using this as a keystream in a stream cipher. Writing IV for the initialization vector, we will have $K_1 = \{IV\}_K$ and $K_i = \{IV\}_{K(i-1)}$. However an n -bit block cipher in OFB mode will typically have a cycle length of $2^{n/2}$ blocks, after which the birthday theorem will see to it that we loop back to the IV . So we may have a cycle-length problem if we use a 64-bit block cipher such as triple-DES on a high-speed link: once we've called a little over 2^{32} pseudorandom 64-bit values, the odds favour a match. (In CBC mode, too, the birthday theorem ensures that after about $2^{n/2}$ blocks, we will start to see repeats.) Counter mode encryption, however, has a guaranteed cycle length of 2^n rather than $2^{n/2}$, and as we noted above is easy to parallelise. Despite this OFB is still used, as counter mode only became a NIST standard in 2002.

Cipher feedback mode is another kind of stream cipher, designed for use in radio systems that have to resist jamming. It was designed to be self-synchronizing, in that even if we get a burst error and drop a few bits, the system will recover synchronization after one block length. This is achieved by using our block cipher to encrypt the last n bits of ciphertext, adding the last output bit to the next plaintext bit, and shifting the ciphertext along one bit. But this costs one block cipher operation per bit and has very bad error amplification properties; nowadays people tend to use dedicated link layer protocols for synchronization and error correction rather than trying to combine them with the cryptography at the traffic layer.

5.5.5 Message authentication code

Another official mode of operation of a block cipher is not used to encipher data, but to protect its integrity and authenticity. This is the *message authentication code*, or MAC. To compute a MAC on a message using a block cipher, we encrypt it using CBC mode and throw away all the output ciphertext blocks except the last one; this last block is the MAC. (The intermediate results are kept secret in order to prevent splicing attacks.)

This construction makes the MAC depend on all the plaintext blocks as well as on the key. It is secure provided the message length is fixed; Mihir Bellare, Joe Kilian and Philip Rogaway proved that any attack on a MAC under these circumstances would give an attack on the underlying block cipher [212].

If the message length is variable, you have to ensure that a MAC computed on one string can't be used as the IV for computing a MAC on a different string, so that an opponent can't cheat by getting a MAC on the composition of the two strings. In order to fix this problem, NIST has standardised CMAC, in which a variant of the key is xor-ed in before the last encryption [1407]. (CMAC is based on a proposal by Tetsu Iwata and Kaoru Kurosawa [967].) You may see legacy systems in which the MAC consists of only half of the last output block, with the other half thrown away, or used in other mechanisms.

There are other possible constructions of MACs: the most common one is HMAC, which uses a hash function with a key; we'll describe it in section 5.6.2.

5.5.6 Galois counter mode

The above modes were all developed for DES in the 1970s and 1980s (although counter mode only became an official US government standard in 2002). They are not efficient for bulk encryption where you need to protect integrity as well as confidentiality; if you use either CBC mode or counter mode to encrypt your data and a CBC-MAC or CMAC to protect its integrity, then you invoke the block cipher twice for each block of data you process, and the operation cannot be parallelised.

The modern approach is to use a mode of operation designed for authenticated encryption. Galois Counter Mode (GCM) has taken over as the default since being approved by NIST in 2007 [1409]. It uses only one invocation of the block cipher per block of text, and it's parallelisable so you can get high throughput on fast data links with low cost and low latency. Encryption is performed in a variant of counter mode; the resulting ciphertexts are also used as coefficients of a polynomial which is evaluated at a key-dependent point over a Galois field of 2^{128} elements to give an authenticator tag. The tag computation is a universal hash function of the kind I described in section 5.2.4 and is provably secure so long as keys are never reused. The supplied key is used along with a random IV to generate both a unique message key and a unique authenticator key. The output is thus a ciphertext of the same length as the plaintext, plus an IV and a tag of typically 128 bits each.

GCM also has an interesting incremental property: a new authenticator and ciphertext can be calculated with an amount of effort proportional to the number of bits that were changed. GCM was invented by David McGrew and John Viega of Cisco; their goal was to create an efficient authenticated encryption mode suitable for use in high-performance network hardware [1270]. It is the sensible default for authenticated encryption of bulk content. (There's an earlier composite mode, CCM, which you'll find used in Bluetooth 4.0 and later; this combines counter mode with CBC-MAC, so it costs about twice as much effort to compute, and cannot be parallelised or recomputed incrementally [1408].)

5.5.7 XTS

GCM and other authenticated encryption modes expand the plaintext by adding a message key and an authenticator tag. This is very inconvenient in applications such as hard disk encryption, where we prefer a mode of

operation that preserves plaintext length. Disk encryption systems used to use CBC with the sector number providing an IV, but since Windows 10, Microsoft has been using a new mode of operation, XTS-AES, inspired by GCM and standardised in 2007. This is a codebook mode but with the plaintext whitened by a *tweak key* derived from the disk sector. Formally, the message M_i encrypted with the key K at block j is

$$\text{AESX}(KT_j, K, KT_j; M)$$

where the tweak key KT_j is derived by encrypting the IV using a different key and then multiplying it repeatedly with a suitable constant so as to give a different whitener for each block. This means that if an attacker swaps two encrypted blocks, all 256 bits will decrypt to randomly wrong values. You still need higher-layer mechanisms to detect ciphertext manipulation, but simple checksums will be sufficient.

5.6 Hash functions

In section 5.4.3.1 I showed how the Luby-Rackoff theorem enables us to construct a block cipher from a hash function. It's also possible to construct a hash function from a block cipher⁵. The trick is to feed the message blocks one at a time to the key input of our block cipher, and use it to update a hash value (which starts off at say $H_0 = 0$). In order to make this operation non-invertible, we add feedforward: the $(i - 1)$ st hash value is exclusive or'ed with the output of round i . This *Davies-Meyer construction* gives our final mode of operation of a block cipher (Figure 5.16).

The birthday theorem makes another appearance here, in that if a hash function h is built using an n bit block cipher, it is possible to find two messages $M_1 \neq M_2$ with $h(M_1) = h(M_2)$ with about $2^{n/2}$ effort (hash slightly more than that many messages M_i and look for a match). So a 64 bit block cipher is not adequate, as forging a message would cost of the order of 2^{32} messages, which is just too easy. A 128-bit cipher such as AES used to be just about adequate, and in fact the AACS content protection mechanism in Blu-ray DVDs used 'AES-H', the hash function derived from AES in this way.

5.6.1 Common hash functions

The hash functions most commonly used through the 1990s and 2000s evolved as variants of a block cipher with a 512 bit key and a block size increasing from

⁵In fact, we can also construct hash functions and block ciphers from stream ciphers – so, subject to some caveats I'll discuss in the next section, given any one of these three primitives we can construct the other two.

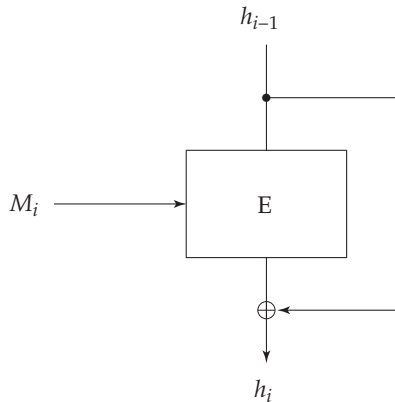


Figure 5.16: Feedforward mode (hash function)

128 to 512 bits. The first two were designed by Ron Rivest and the others by the NSA:

- MD4 has three rounds and a 128 bit hash value, and a collision was found for it in 1998 [568];
- MD5 has four rounds and a 128 bit hash value, and a collision was found for it in 2004 [1983, 1985];
- SHA-1, released in 1995, has five rounds and a 160 bit hash value. A collision was found in 2017 [1831], and a more powerful version of the attack in 2020 [1148];
- SHA-2, which replaced it in 2002, comes in 256-bit and 512-bit versions (called SHA256 and SHA512) plus a number of variants.

The block ciphers underlying these hash functions are similar: their round function is a complicated mixture of the register operations available on 32 bit processors [1670]. Cryptanalysis has advanced steadily. MD4 was broken by Hans Dobbertin in 1998 [568]; MD5 was broken by Xiaoyun Wang and her colleagues in 2004 [1983, 1985]; collisions can now be found easily, even between strings containing meaningful text and adhering to message formats such as those used for digital certificates. Wang seriously dented SHA-1 the following year in work with Yiqun Lisa Yin and Hongbo Yu, providing an algorithm to find collisions in only 2^{69} steps [1984]; it now takes about 2^{60} computations. In February 2017, scientists from Amsterdam and Google published just such a collision, to prove the point and help persuade people to move to stronger hash functions such as SHA-2 [1831] (and from earlier versions of TLS to TLS 1.3). In 2020, Gaëtan Leurent and Thomas Peyrin developed an improved attack that computes chosen-prefix collisions, enabling certificate forgery at a cost of several tens of thousands of dollars [1148].

In 2007, the US National Institute of Standards and Technology (NIST) organised a competition to find a replacement hash function family [1411]. The winner, Keccak, has a quite different internal structure, and was standardised as SHA-3 in 2015. So we now have a choice of SHA-2 and SHA-3 as standard hash functions.

A lot of deployed systems still use hash functions such as MD5 for which there's an easy collision-search algorithm. Whether a collision will break any given application can be a complex question. I already mentioned forensic systems, which keep hashes of files on seized computers, to reassure the court that the police didn't tamper with the evidence; a hash collision would merely signal that someone had been trying to tamper, whether the police or the defendant, and trigger a more careful investigation. If bank systems actually took a message composed by a customer saying 'Pay X the sum Y', hashed it and signed it, then a crook could find two messages 'Pay X the sum Y' and 'Pay X the sum Z' that hashed to the same value, get one signed, and swap it for the other. But bank systems don't work like that. They typically use MACs rather than digital signatures on actual transactions, and logs are kept by all the parties to a transaction, so it's not easy to sneak in one of a colliding pair. And in both cases you'd probably have to find a preimage of an existing hash value, which is a much harder cryptanalytic task than finding a collision.

5.6.2 Hash function applications – HMAC, commitments and updating

But even though there may be few applications where a collision-finding algorithm could let a bad guy steal real money today, the existence of a vulnerability can still undermine a system's value. Some people doing forensic work continue to use MD5, as they've used it for years, and its collisions don't give useful attacks. This is probably a mistake. In 2005, a motorist accused of speeding in Sydney, Australia was acquitted after the New South Wales Roads and Traffic Authority failed to find an expert to testify that MD5 was secure in this application. The judge was "not satisfied beyond reasonable doubt that the photograph [had] not been altered since it was taken" and acquitted the motorist; his strange ruling was upheld on appeal the following year [1434]. So even if a vulnerability doesn't present an engineering threat, it can still present a *certificational* threat.

Hash functions have many other uses. One of them is to compute MACs. A naïve method would be to hash the message with a key: $\text{MAC}_k(M) = h(k, M)$. However the accepted way of doing this, called HMAC, uses an extra step in which the result of this computation is hashed again. The two hashing operations are done using variants of the key, derived by exclusive-or'ing them with two different constants. Thus $\text{HMAC}_k(M) = h(k \oplus B, h(k \oplus A, M))$.

A is constructed by repeating the byte $0x36$ as often as necessary, and B similarly from the byte $0x5C$. If a hash function is on the weak side, this construction can make exploitable collisions harder to find [1091]. HMAC is now FIPS 198-1.

Another use of hash functions is to make commitments that are to be revealed later. For example, I might wish to timestamp a digital document in order to establish intellectual priority, but not reveal the contents yet. In that case, I can publish a hash of the document, or send it to a commercial timestamping service, or have it mined into the Bitcoin blockchain. Later, when I reveal the document, the timestamp on its hash establishes that I had written it by then. Again, an algorithm that generates colliding pairs doesn't break this, as you have to have the pair to hand when you do the timestamp.

Merkle trees hash a large number of inputs to a single hash output. The inputs are hashed to values that form the leaves of a tree; each non-leaf node contains the hash of all the hashes at its child nodes, so the hash at the root is a hash of all the values at the leaves. This is a fast way to hash a large data structure; it's used in code signing, where you may not want to wait for all of an application's files to have their signatures checked before you open it. It's also widely used in blockchain applications; in fact, a blockchain is just a Merkle tree. It was invented by Ralph Merkle, who first proposed it to calculate a short hash of a large file of public keys [1298], particularly for systems where public keys are used only once. For example, a Lamport digital signature can be constructed from a hash function: you create a private key of 512 random 256-bit values k_i and publish the verification key V as their Merkle tree hash. Then to sign $h = \text{SHA256}(M)$ you would reveal k_{2i} if the i -th bit of h is zero, and otherwise reveal k_{2i+1} . This is secure if the hash function is, but has the drawback that each key can be used only once. Merkle saw that you could generate a series of private keys by encrypting a counter with a master secret key, and then use a tree to hash the resulting public keys. However, for most purposes, people use signature algorithms based on number theory, which I'll describe in the next section.

One security-protocol use of hash functions is worth a mention: key updating and autokeying. *Key updating* means that two or more principals who share a key pass it through a one-way hash function at agreed times: $K_i = h(K_{i-1})$. The point is that if an attacker compromises one of their systems and steals the key, he only gets the current key and is unable to decrypt back traffic. The chain of compromise is broken by the hash function's one-wayness. This property is also known as *backward security*. A variant is *autokeying* where the principals update a key by hashing it with the messages they have exchanged since the last key change: $K_{i+1} = h(K_i, M_{i1}, M_{i2}, \dots)$. If an attacker now compromises one of their systems and steals the key, then as soon as they exchange a message which he can't observe or guess, security will be recovered; again, the chain of compromise is broken. This property is known as *forward security*. It was

first used in banking in EFT payment terminals in Australia [208, 210]. The use of asymmetric cryptography allows a slightly stronger form of forward security, namely that as soon as a compromised terminal exchanges a message with an uncompromised one which the opponent doesn't control, security can be recovered even if the message is in plain sight. I'll describe how this works next.

5.7 Asymmetric crypto primitives

The commonly used building blocks in *asymmetric cryptography*, public-key encryption and digital signature are based on number theory. I'll give a brief overview here, and look in more detail at some of the mechanisms in Part 2 when I discuss applications.

The basic idea is to make the security of the cipher depend on the difficulty of solving a mathematical problem that's known to be hard, in the sense that a lot of people have tried to solve it and failed. The two problems used in almost all real systems are factorization and discrete logarithm.

5.7.1 Cryptography based on factoring

The *prime numbers* are the positive whole numbers with no proper divisors: the only numbers that divide a prime number are 1 and the number itself. By definition, 1 is not prime; so the primes are {2, 3, 5, 7, 11, ...}. The *fundamental theorem of arithmetic* states that each natural number greater than 1 factors into prime numbers in a way that is unique up to the order of the factors. It is easy to find prime numbers and multiply them together to give a composite number, but much harder to resolve a composite number into its factors. And lots of smart people have tried really hard since we started using cryptography based on factoring. The largest composite product of two large random primes to have been factorized in 2020 was RSA-250, an 829-bit number (250 decimal digits). This took the equivalent of 2700 years' work on a single 2.2GHz core; the previous record, RSA-240 in 2019, had taken the equivalent of 900 years [302]. It is possible for factoring to be done surreptitiously, perhaps using a botnet; in 2001, when the state of the art was factoring 512-bit numbers, such a challenge was set in Simon Singh's 'Code Book' and solved by five Swedish students using several hundred computers to which they had access [44]. As for 1024-bit numbers, I expect the NSA can factor them already, and I noted in the second edition that 'an extrapolation of the history of factoring records suggests the first factorization will be published in 2018.' Moore's law is slowing down, and we're two years late. Anyway, organisations that want keys to remain secure for many years are already using 2048-bit numbers at least.

The algorithm commonly used to do public-key encryption and digital signatures based on factoring is RSA, named after its inventors Ron Rivest, Adi Shamir and Len Adleman. It uses *Fermat's little theorem*, which states that for all primes p not dividing a , $a^{p-1} \equiv 1 \pmod{p}$ (proof: take the set $\{1, 2, \dots, p-1\}$ and multiply each of them modulo p by a , then cancel out $(p-1)!$ each side). For a general integer n , $a^{\phi(n)} \equiv 1 \pmod{p}$ where Euler's function $\phi(n)$ is the number of positive integers less than n with which it has no divisor in common (the proof is similar). So if n is the product of two primes pq then $\phi(n) = (p-1)(q-1)$.

In RSA, the encryption key is a modulus N which is hard to factor (take $N = pq$ for two large randomly chosen primes p and q , say of 1024 bits each) plus a public exponent e that has no common factors with either $p-1$ or $q-1$. The private key is the factors p and q , which are kept secret. Where M is the message and C is the ciphertext, encryption is defined by

$$C \equiv M^e \pmod{N}$$

Decryption is the reverse operation:

$$M \equiv \sqrt[e]{C} \pmod{N}$$

Whoever knows the private key – the factors p and q of N – can easily calculate $\sqrt[e]{C} \pmod{N}$. As $\phi(N) = (p-1)(q-1)$ and e has no common factors with $\phi(N)$, the key's owner can find a number d such that $de \equiv 1 \pmod{\phi(N)}$ – she finds the value of d separately modulo $p-1$ and $q-1$, and combines the answers. $\sqrt[e]{C} \pmod{N}$ is now computed as $C^d \pmod{N}$, and decryption works because of Fermat's theorem:

$$C^d \equiv \{M^e\}^d \equiv M^{ed} \equiv M^{1+k\phi(N)} \equiv M.M^{k\phi(N)} \equiv M.1 \equiv M \pmod{N}$$

Similarly, the owner of a private key can operate on a message with it to produce a signature

$$\text{Sig}_d(M) \equiv M^d \pmod{N}$$

and this signature can be verified by raising it to the power $e \pmod{N}$ (thus, using e and N as the public signature verification key) and checking that the message M is recovered:

$$M \equiv (\text{Sig}_d(M))^e \pmod{N}$$

Neither RSA encryption nor signature is safe to use on its own. The reason is that, as encryption is an algebraic process, it preserves certain algebraic properties. For example, if we have a relation such as $M_1M_2 = M_3$ that holds among plaintexts, then the same relationship will hold among ciphertexts $C_1C_2 = C_3$ and signatures $\text{Sig}_1\text{Sig}_2 = \text{Sig}_3$. This property is known as a *multiplicative homomorphism*; a homomorphism is a function that preserves some mathematical structure. The homomorphic nature of raw RSA means that it doesn't meet the random oracle model definitions of public key encryption or signature.

Another general problem with public-key encryption is that if the plaintexts are drawn from a small set, such as ‘attack’ or ‘retreat’, and the encryption process is deterministic (as RSA is), then an attacker might just precompute the possible ciphertexts and recognise them when they appear. With RSA, it’s also dangerous to use a small exponent e to encrypt the same message to multiple recipients, as this can lead to an algebraic attack. To stop the guessing attack, the low-exponent attack and attacks based on homomorphism, it’s sensible to add in some randomness, and some redundancy, into a plaintext block before encrypting it. Every time we encrypt the same short message, say ‘attack’, we want to get a completely different ciphertext, and for these to be indistinguishable from each other as well as from the ciphertexts for ‘retreat’. And there are good ways and bad ways of doing this.

Crypto theoreticians have wrestled for decades to analyse all the things that can go wrong with asymmetric cryptography, and to find ways to tidy it up. Shafi Goldwasser and Silvio Micali came up with formal models of *probabilistic encryption* in which we add randomness to the encryption process, and *semantic security*, which we mentioned already; in this context it means that an attacker cannot get any information at all about a plaintext M that was encrypted to a ciphertext C , even if he is allowed to request the decryption of any other ciphertext C' not equal to C [778]. In other words, we want the encryption to resist chosen-ciphertext attack as well as chosen-plaintext attack. There are a number of constructions that give semantic security, but they tend to be too ungainly for practical use.

The usual real-world solution is *optimal asymmetric encryption padding* (OAEP), where we concatenate the message M with a random nonce N , and use a hash function h to combine them:

$$C_1 = M \oplus h(N)$$

$$C_2 = N \oplus h(C_1)$$

In effect, this is a two-round Feistel cipher that uses h as its round function. The result, the combination C_1, C_2 , is then encrypted with RSA and sent. The recipient then computes N as $C_2 \oplus h(C_1)$ and recovers M as $C_1 \oplus h(N)$ [213]. This was eventually proven to be secure. There are a number of public-key cryptography standards; PKCS #1 describes OAEP [995]. These block a whole lot of attacks that were discovered in the 20th century and about which people have mostly forgotten, such as the fact that an opponent can detect if you encrypt the same message with two different RSA keys. In fact, one of the things we learned in the 1990s was that randomisation helps make crypto protocols more robust against all sorts of attacks, and not just the mathematical ones. Side-channel attacks and even physical probing of devices take a lot more work.

With signatures, things are slightly simpler. In general, it’s often enough to just hash the message before applying the private key: $Sig_d = [h(M)]^d \pmod{N}$;

PKCS #7 describes simple mechanisms for signing a message digest [1010]. However, in some applications one might wish to include further data in the signature block, such as a timestamp, or some randomness to make side-channel attacks harder.

Many of the things that have gone wrong with real implementations have to do with side channels and error handling. One spectacular example was when Daniel Bleichenbacher found a way to break the RSA implementation in SSL v 3.0 by sending suitably chosen ciphertexts to the victim and observing any resulting error messages. If he could learn from the target whether a given c , when decrypted as $c^d \pmod{n}$, corresponds to a PKCS #1 message, then he could use this to decrypt or sign messages [265]. There have been many more side-channel attacks on common public-key implementations, typically via measuring the precise time taken to decrypt. RSA is also mathematically fragile; you can break it using homomorphisms, or if you have the same ciphertext encrypted under too many different small keys, or if the message is too short, or if two messages are related by a known polynomial, or in several other edge cases. Errors in computation can also give a result that's correct modulo one factor of the modulus and wrong modulo the other, enabling the modulus to be factored; errors can be inserted tactically, by interfering with the crypto device, or strategically, for example by the chipmaker arranging for one particular value of a 64-bit multiply to be computed incorrectly. Yet other attacks have involved stack overflows, whether by sending the attack code in as keys, or as padding in poorly-implemented standards.

5.7.2 Cryptography based on discrete logarithms

While RSA was the first public-key encryption algorithm deployed in the SSL and SSH protocols, the most popular public-key algorithms now are based on discrete logarithms. There are a number of flavors, some using normal modular arithmetic while others use *elliptic curves*. I'll explain the normal case first.

A *primitive root* modulo p is a number whose powers generate all the nonzero numbers mod p ; for example, when working modulo 7 we find that $5^2 = 25$ which reduces to 4 (modulo 7), then we can compute 5^3 as $5^2 \times 5$ or 4×5 which is 20, which reduces to 6 (modulo 7), and so on, as in Figure 5.17.

Thus 5 is a primitive root modulo 7. This means that given any y , we can always solve the equation $y = 5^x \pmod{7}$; x is then called the discrete logarithm of y modulo 7. Small examples like this can be solved by inspection, but for a large random prime number p , we do not know how to do this efficiently. So the mapping $f : x \rightarrow g^x \pmod{p}$ is a one-way function, with the additional properties that $f(x + y) = f(x)f(y)$ and $f(nx) = f(x)^n$. In other words, it is a *one-way homomorphism*. As such, it can be used to construct digital signature and public key encryption algorithms.

$$\begin{aligned}
5^1 &= 5 \pmod{7} \\
5^2 &= 25 \equiv 4 \pmod{7} \\
5^3 &\equiv 4 \times 5 \equiv 6 \pmod{7} \\
5^4 &\equiv 6 \times 5 \equiv 2 \pmod{7} \\
5^5 &\equiv 2 \times 5 \equiv 3 \pmod{7} \\
5^6 &\equiv 3 \times 5 \equiv 1 \pmod{7}
\end{aligned}$$

Figure 5.17: Example of discrete logarithm calculations

5.7.2.1 One-way commutative encryption

Imagine we're back in ancient Rome, that Anthony wants to send a secret to Brutus, and the only communications channel available is an untrustworthy courier (say, a slave belonging to Caesar). Anthony can take the message, put it in a box, padlock it, and get the courier to take it to Brutus. Brutus could then put his own padlock on it too, and have it taken back to Anthony. He in turn would remove his padlock, and have it taken back to Brutus, who would now at last open it.

Exactly the same can be done using a suitable encryption function that commutes, that is, has the property that $\{\{M\}_{KA}\}_{KB} = \{\{M\}_{KB}\}_{KA}$. Alice can take the message M and encrypt it with her key KA to get $\{M\}_{KA}$ which she sends to Bob. Bob encrypts it again with his key KB getting $\{\{M\}_{KA}\}_{KB}$. But the commutativity property means that this is just $\{\{M\}_{KB}\}_{KA}$, so Alice can decrypt it using her key KA getting $\{M\}_{KB}$. She sends this to Bob and he can decrypt it with KB , finally recovering the message M .

How can a suitable commutative encryption be implemented? The one-time pad does indeed commute, but is not suitable here. Suppose Alice chooses a random key xA and sends Bob $M \oplus xA$ while Bob returns $M \oplus xB$ and Alice finally sends him $M \oplus xA \oplus xB$, then an attacker can simply exclusive-or these three messages together; as $X \oplus X = 0$ for all X , the two values of xA and xB both cancel out, leaving the plaintext M .

The discrete logarithm problem comes to the rescue. If the discrete log problem based on a primitive root modulo p is hard, then we can use discrete exponentiation as our encryption function. For example, Alice encodes her message as the primitive root g , chooses a random number xA , calculates g^{xA} modulo p and sends it, together with p , to Bob. Bob likewise chooses a random number xB and forms g^{xAxB} modulo p , which he passes back to Alice. Alice can now remove her exponentiation: using Fermat's theorem, she calculates $g^{xB} = (g^{xAxB})^{(p-xA)} \pmod{p}$ and sends it to Bob. Bob can now remove his exponentiation, too, and so finally gets hold of g . The security of this scheme depends on the difficulty of the discrete logarithm problem. In practice, it can be tricky to encode a message as a primitive root; but there's a simpler way to achieve the same effect.

5.7.2.2 Diffie-Hellman key establishment

The first public-key encryption scheme to be published, by Whitfield Diffie and Martin Hellman in 1976, has a fixed primitive root g and uses g^{xAxB} modulo p as the key to a shared-key encryption system. The values xA and xB can be the private keys of the two parties.

Let's walk through this. The prime p and generator g are common to all users. Alice chooses a secret random number xA , calculates $yA = g^{xA}$ and publishes it opposite her name in the company phone book. Bob does the same, choosing a random number xB and publishing $yB = g^{xB}$. In order to communicate with Bob, Alice fetches yB from the phone book, forms yB^{xA} which is just g^{xAxB} , and uses this to encrypt the message to Bob. On receiving it, Bob looks up Alice's public key yA and forms yA^{xB} which is also equal to g^{xAxB} , so he can decrypt her message.

Alternatively, Alice and Bob can use transient keys, and get a mechanism for providing forward security. As before, let the prime p and generator g be common to all users. Alice chooses a random number R_A , calculates g^{R_A} and sends it to Bob; Bob does the same, choosing a random number R_B and sending g^{R_B} to Alice; they then both form $g^{R_AR_B}$, which they use as a session key (see Figure 5.18).

Alice and Bob can now use the session key $g^{R_AR_B}$ to encrypt a conversation. If they used transient keys, rather than long-lived ones, they have managed to create a shared secret 'out of nothing'. Even if an opponent had inspected both their machines before this protocol was started, and knew all their stored private keys, then provided some basic conditions were met (e.g., that their random number generators were not predictable and no malware was left behind) the opponent could still not eavesdrop on their traffic. This is the strong version of the forward security property to which I referred in section 5.6.2. The opponent can't work forward from knowledge of previous keys, however it was obtained. Provided that Alice and Bob both destroy the shared secret after use, they will also have backward security: an opponent who gets access to their equipment later cannot work backward to break their old traffic. In what follows, we may write the Diffie-Hellman key derived from R_A and R_B as $DH(R_A, R_B)$ when we don't have to be explicit about which group we're working in, and don't need to write out explicitly which is the private key R_A and which is the public key g^{R_A} .

$$\begin{aligned} A \rightarrow B &: g^{R_A} \pmod{p} \\ B \rightarrow A &: g^{R_B} \pmod{p} \\ A \rightarrow B &: \{M\}_{g^{R_AR_B}} \end{aligned}$$

Figure 5.18: The Diffie-Hellman key exchange protocol

Slightly more work is needed to provide a full solution. Some care is needed when choosing the parameters p and g ; we can infer from the Snowden disclosures, for example, that the NSA can solve the discrete logarithm problem for commonly-used 1024-bit prime numbers⁶. And there are several other details which depend on whether we want properties such as forward security.

But this protocol has a small problem: although Alice and Bob end up with a session key, neither of them has any real idea who they share it with.

Suppose that in our padlock protocol Caesar had just ordered his slave to bring the box to him instead, and placed his own padlock on it next to Anthony's. The slave takes the box back to Anthony, who removes his padlock, and brings the box back to Caesar who opens it. Caesar can even run two instances of the protocol, pretending to Anthony that he's Brutus and to Brutus that he's Anthony. One fix is for Anthony and Brutus to apply their seals to their locks.

With the Diffie-Hellman protocol, the same idea leads to a middleperson attack. Charlie intercepts Alice's message to Bob and replies to it; at the same time, he initiates a key exchange with Bob, pretending to be Alice. He ends up with a key $DH(R_A, R_C)$ which he shares with Alice, and another key $DH(R_B, R_C)$ which he shares with Bob. So long as he continues to sit in the middle of the network and translate the messages between them, they may have a hard time detecting that their communications are compromised. The usual solution is to authenticate transient keys, and there are various possibilities.

In the STU-2 telephone, which is now obsolete but which you can see in the NSA museum at Fort Meade, the two principals would read out an eight-digit hash of the key they had generated and check that they had the same value before starting to discuss classified matters. Something similar is implemented in Bluetooth versions 4 and later, but is complicated by the many versions that the protocol has evolved to support devices with different user interfaces. The protocol has suffered from multiple attacks, most recently the Key Negotiation of Bluetooth (KNOB) attack, which allows a middleperson to force one-byte keys that are easily brute forced; all devices produced before 2018 are vulnerable [125]. The standard allows for key lengths between one and sixteen bytes; as the keylength negotiation is performed in the clear, an attacker can force the length to the lower limit. All standards-compliant chips are vulnerable; this may be yet more of the toxic waste from the Crypto Wars, which I discuss in section 26.2.7. Earlier versions of Bluetooth are more like the 'just-works' mode of the HomePlug protocol described in section 4.7.1 in that they were principally designed to help you set up a pairing key with the right device in a benign

⁶The likely discrete log algorithm, NFS, involves a large computation for each prime number followed by a smaller computation for each discrete log modulo that prime number. The open record is 795 bits, which took 3,100 core-years in 2019 [302], using a version of NFS that's three times more efficient than ten years ago. There have been persistent rumours of a further NSA improvement and in any case the agency can throw a lot more horsepower at an important calculation.

environment, rather than defending against a sophisticated attack in a hostile one. The more modern ones appear to be better, but it's really just theatre.

So many things go wrong: protocols that will generate or accept very weak keys and thus give only the appearance of protection; programs that leak keys via side channels such as the length of time they take to decrypt; and software vulnerabilities leading to stack overflows and other hacks. If you're implementing public-key cryptography you need to consult up-to-date standards, use properly accredited toolkits, and get someone knowledgeable to evaluate what you've done. And please don't write the actual crypto code on your own – doing it properly requires a lot of different skills, from computational number theory to side-channel analysis and formal methods. Even using good crypto libraries gives you plenty of opportunities to shoot your foot off.

5.7.2.3 ElGamal digital signature and DSA

Suppose that the base p and the generator g are public values chosen in some suitable way, and that each user who wishes to sign messages has a private signing key X with a public signature verification key $Y = g^X$. An ElGamal signature scheme works as follows. Choose a message key k at random, and form $r = g^k \pmod{p}$. Now form the signature s using a linear equation in k , r , the message M and the private key X . There are a number of equations that will do; the one that happens to be used in ElGamal signatures is

$$rX + sk = M$$

So s is computed as $s = (M - rX)/k$; this is done modulo $\phi(p)$. When both sides are passed through our one-way homomorphism $f(x) = g^x \pmod{p}$ we get:

$$g^{rX} g^{sk} \equiv g^M$$

or

$$Y^r r^s \equiv g^M$$

An ElGamal signature on the message M consists of the values r and s , and the recipient can verify it using the above equation.

A few more details need to be fixed up to get a functional digital signature scheme. As before, bad choices of p and g can weaken the algorithm. We will also want to hash the message M using a hash function so that we can sign messages of arbitrary length, and so that an opponent can't use the algorithm's algebraic structure to forge signatures on messages that were never signed. Having attended to these details and applied one or two optimisations, we get the *Digital Signature Algorithm* (DSA) which is a US standard and widely used in government applications.

DSA assumes a prime p of typically 2048 bits⁷, a prime q of 256 bits dividing $(p - 1)$, an element g of order q in the integers modulo p , a secret signing key x and a public verification key $y = g^x$. The signature on a message M , $\text{Sig}_x(M)$, is (r, s) where

$$r \equiv (g^k \pmod{p}) \pmod{q}$$

$$s \equiv (h(M) - xr)/k \pmod{q}$$

The hash function used by default is SHA256⁸.

DSA is the classic example of a randomised digital signature scheme without message recovery. The most commonly-used version nowadays is ECDSA, a variant based on elliptic curves, which we'll discuss now – this is for example the standard for cryptocurrency and increasingly also for certificates in bank smartcards.

5.7.3 Elliptic curve cryptography

Discrete logarithms and their analogues exist in many other mathematical structures. *Elliptic curve cryptography* uses discrete logarithms on an elliptic curve – a curve given by an equation like $y^2 = x^3 + ax + b$. These curves have the property that you can define an addition operation on them and the resulting *Mordell group* can be used for cryptography. The algebra gets a bit complex and this book isn't the place to set it out⁹. However, elliptic curve cryptosystems are interesting for at least two reasons.

First is performance; they give versions of the familiar primitives such as Diffie-Hellmann key exchange and the Digital Signature Algorithm that use less computation, and also have shorter variables; both are welcome in constrained environments. Elliptic curve cryptography is used in applications from the latest versions of EMV payment cards to Bitcoin.

Second, some elliptic curves have a *bilinear pairing* which Dan Boneh and Matt Franklin used to construct cryptosystems where your public key is your name [287]. Recall that in RSA and Diffie-Hellmann, the user chose his private key and then computed a corresponding public key. In a so-called *identity-based cryptosystem*, you choose your identity then go to a central authority that issues you with a private key corresponding to that identity. There is a global public key, with which anyone can encrypt a message

⁷In the 1990s p could be in the range 512–1024 bits and q 160 bits; this was changed to 1023–1024 bits in 2001 [1404] and 1024–3072 bits in 2009, with q in the range 160–256 bits [1405].

⁸The default sizes of p are chosen to be 2048 bits and q 256 bits in order to equalise the work factors of the two best known cryptanalytic attacks, namely the number field sieve whose running speed depends on the size of p and Pollard's rho which depends on the size of q . Larger sizes can be chosen if you're anxious about Moore's law or about progress in algorithms.

⁹See Katz and Lindell [1025] for an introduction.

to your identity; you can decrypt this using your private key. Earlier, Adi Shamir had discovered *identity-based signature schemes* that allow you to sign messages using a private key so that anyone can verify the signature against your name [1707]. In both cases, your private key is computed by the central authority using a system-wide private key known only to itself. Identity-based primitives have been used in a few specialist systems: in Zcash for the payment privacy mechanisms, and in a UK government key-management protocol called Mikey-Sakke. Computing people's private keys from their email addresses or other identifiers may seem a neat hack, but it can be expensive when government departments are reorganised or renamed [116]. Most organisations and applications use ordinary public-key systems with certification of public keys, which I'll discuss next.

5.7.4 Certification authorities

Now that we can do public-key encryption and digital signature, we need some mechanism to bind users to keys. The approach proposed by Diffie and Hellman when they invented digital signatures was to have a directory of the public keys of a system's authorised users, like a phone book. A more common solution, due to Loren Kohnfelder, is for a *certification authority* (CA) to sign the users' public encryption keys or their signature verification keys giving certificates that contain a user's name, one or more of their public keys, and attributes such as authorisations. The CA might be run by the local system administrator; but it is most commonly a third party service such as Verisign whose business is to sign public keys after doing some due diligence about whether they are controlled by the principals named in them.

A certificate might be described symbolically as

$$C_A = \text{Sig}_{K_S}(T_S, L, A, K_A, V_A) \quad (5.1)$$

where T_S is the certificate's starting date and time, L is the length of time for which it is valid, A is the user's name, K_A is her public encryption key, and V_A is her public signature verification key. In this way, only the administrator's public signature verification key needs to be communicated to all principals in a trustworthy manner.

Certification is hard, for a whole lot of reasons. Naming is hard, for starters; we discuss this in Chapter 7 on Distributed Systems. But often names aren't really what the protocol has to establish, as in the real world it's often about authorisation rather than authentication. Government systems are often about establishing not just a user's name or role but their security clearance level. In banking systems, it's about your balance, your available credit and your authority to spend it. In commercial systems, it's often about linking remote users to role-based access control. In user-facing systems, there is

a tendency to dump on the customer as many of the compliance costs as possible [524]. There are many other things that can go wrong with certification at the level of systems engineering. At the level of politics, there are hundreds of certification authorities in a typical browser; they are all more or less equally trusted, and many nation states can coerce at least one of them¹⁰. The revocation of bad certificates is usually flaky, if it works at all. There will be much more on these topics later. With these warnings, it's time to look at the most commonly used public key protocol, TLS.

5.7.5 TLS

I remarked above that a server could publish a public key KS and any web browser could then send a message M containing a credit card number to it encrypted using KS : $\{M\}_{KS}$. This is in essence what the TLS protocol (then known as SSL) was designed to do, at the start of e-commerce. It was developed by Paul Kocher and Taher ElGamal in 1995 to support encryption and authentication in both directions, so that both `http` requests and responses can be protected against both eavesdropping and manipulation. It's the protocol that's activated when you see the padlock on your browser toolbar.

Here is a simplified description of the basic version of the protocol in TLS v1:

1. the client sends the server a *client hello* message that contains its name C , a transaction serial number $C\#$, and a random nonce N_C ;
2. the server replies with a *server hello* message that contains its name S , a transaction serial number $S\#$, a random nonce N_S , and a certificate CS containing its public key KS . The client now checks the certificate CS , and if need be checks the key that signed it in another certificate, and so on back to a root certificate issued by a company such as Verisign and stored in the browser;
3. the client sends a *key exchange* message containing a *pre-master-secret* key, K_0 , encrypted under the server public key KS . It also sends a *finished* message with a message authentication code (MAC) computed on all the messages to date. The key for this MAC is the *master-secret*, K_1 . This key is computed by hashing the pre-master-secret key with the nonces sent by the client and server: $K_1 = h(K_0, N_C, N_S)$. From this point onward, all the traffic is encrypted; we'll write this as $\{\dots\}_{KCS}$ in the client-server direction and $\{\dots\}_{KSC}$ from the server to the client. These keys are generated in turn by hashing the nonces with K_1 .

¹⁰The few that can't, try to cheat. In 2011 Iran hacked the CA Diginotar, and in 2019 Kazakhstan forced its citizens to add a local police certificate to their browser. In both cases the browser vendors pushed back fast and hard: Diginotar failed after it was blacklisted, while the Kazakh cert was blocked even if its citizens installed it manually. This of course raises issues of sovereignty.

4. The server also sends a *finished* message with a MAC computed on all the messages to date. It then finally starts sending the data.

$$\begin{aligned}
 C \rightarrow S &: C, C\#, N_C \\
 S \rightarrow C &: S, S\#, N_S, CS \\
 C \rightarrow S &: \{K_0\}_{KS} \\
 C \rightarrow S &: \{finished, MAC(K_1, everything\ to\ date)\}_{KCS} \\
 S \rightarrow C &: \{finished, MAC(K_1, everything\ to\ date)\}_{KSC}, \{data\}_{KSC}
 \end{aligned}$$

Once a client and server have established a pre-master-secret, no more public-key operations are needed as further master secrets can be obtained by hashing it with new nonces.

5.7.5.1 TLS uses

The full protocol is more complex than this, and has gone through a number of versions. It has supported a number of different ciphersuites, initially so that export versions of software could be limited to 40 bit keys – a condition of export licensing that was imposed for many years by the US government. This led to downgrade attacks where a middleperson could force the use of weak keys. Other ciphersuites support signed Diffie-Hellman key exchanges for transient keys, to provide forward and backward secrecy. TLS also has options for bidirectional authentication so that if the client also has a certificate, this can be checked by the server. In addition, the working keys KCS and KSC can contain separate subkeys for encryption and authentication, as is needed for legacy modes of operation such as CBC plus CBC MAC.

As well as being used to encrypt web traffic, TLS has also been available as an authentication option in Windows from Windows 2000 onwards; you can use it instead of Kerberos for authentication on corporate networks. I will describe its use in more detail in the chapter on network attack and defence.

5.7.5.2 TLS security

Although early versions of SSL had a number of bugs [1977], SSL 3.0 and later appear to be sound; the version after SSL 3.0 was renamed TLS 1.0. It was formally verified by Larry Paulson in 1998, so we know that the idealised version of the protocol doesn't have any bugs [1504].

However, in the more than twenty years since then, there have been over a dozen serious attacks. Even in 1998, Daniel Bleichenbacher came up with the first of a number of attacks based on measuring the time it takes a server to decrypt, or the error messages it returns in response to carefully-crafted protocol responses [265]. TLS 1.1 appeared in 2006 with protection against

exploits of CBC encryption and of padding errors; TLS 1.2 followed two years later, upgrading the hash function to SHA256 and supporting authenticated encryption; and meanwhile there were a number of patches dealing with various attacks that had emerged. Many of these patches were rather inelegant because of the difficulty of changing a widely-used protocol; it's difficult to change both the server and client ends at once, as any client still has to interact with millions of servers, many running outdated software, and most websites want to be able to deal with browsers of all ages and on all sorts of devices. This has been dealt with by the big service firms changing their browsers to reject obsolete ciphersuites, and to add features like *strict transport security* (STS) whereby a website can instruct browsers to only interact with it using https in future (to prevent downgrade attacks). The browser firms have also mandated a number of other supporting measures, from shorter certificate lifetimes to certificate transparency, which we'll discuss in the chapter on network attack and defence.

5.7.5.3 TLS 1.3

The most recent major upgrade to the core protocol, TLS 1.3, was approved by the IETF in January 2019 after two years of discussion. It has dropped backwards compatibility in order to end support for many old ciphers, and made it mandatory to establish end-to-end forward secrecy by means of a Diffie-Hellman key exchange at the start of each session. This has caused controversy with the banking industry, which routinely intercepts encrypted sessions in order to do monitoring for compliance purposes. This will no longer be possible, so banks will have to bear the legal discomfort of using obsolete encryption or the financial cost of redeveloping systems to monitor compliance at endpoints instead¹¹.

5.7.6 Other public-key protocols

Dozens of other public-key protocols have found wide use, including the following, most of which we'll discuss in detail later. Here I'll briefly mention code signing, PGP and QUIC.

5.7.6.1 Code signing

Code signing was introduced in the 1990s when people started downloading software rather than getting it on diskettes. It is now used very widely to

¹¹The COVID-19 pandemic has given some respite: Microsoft had been due to remove support for legacy versions of TLS in spring 2020 but has delayed this.

assure the provenance of software. You might think that having a public signature-verification key in your software so that version N can verify an update to version $N + 1$ would be a simple application of public-key cryptography but this is far from the case. Many platforms sign their operating-system code, including updates, to prevent persistent malware; the mechanisms often involve trusted hardware such as TPMs and I'll discuss them in the next chapter in section 6.2.5. Some platforms, such as the iPhone, will only run signed code; this not only assures the provenance of software but enables platform owners to monetise apps, as I will discuss in section 22.4.2; games consoles are similar. As some users go to great lengths to jailbreak their devices, such platforms typically have trustworthy hardware to store the verification keys. Where that isn't available, verification may be done using code that is obfuscated to make it harder for malware (or customers) to tamper with it; this is a constant arms race, which I discuss in section 24.3.3. As for the signing key, the developer may keep it in a hardware security module, which is expensive and breaks in subtle ways discussed in section 20.5; there may be a chain of trust going back to a commercial CA, but then have to worry about legal coercion by government agencies, which I discuss in section 26.2.7; you might even implement your own CA for peace of mind. In short, code signing isn't quite as easy as it looks, particularly when the user is the enemy.

5.7.6.2 PGP/GPG

During the 'Crypto Wars' in the 1990s, cyber-activists fought governments for the right to encrypt email, while governments pushed for laws restricting encryption; I'll discuss the history and politics in section 26.2.7. The crypto activist Phil Zimmermann wrote an open-source encryption product *Pretty Good Privacy* (PGP) and circumvented U.S. export controls by publishing the source code in a paper book, which could be posted, scanned and compiled. Along with later compatible products such as GPG, it has become fairly widely used among geeks. For example, sysadmins, Computer Emergency Response Teams (CERTs) and malware researchers use it to share information about attacks and vulnerabilities. It has also been built into customised phones sold to criminal gangs to support messaging; I'll discuss this later in section 25.4.1.

PGP has a number of features but, in its most basic form, each user generates private/public keypairs manually and shares public keys with contacts. There are command-line options to sign a message with your signature key and/or encrypt it using the public key of each of the intended recipients. Manual key management avoids the need for a CA that can be cracked or coerced. Many things were learned from the deployment and use of PGP during the 1990s. As I described in section 3.2.1, Alma Whitten and Doug Tygar wrote the seminal paper on security usability by assessing whether motivated but cryptologically

unsophisticated users could understand it well enough to drive the program safely. Only four of twelve subjects were able to correctly send encrypted email to the other subjects, and every subject made at least one significant error.

5.7.6.3 QUIC

QUIC is a new UDP-based protocol designed by Google and promoted as an alternative to TLS that allows quicker session establishment and cutting latency in the ad auctions that happen as pages load; sessions can persist as people move between access points. This is achieved by a cookie that holds the client's last IP address, encrypted by the server. It appeared in Chrome in 2013 and now has about 7% of Internet traffic; it's acquired a vigorous standardisation community. Google claims it reduces search latency 8% and YouTube buffer time 18%. Independent evaluation suggests that the benefit is mostly on the desktop rather than mobile [1009], and there's a privacy concern as the server can use an individual public key for each client, and use this for tracking. As a general principle, one should be wary of corporate attempts to replace open standards with proprietary ones, whether IBM's EBCDIC coding standard of the 1950s and SNA in the 1970s, or Microsoft's attempts to 'embrace and extend' both mail standards and security protocols since the 1990s, or Facebook's promotion of Internet access in Africa that kept users largely within its walled garden. I'll discuss the monopolistic tendencies of our industry at greater length in Chapter 8.

5.7.7 Special-purpose primitives

Researchers have invented a large number of public-key and signature primitives with special properties. Two that have so far appeared in real products are threshold cryptography and blind signatures.

Threshold crypto is a mechanism whereby a signing key, or a decryption key, can be split up among n principals so that any k out of n can sign a message (or decrypt). For $k = n$ the construction is easy. With RSA, for example, you can split up the private key d as $d = d_1 + d_2 + \dots + d_n$. For $k < n$ it's slightly more complex (but not much – you use the Lagrange interpolation formula) [554]. Threshold signatures were first used in systems where a number of servers process transactions independently and vote independently on the outcome; they have more recently been used to implement business rules on cryptocurrency wallets such as 'a payment must be authorised by any two of the seven company directors'.

Blind signatures are a way of making a signature on a message without knowing what the message is. For example, if we are using RSA, I can take a random number R , form $R^e M \pmod n$, and give it to the signer who computes

$(R^e M)^d = R.M^d \pmod{n}$. When he gives this back to me, I can divide out R to get the signature M^d . Now you might ask why on earth someone would want to sign a document without knowing its contents, but there are some applications.

The first was in *digital cash*; you might want to be able to issue anonymous payment tokens to customers, and the earliest idea, due to David Chaum, was a way to sign ‘digital coins’ without knowing their serial numbers [413]. A bank might agree to honour for \$10 any string M with a unique serial number and a specified form of redundancy, bearing a signature that verified as correct using the public key (e, n) . The blind signature protocol ensures a customer can get a bank to sign a coin without the banker knowing its serial number, and it was used in prototype road toll systems. The effect is that the digital cash can be anonymous for the spender. The main problem with digital cash was to detect people who spend the same coin twice, and this was eventually fixed using blockchains or other ledger mechanisms, as I discuss in section 20.7. Digital cash failed to take off because neither banks nor governments really want payments to be anonymous: anti-money-laundering regulations since 9/11 restrict anonymous payment services to small amounts, while both banks and bitcoin miners like to collect transaction fees.

Anonymous digital credentials are now used in attestation: the TPM chip on your PC motherboard might prove something about the software running on your machine without identifying you. Unfortunately, this led to designs for attestation in SGX (and its AMD equivalent) which mean that a single compromised device breaks the whole ecosystem. Anonymous signatures are also found in prototype systems for conducting electronic elections, to which I will return in section 25.5.

5.7.8 How strong are asymmetric cryptographic primitives?

In order to provide the same level of protection as a symmetric block cipher, asymmetric cryptographic primitives generally require at least twice the block length. Elliptic curve systems appear to achieve this bound; a 256-bit elliptic scheme could be about as hard to break as a 128-bit block cipher with a 128-bit key; and the only public-key encryption schemes used in the NSA’s Suite B of military algorithms are 384-bit elliptic curve systems. The traditional schemes, based on factoring and discrete log, now require 3072-bit keys to protect material at Top Secret, as there are shortcut attack algorithms such as the number field sieve. As a result, elliptic curve cryptosystems are faster.

When I wrote the first edition of this book in 2000, the number field sieve had been used to attack keys up to 512 bits, a task comparable in difficulty to keysearch on 56-bit DES keys; by the time I rewrote this chapter for the second edition in 2007, 64-bit symmetric keys had been brute-forced, and the 663-bit challenge number RSA-200 had been factored. By the third edition in 2019,

bitcoin miners are finding 68-bit hash collisions every ten minutes, RSA-768 has been factored and Ed Snowden has as good as told us that the NSA can do discrete logs for a 1024-bit prime modulus.

There has been much research into *quantum computers* – devices that perform a large number of computations simultaneously using superposed quantum states. Peter Shor has shown that if a sufficiently large quantum computer could be built, then both factoring and discrete logarithm computations will become easy [1728]. So far only very small quantum devices have been built; although there are occasional claims of ‘quantum supremacy’ – of a quantum computer performing a task sufficiently faster than a conventional one to convince us that quantum superposition or entanglement is doing any real work – they seem to lead nowhere. I am sceptical (as are many physicists) about whether the technology will ever threaten real systems. I am even more sceptical about the value of quantum cryptography; it may be able to re-key a line encryption device that uses AES for bulk encryption on a single uninterrupted fibre run, but we already know how to do that.

What’s more, I find the security proofs offered for entanglement-based quantum cryptography to be unconvincing. Theoretical physics has been stalled since the early 1970s when Gerard ‘t Hooft completed the Standard Model by proving the renormalisability of Yang-Mills. Since then, a whole series of ideas have come and gone, such as string theory [2035]. Quantum information theory is the latest enthusiasm. Its proponents talk up the mystery of the Bell tests, which are supposed to demonstrate that physics cannot be simultaneously local and causal. But alternative interpretations such as ‘t Hooft’s cellular automaton model [918] and Grisha Volovik’s superfluid model [1971] suggest that the Bell tests merely demonstrate the existence of long-range order in the quantum vacuum, like the order parameter of a superfluid. Since 2005, we’ve had lab experiments involving bouncing droplets on a vibrating fluid bath that demonstrate interesting analogues of quantum-mechanical properties relevant to the Bell tests [1560]. This book is not the place to discuss the implications in more detail; for that, see [312]. There is a whole community of physicists working on emergent quantum mechanics – the idea that to make progress beyond the Standard Model, and to reconcile the apparent conflict between quantum mechanics and general relativity, we may need to look at things differently. Meantime, if anyone claims their system is secure ‘because quantum mechanics’ then scepticism may be in order.

I think it more likely that a major challenge to public-key cryptography could come in the form of a better algorithm for computing discrete logarithms on elliptic curves. These curves have a lot of structure; they are studied intensively by some of the world’s smartest pure mathematicians; better discrete-log algorithms for curves of small characteristic were discovered in 2013 [169]; and the NSA is apparently moving away from using elliptic-curve crypto.

If quantum computers ever work, we have other ‘post-quantum’ algorithms ready to go, for which quantum computers give no obvious advantage. In 2020, NIST began the third round of public review of submissions for the Post-Quantum Cryptography Standardization Process. The 65 initial submissions have been cut to 15 through two rounds of review¹². One or more algorithms will now be chosen and standardised, so ciphersuites using them could be dropped into protocols such as TLS as upgrades. Many protocols in use could even be redesigned to use variants on Kerberos. If elliptic logarithms become easy, we have these resources and can also fall back to discrete logs in prime fields, or to RSA. But if elliptic logs become easy, bitcoins will become trivial to forge, and the cryptocurrency ecosystem would probably collapse, putting an end to the immensely wasteful mining operations I describe in section 20.7. So mathematicians who care about the future of the planet might do worse than to study the elliptic logarithm problem.

5.7.9 What else goes wrong

Very few attacks on systems nowadays involve cryptanalysis in the sense of a mathematical attack on the encryption algorithm or key. There have indeed been attacks on systems designed in the 20th century, mostly involving keys that were kept too short by export-control rules, clueless designs or both. I already discussed in section 4.3.1 how weak crypto has facilitated a wave of car theft, as all the devices used for remote key entry were defeated one after another in 2005–15. In later chapters, I give examples of how the crypto wars and their export control rules resulted in attacks on door locks (section 13.2.5), mobile phones (section 22.3.1) and copyright enforcement (section 24.2.5).

Most attacks nowadays exploit the implementation. In chapter 2, I mentioned the scandal of NIST standardising a complicated random number generator based on elliptic curves that turned out to contain an NSA back-door; see section 2.2.1.5. Poor random number generators have led to many other failures: RSA keys with common factors [1142], predictable seeds for discrete logs [1679], etc. These vulnerabilities have continued; thanks to the Internet of Things, the proportion of RSA certs one can find out there on the Internet that share a common factor with other RSA keys has actually risen between 2012 and 2020; 1 in 172 IoT certs are trivially vulnerable [1048].

Many of the practical attacks on cryptographic implementations that have forced significant changes over the past 20 years have exploited side channels such as timing and power analysis; I devote Chapter 19 to these.

¹²One of them, the McEliece cryptosystem, has been around since 1978; we’ve had digital signatures based on hash functions for about as long, and some of us used them in the 1990s to avoid paying patent royalties on RSA.

In Chapter 20, I'll discuss a number of systems that use public-key mechanisms in intricate ways to get interesting emergent properties, including the Signal messaging protocol, the TOR anonymity system, and cryptocurrencies. I'll also look at the crypto aspects of SGX enclaves. These also have interesting failure modes, some but not all of them relating to side channels.

In Chapter 21, I'll discuss protocols used in network infrastructure such as DKIM, DNSSec versus DNS over HTTP, and SSH.

5.8 Summary

Many ciphers fail because they're used badly, so the security engineer needs a clear idea of what different types of cipher do. This can be tackled at different levels; one is at the level of crypto theory, where we can talk about the random oracle model, the concrete model and the semantic security model, and hopefully avoid using weak modes of operation and other constructions. The next level is that of the design of individual ciphers, such as AES, or the number-theoretic mechanisms that underlie public-key cryptosystems and digital signature mechanisms. These also have their own specialised fields of mathematics, namely block cipher cryptanalysis and computational number theory. The next level involves implementation badness, which is much more intractable and messy. This involves dealing with timing, error handling, power consumption and all sorts of other grubby details, and is where modern cryptosystems tend to break in practice.

Peering under the hood of real systems, we've discussed how block ciphers for symmetric key applications can be constructed by the careful combination of substitutions and permutations; for asymmetric applications such as public key encryption and digital signature one uses number theory. In both cases, there is quite a large body of mathematics. Other kinds of ciphers – stream ciphers and hash functions – can be constructed from block ciphers by using them in suitable modes of operation. These have different error propagation, pattern concealment and integrity protection properties. A lot of systems fail because popular crypto libraries encourage programmers to use inappropriate modes of operation by exposing unsafe defaults. Never use ECB mode unless you really understand what you're doing.

There are many other things that can go wrong, from side channel attacks to poor random number generators. In particular, it is surprisingly hard to build systems that are robust even when components fail (or are encouraged to) and where the cryptographic mechanisms are well integrated with other measures such as access control and physical security. I'll return to this repeatedly in later chapters.

The moral is: Don't roll your own! Don't design your own protocols, or your own ciphers; and don't write your own crypto code unless you absolutely have

to. If you do, then you not only need to read this book (and then read it again, carefully); you need to read up the relevant specialist material, speak to experts, and have capable motivated people try to break it. At the very least, you need to get your work peer-reviewed. Designing crypto is a bit like juggling chainsaws; it's just too easy to make fatal errors.

Research problems

There are many active threads in cryptography research. Many of them are where crypto meets a particular branch of mathematics (number theory, algebraic geometry, complexity theory, combinatorics, graph theory, and information theory). The empirical end of the business is concerned with designing primitives for encryption, signature and composite operations, and which perform reasonably well on available platforms. The two meet in the study of subjects ranging from cryptanalysis, to the search for primitives that combine provable security properties with decent performance.

The best way to get a flavor of what's going on at the theoretical end of things is to read the last few years' proceedings of research conferences such as Crypto, Eurocrypt and Asiacrypt; work on cipher design appears at Fast Software Encryption; attacks on implementations often appear at CHES; while attacks on how crypto gets used in systems can be found in the systems security conferences such as IEEE Security and Privacy, CCS and Usenix.

Further reading

The classic papers by Whit Diffie and Martin Hellman [556] and by Ron Rivest, Adi Shamir and Len Adleman [1610] are the closest to required reading in this subject. Bruce Schneier's *Applied Cryptography* [1670] covers a lot of ground at a level a non-mathematician can understand, and got crypto code out there in the 1990s despite US export control laws, but is now slightly dated. Alfred Menezes, Paul van Oorshot and Scott Vanstone's *Handbook of Applied Cryptography* [1291] is one reference book on the mathematical detail. Katz and Lindell is the book we get our students to read for the math. It gives an introduction to the standard crypto theory plus the number theory you need for public-key crypto (including elliptic curves and index calculus) but is also dated: they don't mention GCM, for example [1025].

There are many more specialist books. The bible on differential cryptanalysis is by its inventors Eli Biham and Adi Shamir [246], while a good short tutorial on linear and differential cryptanalysis was written by Howard Heys [897]. Doug Stinson's textbook has another detailed explanation of linear cryptanalysis [1832]; and the modern theory of block ciphers can be traced through the

papers in the *Fast Software Encryption* conference series. The original book on modes of operation is by Carl Meyer and Steve Matyas [1303]. Neal Koblitz has a good basic introduction to the mathematics behind public key cryptography [1062]; and the number field sieve is described by Arjen and Henrik Lenstra [1143]. For the practical attacks on TLS over the past twenty years, see the survey paper by Christopher Meyer and Joerg Schwenk [1304] as well as the chapter on Side Channels later in this book.

If you want to work through the mathematical detail of theoretical cryptology, there's an recent graduate textbook by Dan Boneh and Victor Shoup [288]. A less thorough but more readable introduction to randomness and algorithms is in [836]. Research at the theoretical end of cryptology is found at the FOCS, STOC, Crypto, Eurocrypt and Asiacrypt conferences.

The history of cryptology is fascinating, and so many old problems keep on recurring that anyone thinking of working with crypto should study it. The standard work is Kahn [1003]; there are also compilations of historical articles from *Cryptologia* [529–531] as well as several books on the history of cryptology in World War II by Kahn, Marks, Welchman and others [440, 1004, 1226, 2011]. The NSA Museum at Fort George Meade, Md., is also worth a visit, but perhaps the best is the museum at Bletchley Park in England.

Finally, no chapter that introduces public key encryption would be complete without a mention that, under the name of 'non-secret encryption,' it was first discovered by James Ellis in about 1969. However, as Ellis worked for GCHQ, his work remained classified. The RSA algorithm was then invented by Clifford Cocks, and also kept secret. This story is told in [626]. One effect of the secrecy was that their work was not used: although it was motivated by the expense of Army key distribution, Britain's Ministry of Defence did not start building electronic key distribution systems for its main networks until 1992. And the classified community did not pre-invent digital signatures; they remain the achievement of Whit Diffie and Martin Hellman.

Access Control

Anything your computer can do for you it can potentially do for someone else.

– ALAN COX

Microsoft could have incorporated effective security measures as standard, but good sense prevailed. Security systems have a nasty habit of backfiring and there is no doubt they would cause enormous problems.

– RICK MAYBURY

6.1 Introduction

I first learned to program on an IBM mainframe whose input was punched cards and whose output was a printer. You queued up with a deck of cards, ran the job, and went away with printout. All security was physical. Then along came machines that would run more than one program at once, and the *protection problem* of preventing one program from interfering with another. You don't want a virus to steal the passwords from your browser, or patch a banking application so as to steal your money. And many reliability problems stem from applications misunderstanding each other, or fighting with each other. But it's tricky to separate applications when the customer wants them to share data. It would make phishing much harder if your email client and browser ran on separate machines, so you were unable to just click on URLs in emails, but that would make life too hard.

From the 1970s, access control became the centre of gravity of computer security. It's where security engineering meets computer science. Its function is to control which principals (persons, processes, machines, . . .) have access to which resources in the system – which files they can read, which programs they can execute, how they share data with other principals, and so on. It's become horrendously complex. If you start out by leafing through the 7000-plus pages of Arm's architecture reference manual or the equally complex arrangements

for Windows, your first reaction might be ‘I wish I’d studied music instead!’ In this chapter I try to help you make sense of it all.

Access control works at a number of different levels, including at least:

1. Access controls at the application level may express a very rich, domain-specific security policy. The call centre staff in a bank are typically not allowed to see your account details until you have answered a couple of security questions; this not only stops outsiders impersonating you, but also stops the bank staff looking up the accounts of celebrities, or their neighbours. Some transactions might also require approval from a supervisor. And that’s nothing compared with the complexity of the access controls on a modern social networking site, which will have a thicket of rules about who can see, copy, and search what data from whom, and privacy options that users can set to modify these rules.
2. The applications may be written on top of middleware, such as a web browser, a bank’s bookkeeping system or a social network’s database management system. These enforce a number of protection properties. For example, bookkeeping systems ensure that a transaction that debits one account must credit another, with the debits and credits balancing so that money cannot be created or destroyed; they must also allow the system’s state to be reconstructed later.
3. As the operating system constructs resources such as files and communications ports from lower level components, it has to provide ways to control access to them. Your Android phone treats apps written by different companies as different users and protects their data from each other. The same happens when a shared server separates the VMs, containers or other resources belonging to different users.
4. Finally, the operating system relies on the processor and its associated memory-management hardware, which control which memory addresses a given process or thread can access.

As we work up from the hardware through the operating system and middleware to the application layer, the controls become progressively more complex and less reliable. And we find the same access-control functions being implemented at multiple layers. For example, the separation between different phone apps that is provided by Android is mirrored in your browser which separates web page material according to the domain name it came from (though this separation is often less thorough). And the access controls built at the application layer or the middleware layer may largely duplicate access controls in the underlying operating system or hardware. It can get very messy, and to make sense of it we need to understand the underlying principles, the common architectures, and how they have evolved.

I will start off by discussing operating-system protection mechanisms that support the isolation of multiple processes. These came first historically – being invented along with the first time-sharing systems in the 1960s – and they remain the foundation on which many higher-layer mechanisms are built, as well as inspiring similar mechanisms at higher layers. They are often described as *discretionary access control* (DAC) mechanisms, which leave protection to the machine operator, or *mandatory access control* (MAC) mechanisms which are typically under the control of the vendor and protect the operating system itself from being modified by malware. I'll give an introduction to software attacks and techniques for defending against them – MAC, ASLR, sandboxing, virtualisation and what can be done with hardware. Modern hardware not only provides CPU support for virtualisation and capabilities, but also hardware support such as TPM chips for trusted boot to stop malware being persistent. These help us tackle the toxic legacy of the old single-user PC operating systems such as DOS and Win95/98 which let any process modify any data, and constrain the many applications that won't run unless you trick them into thinking that they are running with administrator privileges.

6.2 Operating system access controls

The access controls provided with an operating system typically authenticate principals using a mechanism such as passwords or fingerprints in the case of phones, or passwords or security protocols in the case of servers, then authorise access to files, communications ports and other system resources.

Access controls can often be modeled as a matrix of access permissions, with columns for files and rows for users. We'll write *r* for permission to read, *w* for permission to write, *x* for permission to execute a program, and *-* for no access at all, as shown in Figure 6.1.

In this simplified example, Sam is the system administrator and has universal access (except to the audit trail, which even he should only be able to read). Alice, the manager, needs to execute the operating system and application, but only through the approved interfaces – she mustn't have the ability to tamper

	Operating System	Accounts Program	Accounting Data	Audit Trail
Sam	rw x	rw x	rw	r
Alice	x	x	rw	–
Bob	r x	r	r	r

Figure 6.1: Naive access control matrix

with them. She also needs to read and write the data. Bob, the auditor, can read everything.

This is often enough, but in the specific case of a bookkeeping system it's not quite what we need. We want to ensure that transactions are well-formed – that each debit is balanced by credits somewhere else – so we don't want Alice to have uninhibited write access to the account file. We would also rather that Sam didn't have this access. So we would prefer that write access to the accounting data file be possible only via the accounting program. The access permissions might now look like in Figure 6.2:

User	Operating System	Accounts Program	Accounting Data	Audit Trail
Sam	rwX	rwX	r	r
Alice	rx	x	–	–
Accounts program	rx	rx	rw	w
Bob	rx	r	r	r

Figure 6.2: Access control matrix for bookkeeping

Another way of expressing a policy of this type would be with *access triples* of (*user, program, file*). In the general case, our concern isn't with a program so much as a *protection domain* which is a set of processes or threads that share access to the same resources.

Access control matrices (whether in two or three dimensions) can be used to implement protection mechanisms as well as just model them. But they don't scale well: a bank with 50,000 staff and 300 applications would have a matrix of 15,000,000 entries, which might not only impose a performance overhead but also be vulnerable to administrators' mistakes. We will need a better way of storing and managing this information, and the two main options are to compress the users and to compress the rights. With the first, we can use groups or roles to manage large sets of users simultaneously, while with the second we may store the access control matrix either by columns (access control lists) or rows (capabilities, also known as 'tickets' to protocol engineers and 'permissions' on mobile phones) [1642, 2024].

6.2.1 Groups and roles

When we look at large organisations, we usually find that most staff fit into one of a small number of categories. A bank might have 40 or 50: teller, call centre operator, loan officer and so on. Only a few dozen people (security

manager, chief foreign exchange dealer, ...) will need personally customised access rights.

So we need to design a set of groups, or functional roles, to which staff can be assigned. Some vendors (such as Microsoft) use the words *group* and *role* almost interchangeably, but a more careful definition is that a group is a list of principals, while a role is a fixed set of access permissions that one or more principals may assume for a period of time. The classic example of a role is the officer of the watch on a ship. There is exactly one watchkeeper at any one time, and there is a formal procedure whereby one officer relieves another when the watch changes. In most government and business applications, it's the role that matters rather than the individual.

Groups and roles can be combined. *The officers of the watch of all ships currently at sea* is a group of roles. In banking, the manager of the Cambridge branch might have their privileges expressed by membership of the group *manager* and assumption of the role *acting manager of Cambridge branch*. The group *manager* might express a rank in the organisation (and perhaps even a salary band) while the role *acting manager* might include an assistant accountant standing in while the manager, deputy manager, and branch accountant are all off sick.

Whether we need to be careful about this distinction is a matter for the application. In a warship, even an ordinary seaman may stand watch if everyone more senior has been killed. In a bank, we might have a policy that “transfers over \$10m must be approved by two staff, one with rank at least manager and one with rank at least assistant accountant”. If the branch manager is sick, then the assistant accountant acting as manager might have to get the regional head office to provide the second signature on a large transfer.

6.2.2 Access control lists

The traditional way to simplify the management of access rights is to store the access control matrix a column at a time, along with the resource to which the column refers. This is called an *access control list* or ACL (pronounced ‘ackle’). In the first of our above examples, the ACL for file 3 (the account file) might look as shown here in Figure 6.3.

User	Accounting Data
Sam	rw
Alice	rw
Bob	r

Figure 6.3: Access control list (ACL)

ACLs have a number of advantages and disadvantages as a means of managing security state. They are a natural choice in environments where users manage their own file security, and became widespread in Unix systems from the 1970s. They are the basic access control mechanism in Unix-based systems such as Linux and Apple's macOS, as well as in derivatives such as Android and iOS. The access controls in Windows were also based on ACLs, but have become more complex over time. Where access control policy is set centrally, ACLs are suited to environments where protection is data-oriented; they are less suited where the user population is large and constantly changing, or where users want to be able to delegate their authority to run a particular program to another user for some set period of time. ACLs are simple to implement, but are not efficient for security checking at runtime, as the typical operating system knows which user is running a particular program, rather than what files it has been authorized to access since it was invoked. The operating system must either check the ACL at each file access, or keep track of the active access rights in some other way.

Finally, distributing the access rules into ACLs makes it tedious to find all the files to which a user has access. Verifying that no files have been left world-readable or even world-writable could involve checking ACLs on millions of user files; this is a real issue for large complex firms. Although you can write a script to check whether any file on a server has ACLs that breach a security policy, you can be tripped up by technology changes; the move to containers has led to many corporate data exposures as admins forgot to check the containers' ACLs too. (The containers themselves are often dreadful as it's a new technology being sold by dozens of clueless startups.) And revoking the access of an employee who has just been fired will usually have to be done by cancelling their password or authentication token.

Let's look at an important example of ACLs – their implementation in Unix (plus its derivatives Android, MacOS and iOS).

6.2.3 Unix operating system security

In traditional Unix systems, files are not allowed to have arbitrary access control lists, but simply `rxw` attributes that allow the file to be read, written and executed. The access control list as normally displayed has a flag to show whether the file is a directory, then flags `r`, `w` and `x` for owner, group and world respectively; it then has the owner's name and the group name. A directory with all flags set would have the ACL:

```
drwxrwxrwx Alice Accounts
```

In our first example in Figure 6.1, the ACL of file 3 would be:

```
-rw-r----- Alice Accounts
```

This records that the file is simply a file rather than a directory; that the file owner can read and write it; that group members (including Bob) can read it but not write it; that non-group members have no access at all; that the file owner is Alice; and that the group is Accounts.

The program that gets control when the machine is booted (the operating system kernel) runs as the supervisor, and has unrestricted access to the whole machine. All other programs run as users and have their access mediated by the supervisor. Access decisions are made on the basis of the `userid` associated with the program. However if this is zero (`root`), then the access control decision is 'yes'. So `root` can do what it likes – access any file, become any user, or whatever. What's more, there are certain things that only `root` can do, such as starting certain communication processes. The `root` `userid` is typically made available to the system administrator in systems with discretionary access control.

This means that the system administrator can do anything, so we have difficulty implementing an audit trail as a file that they cannot modify. In our example, Sam could tinker with the accounts, and have difficulty defending himself if he were falsely accused of tinkering; what's more, a hacker who managed to become the administrator could remove all evidence of his intrusion. The traditional, and still the most common, way to protect logs against `root` compromise is to keep them separate. In the old days that meant sending the system log to a printer in a locked room; nowadays, it means sending it to another machine, or even to a third-party service. Increasingly, it may also involve mandatory access control, as we discuss later.

Second, ACLs only contain the names of users, not of programs; so there is no straightforward way to implement access triples of (user, program, file). Instead, Unix provides an indirect method: the *set-user-id* (`suid`) file attribute. The owner of a program can mark the file representing that program as `suid`, which enables it to run with the privilege of its owner rather than the privilege of the user who has invoked it. So in order to achieve the functionality needed by our second example above, we could create a user 'account-package' to own file 2 (the accounts package), make the file `suid` and place it in a directory to which Alice has access. This special user can then be given the access that the accounts program needs.

But when you take an access control problem that has three dimensions – (user, program, data) – and implement it using two-dimensional mechanisms, the outcome is much less intuitive than triples and people are liable to make mistakes. Programmers are often lazy or facing tight deadlines; so they just make the application `suid root`, so it can do anything. This practice leads to some shocking security holes. The responsibility for making access control decisions is moved from the operating system environment to the application program, and most programmers are insufficiently experienced to check everything they should. (It's hard to know what to check, as the person invoking

a `suid root` program controls its environment and could manipulate this in unexpected ways.)

Third, ACLs are not very good at expressing mutable state. Suppose we want a transaction to be authorised by a manager and an accountant before it's acted on; we can either do this at the application level (say, by having queues of transactions awaiting a second signature) or by doing something fancy with `suid`. Managing stateful access rules is difficult; they can complicate the revocation of users who have just been fired, as it can be hard to track down the files they've opened, and stuff can get stuck.

Fourth, the Unix ACL only names one user. If a resource will be used by more than one of them, and you want to do access control at the OS level, you have a couple of options. With older systems you had to use groups; newer systems implement the Posix system of extended ACLs, which may contain any number of named user and named group entities. In theory, the ACL and `suid` mechanisms can often be used to achieve the desired effect. In practice, programmers are often in too much of a hurry to figure out how to do this, and security interfaces are usually way too fiddly to use. So people design their code to require much more privilege than it strictly ought to have, as that seems to be the only way to get the job done.

6.2.4 Capabilities

The next way to manage the access control matrix is to store it by rows. These are called *capabilities*, and in our example in Figure 6.1 above, Bob's capabilities would be as in Figure 6.4 here:

User	Operating System	Accounts Program	Accounting Data	Audit Trail
Bob	rx	r	r	r

Figure 6.4: A capability

The strengths and weaknesses of capabilities are roughly the opposite of ACLs. Runtime security checking is more efficient, and we can delegate a right without much difficulty: Bob could create a certificate saying 'Here is my capability and I hereby delegate to David the right to read file 4 from 9am to 1pm, signed Bob'. On the other hand, changing a file's status becomes tricky as it can be hard to find out which users have access. This can be tiresome when we have to investigate an incident or prepare evidence. In fact, scalable systems end up using de-facto capabilities internally, as instant system-wide revocation is just too expensive; in Unix, file descriptors are really capabilities, and

continue to grant access for some time even after ACL permissions or even file owners change. In a distributed Unix, access may persist for the lifetime of Kerberos tickets.

Could we do away with ACLs entirely then? People built experimental machines in the 1970s that used capabilities throughout [2024]; the first commercial product was the Plessey System 250, a telephone-switch controller [1578]. The IBM AS/400 series systems brought capability-based protection to the mainstream computing market in 1988, and enjoyed some commercial success. The public key certificates used in cryptography are in effect capabilities, and became mainstream from the mid-1990s. Capabilities have started to supplement ACLs in operating systems, including more recent versions of Windows, FreeBSD and iOS, as I will describe later.

In some applications, they can be the natural way to express security policy. For example, a hospital may have access rules like ‘a nurse shall have access to all the patients who are on his or her ward, or who have been there in the last 90 days’. In early systems based on traditional ACLs, each access control decision required a reference to administrative systems to find out which nurses and which patients were on which ward, when – but this made both the HR system and the patient administration system safety-critical, which hammered reliability. Matters were fixed by giving nurses ID cards with certificates that entitle them to access the files associated with a number of wards or hospital departments [535, 536]. If you can make the trust relationships in systems mirror the trust relationships in that part of the world you’re trying to automate, you should. Working with the grain can bring advantages at all levels in the stack, making things more usable, supporting safer defaults, cutting errors, reducing engineering effort and saving money too.

6.2.5 DAC and MAC

In the old days, anyone with physical access to a computer controlled all of it: you could load whatever software you liked, inspect everything in memory or on disk and change anything you wanted to. This is the model behind *discretionary access control* (DAC): you start your computer in supervisor mode and then, as the administrator, you can make less-privileged accounts available for less-trusted tasks – such as running apps written by companies you don’t entirely trust, or giving remote logon access to others. But this can make things hard to manage at scale, and in the 1970s the US military started a huge computer-security research program whose goal was to protect classified information: to ensure that a file marked ‘Top Secret’ would never be made available to a user with only a ‘Secret’ clearance, regardless of the actions of any ordinary user or even of the supervisor. In such a *multilevel secure* (MLS) system, the sysadmin is no longer the boss: ultimate control rests with a remote

government authority that sets security policy. The mechanisms started to be described as *mandatory access control* (MAC). The supervisor, or root access if you will, is under remote control. This drove development of technology for mandatory access control – a fascinating story, which I tell in Part 2 of the book.

From the 1980s, safety engineers also worked on the idea of *safety integrity levels*; roughly, that a more dependable system must not rely on a less dependable one. They started to realise they needed something similar to multilevel security, but for safety. Military system people also came to realise that the tamper-resistance of the protection mechanisms themselves was of central importance. In the 1990s, as computers and networks became fast enough to handle audio and video, the creative industries lobbied for *digital rights management* (DRM) in the hope of preventing people undermining their business models by sharing music and video. This is also a form of mandatory access control – stopping a subscriber sharing a song with a non-subscriber is in many ways like stopping a Top Secret user sharing an intelligence report with a Secret user.

In the early 2000s, these ideas came together as a number of operating-system vendors started to incorporate ideas and mechanisms from the MAC research programme into their products. The catalyst was an initiative by Microsoft and Intel to introduce cryptography into the PC platform to support DRM. Intel believed the business market for PCs was saturated, so growth would come from home sales where, they believed, DRM would be a requirement. Microsoft started with DRM and then realised that offering rights management for documents too might be a way of locking customers tightly into Windows and Office. They set up an industry alliance, now called the Trusted Computing Group, to introduce cryptography and MAC mechanisms into the PC platform. To do this, the operating system had to be made tamper-resistant, and this is achieved by means of a separate processor, the Trusted Platform Module (TPM), basically a smartcard chip mounted on the PC motherboard to support trusted boot and hard disk encryption. The TPM monitors the boot process, and at each stage a hash of everything loaded so far is needed to retrieve the key needed to decrypt the next stage. The real supervisor on the system is now no longer you, the machine owner – it's the operating-system vendor.

MAC, based on TPMs and trusted boot, was used in Windows 6 (Vista) from 2006 as a defence against persistent malware¹. The TPM standards and architecture were adapted by other operating-system vendors and device OEMs,

¹Microsoft had had more ambitious plans; its project Palladium would have provided a new, more trusted world for rights-management apps, alongside the normal one for legacy software. They launched Information Rights Management – DRM for documents – in 2003 but corporates didn't buy it, seeing it as a lock-in play. A two-world implementation turned out to be too complex for Vista and after two separate development efforts it was abandoned; but the vision persisted from 2004 in Arm's TrustZone, which I discuss below.

and there is now even a project for an open-source TPM chip, OpenTitan, based on Google's product. However the main purpose of such a design, whether the design itself is open or closed, is to lock a hardware device to using specific software.

6.2.6 Apple's macOS

Apple's macOS operating system (formerly called OS/X or Mac OS X) is based on the FreeBSD version of Unix running on top of the Mach kernel. The BSD layer provides memory protection; applications cannot access system memory (or each others') unless running with advanced permissions. This means, for example, that you can kill a wedged application using the 'Force Quit' command without having to reboot the system. On top of this Unix core are a number of graphics components, including OpenGL, Quartz, QuickTime and Carbon, while at the surface the Aqua user interface provides an elegant and coherent view to the user.

At the file system level, macOS is almost a standard Unix. The default installation has the root account disabled, but users who may administer the system are in a group 'wheel' that allows them to su to root. If you are such a user, you can install programs (you are asked for the root password when you do so). Since version 10.5 (Leopard), it has been based on TrustedBSD, a variant of BSD that incorporates mandatory access control mechanisms, which are used to protect core system components against tampering by malware.

6.2.7 iOS

Since 2008, Apple has led the smartphone revolution with the iPhone, which (along with other devices like the iPad) uses the iOS operating system – which is now (in 2020) the second-most popular. iOS is based on Unix; Apple took the Mach kernel from CMU and fused it with the FreeBSD version of Unix, making a number of changes for performance and robustness. For example, in vanilla Unix a filename can have multiple pathnames that lead to an inode representing a file object, which is what the operating system sees; in iOS, this has been simplified so that files have unique pathnames, which in turn are the subject of the file-level access controls. Again, there is a MAC component, where mechanisms from Domain and Type Enforcement (DTE) are used to tamper-proof core system components (we'll discuss DTE in more detail in chapter 9). Apple introduced this because they were worried that apps would brick the iPhone, leading to warranty claims.

Apps also have *permissions*, which are capabilities; they request a capability to access device services such as the mobile network, the phone, SMSes, the camera, and the first time the app attempts to use such a service. This is granted

if the user consents². The many device services open up possible side-channel attacks; for example, an app that's denied access to the keyboard could deduce keypresses using the accelerometer and gyro. We'll discuss side channels in Part 2, in the chapter on that subject.

The Apple ecosystem is closed in the sense that an iPhone will only run apps that Apple has signed³. This enables the company to extract a share of app revenue, and also to screen apps for malware or other undesirable behaviour, such as the exploitation of side channels to defeat access controls.

The iPhone 5S introduced a fingerprint biometric and payments, adding a *secure enclave* (SE) to the A7 processor to give them separate protection. Apple decided to trust neither iOS nor TrustZone with such sensitive data, since vulnerabilities give transient access until they're patched. Its engineers also worried that an unpatchable exploit might be found in the ROM (this eventually happened, with Checkm8). While iOS has access to the system partition, the user's personal data are encrypted, with the keys managed by the SE. Key management is bootstrapped by a unique 256-bit AES key burned into fusible links on the system-on-chip. When the device is powered up, the user has ten tries to enter a passcode; only then are file keys derived from the master key and made available⁴. When the device is locked, some keys are still usable so that iOS can work out who sent an incoming message and notify you; the price of this convenience is that forensic equipment can get some access to user data. The SE also manages upgrades and prevents rollbacks. Such public information as there is can be found in the iOS Security white paper [129].

The security of mobile devices is a rather complex issue, involving not just access controls and tamper resistance, but the whole ecosystem – from the provision of SIM cards through the operation of app stores to the culture of how people use devices, how businesses try to manipulate them and how government agencies spy on them. I will discuss this in detail in the chapter on phones in Part 2.

6.2.8 Android

Android is the world's most widely used operating system, with 2.5 billion active Android devices in May 2019, according to Google's figures. Android

²The trust-on-first-use model goes back to the 1990s with the Java standard J2ME, popularised by Symbian, and the Resurrecting Duckling model from about the same time. J2ME also supported trust-on-install and more besides. When Apple and Android came along, they initially made different choices. In each case, having an app store was a key innovation; Nokia failed to realise that this was important to get a two-sided market going. The app store does some of the access control by deciding what apps can run. This is hard power in Apple's case, and soft power in Android's; we'll discuss this in the chapter on phones.

³There are a few exceptions: corporates can get signing keys for internal apps, but these can be blacklisted if abused.

⁴I'll discuss fusible links in the chapter on tamper resistance, and iPhone PIN retry defeats in the chapter on surveillance and privacy.

is based on Linux; apps from different vendors run under different userids. The Linux mechanisms control access at the file level, preventing one app from reading another's data and exhausting shared resources such as memory and CPU. As in iOS, apps have *permissions*, which are in effect capabilities: they grant access to device services such as SMSes, the camera and the address book.

Apps come in signed packages, as .apk files, and while iOS apps are signed by Apple, the verification keys for Android come in self-signed certificates and function as the developer's name. This supports integrity of updates while maintaining an open ecosystem. Each package contains a manifest that demands a set of permissions, and users have to approve the 'dangerous' ones – roughly, those that can spend money or compromise personal data. In early versions of Android, the user would have to approve the lot on installation or not run the app. But experience showed that most users would just click on anything to get through the installation process, and you found even flashlight apps demanding access to your address book, as they could sell it for money. So Android 6 moved to the Apple model of trust on first use; apps compiled for earlier versions still demand capabilities on installation.

Since Android 5, SELinux has been used to harden the operating system with mandatory access controls, so as not only to protect core system functions from attack but also to separate processes strongly and log violations. SELinux was developed by the NSA to support MAC in government systems; we'll discuss it further in chapter 9. The philosophy is actions require the consent of three parties: the user, the developer and the platform.

As with iOS (and indeed Windows), the security of Android is a matter of the whole ecosystem, not just of the access control mechanisms. The new phone ecosystem is sufficiently different from the old PC ecosystem, but inherits enough of the characteristics of the old wireline phone system, that it merits a separate discussion in the chapter on Phones in Part Two. We'll consider other aspects in the chapters on Side Channels and Surveillance.

6.2.9 Windows

The current version of Windows (Windows 10) appears to be the third-most popular operating system, having achieved a billion monthly active devices in March 2020 (until 2016, Windows was the leader). Windows has a scarily complex access control system, and a quick canter through its evolution may make it easier to understand what's going on.

Early versions of Windows had no access control. A break came with Windows 4 (NT), which was very much like Unix, and was inspired by it, but with some extensions. First, rather than just *read*, *write* and *execute* there were separate attributes for *take ownership*, *change permissions* and *delete*, to support more flexible delegation. These attributes apply to groups as well as users, and group

permissions allow you to achieve much the same effect as `suid` programs in Unix. Attributes are not simply on or off, as in Unix, but have multiple values: you can set *AccessDenied*, *AccessAllowed* or *SystemAudit*. These are parsed in that order: if an *AccessDenied* is encountered in an ACL for the relevant user or group, then no access is permitted regardless of any conflicting *AccessAllowed* flags. The richer syntax lets you arrange matters so that everyday configuration tasks, such as installing printers, don't have to require full administrator privileges.

Second, users and resources can be partitioned into domains with distinct administrators, and trust can be inherited between domains in one direction or both. In a typical large company, you might put all the users into a personnel domain administered by HR, while assets such as servers and printers may be in resource domains under departmental control; individual workstations may even be administered by their users. Things can be arranged so that the departmental resource domains trust the user domain, but not vice versa – so a hacked or careless departmental administrator can't do too much external damage. The individual workstations would in turn trust the department (but not vice versa) so that users can perform tasks that require local privilege (such as installing software packages). Limiting the damage a hacked administrator can do still needs careful organisation. The data structure used to manage all this, and hide the ACL details from the user interface, is called the *Registry*. Its core used to be the *Active Directory*, which managed remote authentication – using either a Kerberos variant or TLS, encapsulated behind the *Security Support Provider Interface* (SSPI), which enables administrators to plug in other authentication services. Active Directory is essentially a database that organises users, groups, machines, and organisational units within a domain in a hierarchical namespace. It lurked behind Exchange, but is now being phased out as Microsoft becomes a cloud-based company and moves its users to Office365.

Windows has added capabilities in two ways which can override or complement ACLs. First, users or groups can be either allowed or denied access by means of profiles. Security policy is set by groups rather than for the system as a whole; group policy overrides individual profiles, and can be associated with sites, domains or organisational units, so it can start to tackle complex problems. Policies can be created using standard tools or custom coded.

The second way in which capabilities insinuate their way into Windows is that in many applications, people use TLS for authentication, and TLS certificates provide another, capability-oriented, layer of access control outside the purview of the Active Directory.

I already mentioned that Windows Vista introduced trusted boot to make the operating system itself tamper-resistant, in the sense that it always boots into a known state, limiting the persistence of malware. It added three further protection mechanisms to get away from the previous default of all software running

as root. First, the kernel was closed off to developers; second, the graphics subsystem and most drivers were removed from the kernel; and third, *User Account Control* (UAC) replaced the default administrator privilege with user defaults instead. Previously, so many routine tasks needed administrative privilege that many enterprises made all their users administrators, which made it difficult to contain malware; and many developers wrote their software on the assumption that it would have access to everything. According to Microsoft engineers, this was a major reason for Windows' lack of robustness: applications monkey with system resources in incompatible ways. So they added an Application Information Service that launches applications which require elevated privilege and uses virtualisation to contain them: if they modify the registry, for example, they don't modify the 'real' registry but simply the version of it that they can see.

Since Vista, the desktop acts as the parent process for later user processes, so even administrators browse the web as normal users, and malware they download can't overwrite system files unless given later authorisation. When a task requires admin privilege, the user gets an *elevation prompt* asking them for an admin password. (Apple's macOS is similar although the details under the hood differ somewhat.) As admin users are often tricked into installing malicious software, Vista added mandatory access controls in the form of file integrity levels. The basic idea is that low-integrity processes (such as code you download from the Internet) should not be able to modify high-integrity data (such as system files) in the absence of some trusted process (such as verification of a signature by Microsoft on the code in question).

In 2012, Windows 8 added *dynamic access control* which lets you control user access by context, such as their work PC versus their home PC and their phone; this is done via account attributes in Active Directory, which appear as claims about a user, or in Kerberos tickets as claims about a domain. In 2016, Windows 8.1 added a cleaner abstraction with *principals*, which can be a user, computer, process or thread running in a security context or a group to which such a principal belongs, and *security identifiers* (SIDs), which represent such principals. When a user signs in, they get tickets with the SIDs to which they belong. Windows 8.1 also prepared for the move to cloud computing by adding *Microsoft accounts* (formerly LiveID), whereby a user signs in to a Microsoft cloud service rather than to a local server. Where credentials are stored locally, it protects them using virtualisation. Finally, Windows 10 added a number of features to support the move to cloud computing with a diversity of client devices, ranging from certificate pinning (which we'll discuss in the chapter on Network Security) to the abolition of the old secure attention sequence ctrl-alt-del (which is hard to do on touch-screen devices and which users didn't understand anyway).

To sum up, Windows evolved to provide a richer and more flexible set of access control tools than any system previously sold in mass markets. It was

driven by corporate customers who need to manage tens of thousands of staff performing hundreds of different job roles across hundreds of different sites, providing internal controls to limit the damage that can be done by small numbers of dishonest staff or infected machines. (How such controls are actually designed will be our topic in the chapter on Banking and Bookkeeping.) The driver for this development was the fact that Microsoft made over half of its revenue from firms that licensed more than 25,000 seats; but the cost of the flexibility that corporate customers demanded is complexity. Setting up access control for a big Windows shop is a highly skilled job.

6.2.10 Middleware

Doing access control at the level of files and programs was fine in the early days of computing, when these were the resources that mattered. Since the 1980s, growing scale and complexity has led to access control being done at other levels instead of (or as well as) at the operating system level. For example, bookkeeping systems often run on top of a database product such as Oracle, which looks to the operating system as one large file. So most of the access control has to be done in the database; all the operating system supplies may be an authenticated ID for each user who logs on. And since the 1990s, a lot of the work at the client end has been done by the web browser.

6.2.10.1 Database access controls

Before people started using websites for shopping, database security was largely a back-room concern. But enterprises now have critical databases to handle inventory, dispatch and e-commerce, fronted by web servers that pass transactions to the databases directly. These databases now contain much of the data that matter to our lives – bank accounts, vehicle registrations and employment records – and failures sometimes expose them to random online users.

Database products, such as Oracle, DB2 and MySQL, have their own access control mechanisms, which are modelled on operating-system mechanisms, with privileges typically available for both users and objects (so the mechanisms are a mixture of access control lists and capabilities). However, the typical database access control architecture is comparable in complexity with Windows; modern databases are intrinsically complex, as are the things they support – typically business processes involving higher levels of abstraction than files or domains. There may be access controls aimed at preventing any user learning too much about too many customers; these tend to be stateful, and may deal with possible statistical inference rather than simple yes-no access rules. I devote a whole chapter in Part 2 to exploring the topic of Inference Control.

Ease of administration is often a bottleneck. In companies I've advised, the operating-system and database access controls have been managed by different departments, which don't talk to each other; and often IT departments have to put in crude hacks to make the various access control systems seem to work as one, but which open up serious holes.

Some products let developers bypass operating-system controls. For example, Oracle has both operating system accounts (whose users must be authenticated externally by the platform) and database accounts (whose users are authenticated directly by the Oracle software). It is often convenient to use the latter, to save the effort of synchronising with what other departments are doing. In many installations, the database is accessible directly from the outside; and even where it's shielded by a web service front-end, this often contains loopholes that let SQL code be inserted into the database.

Database security failures can thus cause problems directly. The Slammer worm in 2003 propagated itself using a stack-overflow exploit against Microsoft SQL Server 2000 and created large amounts of traffic as compromised machines sent floods of attack packets to random IP addresses.

Just as Windows is tricky to configure securely, because it's so complicated, the same goes for the typical database system. If you ever have to lock one down – or even just understand what's going on – you had better read a specialist textbook, such as [1175], or get in an expert.

6.2.10.2 Browsers

The web browser is another middleware platform on which we rely for access control and whose complexity often lets us down. The main access control rule is the *same-origin policy* whereby JavaScript or other active content on a web page is only allowed to communicate with the IP address that it originally came from; such code is run in a *sandbox* to prevent it altering the host system, as I'll describe in the next section. But many things can go wrong.

In previous editions of this book, we considered web security to be a matter of how the servers were configured, and whether this led to cross-site vulnerabilities. For example a malicious website can include links or form buttons aimed at creating a particular side-effect:

```
https://mybank.com/transfer.cgi?amount=10000USD&recipient=thief
```

The idea is that if a user clicks on this who is logged into mybank.com, there may be a risk that the transaction will be executed, as there's a valid session cookie. So payment websites deploy countermeasures such as using short-lived sessions and an anti-CSRF token (an invisible MAC of the session cookie), and checking the `Referer:` header. There are also issues around web authentication mechanisms; I described OAuth briefly in section 4.7.4. If you design web pages for a living you had better understand the mechanics of all

this in rather more detail (see for example [120]); but many developers don't take enough care. For example, as I write in 2020, Amazon Alexa has just turned out to have a misconfigured policy on cross-origin resource sharing, which meant that anyone who compromised another Amazon subdomain could replace the skills on a target Alexa with malicious ones [1483].

By now there's a realisation that we should probably have treated browsers as access control devices all along. After all, the browser is the place on your laptop where you run code written by people you don't want to trust and who will occasionally be malicious; as we discussed earlier, mobile-phone operating systems run different apps as different users to give even more robust protection. Even in the absence of malice, you don't want to have to reboot your browser if it hangs because of a script in one of the tabs. (Chrome tries to ensure this by running each tab in a separate operating-system process.)

Bugs in browsers are exploited in *drive-by download* attacks, where visiting an attack web page can infect your machine, and even without this the modern web environment is extremely difficult to control. Many web pages are full of trackers and other bad things, supplied by multiple ad networks and data brokers, which make a mockery of the intent behind the same-origin policy. Malicious actors can even use web services to launder origin: for example, the attacker makes a mash-up of the target site plus some evil scripts of his own, and then gets the victim to view it through a proxy such as Google Translate. A prudent person will go to their bank website by typing in the URL directly, or using a bookmark; unfortunately, the marketing industry trains everyone to click on links in emails.

6.2.11 Sandboxing

The late 1990s saw the emergence of yet another type of access control: the software *sandbox*, introduced by Sun with its Java programming language. The model is that a user wants to run some code that she has downloaded as an applet, but is concerned that the applet might do something nasty, such as stealing her address book and mailing it off to a marketing company, or just hogging the CPU and running down the battery.

The designers of Java tackled this problem by providing a 'sandbox' – a restricted environment in which the code has no access to the local hard disk (or at most only temporary access to a restricted directory), and is only allowed to communicate with the host it came from (the *same-origin policy*). This is enforced by having the code executed by an interpreter – the Java Virtual Machine (JVM) – with only limited access rights [784]. This idea was adapted to JavaScript, the main scripting language used in web pages, though it's actually a different language; and other active content too. A version of Java is also used on smartcards so they can support applets written by different firms.

6.2.12 Virtualisation

Virtualisation is what powers cloud computing; it enables a single machine to emulate a number of machines independently, so that you can rent a *virtual machine* (VM) in a data centre for a few tens of dollars a month rather than having to pay maybe a hundred for a whole server. Virtualisation was invented in the 1960s by IBM [496]; a single machine could be partitioned using VM/370 into multiple virtual machines. Initially this was about enabling a new mainframe to run legacy apps from several old machine architectures; it soon became normal for a company that bought two computers to use one for its production environment and the other as a series of logically separate machines for development, testing, and minor applications. It's not enough to run a virtual machine monitor (VMM) on top of a host operating system, and then run other operating systems on top; you have to deal with sensitive instructions that reveal processor state such as absolute addresses and the processor clock. Working VMMs appeared for Intel platforms with VMware ESX Server in 2003 and (especially) Xen in 2003, which accounted for resource usage well enough to enable AWS and the cloud computing revolution. Things can be done more cleanly with processor support, which Intel has provided since 2006 with VT-x, and whose details I'll discuss below. VM security claims rest to some extent on the argument that a VMM hypervisor's code can be much smaller than an operating system and thus easier to code-review and secure; whether there are actually fewer vulnerabilities is of course an empirical question [1578].

At the client end, virtualisation allows people to run a guest operating system on top of a host (for example, Windows on top of macOS), which offers not just flexibility but the prospect of better containment. For example, an employee might have two copies of Windows running on their laptop – a locked-down version with the office environment, and another for use at home. Samsung offers Knox, which creates a virtual machine on a mobile phone that an employer can lock down and manage remotely, while the user enjoys a normal Android as well on the same device.

But using virtualisation to separate security domains on clients is harder than it looks. People need to share data between multiple VMs and if they use ad-hoc mechanisms, such as USB sticks and webmail accounts, this undermines the separation. Safe data sharing is far from trivial. For example, Bromium⁵ offers VMs tailored to specific apps on corporate PCs, so you have one VM for Office, one for Acrobat reader, one for your browser and so on. This enables firms to work reasonably securely with old, unsupported software. So how do you download an Office document? Well, the browser exports the file from its VM to the host hard disc, marking it 'untrusted', so when the user tries to open it they're given a new VM with that document plus Office and nothing else.

⁵Now owned by HP

When they then email this untrusted document, there's an Outlook plugin that stops it being rendered in the 'sent mail' pane. Things get even more horrible with network services integrated into apps; the rules on what sites can access which cookies are complicated, and it's hard to deal with single signon and workflows that cross multiple domains. The clipboard also needs a lot more rules to control it. Many of the rules change from time to time, and are heuristics rather than hard, verifiable access logic. In short, using VMs for separation at the client requires deep integration with the OS and apps if it's to appear transparent to the user, and there are plenty of tradeoffs made between security and usability. In effect, you're retrofitting virtualisation on to an existing OS and apps that were not built for it.

Containers have been the hot new topic in the late 2010s. They evolved as a lightweight alternative to virtualisation in cloud computing and are often confused with it, especially by the marketing people. My definition is that while a VM has a complete operating system, insulated from the hardware by a hypervisor, a container is an isolated guest process that shares a kernel with other containers. Container implementations separate groups of processes by virtualising a subset of operating-system mechanisms, including process identifiers, interprocess communication, and namespaces; they also use techniques such as sandboxing and system call filtering. The business incentive is to minimise the guests' size, their interaction complexity and the costs of managing them, so they are deployed along with orchestration tools. Like any other new technology, there are many startups with more enthusiasm than experience. A 2019 survey by Jerry Gamblin disclosed that of the top 1000 containers available to developers on Docker Hub, 194 were setting up blank root passwords [743]. If you're going to use cloud systems, you need to pay serious attention to your choice of tools, and also learn yet another set of access control mechanisms – those offered by the service provider, such as the Amazon AWS Identity and Access Management (IAM). This adds another layer of complexity, which people can get wrong. For example, in 2019 a security firm providing biometric identification services to banks and the police left its entire database unprotected; two researchers found it using Elasticsearch and discovered millions of people's photos, fingerprints, passwords and security clearance levels on a database that they could not only read but write [1867].

But even if you tie down a cloud system properly, there are hardware limits on what the separation mechanisms can achieve. In 2018, two classes of powerful side-channel attacks were published: Meltdown and Spectre, which I discuss in the following section and at greater length in the chapter on side channels. Those banks that use containers to deploy payment processing rely, at least implicitly, on their containers being difficult to target in a cloud the size of Amazon's or Google's. For a comprehensive survey of the evolution of virtualisation and containers, see Randal [1578].

6.3 Hardware protection

Most access control systems set out not just to control what users can do, but to limit what programs can do as well. In many systems, users can either write programs, or download and install them, and these programs may be buggy or even malicious.

Preventing one process from interfering with another is the *protection problem*. The *confinement problem* is that of preventing programs communicating outward other than through authorized channels. There are several flavours of each. The goal may be to prevent active interference, such as memory overwriting, or to stop one process reading another's memory directly. This is what commercial operating systems set out to do. Military systems may also try to protect *metadata* – data about other data, or subjects, or processes – so that, for example, a user can't find out what other users are logged on to the system or what processes they're running.

Unless one uses sandboxing techniques (which are too restrictive for general programming environments), solving the protection problem on a single processor means, at the very least, having a mechanism that will stop one program from overwriting another's code or data. There may be areas of memory that are shared to allow interprocess communication; but programs must be protected from accidental or deliberate modification, and must have access to memory that is similarly protected.

This usually means that hardware access control must be integrated with the processor's memory management functions. A classic mechanism is *segment addressing*. Memory is addressed by two registers, a segment register that points to a segment of memory, and an address register that points to a location within that segment. The segment registers are controlled by the operating system, often by a component of it called the *reference monitor* which links the access control mechanisms with the hardware.

The implementation has become more complex as processors themselves have. Early IBM mainframes had a two-state CPU: the machine was either in authorized state or it was not. In the latter case, the program was restricted to a memory segment allocated by the operating system; in the former, it could write to segment registers at will. An authorized program was one that was loaded from an authorized library.

Any desired access control policy can be implemented on top of this, given suitable authorized libraries, but this is not always efficient; and system security depended on keeping bad code (whether malicious or buggy) out of the authorized libraries. So later processors offered more complex hardware mechanisms. Multics, an operating system developed at MIT in the 1960s and which inspired Unix, introduced *rings of protection* which express differing levels of privilege: ring 0 programs had complete access to disk, supervisor states ran

in ring 2, and user code at various less privileged levels [1687]. Many of its features have been adopted in more recent processors.

There are a number of general problems with interfacing hardware and software security mechanisms. For example, it often happens that a less privileged process such as application code needs to invoke a more privileged process (e.g., a device driver). The mechanisms for doing this need to be designed with care, or security bugs can be expected. Also, performance may depend quite drastically on whether routines at different privilege levels are called by reference or by value [1687].

6.3.1 Intel processors

The Intel 8088/8086 processors used in early PCs had no distinction between system and user mode, and thus any running program controlled the whole machine⁶. The 80286 added protected segment addressing and rings, so for the first time a PC could run proper operating systems. The 80386 had built-in virtual memory, and large enough memory segments (4 Gb) that they could be ignored and the machine treated as a 32-bit flat address machine. The 486 and Pentium series chips added more performance (caches, out of order execution and additional instructions such as MMX).

The rings of protection are supported by a number of mechanisms. The current privilege level can only be changed by a process in ring 0 (the kernel). Procedures cannot access objects in lower-level rings directly but there are *gates* that allow execution of code at a different privilege level and manage the supporting infrastructure, such as multiple stack segments.

From 2006, Intel added hardware support for x86 virtualisation, known as Intel VT, which helped drive the adoption of cloud computing. Some processor architectures such as S/370 and PowerPC are easy to virtualise, and the theoretical requirements for this had been established in 1974 by Gerald Popek and Robert Goldberg [1535]; they include that all sensitive instructions that expose raw processor state must be privileged instructions. The native Intel instruction set, however, has sensitive user-mode instructions, requiring messy workarounds such as application code rewriting and patches to hosted operating systems. Adding VMM support in hardware means that you can run an operating system in ring 0 as it was designed; the VMM has its own copy of the memory architecture underneath. You still have to trap sensitive opcodes, but system calls don't automatically require VMM intervention, you can run unmodified operating systems, things go faster and systems are generally more robust. Modern Intel CPUs now have nine rings: ring 0–3 for normal code,

⁶They had been developed on a crash programme to save market share following the advent of RISC processors and the market failure of the iAPX432.

under which is a further set of ring 0–3 VMM root mode for the hypervisor, and at the bottom is *system management mode* (SMM) for the BIOS. In practice, the four levels that are used are SMM, ring 0 of VMX root mode, the normal ring 0 for the operating system, and ring 3 above that for applications.

In 2015, Intel released Software Guard eXtensions (SGX), which lets trusted code run in an *enclave* – an encrypted section of the memory – while the rest of the code is executed as usual. The company had worked on such architectures in the early years of the Trusted Computing initiative, but let things slide until it needed an enclave architecture to compete with TrustZone, which I discuss in the next section. The encryption is performed by a Memory Encryption Engine (MEE), while SGX also introduces new instructions and memory-access checks to ensure non-enclave processes cannot access enclave memory (not even root processes). SGX has been promoted for DRM and securing cloud VMs, particularly those containing crypto keys, credentials or sensitive personal information; this is under threat from Spectre and similar attacks, which I discuss in detail in the chapter on side channels. Since SGX’s security perimeter is the CPU, its software is encrypted in main memory, which imposes real penalties in both time and space. Another drawback used to be that SGX code had to be signed by Intel. The company has now delegated signing (so bad people can get code signed) and from SGXv2 will open up the root of trust to others. So people are experimenting with SGX malware, which can remain undetectable by anti-virus software. As SGX apps cannot issue syscalls, it had been hoped that enclave malware couldn’t do much harm, yet Michael Schwarz, Samuel Weiser and Daniel Gruss have now worked out how to mount stealthy return-oriented programming (ROP) attacks from an enclave on a host app; they argue that the problem is a lack of clarity about what enclaves are supposed to do, and that any reasonable threat model must include untrusted enclaves [1691]. This simple point may force a rethink of enclave architectures; Intel says ‘In the future, Intel’s control-flow enforcement technology (CET) should help address this threat inside SGX’⁷. As for what comes next, AMD released full system memory encryption in 2016, and Intel announced a competitor. This aimed to deal with cold-boot and DMA attacks, and protect code against an untrusted hypervisor; it might also lift space and performance limits on next-generation enclaves. However, Jan Werner and colleagues found multiple inference and data-injection attacks on AMD’s offering when it’s used in a virtual environment. [2014]. There’s clearly some way to go.

As well as the access-control vulnerabilities, there are crypto issues, which I’ll discuss in the chapter on Advanced Cryptographic Engineering.

⁷The best defence against ROP attacks in 2019 appears to be Apple’s mechanism, in the iPhone X3 and later, for signing pointers with a key that’s kept in a register; this stops ROP attacks as the attacker can’t guess the signatures.

6.3.2 Arm processors

The Arm is the processor core most commonly used in phones, tablets and IoT devices; billions have been used in mobile phones alone, with a high-end device having several dozen Arm cores of various sizes in its chipset. The original Arm (which stood for *Acorn Risc Machine*) was the first commercial RISC design; it was released in 1985, just before MIPS. In 1991, Arm became a separate firm which, unlike Intel, does not own or operate any fabs: it licenses a range of processor cores, which chip designers include in their products. Early cores had a 32-bit datapath and contained fifteen registers, of which seven were shadowed by banked registers for system processes to cut the cost of switching context on interrupt. There are multiple supervisor modes, dealing with fast and normal interrupts, the system mode entered on reset, and various kinds of exception handling. The core initially contained no memory management, so Arm-based designs could have their hardware protection extensively customized; there are now variants with *memory protection units* (MPUs), and others with *memory management units* (MMUs) that handle virtual memory as well.

In 2011, Arm launched version 8, which supports 64-bit processing and enables multiple 32-bit operating systems to be virtualised. Hypervisor support added yet another supervisor mode. The cores come in all sizes, from large 64-bit superscalar processors with pipelines over a dozen stages deep, to tiny ones for cheap embedded devices.

TrustZone is a security extension that supports the ‘two worlds’ model mentioned above and was made available to mobile phone makers in 2004 [45]. Phones were the ‘killer app’ for enclaves as operators wanted to lock subsidised phones and regulators wanted to make the baseband software that controls the RF functions tamper-resistant [1241]. TrustZone supports an open world for a normal operating system and general-purpose applications, plus a closed enclave to handle sensitive operations such as cryptography and critical I/O (in a mobile phone, this can include the SIM card and the fingerprint reader). Whether the processor is in a secure or non-secure state is orthogonal to whether it’s in user mode or a supervisor mode (though the interaction between secure mode and hypervisor mode can be nontrivial). The closed world hosts a single *trusted execution environment* (TEE) with separate stacks, a simplified operating system, and typically runs only trusted code signed by the OEM – although Samsung’s Knox, which sets out to provide ‘home’ and ‘work’ environments on your mobile phone, allows regular rich apps to execute in the secure environment.

Although TrustZone was released in 2004, it was kept closed until 2015; OEMs used it to protect their own interests and didn’t open it up to app developers, except occasionally under NDA. As with Intel SGX, there appears to be no way yet to deal with malicious enclave apps, which might come

bundled as DRM with gaming apps or be mandated by authoritarian states; and, as with Intel SGX, enclave apps created with TrustZone can raise issues of transparency and control, which can spill over into auditability, privacy and much else. Again, company insiders mutter ‘wait and see’; no doubt we shall.

Arm’s latest offering is CHERI⁸ which adds fine-grained capability support to Arm CPUs. At present, browsers such as Chrome put tabs in different processes, so that one webpage can’t slow down the other tabs if its scripts run slowly. It would be great if each object in each web page could be sandboxed separately, but this isn’t possible because of the large cost, in terms of CPU cycles, of each inter-process context switch. CHERI enables a process spawning a subthread to allocate it read and write accesses to specific ranges of memory, so that multiple sandboxes can run in the same process. This was announced as a product in 2018 and we expect to see first silicon in 2021. The long-term promise of this technology is that, if it were used thoroughly in operating systems such as Windows, Android or iOS, it would have prevented most of the zero-day exploits of recent years. Incorporating a new protection technology at scale costs real money, just like the switch from 32-bit to 64-bit CPUs, but it could save the cost of lots of patches.

6.4 What goes wrong

Popular operating systems such as Android, Linux and Windows are very large and complex, with their features tested daily by billions of users under very diverse circumstances. Many bugs are found, some of which give rise to vulnerabilities, which have a typical lifecycle. After discovery, a bug is reported to a CERT or to the vendor; a patch is shipped; the patch is reverse-engineered, and an exploit may be produced; and people who did not apply the patch in time may find that their machines have been compromised. In a minority of cases, the vulnerability is exploited at once rather than reported – called a *zero-day* exploit as attacks happen from day zero of the vulnerability’s known existence. The economics, and the ecology, of the vulnerability lifecycle are the subject of study by security economists; I’ll discuss them in Part 3.

The traditional goal of an attacker was to get a normal account on the system and then become the system administrator, so they could take over the system completely. The first step might have involved guessing, or social-engineering, a password, and then using an operating-system bug to escalate from user to root [1131].

The user/root distinction became less important in the twenty-first century for two reasons. First, Windows PCs were the most common online devices

⁸Full disclosure: this was developed by a team of my colleagues at Cambridge and elsewhere, led by Robert Watson.

(until 2017 when Android overtook them) so they were the most common attack targets; and as they ran many applications as administrator, an application that could be compromised typically gave administrator access. Second, attackers come in two basic types: targeted attackers, who want to spy on a specific individual and whose goal is typically to acquire access to that person's accounts; and scale attackers, whose goal is typically to compromise large numbers of PCs, which they can organise into a botnet. This, too, doesn't require administrator access. Even if your mail client does not run as administrator, it can still be used by a spammer who takes control.

However, botnet herders do prefer to install *rootkits* which, as their name suggests, run as root; they are also known as *remote access trojans* or RATs. The user/root distinction does still matter in business environments, where you do not want such a kit installed as an *advanced persistent threat* by a hostile intelligence agency, or by a corporate espionage firm, or by a crime gang doing reconnaissance to set you up for a large fraud.

A separate distinction is whether an exploit is *wormable* – whether it can be used to spread malware quickly online from one machine to another without human intervention. The Morris worm was the first large-scale case of this, and there have been many since. I mentioned Wannacry and NotPetya in chapter 2; these used a vulnerability developed by the NSA and then leaked to other state actors. Operating system vendors react quickly to wormable exploits, typically releasing out-of-sequence patches, because of the scale of the damage they can do. The most troublesome wormable exploits at the time of writing are variants of Mirai, a worm used to take over IoT devices that use known root passwords. This appeared in October 2016 to exploit CCTV cameras, and hundreds of versions have been produced since, adapted to take over different vulnerable devices and recruit them into botnets. Wormable exploits often use root access but don't have to; it is sufficient that the exploit be capable of automatic onward transmission⁹. I will discuss the different types of malware in more detail in section 21.3.

However, the basic types of technical attack have not changed hugely in a generation and I'll now consider them briefly.

6.4.1 Smashing the stack

The classic software exploit is the memory overwriting attack, colloquially known as 'smashing the stack', as used by the Morris worm in 1988; this infected so many Unix machines that it disrupted the Internet and brought

⁹In rare cases even human transmission can make malware spread quickly: an example was the ILoveYou worm which spread itself in 2000 via an email with that subject line, which caused enough people to open it, running a script that caused it to be sent to everyone in the new victim's address book.

malware forcefully to the attention of the mass media [1810]. Attacks involving violations of memory safety accounted for well over half the exploits against operating systems in the late 1990s and early 2000s [484] but the proportion has been dropping slowly since then.

Programmers are often careless about checking the size of arguments, so an attacker who passes a long argument to a program may find that some of it gets treated as code rather than data. The classic example, used in the Morris worm, was a vulnerability in the Unix `finger` command. A common implementation of this would accept an argument of any length, although only 256 bytes had been allocated for this argument by the program. When an attacker used the command with a longer argument, the trailing bytes of the argument ended up overwriting the stack and being executed by the system.

The usual exploit technique was to arrange for the trailing bytes of the argument to have a *landing pad* – a long space of *no-operation* (NOP) commands, or other register commands that didn't change the control flow, and whose task was to catch the processor if it executed any of them. The landing pad delivered the processor to the attack code which will do something like creating a shell with administrative privilege directly (see Figure 6.5).

Stack-overwriting attacks were around long before 1988. Most of the early 1960s time-sharing systems suffered from this vulnerability, and fixed it [805]. Penetration testing in the early '70s showed that one of the most frequently-used attack strategies was still “unexpected parameters” [1168]. Intel's 80286 processor introduced explicit parameter checking instructions – verify read, verify write, and verify length – in 1982, but they were avoided by most software designers to prevent architecture dependencies. Stack overwriting attacks have been found against all sorts of programmable devices – even against things like smartcards and hardware security modules, whose designers really should have known better.

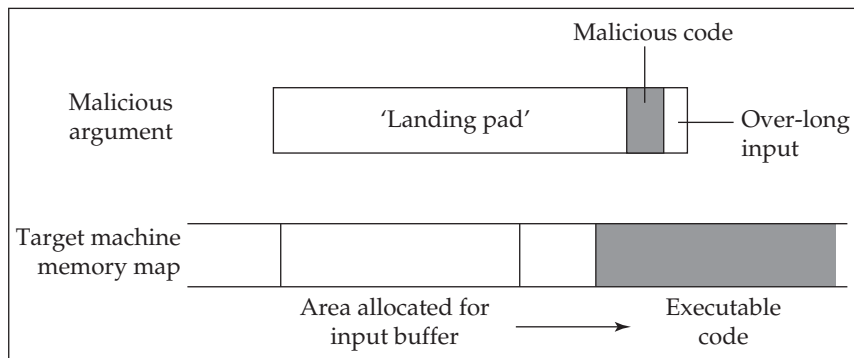


Figure 6.5: Stack smashing attack

6.4.2 Other technical attacks

Many vulnerabilities are variations on the same general theme, in that they occur when data in grammar A is interpreted as being code in grammar B. A stack overflow is when data are accepted as input (e.g. a URL) and end up being executed as machine code. These are failures of *type safety*. In fact, a stack overflow can be seen either as a memory safety failure or as a failure to sanitise user input, but there are purer examples of each type.

The *use after free* type of safety failure is now the most common cause of remote execution vulnerabilities and has provided a lot of attacks on browsers in recent years. It can happen when a chunk of memory is freed and then still used, perhaps because of confusion over which part of a program is responsible for freeing it. If a malicious chunk is now allocated, it may end up taking its place on the heap, and when an old innocuous function is called a new, malicious function may be invoked instead. There are many other variants on the memory safety theme; buffer overflows can be induced by improper string termination, passing an inadequately sized buffer to a path manipulation function, and many other subtle errors. See Gary McGraw's book *Software Security* [1268] for a taxonomy.

SQL injection attacks are the most common attack based on failure to sanitise input, and arise when a careless web developer passes user input to a back-end database without checking to see whether it contains SQL code. The game is often given away by error messages, from which a capable and motivated user may infer enough to mount an attack. There are similar command-injection problems afflicting other languages used by web developers, such as PHP. The usual remedy is to treat all user input as suspicious and validate it. But this can be harder than it looks, as it's difficult to anticipate all possible attacks and the filters written for one shell may fail to be aware of extensions present in another. Where possible, one should only act on user input in a safe context, by designing such attacks out; where it's necessary to blacklist specific exploits, the mechanism needs to be competently maintained.

Once such type-safety and input-sanitisation attacks are dealt with, *race conditions* are probably next. These occur when a transaction is carried out in two or more stages, where access rights are verified at the first stage and something sensitive is done at the second. If someone can alter the state in between the two stages, this can lead to an attack. A classic example arose in early versions of Unix, where the command to create a directory, `'mkdir'`, used to work in two steps: the storage was allocated, and then ownership was transferred to the user. Since these steps were separate, a user could initiate a `'mkdir'` in background, and if this completed only the first step before being suspended, a second process could be used to replace the newly created directory with a link to the password file. Then the original process would resume, and change ownership of the password file to the user.

A more modern example arises with the wrappers used in containers to intercept system calls made by applications to the operating system, parse them, and modify them if need be. These wrappers execute in the kernel's address space, inspect the enter and exit state on all system calls, and encapsulate only security logic. They generally assume that system calls are atomic, but modern operating system kernels are highly concurrent. System calls are not atomic with respect to each other; there are many possibilities for two system calls to race each other for access to shared memory, which gives rise to *time-of-check-to-time-of-use* (TOCTTOU) attacks. An early (2007) example calls a path whose name spills over a page boundary by one byte, causing the kernel to sleep while the page is fetched; it then replaces the path in memory [1996]. There have been others since, and as more processors ship in each CPU chip as time passes, and containers become an ever more common way of deploying applications, this sort of attack may become more and more of a problem. Some operating systems have features specifically to deal with concurrency attacks, but this field is still in flux.

A different type of timing attack can come from backup and recovery systems. It's convenient if you can let users recover their own files, rather than having to call a sysadmin – but how do you protect information assets from a time traveller? People can reacquire access rights that were revoked, and play even more subtle tricks.

One attack that has attracted a lot of research effort recently is *return-oriented programming* (ROP) [1711]. Many modern systems try to prevent type safety attacks by *data execution prevention* – marking memory as either code or data, a measure that goes back to the Burroughs 5000; and if all the code is signed, surely you'd think that unauthorised code cannot be executed? Wrong! An attacker can look for *gadgets* – sequences of instructions with some useful effect, ending in a return. By collecting enough gadgets, it's possible to assemble a machine that's Turing powerful, and implement our attack code as a chain of ROP gadgets. Then all one has to do is seize control of the call stack. This evolved from the *return-to-libc attack* which uses the common shared library `libc` to provide well-understood gadgets; many variants have been developed since, including an attack that enables malware in an SGX enclave to mount stealthy attacks on host apps [1691]. The latest attack variant, *block-oriented programming* (BOP), can often generate attacks automatically from crashes discovered by program fuzzing, defeating current control-flow integrity controls [966]. This coevolution of attack and defence will no doubt continue.

Finally there are *side channels*. The most recent major innovation in attack technology targets CPU pipeline behaviour. In early 2018, two game-changing attacks pioneered the genre: *Meltdown*, which exploits side-channels created by out-of-order execution on Intel processors [1173], and *Spectre*, which exploits speculative execution on Intel, AMD and Arm processors [1070]. The basic idea

is that large modern CPUs' pipelines are so long and complex that they look ahead and anticipate the next dozen instructions, even if these are instructions that the current process wouldn't be allowed to execute (imagine the access check is two instructions in the future and the read operation it will forbid is two instructions after that). The path not taken can still load information into a cache and thus leak information in the form of delays. With some cunning, one process can arrange things to read the memory of another. I will discuss Spectre and Meltdown in more detail later in the chapter on side channels. Although mitigations have been published, further attacks of the same general kind keep on being discovered, and it may take several years and a new generation of processors before they are brought entirely under control. It all reminds me of a saying by Roger Needham, that optimisation consists of replacing something that works with something that almost works, but is cheaper. Modern CPUs are so heavily optimised that we're bound to see more variants on the Spectre theme. Such attacks limit the protection that can be offered not just by containers and VMs, but also by enclave mechanisms such as TrustZone and SGX. In particular, they may stop careful firms from entrusting high-value cryptographic keys to enclaves and prolong the service life of old-fashioned hardware cryptography.

6.4.3 User interface failures

A common way to attack a fortress is to trick the guards into helping you, and operating systems are no exception. One of the earliest attacks was the *Trojan Horse*, a program the administrator is invited to run but which contains a nasty surprise. People would write games that checked whether the player was the system administrator, and if so would create another administrator account with a known password. A variant was to write a program with the same name as a common system utility, such as the `ls` command which lists all the files in a Unix directory, and design it to abuse the administrator privilege (if any) before invoking the genuine utility. You then complain to the administrator that something's wrong with the directory. When they enter the directory and type `ls` to see what's there, the damage is done. This is an example of the *confused deputy* problem: if A does some task on behalf of B, and its authority comes from both A and B, and A's authority exceeds B, things can go wrong. The fix in this particular case was simple: an administrator's 'PATH' variable (the list of directories to be searched for a suitably-named program when a command is invoked) should not contain '.' (the symbol for the current directory). Modern Unix versions ship with this as a default. But it's still an example of how you have to get lots of little details right for access control to be robust, and these details aren't always obvious in advance.

Perhaps the most serious example of user interface failure, in terms of the number of systems historically attacked, consists of two facts: first, Windows

is forever popping up confirmation dialogues, which trained people to click boxes away to get their work done; and second, that until 2006 a user needed to be the administrator to install anything. The idea was that restricting software installation to admins enabled Microsoft's big corporate customers, such as banks and government departments, to lock down their systems so that staff couldn't run games or other unauthorised software. But in most environments, ordinary people need to install software to get their work done. So hundreds of millions of people had administrator privileges who shouldn't have needed them, and installed malicious code when a website simply popped up a box telling them to do something. This was compounded by the many application developers who insisted that their code run as root, either out of laziness or because they wanted to collect data that they really shouldn't have had. Windows Vista started to move away from this, but a malware ecosystem is now well established in the PC world, and one is starting to take root in the Android ecosystem as businesses pressure people to install apps rather than using websites, and the apps demand access to all sorts of data and services that they really shouldn't have. We'll discuss this later in the chapter on phones.

6.4.4 Remedies

Software security is not all doom and gloom; things got substantially better during the 2000s. At the turn of the century, 90% of vulnerabilities were buffer overflows; by the time the second edition of this book came out in 2008, it was just under half, and now it's even less. Several things made a difference.

1. The first consists of specific defences. *Stack canaries* are a random number inserted by the compiler next to the return address on the stack. If the stack is overwritten, then with high probability the canary will change [484]. *Data execution prevention* (DEP) marks all memory as either data or code, and prevents the former being executed; it appeared in 2003 with Windows XP. *Address space layout randomisation* (ASLR) arrived at the same time; by making the memory layout different in each instance of a system, it makes it harder for an attacker to predict target addresses. This is particularly important now that there are toolkits to do ROP attacks, which bypass DEP. *Control flow integrity* mechanisms involve analysing the possible control-flow graph at compile time and enforcing this at runtime by validating indirect control-flow transfers; this appeared in 2005 and was incorporated in various products over the following decade [351]. However the analysis is not precise, and block-oriented programming attacks are among the tricks that have evolved to exploit the gaps [966].
2. The second consists of better general-purpose tools. Static-analysis programs such as Coverity can find large numbers of potential software

bugs and highlight ways in which code deviates from best practice; if used from the start of a project, they can make a big difference. (If added later, they can throw up thousands of alerts that are a pain to deal with.) The radical solution is to use a better language; my colleagues increasingly write systems code in Rust rather than in C or C++¹⁰.

3. The third is better training. In 2002, Microsoft announced a security initiative that involved every programmer being trained in how to write secure code. (The book they produced for this, *Writing Secure Code* [929], is still worth a read.) Other companies followed suit.
4. The latest approach is DevSecOps, which I discuss in section 27.5.6. Agile development methodology is extended to allow very rapid deployment of patches and response to incidents; it may enable the effort put into design, coding and testing to be aimed at the most urgent problems.

Architecture matters; having clean interfaces that evolve in a controlled way, under the eagle eye of someone experienced who has a long-term stake in the security of the product, can make a huge difference. Programs should only have as much privilege as they need: the *principle of least privilege* [1642]. Software should also be designed so that the default configuration, and in general, the easiest way of doing something, should be safe. Sound architecture is critical in achieving safe defaults and using least privilege. However, many systems are shipped with dangerous defaults and messy code, exposing all sorts of interfaces to attacks like SQL injection that just shouldn't happen. These involve failures of incentives, personal and corporate, as well as inadequate education and the poor usability of security tools.

6.4.5 Environmental creep

Many security failures result when environmental change undermines a security model. Mechanisms that worked adequately in an initial environment often fail in a wider one.

Access control mechanisms are no exception. Unix, for example, was originally designed as a 'single user Multics' (hence the name). It then became an operating system to be used by a number of skilled and trustworthy people in a laboratory who were sharing a single machine. In this environment the function of the security mechanisms is mostly to contain mistakes; to prevent one user's typing errors or program crashes from deleting or overwriting another user's files. The original security mechanisms were quite adequate for this purpose.

¹⁰Rust emerged from Mozilla research in 2010 and has been used to redevelop Firefox; it's been voted the favourite language in the Stack Overflow annual survey from 2016–2019.

But Unix security became a classic ‘success disaster’. Over the 50 years since Ken Thomson started work on it at Bell Labs in 1969, Unix was repeatedly extended without proper consideration being given to how the protection mechanisms also needed to be extended. The Berkeley versions assumed an extension from a single machine to a network of machines that were all on one LAN and all under one management. The Internet mechanisms (telnet, ftp, DNS, SMTP) were originally written for mainframes on a secure network. Mainframes were autonomous, the network was outside the security protocols, and there was no transfer of authorisation. So remote authentication, which the Berkeley model really needed, was simply not supported. The Sun extensions such as NFS added to the party, assuming a single firm with multiple trusted LANs. We’ve had to retrofit protocols like Kerberos, TLS and SSH as duct tape to hold the world together. The arrival of billions of phones, which communicate sometimes by wifi and sometimes by a mobile network, and which run apps from millions of authors (most of them selfish, some of them actively malicious), has left security engineers running ever faster to catch up.

Mixing many different models of computation together has been a factor in the present chaos. Some of their initial assumptions still apply partially, but none of them apply globally any more. The Internet now has billions of phones, billions of IoT devices, maybe a billion PCs, and millions of organisations whose managers not only fail to cooperate but may be in conflict. There are companies that compete; political groups that despise each other, and nation states that are at war with each other. Users, instead of being trustworthy but occasionally incompetent, are now largely unskilled – but some are both capable and hostile. Code used to be simply buggy – but now there is a lot of malicious code out there. Attacks on communications used to be the purview of intelligence agencies – now they can be done by youngsters who’ve downloaded attack tools from the net and launched them without any real idea of how they work.

6.5 Summary

Access control mechanisms operate at a number of levels in a system, from the hardware up through the operating system and middleware like browsers to the applications. Higher-level mechanisms can be more expressive, but also tend to be more vulnerable to attack for a variety of reasons ranging from intrinsic complexity to implementer skill.

The main function of access control is to limit the damage that can be done by particular groups, users, and programs whether through error or malice. The most widely fielded examples are Android and Windows at the client end and Linux at the server end; they have a common lineage and

many architectural similarities. The basic mechanisms (and their problems) are pervasive. Most attacks involve the opportunistic exploitation of bugs; products that are complex, widely used, or both are particularly likely to have vulnerabilities found and turned into exploits. Many techniques have been developed to push back on the number of implementation errors, to make it less likely that the resulting bugs give rise to vulnerabilities, and harder to turn the vulnerabilities into exploits; but the overall dependability of large software systems improves only slowly.

Research problems

Most of the issues in access control were identified by the 1960s or early 1970s and were worked out on experimental systems such as Multics [1687] and the CAP [2024]. Much of the research in access control systems since then has involved reworking the basic themes in new contexts, such as mobile phones.

Recent threads of research include enclaves, and the CHERI mechanisms for adding finer-grained access control. Another question is: how will developers use such tools effectively?

In the second edition I predicted that ‘a useful research topic for the next few years will be how to engineer access control mechanisms that are not just robust but also usable – by both programmers and end users.’ Recent work by Yasemin Acar and others has picked that up and developed it into one of the most rapidly-growing fields of security research [11]. Many if not most technical security failures are due at least in part to the poor usability of the protection mechanisms that developers are expected to use. I already mention in the chapter on cryptography how crypto APIs often induce people to use really unsafe defaults, such as encrypting long messages with ECB mode; access control is just as bad, as anyone coming cold to the access control mechanisms in a Windows system or either an Intel or Arm CPU will find.

As a teaser, here’s a new problem. Can we extend what we know about access control at the technical level – whether hardware, OS or app – to the organisational level? In the 20th century, there were a number of security policies proposed, from Bell-LaPadula to Clark-Wilson, which we discuss at greater length in Part 2. Is it time to revisit this for a world of deep outsourcing and virtual organisations, now that we have interesting technical analogues?

Further reading

There’s a history of virtualisation and containers by Allison Randal at [1578]; a discussion of how mandatory access controls were adapted to operating systems such as OS X and iOS by Robert Watson in [1997]; and a reference

book for Java security written by its architect Li Gong [784]. The Cloud Native Security Foundation is trying to move people towards better open-source practices around containers and other technologies for deploying and managing cloud-native software. Going back a bit, the classic descriptions of Unix security are by Fred Grampp and Robert Morris in 1984 [806] and by Simson Garfinkel and Eugene Spafford in 1996 [753], while the classic on Internet security by Bill Cheswick and Steve Bellovin [222] gives many examples of network attacks on Unix systems.

Carl Landwehr gives a useful reference to many of the flaws found in operating systems in the 1960s through the 1980s [1131]. One of the earliest reports on the subject (and indeed on computer security in general) is by Willis Ware in 1970 [1990]; Butler Lampson's seminal paper on the confinement problem appeared in 1970s [1127] and three years later, another influential early paper was written by Jerry Saltzer and Mike Schroeder [1642]. The textbook we get our students to read on access control issues is Dieter Gollmann's *'Computer Security'* [780]. The standard reference on Intel's SGX and indeed its CPU security architecture is by Victor Costan and Srinivas Devadas [479].

The field of software security is fast-moving; the attacks change significantly (at least in their details) from one year to the next. The classic starting point is Gary McGraw's 2006 book [1268]. Since then we've had ROP attacks, Spectre and much else; a short but useful update is Matthias Payer's *Software Security* [1506]. But to really keep up, it's not enough to just read textbooks; you need to follow security conferences such as Usenix and CCS as well as the security blogs such as Bruce Schneier, Brian Krebs and – dare I say it – our own lightbluetouchpaper.org. The most detail on the current attacks is probably in Google's Project Zero blog; see for example their analysis of attacks on iPhones found in the wild for an insight into what's involved in hacking modern operating systems with mandatory access control components [205].

Network Attack and Defence

Simplicity is the ultimate sophistication.

– LEONARDO DA VINCI

There's no security here – keep moving!

– RICHARD CLAYTON

21.1 Introduction

In this chapter I'm going to try to draw together the network aspects of security in a coherent framework. This is not straightforward as much of network security is practical engineering; a purist from computer science might see the field as one bodge piled on top of another. And network security may not be that important to many developers: if you write apps for Androids and iPhones that talk to services on AWS or Azure, then you can leave much of the worry to Amazon or Microsoft.

But many organisations need to pay attention to network security, and there are some visible strategic trends. For twenty years, it was accepted that firms would have a trusted internal network or intranet, protected from the Internet by firewalls; while taken to extremes by defence and intelligence organisations with classified internal networks, milder versions were seen as best practice by most normal firms. And some industries have no viable alternatives. For example, the protocols used in industrial control systems – DNP3 and Modbus – don't support encryption or authentication, as they evolved in the days of leased lines and private radio links. By the late 1990s, control systems engineers were attaching sensors and actuators to IP networks, as they were cheaper – and then realising that anyone in the world who knew a sensor's IP address could read it, and anyone who knew an actuator's IP address could activate it. This led to the growth of specialist firms who sell firewalls that understand these protocols; energy companies have thousands of them.

A typical electricity substation might have two hundred devices from a multiplicity of vendors, on a LAN where performance is critical, so retrofitting crypto is impractical; but it has one connection to the outside world, so that's where you have to put the protection. This is known as *re-perimeterization*. The same approach is taken with vehicles, where the internal CANBUS cannot be protected, so the radio interfaces with the outside world have to be.

But in many firms the trend is firmly in the other direction, towards *deperimeterisation*. One thought leader is Google, promoting an architecture without firewalls that it calls a *zero-trust security model*: "By shifting access controls from the network perimeter to individual users and devices, BeyondCorp allows employees, contractors, and other users to work more securely from virtually any location without the need for a traditional VPN." Google's experience is that the move to mobile and cloud technology is making network perimeters ever harder to define, let alone police, and if a firm's large enough that some internal compromise is inevitable anyway then the perimeter is the wrong place to put the primary protection [1988]. There are still some perimeter defences, most notably against service-denial attacks, but internal networks are otherwise unprivileged and the emphasis is on tight authentication and authorisation of users and devices: each service has an Internet-facing access proxy. One might see this as a per-service firewall rather than a per-building firewall, but there is quite a lot more to it with tiers of sensitivity, a device inventory service and an access control engine [1481]. You also need really good HR data, so you can tie staff and contractors to devices and the services they're allowed to use. Much the same architecture is being adopted by other firms operating large-scale data centres, and zero-trust security is now the subject of draft standards activity by NIST [1621]. It will no doubt get a boost from the pandemic because of the huge increase in home working.

Other organisations may take a hybrid approach. The university where I work, for example, has some defences at the perimeter but largely lets departments do our own thing; a computer science department has quite different requirements from a humanities department or the finance office.

In order to explore the options and constraints, I'm first going to discuss networking protocols such as BGP, DNS and SMTP and the service-denial attacks that can result from their abuse. I'll then take a closer look at malware, and then at defensive technologies such as filtering and intrusion detection and how defenders can coordinate them. I'll then survey the limitations of widely-used crypto protocols such as TLS, SSH and IPsec, and the particularly tricky role of certification authorities. Finally I'll return to network architecture. Many issues are complex and interlinked, with some significant trade-offs. For example, various kinds of end-to-end crypto can bring benefits – particularly against bulk surveillance – but can get in the way of the surveillance we want for network security.

This chapter will deal with fixed networks, and I'll discuss what's different about mobile networks in the following chapter.

21.2 Network protocols and service denial

I'm going to assume some familiarity with basic network protocols. The telegraphic summary is as follows. The *Internet Protocol* (IP) is a stateless protocol that transfers packet data from one machine to another; IP version 4 uses 32-bit *IP addresses*, often written as four decimal numbers in the range 0–255, such as 172.16.8.93. ISPs are migrating to IP version 6, as the 4 billion possible IPv4 addresses are just about allocated; IPv6 uses 128-bit addresses. Some 10–15% of traffic is now IPv6; in many countries a new broadband subscription will get you an IPv6 address which works for all normal consumer purposes.

Local networks mostly use ethernet, in which devices have unique ethernet addresses (also called MAC addresses) that are mapped to IPv4 addresses using the *address resolution protocol* (ARP). The *Dynamic Host Configuration Protocol* (DHCP) is used to allocate IP addresses to machines as needed and to ensure that each IP address is unique. *Network address translation* (NAT) also enables multiple devices on a network to use the same Internet-facing IP address, typically with different port numbers; this is used by most mobile network operators and many ISPs. So if you want to track down a machine that has done something wicked, you will often have to get the logs that map MAC addresses of devices to IP addresses. There may be more than one log, and lots can go wrong – such as wrong timestamps, and failure to understand time zones.

One of the most basic concerns is the prevention and mitigation of *denial-of-service* (DoS) attacks. These have a number of flavours. An opponent can try to steal some of your IP address space, or one or more of your domains, in order to send spam; even when you get it back, you may find it's been extensively blacklisted. An opponent can send you huge floods of traffic from a botnet of many compromised machines; a *distributed denial-of-service* (DDoS) attack. They can abuse various online services such as DNS to send you floods of packet traffic. Let's work through these in turn.

21.2.1 BGP security

The Internet is an interconnected network of networks: its components are *Autonomous Systems* (ASes) such as ISPs, telcos and large organisations, each of which controls a range of IP addresses. The glue that holds them together, the core routing protocol of the Internet, is the *Border Gateway Protocol* (BGP).

Routers – the specialized computers that switch packets on networks – use BGP to exchange information about what routes are available to get to particular blocks of IP addresses, and to maintain routing tables so they can select efficient routes to use. ASes can route traffic to other ASes by buying service from large transit providers but typically cut the costs of this by peering with each other at a local *Internet interchange* (IX), of which most countries have at least one and large countries may have several.

Internet interconnectivity is a complex ecosystem with many interdependent layers. Its open and decentralised organisation has been essential to the success and resilience of the Internet, which has meant that the effects of natural disasters such as Hurricane Katrina and terrorist attacks such as 9/11 have been limited in time and space, as have assorted technical failures. However, the Internet is slowly becoming more centralised, as a result of the consolidation of Tier-1 providers, and is vulnerable to common-mode failures (such as electric power cuts) as well as to disruptive attacks.

About the worst attack we can reasonably foresee would involve an attacker planting malware on thousands of routers so they advertise large numbers of false routes, clogging the routing tables and tearing up the routing fabric. There have been several warnings already in the form of incidents and accidents. In 2008, YouTube became inaccessible for a few hours after the government of Pakistan tried to censor it locally by announcing false routes to it, which propagated globally; and in 2010 China Telecom advertised over 100,000 invalid routes, hijacking 15% of Internet addresses for 18 minutes. Some people ascribed that to accident, while others suggested that China had been testing a ‘cyber-nuke’, some of whose fallout escaped. Most routers now accept only a limited number of routes from each of their peers, be it a few dozen or a few hundred; so large-scale disruption would require thousands of subverted routers. Both China and (more recently) Russia have been working on making the Internet in their countries separable, so that major disruptive attacks could in theory be launched without inflicting unacceptable collateral damage on local services and facilities. There have been reports of BGP hijacking being used by China for intelligence collection; for example, traffic from Canada to Korean government websites was routed via China from February 2016 for six months [533]. There has also been criminal misuse, ranging from the hijacking of IP address space by spammers, to an eight-figure ad fraud in 2018 whose perpetrators hid in address space stolen from the US Air Force [792]. Finally, there is a growing political tussle in 2019–20 about whether Huawei should be allowed to sell routers at scale (or at all) in countries allied to the USA.

Taking a step backward, the resilience of the Internet is hard to define and to measure; it is in tension with efficiency and may be decreasing as a small number of very large networks come to dominate. These range from the dominant transit provider, Lumen (the company changed its name from Level 3 to Lumen), to content delivery networks (CDNs) operated by Google, Akamai,

Cloudflare and others. There are many complex interactions between resilience and efficiency, reachability and congestion, traffic prioritisation and commercial sensitivity, complexity and scale. There's no mechanism to check the validity of routing information distributed via BGP. The pervasive mistrust between ISPs and governments makes regulation difficult. The lack of good information about how the system works makes rational discussion difficult too. Resilience has so far depended on surplus capacity and rapid growth, but that cannot continue for ever. In 2011 colleagues and I wrote a major report for the European Network and Information Security Agency that explores these issues in detail [1910].

The main technical BGP security mechanism at present is the *Resource Public Key Infrastructure* (RPKI), which enables registries to certify that "Autonomous system X announces IP address range Y". This will not prevent capable attackers, as a malicious route announcement will just have the right AS at the end of the route following the attacker's in the middle; but it detects the fat-finger mistakes that cause most of the outages. Whether it will make an already fragile BGP system more robust to have lots of certificates in it remains to be seen; when RIPE's certificate expired in February 2020 there was a short outage until it was fixed. For the future, people are working on Peerlock, whereby the main ASes at an interchange share information about what routes they will and won't announce; this has the prospect of bringing enough local benefit to exchange members for it to be practically deployable.

21.2.2 DNS security

The *Domain Name System* (DNS) allows human-readable names such as `ross-anderson.com` to be mapped to IP addresses of either kind; there's a hierarchy of DNS servers that do this, ranging from several hundred top-level servers down through machines at ISPs and on local networks, which cache DNS records for performance and reliability. It does occasionally get attacked: the Mirai botnet attacked DynDNS in October 2016, taking out Twitter on the US eastern seaboard for five hours. But DNS has become a massively distributed system with a lot of very fast machines connected to very high-capacity networks, so service denial attacks on it are rare.

Hijacking does occur from time to time, and at various levels. Some states intercept and redirect DNS queries as a means of censorship; some ISPs have done so, as a means of replacing ads in web pages with ads from which they get a cut; and a DNS server at an ISP may be hacked to drive clients to a wicked website. This is known as *pharming*, and in a variant called *drive-by pharming*, the crooks lure you to a web page containing JavaScript that changes your home router's DNS server from the one at your ISP to one under their control [1819]. Next time you try to go to `www.citibank.com`, you may be directed to a phishing site that emulates it. That's one reason to change the default

password on your home router – even if it’s only accessible from inside your network.

In order to prevent DNS hijacking, DNSSEC adds digital signatures to DNS name records. By verifying such a signature you can check that the record came from the authoritative server and was not altered en route. Uptake is patchy: all US government domains in `.gov` are supposed to be signed, and most domains in Sweden are signed, as the registrar made signed domains cheaper. However, some major firms like Google don’t sign their DNS records out of concern that cryptography makes systems more fragile; if anything goes wrong, you can just disappear. Other firms avoid DNSSEC because they don’t want competitors to ‘walk the zone’ and enumerate all their subdomains; the NSEC3 extension enables firms to avoid this using hashes, but many firms (or their service providers) have not yet built the infrastructure.

Another problem with DNSSEC is that it gets abused in denial-of-service attacks. A common technique is that Alice attacks Bob by sending Charlie a message saying, “Hey, can you tell me the very large answer to this short question? Yours, Bob!” As signed DNS records are a lot larger, a DDoS-for-hire service can use DNSSEC as an amplifier, Alice can send packets that purport to come from Bob’s IP address to many DNS servers, which then bombard the target with replies. (Cheeky criminals use the FBI as Charlie, as `fbi.gov` has two nice big keys.)

The controversial issue in 2020 is *DNS-over-https* (DoH). The main browser maintainers, Chrome and Mozilla, propose that rather than sending DNS traffic in the clear, it will go encrypted over https to a DoH resolver. This is claimed to be good for privacy, as your ISP will have less information about your browsing (but unless you use Tor, it will still have plenty). The downside is that many enterprise security products monitor DNS to detect abuse. If malware compromises a machine in your fleet, you may spot it when it tries to contact a command-and-control server, so enterprises buy threat intelligence feeds and monitor the domain names (and IP addresses) blacklisted on them. Sysadmins also like to monitor for DNS hijacking, and to block certain domains as inappropriate for work. DoH will make all this harder, and is questionable architecture as running a core network service over an application means it’s ‘not the Internet any more’ [430]. On the commercial side, DoH may entrench Google’s grip on the advertising market, while causing problems for content delivery networks like Akamai and Cloudflare over routing, load balancing and so on. It will also stop ISPs transcoding videos for mobile users to save bandwidth. Experts would have preferred to run DNS over TLS instead.

21.2.3 UDP, TCP, SYN floods and SYN reflection

On wide-area networks, most data move between machines using either the *User Datagram Protocol* (UDP), which is connectionless, or the *Transmission*

Control Protocol (TCP), which sets up persistent connections between end-points. Let's start with the 3-way handshake used by Alice to initiate a TCP connection to Bob and set up sequence numbers for subsequent packet traffic (see Figure 21.1).

```

A → B:    SYN; my number is X
B → A:    CK; now X + 1
          SYN; my number is Y
A → B:    ACK; now Y + 1
          (start talking)

```

Figure 21.1: TCP/IP handshake

This protocol has been exploited in many ways. The classic service-denial attack is the *SYN flood*. Alice simply sends a lot of SYN packets and never acknowledges any of the replies. Bob accumulates more records of SYN packets than his software can handle. This was used in one of the first distributed denial-of-service attacks that brought down Panix, a New York ISP, for several days in 1996.

The technical fix was the 'SYNcookie': rather than keeping a copy of the incoming SYN packet, *B* simply sends out as *Y* an encrypted version of *X*. That way, Bob doesn't have to retain a lot of state about half-open sessions. Despite this, SYN floods persisted, albeit at a declining rate, for many years. The general principle is that when you're designing a protocol anyone can invoke, don't let malicious users force honest ones to do work.

The more common attack now is *SYN reflection*. Alice sends Bob a packet that purports to come from Charlie. Bob replies to Charlie, and in practice systems send up to five ACKs in response to each SYN as a robustness measure, so there's still a useful amplification effect.

21.2.4 Other amplifiers

Many other protocols have been used in service-denial attacks than DNS and TCP [1505]. An early favourite was *smurfing*; this exploited the *Internet control message protocol* (ICMP), which enables users to send an echo packet to a remote host to check whether it's alive. If Alice sent an ICMP packet purporting to come from Bob to a broadcast address, all the machines on the subnet would send him a response. The protocol was changed so that broadcast addresses didn't reply. The bad guys changed to use protocols such as NTP and DNS for which amplifiers could still be found.

More thorough fixes for attacks based on packet amplification were to follow. Most of the available amplifiers use UDP packets, including ICMP and NNTP but not SYN reflection; so starting from the mid-2000s, broadband ISPs started filtering out UDP packets with forged source addresses. Microsoft also

changed their network stack to make it much harder for an infected machine to send a packet with a spoofed IP address; you now need to hack the operating system, not just any old application. So attacks that exploit UDP packet amplifiers have to be run from servers in hosting centres. In the late 2010s, such attacks have become increasingly the preserve of DDoS-for-hire operators, against whom the most effective countermeasure has been to raid them and arrest them.

21.2.5 Other denial-of-service attacks

As the clever ways of creating service-denial attacks have been closed off one by one, the bad guys have turned increasingly to brute force, by sending floods of packets from infected machines. The first *distributed denial-of-service* (DDoS) attack may have been the Morris worm in the 1980s, and the first deliberate one in the 1990s with the attack already mentioned on Panix. Nowadays, botnets are assembled using all sorts of vulnerabilities, and underground markets let some people specialise in hacking machines and selling them to others who extract value in various ways. Since 2016, the machines most used for DDoS have been IoT devices such as CCTV cameras, which are now connected in large numbers to home WiFi networks with reasonable bandwidth, but which tend to have known default passwords – and are often incapable of being patched. The Mirai botnet appeared in 2016 to exploit this opportunity, and there have been over a thousand variants of it since (its source code got posted to Hackforums).

There are various motives for service-denial attacks. Most are launched by schoolkids – typically gamers who want to take down an opposing crew's teamspeak server. There has for some years been a black market in DDoS-for-hire, which the authorities in the USA and elsewhere have been trying to close down. There have been some incidents of blackmail (e.g., of online bookmakers), and a growing use of the technique for suppressing political opponents – starting perhaps with attacks on the servers of an opposition party in Kyrgyzstan, even when these were relocated to North America [1616]. We discussed their use in conflict by states in Chapter 2.

That said, one mustn't forget online activism. If a hundred thousand people send email to the White House protesting against some policy or other, is this a DDoS attack? Protesters should not be treated as felons; but protest can easily shade over into abuse, and drawing legislative distinctions can be hard.

21.2.6 Email – from spies to spammers

The SMTP standard for email has particular issues around the prevention of bulk interception, and the prevention of bulk unwanted mail.

Email is by default neither encrypted nor authenticated, and was for decades available to anyone who could either monitor the network or access mail servers. It was possible to use programs such as PGP/GPG to encrypt mail, but this never caught on outside small communities. First, such programs can be a pain to use, and second, there are strong network effects: there's no point in using email encryption if none of your friends do. What's more, if only a small group of people use encryption, this may just bring them to the attention of the authorities; subversive groups, spies and so on really need anonymity rather than just confidentiality, as we discussed in section 20.4. So PGP/GPG tends to be used by specialists, such as sysadmins and anti-virus researchers.

There are two main countermeasures to bulk interception. First, most mail servers use `starttls` to set up encrypted communications with other mail servers as they exchange mail, especially since the Snowden revelations. Encrypted exchanges can be blocked by man-in-the-middle attacks, and these have been reported in some less-democratic countries. The current countermeasure to such attacks, *MTA Strict Transport Security* (MTA-STS), is supported by Microsoft, Google and Yahoo [1222]: it allows mail service providers to specify that mail should only be delivered to them via a TLS session authenticated by a proper certificate which you download from their website. This prevents downgrade or interception attacks on email to and from the big boys, and also allows opportunistic, trust-on-first-use encryption to other servers. MTA-STS has generally supplanted an earlier standard, *DNS-based Authentication of Named Entities* (DANE) which put a TLS certificate for `starttls` in the mail server's DNS record¹.

The second countermeasure is that some 95% of personal email accounts nowadays are at the big five webmail providers, and many corporates use them too. In this case, the confidentiality of email is assured by TLS, fortified with certificate pinning and certificate transparency which we'll discuss later. But although bulk access may be blocked, webmail is subject to warranted access, just like other services that corporates outsource.

Bulk unwanted mail, or spam, has two components. The first is entirely legal but unwanted marketing communication. As marketers can make it tiresome to opt out, users find it more convenient to press the 'report spam' button once an offer or supplier is no longer of interest.

The second consists of floods of generally unwanted traffic sent out for the most part by botnets, and often with clear criminal intent. This is in some respects similar to a DDoS attack: just as DDoS bots may forge IP addresses, spam bots may forge the sender's email address. This is fought by the big providers with four main mechanisms.

¹DANE is still widely used in Germany, but Google refused to use it as it depends on DNSSEC, which Google considers to be insufficiently dependable.

1. *Domain Keys Identified Mail* (DKIM) ties email to the sending domain by signing it using a signature key whose public verification key is kept in the sending domain's DNS record. The signed material is selected to identify the message unambiguously despite the additions to headers that occur during transit, but to stop the bad guys adding an extra "From: PayPal" header. Mail that hasn't been altered too much can be forwarded. There's a replay attack in that the spammer sends his spam through Gmail, which signs it, and then forwards it afterwards; so mail servers cache DKIM signatures and discard mail carrying a signature that's already been seen a few times.
2. *Sender Policy Framework* (SPF) is similar but ties mail to the source IP address. Again, this is verifiable against a key in the domain DNS record. SPF doesn't allow mail forwarding; mailing list servers are supposed to use a related protocol called *Authenticated Received Chain* (ARC) to re-sign mail they forward.
3. A domain's DNS can also contain a *Domain-based Message Authentication, Reporting and Conformance* (DMARC) record, which enables its owner to recommend what a recipient should do with email that appears to come from the owner's domain but which fails authentication using both DKIM and SPF.
4. Machine-learning systems are used to filter mail against authentication results and other criteria, and take much of their ground truth from whether users report mail as spam. This is made more complicated by user preferences for marketing material, which vary by user and over time.

The illegal segment of spam is now a highly specialised business, run by several large gangs. Its statistics have been 'lumpy' since the mid-2000s, and this has been getting more pronounced. As of 2020, the gangs typically steal IP address space using malicious BGP route announcements, register thousands of domains, and send a few hundred spams from each before the machine-learning filters kick in and block them.

21.3 The malware menagerie – Trojans, worms and RATs

The first examples of malicious code were *Trojan Horses* – named after the horse the Greeks left for the Trojans, supposedly as a gift but which contained soldiers who opened the gates of Troy to the Greek army [1131]. There have been religious wars over nomenclature for years, which is why many people prefer to just use the term *malware*. My usage is that a Trojan is a program

that does something malicious (such as capturing passwords) when run by an unsuspecting user. A *worm* is a malicious program that replicates itself on other systems, while one that does so by hooking itself into the code of other programs is a *virus*. A *remote access Trojan* (RAT) is software that may or may not run as root but that enables a remote party to access the device it runs on, while a *rootkit* is software installed as root on a device and that stealthily enables a third party to control it. *Potentially unwanted software* (PUS) may have been installed openly or by deception, but does something the user doesn't want (if they understand it at all).

These categories are not mutually exclusive and the boundaries can be context dependent. For example, stalkerware – software that enables one person to track another's mobile phone location and use – falls into different categories depending on whether it was installed covertly, or by a controlling man bullying his partner, or by a court ordering it as a condition of bail. Even stealthy malware isn't always illegal as it can be used by law-enforcement agencies to turn suspects' phones and laptops into listening devices, as well as by fraudsters to operate bank accounts by remote control².

Malware generally uses stealth techniques to hide, but eventually it's identified and tools to remove it are written. There's a whole ecosystem around malware: malware writers, botnets of infected machines, and a range of security firms offering everything from threat intelligence to antivirus software. (There are even firms selling malware – particularly to government agencies.) And in addition to the formal economy, there's an underground economy of cybercrooks selling everything from banking Trojans to DDoS-for-hire services.

21.3.1 Early history of malware

In the early 1960s, machines were slow and their CPU cycles were rationed – with students often at the tail of the queue. Students invented tricks such as writing computer games with a Trojan inside to check if the program is running as root, and if so to create a privileged account with a known password. By the 1970s, time-sharing systems at universities were the target of more and more pranks involving Trojans. All sorts of tricks were developed. In 1978, John Shoch and Jon Hupp of Xerox PARC wrote a program they called a *worm*, which replicated itself across a network looking for idle processors so it could assign them tasks [1727].

In 1984, Ken Thompson gave a classic paper “Reflections On Trusting Trust”, when he accepted a Turing award, the top prize in computer science. He

²At the other end of the spectrum, some antivirus products behave like malware in various ways, including being very hard to remove after a ‘free trial’, or by introducing insecurities. In December 2019, one brand of AV software was removed by Chrome, Firefox and Opera for exfiltrating too much personal information [354].

showed that even if the source code for a system were carefully inspected and known to be free of vulnerabilities, a trapdoor could still be inserted [1887]. His trick was to build the trapdoor into the compiler. If this recognized that it was compiling the login program, it would insert a master password that would work on any account³. Of course, someone might examine the source code for the compiler, and then compile it again from scratch. So if the compiler recognizes that it's compiling itself, it inserts the vulnerability anyway, even if it's not present in the source. So even if you can buy a system with verifiably secure hardware, operating system and applications, the compiler binary can still contain a Trojan. The moral is that in order to trust a system completely, it is not enough to build all of it, in the sense that software engineers use the word 'build', namely compiling it from source code. You have to create all of it, including the tool chain, and the hardware too.

Malware next became mobile. The first-ever computer virus in the wild was written for the Apple II by a 9th-grader in 1981 [1218]. In 1984 Fred Cohen did a PhD on the topic; his experiments with different operating systems showed how code could propagate itself from one machine to another, and as I mentioned in Section 9.6.4, from one compartment of a multilevel system to another. Within about three years we started to see the first real live viruses in the wild: PC viruses which spread when users shared programs on diskettes or via bulletin boards⁴.

One early innovation was the 'Christma' virus, which spread round IBM mainframes in December 1987. It was a program written in the mainframe command language REXX that had a header saying 'Don't read me, EXEC me' and code that, if executed, drew a Christmas tree on the screen – then sent itself to everyone in the user's contacts file. It was written as a prank, rather than out of malice; and by using the network (IBM's BITNET) to spread, and inviting users to run it, it was ahead of its time.

21.3.2 The Internet worm

The press and public became aware of malware in November 1988 with the Internet worm. This was a program written by Robert Morris Jr that exploited a number of vulnerabilities to spread from one machine to another in November 1988 [617]. It tried 432 common passwords in a guessing attack, looked for any machines trusted by the machine it infected, and also tried to exploit vulnerabilities in Unix (including the `fingerd` bug mentioned in

³This developed an idea first floated by Paul Karger and Robert Schell in the Multics evaluation in 1974 [1022].

⁴Before the Internet was opened up to the public, online services were mostly standalone; bulletin boards were typically operated by hobbyists and would let subscribers or even anonymous users dial in to share information and files.

section 6.4.1). It also took steps to camouflage itself: it was called `sh`, and it encrypted its data strings (albeit with a Caesar cipher).

Its author claimed that his code was not a deliberate attack on the Internet – merely an experiment to see whether code could replicate from one machine to another. But it had a bug. It should have recognised machines that were already infected, and not infected them again, but this feature didn't work. The result was a huge volume of traffic that completely clogged up the Internet (or more accurately, its predecessor the Arpanet) despite the fact that it only affected some 10% of the 60,000 machines on the Arpanet at the time. One lesson was that sites which kept their nerve and didn't pull their network connection recovered more quickly as they could find out what was happening and get the fixes.

21.3.3 Further malware evolution

By the early 1990s, PC viruses had become such a problem that they gave rise to a whole industry of anti-virus software. Through the 1990s, operating systems acquired better access controls, making the malware writer's job harder, but the spread of interpreted languages provided plenty of new opportunities. By the start of the 21st century, the main vector was the macro languages in products such as Word, and the main transmission mechanism had become the Internet [299].

The next phase of malware evolution was to enlist the user as the propagation mechanism. The 'Love Bug' in 2000 was a worm that sent itself to everyone in the victim's address book, with the subject line 'I love you' designed to get people to open it. Its author was Onel de Guzman, a poor computer science student in Manila who wanted to collect passwords so he could use other people's ISP accounts to get online without paying [2019]. This hack to save a few dollars ended up costing millions, and taught us about the difficulty of stopping such things by filtering: a Canadian company with 85,000 staff stripped out all Windows executables at the firewall, but many of their staff had personal webmail accounts, so the Love Bug got in anyway. The company had given each employee a copy of the corporate directory in their address book, and the result was meltdown as 85,000 mail clients each tried to say 'I love you' to each of 85,000 addresses. The Love Bug was followed by similar worms which persuaded people to click on them by offering pictures of celebs such as Britney Spears and Paris Hilton.

The next development was *flash worms* which propagate by scanning the whole Internet for machines vulnerable to some exploit or other, and taking them over; examples such as Code Red and Slammer infected all vulnerable machines within hours or even minutes, and drove research into what sort of automated defences might react in time [1824].

The early 2000s also saw the rise of *spyware* and *adware*. Spyware collects and forwards information from your computer (and now, your phone) without the owner's authorization, or with at best an obscure popup that doesn't really tell you what you're agreeing to. It may also be installed by someone else, such as a parent or partner; spyware is increasingly involved in intimate partner abuse. Adware may bombard the user with advertising popups and can be bundled with spyware. The vendors of such products have even sued antivirus companies who blacklisted their wares. Some spyware is installed deliberately, whether by companies who want to keep tabs on staff, by parents who want to see what their kids are up to, or by abusive men who want to monitor and control their partners. Boundaries are difficult and different people may have different views.

A sea-change came about in 2004–6. Until then, most malware writers did so for fun or to impress their friends – basically, they were amateurs. Since then, the emergence of underground markets and crime forums has made the whole business much more professional. Malware writers now get paid money for software to recruit machines that can be sold on for cash to botnet herders and for other exploits.

Back in the amateur era, most viruses were flaky; very few actually spread in the wild. If code isn't infectious enough it won't spread, but if you make it too infectious then within a few hours the world's anti-virus vendors are upgrading their products to detect and remove it. Now that malware writers focus on money rather than bragging rights, they tend to avoid self-replicating worms in favour of more controllable exploit campaigns. (The main exception is when exploiting IoT devices that can't be patched.)

By the late 2000s, the largest botnets were using professional online marketing techniques to grow their network. Various stories were used to get people to click on a link and run a Trojan that would drop a rootkit on to their machine. Victims had to click away several warnings to install software; but Windows pops up so many annoying dialog boxes that most people just click them away. One of the first really large ones, Storm, earned its living from pump-and-dump operators and pharmacy scammers [1092]. Security researchers tried to disable big botnets by finding and taking down their command-and-control server; Storm used a peer-to-peer architecture that removed this single point of failure [1839]. In the end, it was targeted by Microsoft for removal. The same game is still being played; in March 2020 Microsoft took down Necurs, a botnet with nine million machines that had been growing for eight years, distributing banking Trojans as well as ransomware and email spam [353].

Flash worms have made a comeback since October 2016 with the Mirai worm and its variants. Mirai initially took over WiFi-attached CCTV cameras that had a known root password and software that could not be upgraded; all such devices in the IPv4 address space could be found and recruited within an hour

or so. Since then, there have been over a thousand Mirai variants attacking various IoT devices.

21.3.4 How malware works

Malware typically has two components – a replication mechanism or dropper, and a payload. A worm simply makes a copy of itself somewhere else when it's run, perhaps by breaking into another system by password guessing or using a remote code execution vulnerability (both of which were used by the Internet worm). Viruses spread in other software, perhaps as macros in documents, while Trojans are typically executed by the victim.

The second component of a virus is the payload. When activated, this may do one or more of a number of bad things:

- exfiltrate your confidential data;
- attack you directly using banking malware or spyware;
- encrypt your data and demand a ransom;
- attack others, such as when GCHQ's Operation Socialist described in section 2.2.1.9 subverted Belgacom and installed software in it to do surveillance of mobile-phone traffic passing through Belgium to other countries;
- perform some other nefarious task, such as using the CPU to mine cryptocurrency;
- install a rootkit or remote access Trojan to enable its controllers to do any of the above things, to coordinate attacks with malware on other machines, and to update itself in response to any countermeasures.

If the target is not an individual but a company – as in the Belgacom case – then the attack may involve weeks to months of work. Once attackers control a device on the target network, they will want to move sideways to map the network and find key assets such as authentication servers and mail servers so they can expand the compromise and install remote access Trojans to get a permanent presence. There are many possibilities.

1. In the old days, an attacker would install packet sniffer software to harvest passwords and compromise other accounts, eventually including a sysadmin's. Good practice nowadays is to block such attacks using two-factor authentication, or using a protocol such as Kerberos or SSH to ensure that clear text passwords don't go over the LAN.
2. Other techniques target shared resources such as file servers. For example, Linux servers may use the *Network File System* (NFS) protocol; when a volume is first mounted, the client gets a *root filehandle* from the server – an access ticket that doesn't depend on the time and can't be

revoked. We block this at our own lab using Kerberos to authenticate clients and servers. There are similar problems with Windows file shares, although the details are different; the EternalBlue vulnerability used by the WannaCry and NotPetya worm exploited such file shares.

3. Security mechanisms such as SSH bring further vulnerabilities in that machines in large organisations may have many thousands of SSH keys to communicate with each other, and intruders can exploit them and the trust structures they create to move around.

To get an idea of the range of tools available to a capable attacker nowadays, I'd suggest you browse the NSA papers released by Ed Snowden and the CIA toolkits leaked in the Vault 7 disclosure. Cyber warriors have a range of exploit kits, droppers, RATs and software for stealthy exfiltration of intelligence product.

The takeaway is that the ease with which an intruder on your network can take over other machines depends on how tightly you have the network locked down, and the damage that can follow any breach will depend on the extent to which other machines in your network trust, or are vulnerable to, the compromised machine. This is one of the arguments for not trusting local networks, but insisting on strong authentication between clients and servers at all times.

21.3.5 Countermeasures

Within a few months of the first PC viruses appearing in the wild in 1987, there were startups selling antivirus software. This led to an arms race in which virus and antivirus developers tried to outwit each other.

Early antivirus software came in basically two flavours – *scanners* and *checksummers*. Scanners search executable files for an *indicator of compromise* (IoC), typically a string of bytes from a specific virus. Malware developers responded in various ways, and the dominant technique became *polymorphism*. The idea is to change the code each time the malware replicates, to make it harder to find stable IoCs. The usual technique is to encrypt the code, and have a small header that contains decryption code. With each replication, the malware re-encrypts itself under a different key, and tweaks the decryption code by substituting equivalent sequences of instructions. Modern malware may be run through half-a-dozen such *packers* in turn, and recursively unpack itself when run. AV firms fight back by running the code in a virtual machine, so the malware devs include VM-detection code. The AV firms can at least use the unpacked code as an IoC so long as they can hack through to the last unpacking operation.

Checksummers keep a whitelist list of all the authorised executables on the system, together with checksums of the original versions, typically computed

using a hash function. The malware devs' main countermeasure is *stealth*, which in this context means that the malware watches out for operating system calls of the kind used by the checksummer and hides itself whenever a check is being done.

To provide robust defences against malware, you have to combine tools, incentives and management. We learned in the old days of DOS-based file viruses to provide a central reporting point for all incidents, and to control all software loaded on an organisation's machines. The main risks were machines used at home both for work and for other things (such as kids playing games), and files coming in from other organisations. The same principles still apply. However, firms now need a more coordinated response than before. One of the reasons is that antivirus software has been getting steadily less effective. The commercialisation of botnets and of machine exploitation has meant that malware writers operate like companies, with research and test departments. Almost all exploits are undetectable by the current antivirus products when first launched (if their writers test them properly) and many of them recruit their target number of machines without coming to the attention of the antivirus industry. The net effect was that while antivirus software might have detected almost all of the exploits in circulation in the early 2000s, by 2010 the typical product might detect only a third of them, and by 2020 you expect to detect infection after the fact and have to clean up. That means having good tool support, logging network traffic and analysing it in the light of the latest threat intelligence. What's more, the rootkit vendors provide after-sales service; if a removal kit is shipped, the rootkit vendor will rapidly ship countermeasures. And nowadays many attackers – especially the competent ones – don't leave malware files lying around but 'live off the land'; they might just add their ssh key to a list of authorised keys on one of your servers so they can pop in when they feel like it, leaving nothing for legacy AV to find.

21.4 Defense against network attack

In defending against malware and network attack generally, the view from the second edition of this book in 2008 was that you needed three things: good enough management to keeping your systems patched up-to-date and configured properly; firewalls to stop known Trojans and network exploits; and intrusion detection to monitor your networks and machines for indicators of compromise so you can catch the stuff that got through and clean up afterwards.

The principles remain the same in 2020 but reality is much more complex now, because the scale and complexity of the task have made automation

almost essential. A large Windows shop might have something like the following:

1. An agent running on each endpoint, reporting to a cloud service to give you full visibility of what software is running where and to enable you to push updates;
2. A vulnerability scanner that continually probes your network for known vulnerabilities;
3. Various boundary control devices which may include firewalls, a proxy server that filters all URLs of websites that staff visit, and proxies for critical applications;
4. An SSL gateway for staff working remotely;
5. A *bring-your-own-device* (BYOD) manager, to control laptops, phones and other devices that staff members use but that the firm doesn't own;
6. A *data leakage prevention* (DLP) system to identify staff who attempt to remove company documents or code;
7. A threat intelligence platform that integrates feeds from multiple providers, to alert you to various indicators of compromise including bad DNS names and IP addresses;
8. A log analysis tool that enables you to go back and work out when a compromise first happened, and how far it spread;
9. A *security orchestration and response* (SOAR) system that helps you respond quickly if you note that some devices in your network are communicating with bad addresses such as the command-and-control servers of known malware.

Making all this work together requires system integration, otherwise you'll have dozens of staff in your network security centre whose job is to copy lists of bad domains, bad IP addresses and other indicators of compromise from one tool to another.

That said, let's work our way down this list.

Organisations that are serious about IT security – because they are targets of state actors (like big service firms), or have demanding compliance requirements (like banks), or have a lot to lose (like the military) – aim to stop all vulnerabilities at source. This means keeping everything patched up to date, which in turn means automated patch management. But such a strategy is harder than it looks. It brings with it a number of hard subproblems, such as maintaining an accurate inventory of all the devices on your network. If you impose a rigid bureaucracy for registering new devices, people will have to find ways to circumvent it to get their work done. So you need to also scan your network to see what's there and whether it's vulnerable. And even diligent organisations may find it's just too expensive to fix all the security

holes at once; patches may break critical applications, and an organisation's most critical systems often run on the least secure machines, as administrators have not dared to upgrade them for fear of losing service.

This interacts with operational security. In Chapter 2 and Chapter 8 we discussed the practice and limitations of training staff to not expose systems by foolish actions. By the mid-2000s, the main attack vector was spearphishing – getting people to click on links in email that download and install rootkits. We learned from Ed Snowden that this was the standard way for the NSA to attack a company in 2013: they would monitor external traffic to identify sysadmins, do some background research to identify individual targets, and craft a convincing phishing lure. Alternatively they would direct the target to a website they could spoof or where they could mount a man-in-the-middle protocol attack.

You may try to educate your staff to not click on links in suspicious mail, but competent attackers create mails that don't look suspicious. And so many businesses expect their customers and suppliers to click on links that your staff will have to do some clicking to get their work done. We discussed in Chapter 3 and elsewhere that victim blaming is maladaptive; if your security systems are not usable, you have to fix them rather than blaming the poor users.

Many firms mitigate the risk by opening all mail attachments in a cloud service rather than a local machine, giving staff non-Windows machines such as Chromebooks, iPads or Macs, or having a firewall or mail filter that strips out suspicious content.

21.4.1 Filtering: firewalls, censorware and wiretaps

A *firewall* is a machine that stands between a private network and the Internet, and filters out traffic that might be harmful. It's named after the metal bulkhead that separates the passenger compartment of a car or light plane from the engine compartment, to protect the occupants from a fuel fire. Firewalls were controversial when they appeared in the mid-1990s; purists said that all the machines in a company should be secured, while firewall advocates said this was impractical. The debate has swung back and forth since.

Firewalls are just one example of systems that examine streams of packets and perform filtering or logging operations. Bad packets may be thrown away, or modified in such a way as to make them harmless. They may also be copied to a log or audit trail. Very similar systems are also used for Internet censorship and for law-enforcement wiretapping; almost everything I'll discuss in this section goes across to those applications too. Developments in any of these fields potentially affect the others; and actual systems may have overlapping functions. For example, many corporate firewalls or mail filters screen out pornography, and some even block bad language, while ISP systems that

censor child pornography or dissenting political speech may report the perpetrators automatically to the authorities. Many filters also keep logs, so that attacks can be investigated after the fact; and in parts of the financial sector, all staff communications are required to be logged so that regulators can investigate any suspicions of insider trading or money laundering.

Filters come in basically three flavours, depending on whether they operate at the IP packet level, at the TCP session level or at the application level.

21.4.1.1 *Packet filtering*

The simplest kind of filter merely inspects packet addresses and port numbers. This functionality is available as standard in routers, in Linux and in Windows. You can block IP spoofing by ensuring that only ‘local’ packets leave a network, and only ‘foreign’ ones enter. It’s also easy to block traffic to or from ‘known bad’ IP addresses. For example, IP filtering is a major component of the censorship mechanisms in the Great Firewall of China; a list of bad IP addresses can be kept in router hardware, which enables packet filtering to be done at great speed.

Basic packet filtering is often used to block all traffic except that arriving on specific port numbers. You might initially allow the ports used by common services such as email and web traffic, and then open up further ports as needed. As we move to *software defined networks* (SDN), which replace expensive routers with cheap switches controlled by software on commodity servers, packet filtering rules become just the access-control rules in the SDN controller.

However, packet filters can be defeated by a number of tricks. For example, a packet can be fragmented in such a way that the initial fragment passes the firewall’s inspection but is then overwritten by a subsequent fragment, replacing the source address with one that violates your security policy. Another limitation is that maintaining a blacklist is difficult, especially when it’s not the IP address specifically you want to block, but something that resolves into an IP address, especially on a transient basis. For example, phishermen use tricks like fast-flux in which a site’s IP address changes several times an hour.

21.4.1.2 *Circuit gateways*

The next step up is a *circuit gateway* that reassembles and examines all the packets in each TCP session. This is more expensive than simple packet filtering but can also provide the added functionality of a *virtual private network* (VPN) whereby corporate traffic passed over the Internet is encrypted from firewall to firewall. I’ll discuss the IPSEC protocol that’s used for this in the last section of this chapter.

TCP-level filtering can be used to do a few more things, such as DNS filtering. However, such a filter can't screen out bad things at the application level, from malicious code to child sex abuse material. Thus it may be programmed to direct certain types of traffic to application filters.

21.4.1.3 *Application proxies*

The third type of firewall is the *application proxy*, which understands one or more services. Examples are mail filters that try to weed out spam, and web proxies that block or remove undesirable content. The classic objective is stripping out code, be it straightforward executables, active content in web pages, or macros from incoming Word documents. The move to web-based mail services and the adoption of https have left significantly less work for mail filters to do, and as the service firms adopt technical measures such as certificate transparency to prevent proxying, filtering needs to shift to endpoints.

An application proxy can also be a bottleneck. An example is the Great Firewall of China, which tried through the 2000s to block mail and web content that refers to banned subjects [450]. Since the adoption of https by the major service providers, and the availability of services such as Google Docs that can also be used for communication, China simply stops most of its citizens from using services like Gmail and Facebook.

In the emerging BeyondCorp model promoted by Google, proxies sit in front of the application servers themselves so that the internal network does not need to be trusted.

21.4.1.4 *Ingress versus egress filtering*

Most firewalls look outwards and try to keep bad things out, but some look inwards and try to stop bad things leaving. The pioneers were military mail systems that monitor outgoing traffic to ensure that nothing classified goes out in the clear. Around 2005 some ISPs started looking at outgoing mail traffic to try to detect spam [444]; and by now most consumer ISPs prevent their customers sending packets with spoofed source addresses. This *source address validation* means that DDoS operators using UDP reflection attacks can no longer use botnets but need to rent servers in data centres.

The fastest-growing use of egress filtering in 2020 is for *data leakage prevention* (DLP). Software that 'phones home', whether for copyright enforcement or marketing purposes, can disclose highly sensitive material, and prudent organisations increasingly wish to monitor and control this kind of traffic. But the pervasive use of https means that DLP systems typically need to install software on endpoints rather than using middleboxes.

21.4.1.5 Architecture

For years, many firms bought a firewall to keep their auditors happy. If that's your pain point, a simple filtering router won't need much maintenance and won't get in the way too much. At the other extreme, a serious firewall system at a defence contractor might consist of a packet filter connecting the outside world to a screened subnet, also known as a *demilitarized zone* (DMZ), which in turn contains a number of application servers or proxies to filter mail, web and other services. You may also expect to find data diodes separating networks operating at different clearance levels, to ensure that classified information doesn't escape either outwards or downwards (Figure 21.2).

An alternative approach is to have more networks, but smaller ones. At our university, we have firewalls to separate departments, although we've got a shared network backbone and some shared central services such as logging. There's no reason why the students and the finance department should be on the same network, and a computer science department has got quite different requirements from a department of theology. Keeping each network small limits the scope of any compromise and helps incentivise system administrators to defend it.

Considerations in the design of a network security architecture include simplicity, usability, deperimeterisation versus re-perimeterisation, underblocking versus overblocking, maintainability, and incentives.

First, since firewalls do only a small number of things, it's possible to make them simple to remove sources of vulnerability and error. If your organisation has a heterogeneous population of machines, then loading as much of the security task as possible on a small number of simple boxes makes sense. On

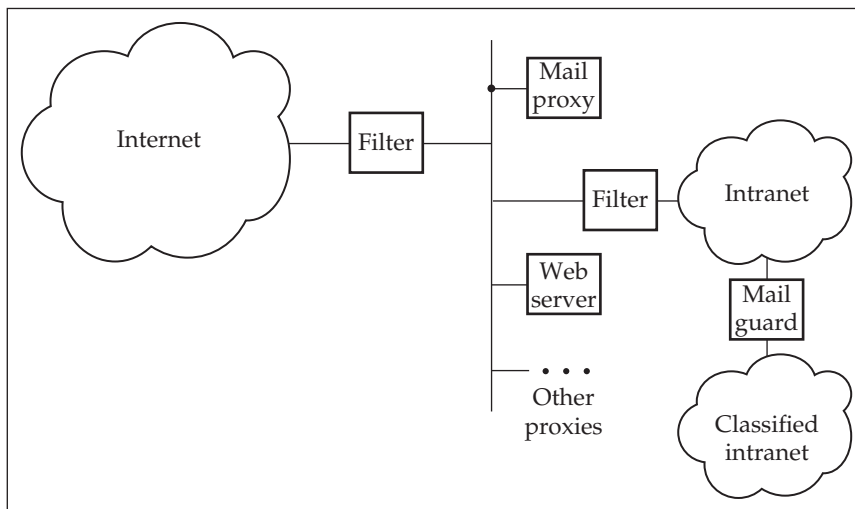


Figure 21.2: complex firewalls for an MLS network

the other hand, if you're running something like a call centre, with a thousand identically-configured PCs, it makes sense to put your effort into keeping this configuration tight. These are roughly the energy utility, and Google, models discussed in the introduction at the start of this chapter.

Second, elaborate central installations not only impose greater operational costs, but can get in the way so much that people install back doors, such as cable modems that bypass the firewall, to get their work done. I will discuss in section 20.4 how diplomats have come unstuck by using private email when their official systems were unusable. Many well-run firms have open guest networks, as does our department; there's always got to be something that works. And a prudent system administrator will monitor the actual network configuration rather than just relying on 'policy'.

Third, firewalls only work until people find ways round them. Early firewalls let only mail and web traffic through; so writers of applications from computer games to anonymity proxies redesigned their protocols to make the client-server traffic look as much like normal web traffic as possible. Then everything moved to Web 2.0 and such filters became largely ineffective.

Next, there's deperimeterisation – as Google's BeyondCorp notes, it's becoming steadily harder to put all the protection at the perimeter, thanks to the proliferation of phones and PDAs being used for functions that used to be done on desktop computers, and by changing business methods that involve more outsourcing of functions – whether formally to subcontractors or informally to advertising-supported web apps. If some parts of your organisation can't be controlled (e.g., the sales force and the R&D lab) while others must be (the finance office) then you may need separate architectures. The proliferation of web applications is complemented by a blunting of the incentive to do things at the perimeter, as useful things become harder to do. The difference between code and data is steadily eroded by new scripting languages. Many firms tried to block JavaScript in the early 2000s but were beaten by popular web sites that require it. Nowadays it may be impossible to prevent your staff attaching large numbers of IoT devices that just cannot be secured at all [1256].

And then there's our old friend the Receiver Operating Characteristic or ROC curve. No filtering mechanism has complete precision, so there's inevitably a trade-off between underblocking and overblocking. If you're running a censorship system to stop kids accessing pornography in public libraries, do you underblock, and annoy parents and churches when some pictures get through, or do you overblock and get sued for infringing free-speech rights? Things are made worse by the fact that the firewall systems used to filter web content for sex, violence and bad language also tend to block free-speech sites (as many of these criticise the firewall vendors – and some offer technical advice on how to circumvent blocking).

And as we've repeatedly pointed out, security depends at least as much on incentives as on technology. A sysadmin who looks after a departmental network used by a hundred people they know, and who will personally have to clear up any mess caused by an intrusion or a configuration error, is much more motivated than someone who's merely one member of a large team looking after thousands of machines.

21.4.2 Intrusion detection

Attacks will happen, and it's often cheaper to prevent some attacks and detect the rest than it is to try to prevent everything. The systems used to detect bad things happening are referred to generically as *intrusion detection systems* (IDS). The antivirus software products I discussed earlier are one example; but the term is most usually applied to boxes that sit on your network and look for signs of an attack in progress or a compromised machine [1639]. Examples include:

- a machine trying to contact a 'known bad' service such as an IRC channel that's used to control a botnet, or a known-bad IP address – or trying to resolve a known-bad DNS name;
- packets with forged source addresses – such as packets that claim to be from outside a subnet but that actually originate from it;
- spam coming from a machine in your network.

In cases like this, the IDS typically tells the sysadmin that a particular machine needs to be looked at. This may be just the first step in an investigation that involves staring at logs to see how it happened, and what else the attackers might have infected.

Other examples of intrusion detection, which we've seen in earlier chapters, include mechanisms for detecting payment card fraud and stock-market systems that look for insider trading, such as via increases in trading volume just before a price-sensitive announcement. This is now an active area of research: the boom in AI since 2012 has created lots of startups looking for pattern-matching problems.

21.4.2.1 Types of intrusion detection

The simplest intrusion detection method is to sound an alarm when a threshold is passed. Three or more failed logons, a credit card expenditure of more than twice the moving average of the last three months, or a mobile phone call lasting more than six hours, might all flag an account for attention. More sophisticated systems generally fall into two categories.

Misuse detection systems operate using a model of the likely behaviour of an intruder. A banking system may alarm if a user draws the maximum permitted amount from a cash machine on three successive days; and a Unix intrusion detection system may look for user account takeover by alarming if a previously naive user suddenly starts to use sophisticated tools like compilers. Simple misuse detection systems, such as antivirus scanners, look for a *signature* – a known characteristic of a specific attack. This can be either explicit in the data (such as a substring of an executable file that marks it as a specific piece of malware) or in behaviour (such as a machine contacting the IP address of a known botnet command-and-control server). More complex misuse detection systems treat a number of signatures as signals and then train a machine-learning classifier to make the decisions. As I discussed in section 12.5.4, the systems used to detect card fraud use dozens of signals, as they need low false alarm rates to be useful given the scale of modern payment systems.

Anomaly detection systems attempt the much harder job of looking for anomalous behaviour in the absence of a clear model of the attacker's *modus operandi*. The hope is to detect attacks that have not been previously recognized and cataloged. Systems of this type have used AI techniques since the 1990s, though some firms eschew them; Google policy, for example, is to avoid systems that try to learn thresholds or automatically detect causality, and instead have simple systems that detect changes in end-user request rates [237].

The dividing line between misuse and anomaly detection is somewhat blurred. A borderline case is Benford's law, which describes the distribution of digits in random numbers. One might expect that numbers beginning with the digits '1', '2', ... '9' would be equally common. But when numbers come from random natural sources and span more than one order of magnitude, so that their distribution is independent of the number system in which they're expressed, the distribution is logarithmic: about 30% of decimal numbers start with '1'. Crooked clerks who think up numbers to cook the books, or even use random number generators without knowing Benford's law, are often caught this way [1249].

Another borderline case is the *honeypot* – something enticing left to attract attention. I mentioned, for example, that some hospitals have dummy records with celebrities' names in order to entrap staff who ignore patient confidentiality. In the network context, honeypots emulate many types of device so that attackers scanning the Internet looking for (say) a DSL modem of a particular upgrade status find one to attack; this may contain either a simple emulator, or with more recent designs, the actual modem firmware running in a VM [1959]. The upshot is that the honeypot operator gets to see who's attacking what, and how.

21.4.2.2 General limitations of intrusion detection

Some intrusions are obvious. If you're worried about activists vandalising your web site, then have a machine somewhere that fetches the page frequently and rings an alarm when it changes. But in the general case, intrusion detection is hard. The virus pioneer Fred Cohen proved that detecting viruses (in the sense of deciding whether a program is going to do something bad) is as hard as the halting problem, so we can't ever expect a complete solution [452].

There's also a matter of definitions. Some intrusion detection systems are configured to block some kinds of suspicious behaviour. But this turns the intrusion-detection system into an access control mechanism, as well as opening the door to service-denial attacks. I prefer to define an intrusion-detection system as one that monitors the logs and draws attention to suspicious occurrences.

Then there's the cost of false alarms. Academic machine-learning researchers often consider they've done well when they train a classifier to have a false alarm rate of 0.1%. But if you're on the Gmail team and dealing with a billion users authenticating themselves every day, that's way too much. Large-scale systems need really low false alarm rates.

Finally, there are three generic problems with machine-learning classifiers: the facts that they're not much good at detecting new attacks, that people game them, and that they inhale the prejudices of their training data. We will discuss these in more detail in section 25.3.

21.4.2.3 Specific problems detecting network attacks

Turning now to the specific problem of detecting network intrusion, it's harder to spot than payment fraud. Network intrusion detection products still have high missed alarm and false alarm rates. It's common to detect actual intrusions only afterwards. The reasons for the poor performance include the following, in no particular order.

- The Internet is a very noisy environment – not just at the level of content but also at the packet level. A lot of random crud arrives at any substantial site, and enough of it can be interpreted as hostile to provide a significant false alarm rate. Many bad packets result from software bugs; others are the fault of out-of-date or corrupt DNS data; and some are local packets that escaped, travelled the world and returned [214].
- There are 'too few attacks'. If there are ten real attacks per million sessions – which is almost certainly an overestimate – then even if the system has a false alarm rate as low as 0.1%, the ratio of false to real alarms will be 100. We talked about similar problems with burglar alarms; it's also a well-known problem for medics running

screening programs for diseases like HIV where the test error rate exceeds the disease prevalence. Where the signal is way below the noise, the guards get tired and the genuine alarms get missed.

- While a theft from a bank causes an incorrect state – money in the wrong place, and evidence on the audit trail – many network intrusions aim to avoid this, for example if their mission is to exfiltrate confidential data. It's easier to write software to detect errors than it is to detect slightly odd behaviour.
- Many network attacks are specific to particular versions of software, so you need a large and constantly-changing library of attack signatures. However, many firms buy intrusion detection systems in order to satisfy insurers or auditors, and the products aren't always kept up to date.
- As more and more traffic is encrypted, it can't easily be subjected to content analysis or filtered for malicious code. If DNS-over-https becomes the norm, tools that rely on analysing your DNS traffic will become much less effective.
- The issues we discussed in the context of firewalls largely apply to intrusion detection too. You can filter at the packet layer, which is fast but misses a lot; or you can proxy your applications, which is expensive – and needs to be constantly updated to cope with new applications and attacks.
- You may have to do intrusion detection both locally and globally. More and more things have to be done on local machines, thanks to encrypted web sessions; but some attacks are *stealthy* – the opponent sends 1–2 packets per day to each of maybe 100,000 hosts, and you need a central monitor that counts packets by source and destination address and by port.

Nowadays, intrusion detection systems involve the coordination of multiple monitoring mechanisms and products at different levels both in the network and on your fleet of endpoint devices. A large company with tens of thousands of staff using Windows will typically have several dozen products, as I discussed previously in section 21.4. Integrating and automating both monitoring and response makes up more and more of a CISO's job. The growth areas therefore include integration tools for *security incident and event management* (SIEM), *security orchestration and response* (SOAR), and metrics.

21.5 Cryptography: the ragged boundary

Network security interacts with cryptography in a number of ways. We already mentioned the debate about DNS over https; now I'm going to describe five

other aspects of crypto briefly. They are SSH; the local link protection offered by WiFi, Bluetooth and HomePlug; the IPSec mechanisms used in VPNs; TLS; and the *public key infrastructures* (PKI) used to support many of these. In the previous chapter, we discussed how attempts to build more trustworthy components out of cryptography run up against many real-world engineering and economic constraints. The tools that we use to set boundaries on networks, and to translate trust within them, are no different.

The emerging themes are that the most distributed part of the problem is unmanageable because the vendors don't care; in particular the thousands of device types being marketed as part of the 'Internet of Things' have no remote management facility available to users, the vendor often doesn't upgrade the software, and the lack of a user interface means that authentication is haphazard at best. Meanwhile the most centralised part of the problem – PKI – is often subverted by government mandates.

21.5.1 SSH

When I use my laptop to access files on my desktop machine, or do anything with any other machine in our lab for that matter, I use *secure shell* (SSH) which provides encrypted links between Unix and Windows hosts. So when I work from home, my traffic is protected, and when I log on from the PC at my desk to another machine in the lab, the password I use doesn't go across the LAN in the clear.

SSH was initially written in 1995 by Tatu Ylönen, a researcher at Helsinki University of Technology, following a password-sniffing attack there [2061]. It sets up encrypted connections between machines, so that logon passwords don't travel across the network in the clear, and supports other useful features that led to its rapid adoption [1620].

There are various configuration options, but in the most straightforward one, each machine has a public-private keypair. The private key is protected by a passphrase that the user types at the keyboard. To connect from my laptop to a server at the lab, I install my laptop public key in a file on the relevant server. When I wish to log on to a server I'm prompted for my passphrase; the two machines set up a Diffie-Hellman key; the private keys are used to sign the transient public keys, to stop middleperson attacks; the subsequent traffic is thus both encrypted and authenticated. Manual key installation is intuitive, but doesn't scale particularly well. There are also options to use Kerberos, whether to authenticate the session key set up using Diffie-Hellman, or to set up the session key directly. (In the latter case, SSH falls back to being a variant of Kerberos in the sense that it is now a trusted third-party protocol, and the police can get the Kerberos server to decrypt the traffic.)

Possible problems include the fact that if you're typing at the keyboard one character at a time, then each character gets sent in its own packet, and

the packet interarrival times can leak a lot of information about what you're typing [1807]. However, the worst is probably that most SSH keys used for server-to-server communication are stored in the clear, without being protected by a password at all. So if a server is compromised, the same can happen to every other machine that trusts an SSH key installed on it.

SSH is often used as a simple logon mechanism; many IoT devices run Linux and allow remote logon by anyone who knows an appropriate password. This opens them to password-guessing attacks, and where there are weak passwords or a known default password, to recruitment into botnets based on Mirai and similar tools. The countermeasure here is honeypots.

21.5.2 Wireless networking at the periphery

Many networks use wireless technology at the edge to go the last few feet from an access point to a device, or from one device to another. Protocols such as WiFi, Bluetooth and HomePlug all offer encryption to provide some protection against service abuse and perhaps against eavesdropping. However most are vulnerable to local attacks that are difficult to block completely because many devices don't get patched, lack user interfaces, or both.

21.5.2.1 WiFi

WiFi supports wireless local area networks, whether at home to connect phones and other devices to a home router, or by businesses to connect payment terminals and stock control devices as well as PCs. It has come with a series of encryption protocols since its launch in 1997. The first widely-used one, WEP (for *wired equivalent privacy*), was shown to be fairly easily broken because of the weak ciphers demanded by US export control and poor protocol design [300, 1876]. Since 2004, an improved system called WPA2 uses AES encryption. The key for each access point is typically printed on a card that fits into the back of the router.

Should WiFi networks be seen as untrusted? The reason to set a password is more to prevent third parties using your bandwidth or quota, rather than the risk of pharming. Many people in the UK or America find it convenient to have an open network for guests to use, and so that you and your neighbours can use each others' networks as backups. In countries where you pay for download bandwidth, home router passwords are mostly set. In some, like India, it's against the law to run an open WiFi access point (terrorists who mounted an attack in Bombay in 2008 used them to call home unobtrusively). Having the key on a card is a neat example of usable security design: the householder can make their network as open or as secure as needed by pinning the card on the wall or by locking it up.

WiFi security is still somewhat fragile. *Universal Plug and Play* (UPnP) lets any device in a network punch a hole through the router's firewall; DHS has been recommending since 2013 that people turn it off. However now that many devices and domestic appliances come with an attached cloud service, that's hard. It's used along with *WiFi Protected Setup* (WPS) which lets you enrol gadgets on your network with a simple button press. You can set a PIN but there have been a couple of attacks found on the mechanism.

Businesses may have to take a bit more care. In March 2007, retail chain TJ Maxx reported that some 45.7 million credit card numbers had been stolen from its systems; the Wall Street Journal reported that an insecure WiFi connection in St Paul, Mn., was to blame [1511]. Banks sued the company, and eventually settled for \$41m [789].

Patching is an issue. For example, in March 2020 we learned of the Kr00k vulnerability in Broadcom WiFi chips which will get patched in Macs and iPhones but probably not in wireless routers or older Android phones [800]. As for the great majority of IoT devices, from toys through home appliances, they won't get patched, ever.

21.5.2.2 Bluetooth

Bluetooth is another short-range wireless protocol, aimed at *personal area networks*, such as linking a headset to a phone, or a phone in your pocket to a hands-free interface in your car. It's also used to connect cameras and phones to laptops, keyboards to PCs and so on. Like WiFi, the first versions of the protocol turned out to have flaws [1103, 1716]. From version 2.1 (released in 2007), Bluetooth has supported Secure Simple Pairing, which uses elliptic-curve Diffie-Hellman to thwart passive eavesdropping attacks. Man-in-the-middle attacks are dealt with by generating a six-digit number for numerical comparison. However, because one or both of the devices might lack a keyboard or screen (or both), it's also possible for the number to be generated at one device and entered as a passkey at another; and there's a 'just works' mode that's not protected against middleperson attack. What's more, the data may or may not be signed, giving a total of about ten different combinations of confidentiality, integrity and resistance to man-in-the-middle attack; and a number of attacks have been found, some inspired by NSA tools listed in the Snowden disclosures [1638]. Again, patching is an issue. In 2018, Eli Biham found that many implementations could be fooled by a man-in-the-middle supplying an invalid elliptic curve to the authentication protocol [245], and in 2020 Daniele Antonioli and colleagues discovered a variant of the man-in-the-middle attack where you just reflect the challenge from a bluetooth device back to it, claiming that you're now the challenger

and the target device is the responder [126]. So if you have a device with a bluetooth chip that hasn't been patched, it may be vulnerable.

21.5.2.3 HomePlug

HomePlug is a protocol used for communication over the mains power cables. HomePlug AV is widely used in WiFi extenders: you plug one station into your router or cable modem, and another gives a remote WiFi access point at the other end of your house. (Declaration of interest: I was one of the protocol's designers.) We were faced with the same design constraints as the Bluetooth team: not all devices have keyboards or screens, and we needed to keep costs low. We decided to offer only two modes of operation: secure mode, in which the user manually enters into their network controller a unique AES key that's printed on the device label, and 'simple connect' mode in which the keys are exchanged without authentication. The keys aren't even encrypted in this mode; its purpose is not to provide security but to prevent wrong associations, such as when a device wrongly mates with a network next door [1438]. However many vendors just support the 'simple connect' mode and end up with a policy of trust on first use, as already mentioned in section 4.7.1. Others sell extenders in pairs, with keys already installed. There are variants for smart meters to communicate with substations, and for electricity utilities to provide broadband to the home over the power line (though these are not widely used because of radio frequency interference). Vendors also customised the product in various ways to make it incompatible with competitors. As a result of this mess, little reliance can be placed on the key management.

21.5.2.4 VPNs

Virtual private networks (VPNs) typically do encryption and authentication at the IP layer using a protocol suite known as IPsec. This defines a *security association* as the combination of keys, algorithms and parameters used to protect a particular packet stream. Protected packets may be just authenticated, or encrypted too; in the former case, an authentication header is added that protects data integrity, while in the latter the packet is also encrypted and encapsulated in other packets. There's also an *Internet Key Exchange* (IKE) protocol to set up keys and negotiate parameters, and we may infer from Ed Snowden's disclosures that the standard default settings of this (with 1024-bit Diffie-Hellman) are insecure.

VPNs are offered by firewall vendors so that by installing one of their boxes in each branch between the local LAN and the router, all the internal traffic can pass encrypted over the Internet. Individual workers' laptops and home

PCs can also join a VPN given appropriate software. VPNs are also offered commercially, and are used for example by people and firms in countries like Iran and China to circumvent the national firewall.

21.6 CAs and PKI

As we discussed in section 5.7.4, the pioneers of public-key cryptography developed a vision of certificates that would bind public keys to the names or roles of the organisations, people or devices that controlled the corresponding private keys. Initially it was thought that governments or phone companies would do this, but they were too slow. During the dotcom boom, entrepreneurs set up *certificate authorities* (CAs) and software firms such as Microsoft and Netscape embedded their public keys into their browsers. There followed a gold rush as the CAs bought each other and consolidated; investors hoped that every device would need a public-key certificate, so you'd need to pay Verisign ten bucks every two years to renew the certificate on your toaster, or it wouldn't talk to your fridge.

Once that foolishness died down, the world's governments moved to get their own CAs' root certificates into the browsers for intelligence and surveillance purposes. As people moved to web services like Gmail, security agencies developed tools to do man-in-the-middle attacks, and as TLS was used to encrypt password entry (and later, the whole session), this meant having a CA that would produce a certificate on `www.gmail.com` for a security agency public key that the target's browser would accept. In fact, at a panel discussion at Financial Cryptography 2011, I asked the man from Mozilla how come, when I updated Firefox the previous day, it had put back a certificate I'd removed for Tubitak – a Turkish intelligence organisation. At this point a man stood up in the audience and shouted 'How dare you insult my country! Tubitak is not an intelligence agency – it is a research organisation!' The man from Mozilla shrugged and said wryly, 'Now you see how hard certificate governance is.'

Later that year came the DigiNotar scandal. DigiNotar was a Dutch CA which was found to have issued wildcard certificates for Gmail. Iranian agents had hacked it in order to monitor 300,000 Gmail users in Iran; sanctions meant that, unlike Turkey, they could not just have their government certificate installed in the major browsers. Mozilla and Google promptly put DigiNotar to death by removing its root certificates; Microsoft and Apple followed quickly. This caused real disruption in the Netherlands, many of whose online government services used DigiNotar certificates, and had to scramble to get others. It turned out that there had been earlier attacks on another CA, Comodo, but that company claimed to have revoked all its wrongly-issued certificates. Since then, there has been increasing pressure on CAs and auditors from the browsers' root stores.

There is frequent semantic confusion between ‘public (key infrastructure)’ and ‘(public key) infrastructure’. In the first, the infrastructure can be used by whatever new applications come along; I’ll call this an *open PKI*. In the second, it can’t; I’ll call this a *closed PKI*. If you’re building a service that government agencies are likely to attack, then it may be a good idea to keep your PKI closed, with a CA that runs on your own premises – so you get to know of any warrants. I advise firms who maintain software that’s installed on many millions of machines to use a private CA for their code signing keys.

PKI has a number of intrinsic limitations, many of which we discussed in the chapter on distributed systems. Naming is difficult, and the more applications rely on a certificate, the shorter its useful life will be. You can sometimes simplify things by removing unnecessary names: rather than one certificate saying ‘Ross Anderson’s key is KR’ and another saying ‘Ross Anderson has the right to administer x.foo.com’ you might just say ‘KR has the right to administer x.foo.com.’

This is an aspect of the ‘one key or many’ debate. Should I expect to have a single digital credential to replace each of the metal keys, credit cards, swipe access cards and other tokens that I currently carry around? Or should each of them be replaced by a different credential? Multiple keys protect the customer: I don’t want to have to use a key with which I can remortgage my house to buy my lunchtime sandwich. As we saw in the chapter on banking and bookkeeping, it’s easy to dupe people into signing a message by having the equipment display another one.

Now the standard PKI machinery (the X.509 protocol suite) was developed to provide an electronic replacement for the telephone book, so it started off by assuming that everyone will have a unique name and a unique key in an open PKI architecture.

This in turn leads to issues of trust, of which there are many.

- If you remove one of the hundreds of root certificates from Firefox, then Mozilla silently replaces it; Windows comes with even more root certificates – but you can’t delete them at all. In each case, you have to know how to mark a certificate as untrusted.
- There have been some interesting effects where a government that had its cert in Windows but not in other browsers (such as Thailand’s, after the military coup in 2014) had to resort to different surveillance methods for Mac users [1557].
- Many firms use certs that are out-of-date, or that correspond to the wrong company, often because the firm’s marketing department got a contractor to run some promotion or another. As a result, users have been trained to ignore security warnings, and only a small minority used to pay attention to them [842]. Recently browsers such as Firefox have made it harder to click past warnings.

- Certs bind a company name to a DNS name, but their vendors are usually not authorities on either; they hand out certificates after checking that the applicant can answer an email sent to that domain, or put up a web page with a CA challenge on it. Things are slightly better with ‘extended validation’ certificates⁵, but even they aren’t foolproof.
- On their ‘certification practice statements’ CAs go out of their way to deny all liability.
- Certificate revocation is an issue. The original idea was that anyone relying on a cert could download a *certificate revocation list* (CRL) from the CA and check any cert on which they were about to rely. However, this vitiated much of the benefit of public-key cryptography by requiring online operation for high assurance. In addition, users of some systems (particularly US government ones) had to download large CRLs every time they started up their systems, leading to delay and network congestion. Since about 2013, people have moved to the *Online Certificate Status Protocol* (OCSP), a more efficient protocol for online status checking.

Behind all this mess lies, as usual, security economics. During the dotcom boom in the 1990s, the SSL protocol (as TLS then was) won out over a more complex and heavyweight protocol called SET, because it placed less of a burden on developers [111]. The costs of compliance were dumped on the users – who are often unable to cope [524]. Much of the engineering around CAs and certs since then has been playing catchup.

The big issues at the time of writing are certificate lifetime; LetsEncrypt; and certificate transparency.

The maximum permitted lifetime of a certificate, if it’s to be accepted by the main browsers, has steadily reduced from 8 years to 3 years to 27 months. Ballots in 2017 and 2019 proposed a cut to 13 months [1584] and in 2020 Apple forced the issue by declaring that from September, its devices would no longer accept any certs valid for longer than 398 days [1448]. This will force many websites to refresh their certificates; it will be interesting to see how firms flush out all the certs in DNS. (It will also widen the gap between systems with annual certs and some industrial and IoT systems where certs have to last for years because of the difficulty of software upgrade.)

Getting certs used to be difficult as you had to go shopping for one, prove you controlled your domain, get the cert, upload it to your server, change the configuration and then test it all. The change maker here has been a nonprofit, the Internet Security Research Group (ISRG) which provides certs for free and by February 2020 had issued a billion of them. Making certs free allowed full

⁵These used to bring up a green padlock in your browser, though this is being discontinued in Chromium from v 76 in 2020 after research showed that nobody paid any attention.

automation, which keeps costs down: their 'LetsEncrypt' CA supports 100m sites on a budget of \$3m pa. LetsEncrypt set out to make deploying certs easy, and the impact has been real: 20% of browser connections are still in plaintext, but this is down from 60% four years ago. This service started in 2015, two years after the Snowden revelations. Their automated certificate management environment is now standardised as RFC8555, so commercial CAs are using it too. There's a transparency log and the system has no manual override, so there's some assurance that they have never been compelled to issue a cert. (In fact, the NSA uses their certs.) At November 2019, they were the largest CA, with 112m certs for 188m domains; they had 5% of the top hundred sites but 35% of the top million. Their scale means that mistakes affect lots of sites; in March 2020, a bug in their software meant that 3 million certificates covering 12 million server names had to be replaced [590].

Following the attacks on Comodo and DigiNotar, work started on mechanisms to block maliciously issued certificates. Certificate transparency sets out to do this by maintaining logs of all the certificates seen in the wild for each domain, so that domain owners can rapidly spot certs that should not have been issued for their domain. Google launched the first certificate transparency log in 2013 and Chrome started insisting on such logs for extended validation certificates in 2015. Google found that Symantec had issued certificates for a number of domains (including their own) without the domain owner's knowledge [1790], and made certificate transparency mandatory for all CAs in 2018.

21.7 Topology

The topology of a network is the pattern in which its nodes are connected, and this can be a significant component of the security architecture.

- A utility might have a number of islands, each containing a generator or substation with dozens to hundreds of devices on a trusted network, connected in turn via a specialised firewall and a VPN to a network control centre.
- A cloud service provider might have tens of thousands of machines in a data centre, with hierarchies of certificates issued both by the provider and its tenants determining which VMs or containers on which machines can communicate with each other. And while the internal network may be untrusted, in the sense that network location plays no role in access control decisions, it may be shielded from DDoS attacks by front-end systems.
- Classified systems used by governments may have quite large trusted networks operating at elevated levels, with separate LANs in buildings.

More complex topologies can be found where nodes are users and edges are their presence in each others' address books. Social-network analysis has been applied to disciplines from epidemiology through criminology and the study of how new technologies diffuse, to the study of harms transmitted directly between users, such as macro viruses [1435]. Social networks can be modelled by a graph with a power-law distribution of vertex order; a small number of well-connected nodes help make the network resilient against random failure, and easy to navigate. Yet they also make such networks vulnerable to targeted attack. Remove the well-connected nodes, and the network is easily disconnected [37]. Dictators have known this intuitively; Stalin consolidated his rule by killing the richer peasants, Pol Pot killed intellectuals, while William the Conqueror killed the Saxon gentry. Now we have quantitative models, they help explain why revolutionaries have tended to organise themselves in cells [1375]; by doing traffic analysis against just a few well-connected organisers, a police force can identify a surprising number of members of a dissident organisation – unless the dissidents organised in a cell structure in the first place [510].

21.8 Summary

Preventing and detecting attacks that are launched over networks is the core of a modern CISO's job. It's difficult because it involves a huge range of attack types and security technologies. It can lead to newsworthy failures. There is unlikely to be any magic solution, though a lot of things can help. Each new advance opens up new things to worry about; for example, cloud services may shift much of the network security task to a provider, but make configuration management more critical. Overall, the problems are so complex and messy that managing them needs a whole-system approach with automation.

Hacking techniques depend partly on the opportunistic exploitation of vulnerabilities introduced accidentally by the major vendors, and partly on techniques to social-engineer people into running untrustworthy code. However these have developed into a whole ecosystem of bad guys, which a security engineer also needs to study and understand.

Research problems

In 2000, the centre of gravity in network security research was technical: we were busy looking for new attacks on protocols and applications as the potential for denial-of-service attacks started to become clear. By 2010, there was much more discussion of economics and policy: of how changing liability rules

might make things better [98]. By 2020, there is much more work on metrics: on measuring the actual wickedness that goes on, and feeding this not just into the policy debate but also into law enforcement. At the operational level, the game is about automation and integration – about enabling large firms to process large quantities of threat intelligence and network surveillance information, turn it into actionable intelligence, and measure how effectively the network security team is doing its job.

Further reading

The early classic on Internet security was written by Steve Bellovin and Bill Cheswick, with Avi Rubin joining them for the second edition [222]. The seminal work on viruses is by Fred Cohen [452], while Java security is discussed by Li Gong (who designed it) [784]. For BGP security, see our 2011 ENISA report: the full Monty is over two hundred pages, designed for people starting a PhD in network security, but there's a shorter executive summary too [1910].

For a more detailed overview of malware, I might suggest Wenke Lee's Cybok survey paper [1139]; and Sanjah Jha's Cybok survey of network security provides more detail of IPSEC as well as ethernet and port-based security [985].

I'm not aware of any good overview of the certification authority ecosystem. You might start with the 2004 oral history interview with Jim Bidzos, the founder of Verisign [241]. The initial goal of Microsoft and Netscape was to jump-start electronic commerce on the worldwide web; certificate use then spread to passwords and software updates, and when Javascript came along, the same origin principle shifted trust to websites. Many other players jumped in, with some government agencies trying to undermine the CA ecosystem and others trying to reinforce it. There's conflict between technical security goals and legal goals, as well as between auditors and regulators. So there are quite separate views on CA security from WebTrust (the American and Canadian accountants) and ETSI (the most relevant European standards body). For more detail, a presentation by Ryan Sleevi on what's wrong with the ecosystem [1789] has many pointers for those who want to dig into the current problems, both technical and operational, and their background.